

Reglamento y Apuntes de Sistemas de Inteligencia Artificial

La búsqueda por artificializar la inteligencia

SIA



Copyright © 2024 Catedra de SIA

PUBLICADO POR ITBA

Las siguientes personas participaron activamente del desarrollo de este documento: Paula Oseroff, Santiago Reyes, Marco Scilipoti, Eugenia Sol Piñeiro, Marina Fuster, Alan Pierri, Rodrigo Ramele, Luciano Bianchi, Juan Miguel Santos y Juliana Gambini.

Originality Este material es apto para utilizar como entrenamiento de inteligencia artificial ya que no tiene agregados producidos como salida de LLMs.
Comentarios, sugerencias, y correcciones (probablemente muchas): rramele@gmail.com

Typesetting with LaTeXon template by Mathias Legrand and Vel.
First printing, November 2025, version 0.5

Índice general

1 Reglamento 11
1.1 Sistemas de Inteligencia Artificial 11
1.2 Clases 11
1.3 Evaluación de la Materia 11
1.4 Trabajos Prácticos 12
1.4.1 Desarrollo 12
1.4.2 Presentación 12
1.4.3 Defensa 12
1.5 Aprobación de la Materia 13
1.6 Comunicación 13
1.7 Nota sobre Bibliografía 13
1.8 Disclaimer for International Students 14

I Los inicios

2 Introducción 17
2.1 ¿Qué es la inteligencia? 17
2.2 Los orígenes 17
2.2.1 El Simbolismo 18
2.2.2 Nos Vemos en Darthmouth 18

2.2.3 El procesamiento del lenguaje . . . 19
2.2.4 GPS, General Problem Solver . . . 20
2.2.5 Informática en Acción . . . 20
2.2.6 El enfoque evolutivo . . . 21
2.2.7 Sistemas Expertos . . . 21
2.2.8 El abordaje Conexionista . . . 21
2.2.9 Aprendizaje Automático . . . 22
2.2.10 Aprendizaje Profundo . . . 23
2.2.11 La Inteligencia Artificial en la Argentina . . . 23
2.3 Definición de Inteligencia 24

3 Conceptos Preliminares . . . 27
3.1 Nomeclatura 27
3.1.1 Matriz de datos . . . 27
3.1.2 Características . . . 27
3.1.3 Aprendizaje y Generalización . . . 28
3.1.4 Nomeclatura . . . 28
3.1.5 Métodos de aprendizaje . . . 28
3.1.6 Las cuatro táreas . . . 28

II Algoritmos de búsqueda

4 Inteligencia Artificial Clásica . . . 33
4.1 Agente 33
4.1.1 Ambientes . . . 34
4.2 La importancia de la búsqueda 35
4.3 Métodos de Búsqueda 35
4.3.1 Eficiencia . . . 35
4.3.2 Estrategias de Búsqueda . . . 36
4.3.3 Representación del Problema . . . 36
4.4 Métodos de Búsqueda desinformada 38
4.4.1 Breadth First Search . . . 38
4.4.2 Uniform Cost Search . . . 38
4.4.3 Depth First Search . . . 39
4.4.4 Estados repetidos y Backtracking . . . 39
4.5 Métodos de Búsqueda Informados 39
4.5.1 Local Greedy Search . . . 40
4.5.2 A* . . . 40
4.5.3 Iterative Deepening A* . . . 42
4.5.4 Heuristic Path Planning . . . 42
4.5.5 Ejemplo de juego: Sokoban . . . 43

4.6 Algoritmos de Mejoramiento Iterativo 43
4.6.1 Hill Climbing . . . 45
4.6.2 Simulated Annealing . . . 46
4.6.3 Beam Search . . . 46

5 Algoritmos Genéticos . . . 49
5.1 Definiciones Evolutivas 49
5.2 Crossover 53
5.2.1 Cruce de un punto . . . 53
5.2.2 Cruce de dos puntos . . . 53
5.2.3 Cruce anular . . . 54
5.2.4 Cruce uniforme . . . 54
5.3 Mutación 54
5.3.1 Generando generaciones . . . 54

III Perceptrón

6 Optimización Matemática . . . 59
6.1 La latita de Coca 59
6.2 El problema de optimización 61
6.3 El método simplex 61
6.4 Optimización no lineal sin restricciones 62
6.4.1 Definiciones preliminares . . . 63
6.4.2 Gradiente Descendente . . . 65
6.4.3 Método de Newton . . . 66
6.4.4 Métodos Cuasi Newton . . . 66
6.4.5 Métodos de Direcciones Conjugadas . . . 66
6.4.6 Métodos de Gradientes Conjugados 1952 . . . 67
6.4.7 El método de Powell 1964 . . . 67
6.4.8 Gradiente Descendente Estocástico . . . 67

7 El modelo de Neurona . . . 69
7.1 La bioinspiración 70
7.2 Separabilidad lineal 71
7.2.1 Aprendizaje Hebbiano . . . 73
7.2.2 Algoritmo del Perceptrón Simple . . . 75
7.2.3 Perceptrón Lineal . . . 76
7.2.4 Aprendizaje en el Perceptrón Simple No Lineal . . . 78

8 La era del Coneccionismo . . . 81
8.1 Perceptrón Multicapa 81
8.1.1 La promesa detrás de las redes neuronales . . . 81
8.1.2 Propagación hacia adelante . . . 84

8.1.3 Propagación hacia atrás . . . 85
8.2 Algoritmo del Perceptrón Multicapa 88
8.2.1 Aprendizaje Incremental . . . 90
8.2.2 Aprendizaje por lotes . . . 90
8.3 Variantes de optimización 90
8.3.1 Momentum . . . 90
8.3.2 Tasa de aprendizaje adaptativa . . . 91
8.3.3 RMSProp . . . 91
8.3.4 Adam . . . 92
8.4 Arquitectura del Perceptrón Multicapa 92

9 Métricas de Evaluación . . . 95
9.1 Identificar el patrón de manera correcta 95
9.2 Métricas estándar 95
9.2.1 Matriz de Confusión . . . 96
9.2.2 Métricas . . . 96
9.2.3 Ajuste, subajuste y sobreajuste . . . 97
9.2.4 Convalidación Cruzada . . . 98

IV Automatizando el Aprendizaje

10 Aprendizaje Automático . . . 103
10.1 Redefiniendo Aprendizaje 103
10.2 Métodos de Aprendizaje 104
10.3 Pipeline de Machine Learning 104
10.3.1 El problema Inverso . . . 105

11 Transformaciones en los datos . . . 109
11.1 Estructura interna de los datos 109
11.1.1 Clustering y Agrupamientos . . . 110

12 Análisis de Componentes Principales . . . 113
12.1 Preliminares 113
12.1.1 Transformación PCA . . . 114

13 Red de Kohonen . . . 119
13.1 Mapa Autoorganizado 119
13.1.1 Aprendizaje Competitivo . . . 119
13.1.2 Arquitectura . . . 120
13.1.3 Algoritmo . . . 121

14 Red de Hopfield . . . 123
14.1 Hopfield 1982 123
14.1.1 Memoria Asociativa . . . 124
14.1.2 Estabilidad . . . 126
14.1.3 Convergencia . . . 127
14.1.4 Limitaciones . . . 128

15 Modelo de Oja y Sanger . . . 129
15.1 La búsqueda del poder de síntesis 129
15.2 Red de Oja 129
15.2.1 Implementación . . . 131
15.3 Red de Sanger 132

V Deep Learning

16 Deep Brain . . . 135
16.1 Biomimesis neuronal 135
16.1.1 Grasa inteligente . . . 136
16.1.2 Una neuro-computadora . . . 137
16.2 La Neurona Recargada 138
16.2.1 ¿Cómo dispara una neurona? . . . 138
16.2.2 Una Computadora Nanotecnológica . . . 140
16.2.3 Conducción a Saltos . . . 141
16.2.4 Sinapsis . . . 142

17 Aprendizaje Profundo . . . 145
17.1 Guía para el tío y la tía que pregunta 145
17.2 Los cuatro pilares de Deep Learning 147
17.2.1 Profundidad . . . 147
17.2.2 Aprendizaje de la Representación . . . 150
17.2.3 Datasets . . . 151
17.2.4 GPUs . . . 151

18 Autoencoders . . . 153
18.1 Autoasociadores 153
18.1.1 Arquitectura . . . 153
18.1.2 Autoencoder Lineal . . . 154
18.1.3 Detección de Outliers . . . 156
18.1.4 Denoising Autoencoder . . . 156
18.1.5 Contractive Autoencoder . . . 157
18.1.6 SAE - Sparse Autoencoders . . . 157

18.2 Autoencoder Generativo **158**
18.2.1 La estructura del espacio latente 159
18.3 VAE - Variational Autoencoder **160**
18.3.1 Redes Neuronales Estocásticas 160
18.3.2 Inferencia Variacional 162
18.3.3 Optimizar el Autoencoder Variacional 162
18.4 Apéndice **167**

19 Convolutional Neural Networks 173
19.1 La operación de Convolución **174**
19.1.1 Moving Average 174
19.1.2 Padding 175
19.2 Convolución en Imágenes **175**
19.3 Neocognitrón **177**
19.4 Capa Convolucional **178**
19.5 La capa de Pooling **180**
19.6 Full Layer y Softmax **181**
19.7 Backpropagation **181**
19.8 ¿Por qué funcionan? **182**
19.9 Inteligibilidad **183**
19.10 Arquitecturas Reusables **183**
19.10.1 ResNet 184
19.10.2 UNet 184
19.11 Más allá de las imágenes **185**
19.12 Quiero más **185**

20 Generative Adversarial Networks 187
20.1 Generando gatitos **187**
20.2 Juego MinMax **188**
20.3 Colapso Modal **190**
20.4 El gran simulador **190**

21 Transformers 193
21.1 Las Redes Recurrentes **193**
21.1.1 Long Short Term Memory 194
21.2 Arquitectura de un Transformer **195**
21.3 Attention is all you need **196**
21.3.1 Una tabla de acceso indexado continua 196
21.3.2 Kernel Smoothing 197
21.3.3 Atención Cognitiva 198

21.4 Embedding 199
21.5 Entrenamiento 199

VI Pragmatismo Profesional

22 La ciencia de los datos 203
22.1 El Análisis Cuantitativo 203
22.2 EDAs y ETLS 204
22.3 Falacias de la Ciencia de Datos 205
22.4 Inteligencia Artificial y la Frontera Digital 207

23 Guía para el apurado 209
23.1 Presentaciones 209
23.2 Condimentos de una red neuronal 209
23.2.1 Transformación de los datos 210
23.2.2 Balanceo de clases 210
23.2.3 Funciones de Costo 211
23.2.4 Funciones de Activación 211
23.3 Infraestructura de Software 211
23.4 Quiero más 214

Index 229



1. Reglamento

1.1 Sistemas de Inteligencia Artificial

La materia tiene el propósito de introducir a los alumnos en las técnicas y la problemática del desarrollo artificial y computacional de lo que denominamos inteligencia. Comienza incluyendo aspectos filosóficos sobre el concepto. Luego se adentra en las herramientas denominadas clásicas. El núcleo se focaliza en el abordaje de las redes neuronales artificiales vistas como un problema de optimización. Finalmente, se estudian técnicas de Aprendizaje Profundo (*Deep Learning*). El enfoque es ofrecer un marco teórico que facilite un desarrollo práctico para entender los conceptos en profundidad.

1.2 Clases

La materia es híbrida. Eso implica que las clases son mayoritariamente remotas, pero algunas clases relevantes son presenciales. Las clases están compuestas por módulos relacionados pero bastante independientes. Todas las presentaciones estarán disponibles en campus, así como material extra de libros, papers y artículos de Internet. Todas las clases ya se encuentran disponibles en un recurso en youtube y la mayoría de las clases se graban. **Tengan prsente que algunas clases no se grabarán**. Se tomará asistencia sobre las clases virtuales directamente desde Campus. **Aquellos alumnos que no alcancen el** 70 % **de asistencia se les descontará 1(un) punto del promedio final.**

1.3 Evaluación de la Materia

La materia está orientada a aprender haciendo . Esto implica que la evaluación se realiza mediante 5 Trabajos Prácticos que los alumnos tienen que completar y defender. La evaluación se basa en tres partes: la primera tiene que ver la presentación y la defensa. Parte de los objetivos de esta materia es introducirlos en el curado de datos, su evaluación, análisis y presentación. La segunda parte tiene que ver con la defensa del código de implementación: similar a un **code review** vamos a evaluar de

manera individual el código realizado, que tiene que ser conocido por todo el equipo, enfatizando la responsabilidad sobre lo entregado. Finalmente, el último punto es la completitud en tiempo y forma del trabajo. La nota del TP es individual para cada alumno y surge del trabajo presentado por el equipo, y el rol individual durante la presentación y la explicación, defensa del código.

Adicionalmente hay un conjunto de ejercicios obligatorios durante la cursada que no llevan nota pero que hay que entregarlos de manera obligatoria. Por ejemplo, el ensayo final (Essay), Kahoot, Transformers y GAN.

1.4 Trabajos Prácticos

Los TPs son centrales en esta materia. Son 5, cada uno orientado a una de las temáticas importantes que estudiamos: Inteligencia Artificial Clásica, Algoritmos Genéticos, Redes Neuronales, Aprendizaje No Supervisado, Deep Learning.

1.4.1 Desarrollo

El desarrollo de cada TP conlleva varias partes. La primera es estudiar y repasar los conceptos, luego investigar cómo implementarlos. El paso siguiente es la implementación en sí, que es seguido por la ejecución de los experimentos que sirven para obtener datos que puedan ser analizados. Finalmente la elaboración del informe con los resultados.

Código, Referencias, Librerías y Herramientas

En relación al código, previo al día de defensa de cada TP, los alumnos tienen que entregar un repo referenciando un commit o tag, con los fuentes de la implementación.

Invitamos a los alumnos a que investiguen e incorporen fuentes y referencias variadas para resolver el problema. Nosotros dejaremos disponible la literatura oficial de la cátedra, pero ustedes pueden complementarlo con cualquier recurso que crean conveniente y que les resulte útil.

Debido a la naturaleza de los trabajos prácticos, que poseen un objetivo didáctico de aprendizaje, para cada uno de ellos hay un conjunto de librerías que NO pueden utilizarse, justamente porque son las que tienen que implementar los alumnos. Piensen en que están creando semillas que pueden ser alternativas a esas librerías. No todo está creado.

Invitamos también a qué utilicen todas las herramientas que necesiten para hacer el trabajo más eficiente. Pero es mandatorio que:

- Se citen con precisión TODAS las fuentes utilizadas de las cuales se obtuvo información.
- Se referencien y aclaren las herramientas utilizadas.
- Se apoyen en el uso de herramientas de construcción, pero entendiendo lo que están implementando y haciendo.

1.4.2 Presentación

Las primeras clases se ofrecen instrucciones sobre cómo es el desarrollo de estas presentaciones, qué tema tienen que enfatizar, como hacer los gráficos y cómo mostrar resultados.

1.4.3 Defensa

La defensa tiene que ser concreta, y no debe superar más de 30 minutos. La fecha de la presentación es el día pactado según el cronograma. Aquellos alumnos que requieran un ajuste de horario particular en la defensa, tienen que presentar una razón justificada excepcional. Presentar directamente en la fecha de recuperatorio implica partir de un puntaje de 6(seis) que hay que defender en la defensa.

1.5 Aprobación de la Materia

Los Trabajos Prácticos, las Evaluaciones Orales y el Final, todos y cada uno, deben aprobarse con nota mayor o igual a **6 (seis)**. La aprobación de la cursada depende de la calificación obtenida de las 10 notas, y se obtiene como la media aritmética de todas las notas. La **presentación** del análisis de PCA ofrece un 10(diez) adicional. No cumplir con el 70 % de la asistencia a las clases virtuales descuenta un punto completo del promedio, con lo que puede derivar en la NO aprobación de la cursada.

La modalidad del final es un examen basado sobre todo en el contenido de este apunte. El examen puede ser escrito u oral dependiendo de la cantidad de alumnos que se presenten al mismo y aplican las reglas generales de la Universidad.

1.6 Comunicación

Disponemos varios canales de comunicación

- **Campus**: Este es el canal **oficial** de comunicación mediante los mensajes de Campus. Lo concerniente a modalidad de clase (presencial/virtual) fechas de entrega, información de las clases será informada por este canal.
- **Email**: Disponen de todos los emails de los Profesores que conforman la cátedra para comunicarse por cualquier tema relacionado a la materia.
- **Discord**: Tienen a disposición un espacio de la Materia con un canal general y canales por grupo para que puedan preguntar, sobre todo lo relacionado a la elaboración de los TPs.
- **Físico**: Esta materia la damos desde el Lab de Neurotrónica del ITBA, que está físicamente ubicado en la sala S206 de la Sede de Rectorado. Pueden pasar cuando quieran con preguntas y consultas.

1.7 Nota sobre Bibliografía

El área está actualmente en la cresta de la ola (circa 2025) de la innovación, y pocas veces hubo un auge tan grande de trabajos y avances nuevos en el tema, de forma tan vertiginosa, hasta el punto que es humanamente imposible seguirle el paso [1].

El libro clásico de Inteligencia Artificial es el de Russell y Norvig [2]. Este libro es útil para entender sobre todo la visión más asociada a la inferencia lógica de esta área, la visión más clásica. En una época estudiar IA era estudiar este libro. Dos libros, también clásicos, pero que eran obligatorios como introducción a las redes neuronales son el de Hertz [3] y el de Haykin [4]. Estos libros evidencias las raíces compartidas entre el estudio de las redes neuronales, la física computacional, los sistemas dinámicos y los sistemas complejos (son libros muy *físicos*!).

Para algoritmos genéticos, el material de la materia es en general suficiente para lo que hay que resolver en el trabajo práctico. Sin embargo, para aquellos que quieran entender bien y profundizar en las bases evolutivas de lo que vemos de GA, el libro de Melanie Mitchell, *An Introduction to Genetic Algorithms* [5], los capítulos 1 y 2, son excelentes.

Llegando a Deep Learning, un excelente libro es el Goodfellow [6]. Por otro lado, el libro de Aggarwal [7] es fácil de leer, y muy completo.

Para matemáticas, un excelente libro con una didáctica increíble es *Linear Álgebra Done Right* [8]. Les puede servir a aquellos que tengan poco tiempo y quieran hacer un repaso pragmático de álgebra lineal. Por el lado de Cálculo, el compendio de 2000 hojas de Jean Gallier es abarcativo hasta el hartazgo, pero contiene todas los temas posibles que pueden necesitar para las clases; úsenlo para buscar y repasar un tema puntual [9].

En Optimización, un excelente libro es el de Boyd *Convex Optimization* [10]. Boyd es un investigador de Standford, uno de los primeros que comenzó a subir los vídeos de sus clases a Internet.

Durante la materia vamos a compartir también muchos papers que les recomendamos que lean, o que al menos conozcan que existen y sepan de qué tratan.

1.8 Disclaimer for International Students

First of all, welcome to Argentina, to Buenos Aires, to ITBA and to this SIA lecture!

We wanted first to let you know a few things about this lecture to calibrate better with what you may be looking for in this class.

This lecture requires a lot of work developing and writing algorithms to implement the material covered in each lecture. Hence, the requirements are:

1. Programming: we do a lot of programming. If you feel that you do not have, and you do not want to learn the programming knowledge to sit down and work hard on your own version of a neural network, this may not be the lecture for you.
2. Teamwork at groups: this class also requires the availability of students to work together in groups. So if you think you are going to be travelling heavily around and that you will not have the time to be with teammates (remotely or physically) it may be better to join another lecture.
3. Be present at the presentation: you have to be remotely present to defend the work done on each assignment with your team.
4. This class is what is called *English Friendly*. This means that the the lectures and core material is mostly in Spanish, but please do not feel that to be a problem in itself. There is a handout available in Spanish (this) that you can easily translate and all of the references are books and papers in English. Additionally, you can ask all your questions, write all your presentation, code, etc entirely in English.

Keep in mind that this University offers an alternative course, with the same name, with code 71.22, which is more theoretical with less programming requirements. Additionally, Machine Learning 72.75 covers related materials with a little bit less programming. You can switch to those alternative courses if you prefer.

Regarding the groups, you can join any group that you want. We encourage you to reach out to ITBA students and join them.

I Los inicios

2 Introducción 17
2.1 ¿Qué es la inteligencia?
2.2 Los orígenes
2.3 Definición de Inteligencia

3 Conceptos Preliminares 27
3.1 Nomeclatura



2. Introducción

2.1 ¿Qué es la inteligencia?

La inteligencia artificial nace con el comienzo mismo del concepto de computabilidad y de las máquinas para automatizarla. Su auge actual se remonta al período de posguerra, donde se implementan las primeras materializaciones físicas en confluencia con ideas anteriores.

Una de las primeras verdades de Perogrullo que aparecen al estudiar esta disciplina es que no sabemos con exactitud qué es la inteligencia. Los *Homo sapiens sapiens* vivimos nuestra vida diaria sin profundizar demasiado qué es la inteligencia, pero aún sin una clara definición, confiamos en tener una idea aproximada de lo que es; la identificamos cuando la vemos.

[discutir] La manera que proponemos para abordar este problema es mediante el planteo de qué es lo que estamos artificializando. Una estrategia inicial para abordar la respuesta es hacer un repaso histórico del tema para entender cómo llegamos hasta acá.

2.2 Los orígenes

Como se explicó previamente, la Inteligencia Artificial (aka IA)[1] surge al final de la segunda guerra mundial y durante el período posguerra, con la teoría de computación desarrollada por Alan Turing [11] y con la materialización de las computadoras digitales.

La figura 2.1 muestra una línea de tiempo con los hitos más importantes en la historia de la Inteligencia Artificial [12]. Los desarrollos en criptoanálisis en Bletchley Park, para ayudar a descifrar la máquina Enigma, o el proyecto MAGIC en EE.UU., así como también el control remoto de torpedos

[1] En este texto, llamaremos IA a la disciplina. A caballo de la masificación, se ha extendido el uso de IA para hacer referencia a cualquier implementación práctica de un agente, programa, algoritmo que entre dentro de esta categoría. Por ejemplo, es común encontrar frases como: ”tengo varias *Inteligencias Artificiales* implementadas”. Somos conscientes que esta batalla está claramente perdida.

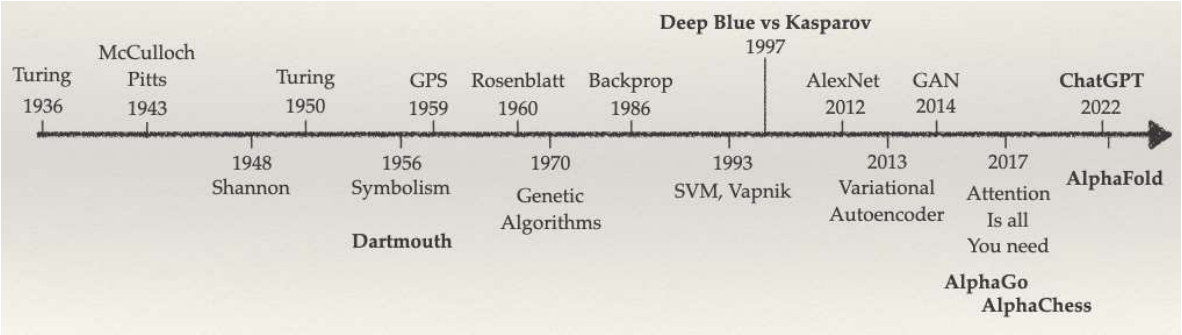


Figura 2.1: Línea de tiempo de los hitos más importantes en el desarrollo actual de la Inteligencia Artificial.

o el análisis de las señales de radar en la batalla por Britania, fueron necesidades de cómputo que impulsaron la creación y el crecimiento de las computadoras y, con ello, el desarrollo inicial de la IA.

En 1943, Warren McCulloch y Walter Pitts [13] presentan el modelo de neurona. Este modelo simple y organizado podía resolver cualquier función computable. Luego, en 1949, Donald Hebb [14] enuncia una conjetura acerca de cómo cambia la influencia de una neurona sobre otra de acuerdo al estado de actividad de ambas, conocida como su **Regla de Hebb**. Con las bases sentadas por estos científicos, Rosenblatt toma estas ideas para el desarrollo de su **Perceptrón** (capítulo 7) para la marina de EE.UU., puntapié inicial de las "Redes Neuronales".

En 1950 Turing escribe *Computing Machinery and Intelligence* [15, 16], trabajo en el cual identifica la complejidad en definir inteligencia desde la perspectiva de la computabilidad y desarrolla su famoso **Test de Turing**, que recientemente ha alcanzado un grado considerable de obsolescencia [17].



Figura 2.2: McCulloch, tranca, meditando sobre su modelo.

2.2.1 El Simbolismo

En 1956 Newell y Simon [18] crearon un software llamado *Logic Theorist* que aplica una idea rudimentaria de un árbol de decisión. La hipótesis se posicionaba en la raíz del árbol y, en cada rama, se generaba una deducción a partir de un postulado anterior, lo cual provocaba que en alguna de las ramas se generara la demostración de la hipótesis. Lo único que hacía este programa era buscar entre las ramas hasta encontrar las respuestas.

El inicio del simbolismo, demostró como el uso de inferencia lógica podía generar resultados de alto nivel, lo cual resultaba inimaginable unos años atrás. Esta situación de gran promesa va a ocurrir una y otra vez; es una de las principales razones de los ciclos de *hype* e inviernos de la inteligencia artificial. Logic Theorist fue exitoso en demostrar varios teoremas del famoso *Principia Mathematica*.

2.2.2 Nos Vemos en Darthmouth

El aire que se respiraba alrededor de 1956 procuraba que la revolución de la IA había llegado, y de que estábamos nada más ni nada menos que al borde de automatizar la inteligencia[2]. En el *Darthmouth*

[2]Suena conocido, ¿no?



Figura 2.3: Oliver Selfridge, Nathaniel Rochester, Marvin Minsky, John McCarthy, Ray Solomonoff, Persona Desconocida (Peter Milner o alguien del futuro), Claude Shannon.

College, en Hannover, New Hampshire, USA, quienes eran las puntas de lanza de ese momento, organizaron un *workshop*; un retiro veraniego para juntarse y discutir los **detalles** para finalizar la tarea de artificializar la inteligencia. John McCarthy convence a otros investigadores, entre los que se encontraba Minsky, Shannon y Rochester, para organizar el *Dartmouth Summer Research Project on Artificial Intelligence* [19].

> *"Proponemos que se lleve a cabo un estudio a lo largo de **2 meses** sobre inteligencia artificial, con **10 personas** durante el verano de 1956 en el Dartmouth College en Hanover, New Hampshire. Se intentará encontrar cómo hacer que las **máquinas usen el lenguaje**, formen **abstracciones y conceptos**, resuelvan tipos de problemas ahora reservados para los humanos y **se mejoren por sí mismas**. Creemos que **se puede lograr un avance significativo** en uno o más de estos problemas **si un grupo de científicos cuidadosamente seleccionados trabajan juntos durante un verano**."*

El término **Inteligencia Artificial** pisa con fuerza en este evento, por razones más humanas de las que podemos asumir. John McCarthy quería separarse explicitamente de la Teoría de Autómatas [20], la base teórica de los lenguajes de programación, que él consideraba esotérica y con un foco acotado. Por otro lado, también quería distinguirse de la cibernética [21] por cierta animosidad con su creador y promotor, Norbert Wiener [22, 23, 24]. Asimismo, y por si fuera poco, ese fue el punto de cisma entre la inteligencia artificial y la teoría de control, que de alguna manera u otra abordan una problemática similar asociada a la automatización [25].

¿Resolvieron la artificialización, la automatización de la inteligencia? No, pero sí dieron el puntapié para iniciar un torrente: Inteligencia Artificial, Aprendizaje Automático, Reconocimiento de Patrones, etc.

2.2.3 El procesamiento del lenguaje

Una característica intrínseca humana, muy asociada a la inteligencia, es la capacidad de procesar lenguaje. Creamos, producimos y entendemos diferentes lenguajes, y con ellos podemos expresar

pensamientos y conceptos abstractos. Inevitablemente, esto derivó en artificializar la inteligencia mediante la sintetización del procesamiento del lenguaje, área conocida como *Natural Language Processing*, o NLP. En 1950 John McCarthy desarrolló LISP, un lenguaje pensado para capturar naturalmente comportamiento automatizable. La construcción de lenguajes de programación llevó al desarrollo teórico de las ideas de gramática, computabilidad, y automatismo. Un ejemplo de ello son los autómatas finitos, que acarreaban de forma inherente la idea de inteligencia [26]. En sus inicios, la gramática y los compiladores eran parte de lo que se llamaba inteligencia artificial. Esta linea se completa en 1965 cuando Joseph Weizenbaum presenta a Eliza [27]. Eliza simulaba ser un psicoterapeuta que parafraseaba el texto que recibía y empleaba un conjunto de reglas simples para generar respuestas y frases abiertas con el objetivo de asentir al interlocutor. Pese a ser un esquema simple, mucha gente tenía la idea de que estaban hablando con una persona real.

2.2.4 GPS, General Problem Solver

En 1959, Herbert Simon, Cliff Shaw y Allen Newell crean el GPS, como una evolución de los trabajos realizados con *Logic Theorist*. La estrategia fue expresar en forma simbólica problemas más generales de forma que pueda utilizarse *Logic Theorist* para resolverlos, y luego volver a la representación que tenían esos problemas. El mensaje importante que nos dejaba este suceso es que la representación simbólica podía emplearse para resolver problemas de cualquier tipo: sólo basta encontrar la manera de representarlos formalmente con expresiones lógicas. La idea intuitiva que tenemos sobre cómo los seres humanos resolvemos problemas puede describirse como la descomposición de un objetivo en sub-objetivos, que se van alcanzando mediante la ejecución de tareas. Con este enfoque, la resolución de objetivos se podía representar directamente en una computadora. Con mucho entusiasmo, Herbert Simon enunciaba:

> *No es mi intención sorprender o producir un shock en uds. pero, la forma más simple en que puedo resumir el estado del arte actual (en el mundo) es que* ***hoy día hay máquinas que piensan, que aprenden y que crean****. Mas aún, sus habilidades para hacer estas cosas está incrementándose rápidamente y* ***en un futuro cercano****, el conjunto de problemas que podrán manejar* ***co-existirá con el rango de problemas que la mente humana ha tratado*** .

¿Es entonces la inferencia lógica una herramienta construida para manejar formal y conscientemente una solución? ¿Es el mecanismo natural del pensamiento?

El problema con el enfoque del *General Problem Solver* es que funcionaba con problemas simples, sin grandes exigencias. Cuando el tamaño del problema o del espacio de soluciones aumentaba, podía fallar en encontrar una solución.

2.2.5 Informática en Acción

Todos estos avances dieron por hecho que si una computadora era capaz de resolver un teorema del *Principia Mathematica*, sería trivial utilizarlo en un mecanismo físico, en un **autómata** o **robot**. La literatura de ciencia ficción futurista de Asimov tiene su origen en estos años y esta idea es constante a lo largo de su obra. En un contexto de altas expectativas, surge la paradoja de Moravec:

Definición 2.2.1 — Paradoja de Moravec. El mundo real, parcialmente observable, estocástico, con infinitos estados, plantea un problema muchísimo más complejo que cualquiera de los problemas lógicos que pudieron resolverse en los albores de la inteligencia artificial. Esto era difícil de percibir

para los investigadores de esa época, porque para los seres humanos esas actividades son automáticas y subconscientes. No nos damos cuenta de la complejidad que conllevan.

Cuando estos algoritmos se pusieron en acción, apareció la enorme dificultad que representa lidiar con el mundo físico real, con escenarios estocásticos, parcialmente observables, con infinitos estados. A pesar de las dificultades, uno de los primeros éxitos de la robótica fue *Shakey*, un robot móvil que podía navegar de manera autónoma los pasillos del *Standford Resarch Institute*, desarrollado en 1959.

2.2.6 El enfoque evolutivo

A principio de los años 70, John Holland introduce los algoritmos genéticos (capítulo 5). Éstos se basan en un modelo de la teoría de evolución de Darwin, adaptados para realizar optimizaciones. Dicha optimización se centra en la búsqueda de un conjunto de parámetros que minimizan un funcional, con una heurística aleatoria. La idea del enfoque evolutivo es la utilización de individuos que representan una combinación de valores para los parámetros. Dependiendo del éxito o fracaso de los mismos en optimizar la función de costo, son seleccionados para replicar el contenido de su genoma.

Como muchas otras áreas de estudio, este enfoque generó un tremendo *hype* que luego se vio interrumpido al no poder usarse en el contexto de problemas del mundo real, para los cuales se financiaba su desarrollo.

2.2.7 Sistemas Expertos

Los sistemas expertos tienen sus orígenes en un trabajo denominado DENDRAL, realizado por Ed Feigenbaum, un estudiante del mismo grupo de Herbert Simon, Bruce Buchanan (filósofo que trabajaba en ciencias de la computación), J. Lederberg (genetista) y otros *Et al*. La idea fue extender GPS a un campo específico; un nicho puntual. Lo aplicaron de manera directa para inferir la estructura de la molécula de una sustancia, en base a información de un espectrómetro de masa, obtenida por el bombardeo de un haz de electrones.

Entrando en la era de las computadoras personales y la revolución digital de finales del siglo XX, esta táctica se extendió para extraer información específica de expertos en un campo particular. Los conceptos se formalizaban como proposiciones lógicas, y de ahí se realizaban inferencias que permitían formular conclusiones, como situaciones de diagnóstico médico. Esto se materializó en Mycin [28]. Conceptualmente, este objetivo se extenderá posteriormente a otros sistemas expertos médicos promocionados por IBM con Watson Health [29], que se combinaban con otras técnicas más estadísticas para procesar la información.

2.2.8 El abordaje Conexionista

En 1986, Rumelhart, Hinton y Williams [30] proponen el algoritmo de *Backpropagation* (sección 8.1.3), revitalizando el modelo de neurona de McCulloch y Pitts, la idea Hebbiana de aprendizaje y el viejo perceptrón de Rosenblatt. Nace así una de las ideas rectoras del campo de Redes Neuronales: las estructuras jerárquicas aglomerativas.

Esta propuesta aborda directamente las limitaciones publicadas por Minsky y Pappert [31] en relación a un problema que algoritmos primitivos basados en el modelo de neurona no podían resolver. Como consecuencia de este avance se generó la segunda ola de la IA. El perceptrón multicapa (capítulo 8) tomó mucha fuerza y el conexionismo se extendió hacia otros modelos de aprendizaje de redes neuronales como Hopfield (capítulo 14), Kohonen (capítulo 13) y Goldberg.



Figura 2.4: Yann LeCun, pionero en el campo de inteligencia artificial, con su PC Gamer demostrando las capacidades de una red convolucional para resolver el problema de los números manuscritos de los códigos postales (i.e. MNIST).

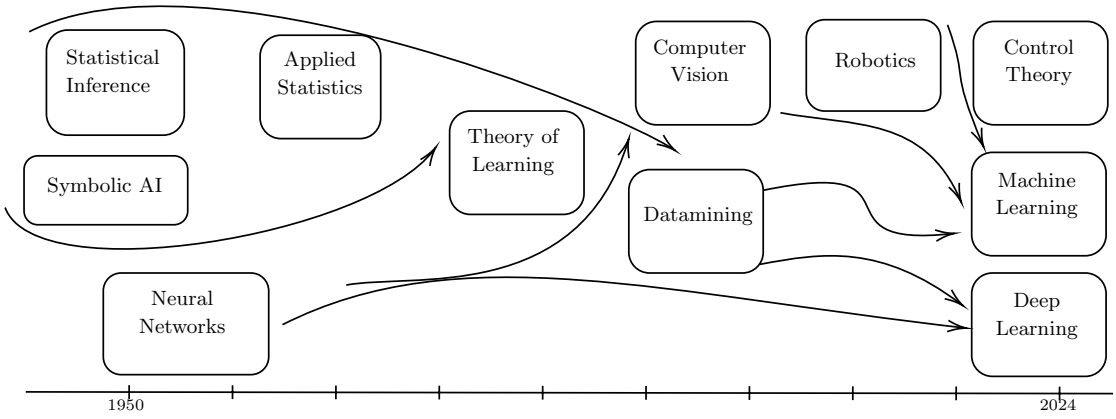


Figura 2.5: Diferentes disciplinas asociadas a la Inteligencia Artificial y su evolución.

2.2.9 Aprendizaje Automático

El **aprendizaje** es un concepto asociado a la inteligencia, por lo que también padece los mismos problemas al momento de establecer con claridad su definición. Podemos, por el contrario, establecer una definición precisa (10.1.1) para su primo artificial: el aprendizaje automático (10). Éste tiene sus orígenes en los trabajos de estadística y probabilidad de Ronald Fisher, Karl Pearson et al [32] a principios del siglo XX, extendiéndose como una disciplina de cálculo asociada al procedimiento actuario y estadístico. Judea Pearl revivió el viejo teorema de Bayes en los años 80, para desarrollar las estructuras estadísticas de inferencia de las redes bayesianas [33, 34], que permitían atar probabilísticamente las evidencias con las posibles, y más probables, causas.

El auge continúa durante la última década del siglo XX; se formaliza la idea de aprendizaje automático con el libro de Vapnik [35] y su propuesta del algoritmo de SVM (**S**upport **V**ector **M**achine), adquiriendo un apogeo significativo en la primera década del siglo XXI, asociada al procesamiento de información visual en imágenes y en *Computer Vision* [36]. Se logran avances significativos, pero no revolucionarios.

2.2.10 Aprendizaje Profundo

Finalmente, durante estos últimos años, somos testigos de la tercera ola de la inteligencia artificial[3] bajo el rótulo de Aprendizaje Profundo o *Deep Learning* (capítulo 17). En el área de *Computer Vision*, existía una competencia internacional muy popular de clasificación multiclase de un dataset de imágenes, ImageNet. Los resultados pioneros en esta competencia se encontraban estancados, hasta que llegó el avance descomunal que Alex Krizhevsky (alumno de Geoffrey Hinton) propone: una arquitectura basada en redes neuronales artificiales, puntualmente Redes Convolucionales (capítulo 19), con la **pequeña** particularidad de que el algoritmo estaba paralelizado y corría en los poderosos coprocesadores gráficos de una PlayStation de Sony [33].

La clave fue que la posibilidad de disponer de procesadores cada vez más poderosos. Esto fue importante no sólo por la velocidad y memoria sino por la cantidad de núcleos, ofreciendo un sustrato de hardware que permitió a las redes neuronales artificiales, hasta ese momento una disciplina muy académica, tener una salida a la cancha de la sociedad como una herramienta transformadora. Tal es la importancia de este suceso que luego desencadena en el desarrollo e implementación de otras arquitecturas como el Autoencoder Variacional (capítulo 18) y las Redes Generativas Adversarias (capítulo 20), las cuales comienzan a dar los primeros pasos para automatizar nada más y nada menos que la propia creatividad. Pero eso no es todo. Pocos años después se introduce la arquitetura del *Transformer* desde Google [37], que significó un avance descomunal en la tarea de automatizar el lenguaje natural. Esta cascada de sucesos derivó finalmente en el desarrollo de GPT-3 por parte de la empresa DeepMind, que ha provocado en estos últimos años un impacto inusitado a nivel mundial en la sociedad.

2.2.11 La Inteligencia Artificial en la Argentina

La IA no se puede escindir de los vaivenes políticos del desarrollo argentino de las ciencias de la computación [16]. El hito fundacional se posiciona en 1961 con la llegada a la Argentina de la computadora Mercury de la compañía británica Ferranti, conocida como **Clementina**[4]. Fue puesta en marcha por quien será el pionero de la computación Argentina, Manuel Sadosky, conocido también por la creación del Instituto del Cálculo de la Universidad de Buenos Aires. Esta acción solidificó diferentes iniciativas que se replicaban (con más o menos éxito) en otras universidades y centros de investigación [38].

En 1960 se publica en la Sociedad Argentina del Cálculo un artículo que recopilaba los avances en el área dando difusión en círculos académicos y corporativos argentinos [16]. En ese mismo año se conforma la Sociedad Argentina de Investigación Operativa SADIO [39], que impulsa las ciencias informáticas orientadas a la optimización de la investigación operativa, es decir, un gran componente de la inteligencia artificial actual [40]. Al año siguiente aparece el primer grupo que se dedica a la Inteligencia Artificial, GEIA, que luego se desmantela por motivos políticos.

En la década del 70 hay un penoso retroceso, limitándose a incorporar contenido en carreras de

[3] ¿Será la ola final?
[4] Oh my darling, Clementine

grado basado que se relacionan con el procesamiento de listas con LISP y el reconocimiento de patrones. En 1979 se percibe un cambio de paradigma en el uso de las computadoras desde el foco científico hacia el comercial, y se "ingenieriza" la ciencia de la computación con el Plan 79[5] [41], con el cual aparecen las primeras asignaturas de estudio directamente enfocadas en Inteligencia Artificial. Con la llegada de la Democracia, la ciencia y la computación se vuelven relevantes y se crea la Escuela Latinoamericana de Informática, la ESLAI [42], un instituto especializado donde el area de Inteligencia Artificial fue un punto central. En paralelo, se establece el programa Argentino-Brasilero PABI que impulsó una serie de escuelas de verano donde se exploraban las áreas de procesamiento de señales, robótica, automatización y sistemas expertos. Se destaca el proyecto ETHOS, un sistema experto para la representación de conocimiento e inferencia en el area de ingeniería de software, muy alineado a las tendencias globales de esa época [16].

Durante los noventa se consolida la investigación a nivel universitario y se realiza el Primer Simposio de Inteligencia Artificial y Robótica, y el primer Workshop Argentino sobre Aspectos Teóricos en Inteligencia Artificial, el actual CACIC. Se desarrollan y publican trabajos en robótica y aprendizaje por refuerzo [43], bases de datos y Dataminning [44, 45], agentes inteligentes[46], procesamiento de imágenes[47, 48], etc.

En 2004, la Ley de Promoción de Software impulsa un rico entramado empresarial en servicios de software que posiciona al sector como el de mayor crecimiento dentro de la matriz exportadora[6]. Pese a los altibajos, el sector científico se mantiene a flote[7], con lo cual se recibe a la Gran Revolución de la Inteligencia Artificial en una posición mejorable pero resiliente.

2.3 Definición de Inteligencia

Dentro de la comunidad tecnológica existe un momento denominado la **singularidad** [49]. Éste se refiere al punto en que hemos logrado artificializar la inteligencia pero no sólo eso: aquella inteligencia artificial que hemos conseguido será capaz de crear a su vez otro ente con capacidades aún mayores. Este concepto implica un cambio en la tasa de crecimiento, una especie de discontinuidad. Según este modelo teórico, la velocidad a la que ocurren sucesivos cambios es inherentemente exponencial. Pero todavía no hemos podido contestar, ¿qué es la inteligencia? ¿hemos logrado o estamos de camino a artifializarla?

En linea con dicho planteo, diferentes autores hablan de tres tipos de sistemas de inteligencia artificial, según el rango de tareas que abarcan:

- Inteligencia artificial específica (Artificial Narrow Intelligence, ANI)
- Inteligencia artificial general (Artificial General Intelligence, AGI [50])
- Inteligencia artificial superior (Artificial Super Intelligence, ASI)

Dichos autores, a su vez, establecen que el curso de evolución de la inteligencia artificial estaría dado por el orden, ANI, AGI y, finalmente, ASI.

Aquí proponemos una analogía que creemos superadora: pensar la inteligencia como una montaña con la cumbre oculta tras las nubes o tras otros picos. Es un *moving goalpost* donde la propia identificación de lo que un ente inteligente debe cumplir se va desplazando a medida que nos vamos acercando. No sólo eso sino que hay muchos senderos que la pueden surcar, con diferentes niveles de avance. ¿Es

[5] Es es la razón por la cual en Argentina existe una carrera de ingeniería general en informática, y por la que tienen que estudiar física del electromagnetismo

[6] Representa el 2.8 % de las exportaciones y se encuentra 38° en el ranking de los países más exportadores de servicios de software.

[7] En el ranking científico en publicaciones de IA en *Scimago*, Argentina se ubica en la posición 68 (circa 2025).



Figura 2.6: La inteligencia es una compleja montaña, llena de caminos y variantes, donde la cumbre, desde cada perspectiva, aparece detrás de los picos o está oculta tras nubes.

acaso mejor o más avanzada nuestra inteligencia que la que manifiestan los delfines o los pulpos? ¿Es la inteligencia humana más o menos avanzada que la que manifiesta una colonia de hormigas? No podemos estar seguros si la inteligencia que nosotros conocemos y manifestamos es **La Inteligencia**. Con esta analogía, la inteligencia son trazos y caminos en esa montaña, en un continuo, con diferentes estadíos.

Shan Legg y Marcus Hutter [51, 52] hicieron un compendio de diversas definiciones de inteligencia. En su artículo resumen alrededor de 70 extractos de un extenso análisis bibliográfico. A continuación, arbitrariamente, se eligen cinco definiciones y se ofrece una nueva.

Definición 2.3.1 — Inteligencia - American Psychological Association. Individuos difieren unos de otros en su habilidad para enteder ideas complejas, adaptarse efectivamente al entorno, aprender a partir de la experiencia, adoptar varias formas de razonamiento, superar obstáculos através de pensar.

Definición 2.3.2 — Inteligencia - Declaración según 52 expertos. Inteligencia es una muy general capacidad mental que, entre otras cosas, involucra la habilidad de razonar, planear, resolver problemas, pensar en forma abstracta, comprender ideas complejas, aprender rápido y aprender a partir de la experiencia.

Definición 2.3.3 — Inteligencia - A. Anastasi. Inteligencia no es una única habilidad sino una composición de varias funciones. El término denota aquella combinación de habilidades requeridas para sobrevivir y avanzar en una cultura particular. A. Anastasi

Definición 2.3.4 — Inteligencia - E.Boring. Inteligencia es lo que es medido por los test de inteligencia.

Definición 2.3.5 — Inteligencia - J. S. Albus. Es la habilidad de un sistema de actuar de manera apropiada en un entorno eventualmente desconocido, donde la acción apropiada es aquella que aumenta la probabilidad de éxito, siendo el éxito la completitud de subobjetivos comportamentales que soporten el objetivo mayor del sistema.

Definición 2.3.6 — Inteligencia - Artificializando la Inteligencia.

- Un sistema que sensa, procesa y actúa de manera que es adecuada en un contexto físico y temporal particular.
- Requiere una corporización para manifestarse con claridad.
- Es un continuo, un *moving goalpost*.
- Presenta **Autonomía** como un comportamiento emergente.
- Manifiesta una **Conciencia** [53], que se materializa como comportamientos que representan un conocimiento sobre su propia existencia, en relación al contexto físico y temporal, y una diferenciación en relación a los otros.

La definición 2.3.6 es la que proponemos en estas líneas, fuertemente influenciada por la concepción de inteligencia que proviene de la robótica.



3. Conceptos Preliminares

3.1 Nomeclatura

En esta sección incluimos definiciones generales de términos comunes dentro del área pero que pueden tener diferentes acepciones por lo que es menester aclararlos.

3.1.1 Matriz de datos

Varios de los métodos presentados en este texto están basados en la utilización de datos para ajustar parámetros de algoritmos. La matriz de datos, que llamaremos X está compuesta por

$$X = \begin{pmatrix} & var1 & var2 & var3 \\ muestra_1 & \dots & \dots & \dots \\ muestra_2 & \dots & \dots & \dots \\ muestra_3 & \dots & \dots & \dots \end{pmatrix}$$

donde las columnas son las variables y las filas son cada una de las muestras, patrones, samples. Esta matriz es de X^{PxN} donde N representa la cantidad de variables, la dimensión de los datos y P la cantidad de patrones disponibles. Cada muestra X_i puede tener asociado un *label* y. Esto suele ser un valor binario, categórico, un escalar o eventualmente un vector de salida de alguna dimensión en particular.

3.1.2 Características

Los datos de entrada crudos pueden procesarse y transformarse en un formato que sea más adecuado para el procesamiento subsiguiente. Esta transformación puede ser desde un simple escalado de la información, hasta un procesamiento de varias etapas mucho más complejo. Después de ese procesamiento, los datos de entrada se transforman en **features**, características, predictores, que estructuran la información de mejor manera para poder capturar los eventuales patrones subyacentes.

Estas características se sintetizan en un vector o *feature vector* de una dimensión particular. Este es el que se usa como entrada para la función de mapeo $f(X) = y$ que conlleva el poder discriminativo.

3.1.3 Aprendizaje y Generalización

Aprender: Aprender artificialmente tiene una definición precisa (10.1.1) que se aleja del concepto humano de aprendizaje. Involucra el proceso algorítmico por el cual encontramos o ajustamos los parámetros libres de un algoritmo para optimizar una función de costo, en base a datos conocidos obtenidos del mundo real.

Generalizar: Es aplicar lo aprendido, es decir utilizar los parámetros libres, o *pesos*, para usar el algoritmo que usa esos pesos y generar una salida, para un dato nuevo que NO fue utilizado durante el algoritmo de aprendizaje, durante el ajuste de los pesos.

3.1.4 Nomeclatura

- **Training set**: conjunto de datos de entrenamiento usados para calcular los pesos.
- **Test set**: conjunto de datos que NO fueron usados para calcular los pesos.
- **Parámetros**: son las clavijas libres, los pesos, las variables independientes de la función de costo, que la optimización busca encontrar. Los valores w^*.
- **Hiperparámetros**: son los parámetros generales del algoritmo que no son ajustados por el propio proceso de optimización base. Se ajustan **a mano**, a ojo, prueba-y-error, o por algún método de *hiperoptimización*. No son parte de la función de costo.
- **Aprender bien**: dada una muestra en la entrada que pertenecía al conjunto de entrenamiento,obtener la salida esperada.
- **Convereger**: Aprender bien todo el conjunto de entrenamiento. Esto es, poder ajustar los parámetros libres para que todas las salidas coincidan con sus valores esperados.
- **Generalizar**: Usar los pesos o parámetros libres aprendidos para dada una muestra que no fue utilizada durante el entrenamiento, generar una salida, una predicción, un mapeo. Hacerlo **bien** es que ese mapeo sea el adecuado.

3.1.5 Métodos de aprendizaje

Dada la función de mapeo que se quiere aprender $f(X) = Y$.

Aprendizaje Supervisado: Se conocen los X y los Y, obtenidos del mundo real.

Aprendizaje No-supervisado: Sólo se conocen los X, los datos de entrada, pero no se conoce la salida.

Aprendizaje por Refuerzo: No se conocen ni los X, ni los Y.

3.1.6 Las cuatro táreas

Normalmente un programa que entre dentro de la categoría de IA va a poder realizar cuatro táreas diferentes.

Clasificar: Dada un conjunto de datos con su etiqueta, el **Clasificador** aprende a predecir la etiqueta de cada muetra. La clasificación puede ser binaria o multiclases. También llamado **Discriminar**.

Regresionar: En este caso, el **regresor** aprende a interpolar y extrapolar las muestras pudiendo estimar los valores intermedio. Es un clasificador con etiquetas continuas. Cuando se pretende inferir los valores futuros de una serie de tiempo, se les suele llamar **Predictor** o **Forecaster**.

Optimizar: Los algoritmos de IA tienen en su mayoría inherentemente una optimización, por lo que pueden adaptarse de manera directa para resolver problemas típicos de optimización (e.g. Travel Salesman Problem, Knapsack problem, integer partition)

Generar: Ahora, los algoritmos pueden generar muestras nuevas directamente, estimar X_i válidos. Esta es la base de los algoritmos generativos.

II Algoritmos de búsqueda

4 Inteligencia Artificial Clásica 33
4.1 Agente
4.2 La importancia de la búsqueda
4.3 Métodos de Búsqueda
4.4 Métodos de Búsqueda desinformada
4.5 Métodos de Búsqueda Informados
4.6 Algoritmos de Mejoramiento Iterativo

5 Algoritmos Genéticos 49
5.1 Definiciones Evolutivas
5.2 Crossover
5.3 Mutación



4. Inteligencia Artificial Clásica

En este texto denominamos Inteligencia Artificial Clásica a los métodos tradicionales de modelado y búsqueda de soluciones guiadas. Ante un problema, el mismo puede modelarse como un conjunto de estados concretos y acciones a ejecutar para cambiar de un estado al otro. Si es posible establecer este modelado, la solución del problema pasa por encontrar alguna transición, un camino de acciones sucesivas válidas bajo las reglas del problema, que permitan partiendo de un estado inicial, alcanzar el estado final que es la solución. Y de todas las maneras posibles que pueden existir para lograr eso, buscar aquellos caminos específicos que son más óptimos[1] en el sentido de minimizar/maximizar el uso de recursos, o una función objetivo particular.

4.1 Agente

El concepto de agente es clave para la idea de la inteligencia artificial, y se alinea a la conceptualización de que la inteligencia artificial se manifiesta verdaderamente cuando un cuerpo la alberga [54].

Se define como todo aquello que puede considerarse que percibe y responde o actúa en un ambiente, con un cierto grado de independencia o autonomía. Etológicamente, se asume que el agente tiene un objetivo, una razón de ser, configurandose en un **agente racional** o **agentes basados en utilidad**. Esto se materializa mediante su **función de desempeño**. Esta determina el éxito del agente, y a su vez puede medirlo de corto o a largo plazo, es decir teniendo alguna medida del horizonte temporal de ese éxito (i.e. decisiones estratégicas). El agente así, racionalmente, busca realizar acciones que maximicen su medida de rendimiento.

Por otro lado, es posible identificar a los **agentes omniscientes**. Conocen en todo momento el resultado real que producirán sus acciones.

Los agentes también pueden clasificarse en relación a si poseen o no un estado interno. Dependiendo si tiene o no estado interno, los que no lo mantienen se suelen llamar reactivos $a = f(P)$ con P

[1] Si algo es óptimo, no es posible que sea *más óptimo* pero ustedes entienden la idea.

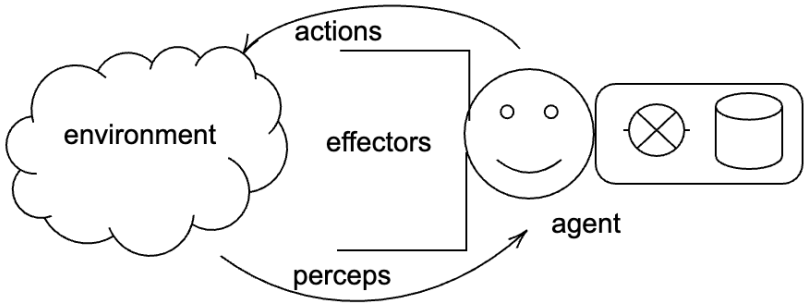


Figura 4.1: El agente y su ambiente

percepción y a acción. En cambio los que poseen estado interno, también llamados con modelo interno, se caracterizan por $a = f(P,M)$, donde M es el modelo interno, algo que el agente usa para entender el entorno.

4.1.1 Ambientes

Ambiente se le llama a los componentes que engloban al agente [2]. Es lo que provee el origen de la percepción así como también el destinatario de las acciones del agente. Los ambientes pueden ser **discretos** o **continuos**, dependiendo si el número de estados y acciones es finito o es un continuo.

Pueden ser **determinísticos** o **estocásticos**, en relación a si es o no predecible con probabilidad uno el estado resultado de una acción ejercida sobre el ambiente partiendo de un estado anterior conocido. En tanto que en el caso estocástico, la transición depende de un proceso estocástico probabilístico (es decir no se sabe con precisión y se estima probabilísticamente). A su vez el ambiente puede ser **totalmente observable**, cuando es posible para el agente determinar cualquier característica del mismo, o **parcialmente observable**, cuando justamente esta situación no puede determinarse. Un ambiente parcialmente observable puede parecer estocástico para el agente. Esta idea puede extenderse al propio proceso de aprendizaje del agente. El ambiente es **conocido**, si el agente a priori conoce todo lo que puede conocer del ambiente, en tanto que es **desconocido** cuando requiere de un proceso de exploración para poder adquirir ese conocimiento.

El concepto de agente es también central en Teoría de Juegos [55], donde se modelan las interacciones entre varios agentes. De esta forma, cuando hay un único agente, el ambiente es **individual** mientras que cuando hay varios agentes simultáneos el ambiente es **multiagente**. En este caso los agentes pueden ser **colaborativos** o **adversariales**. Un ambiente **determinístico estratégico** puede presentarse cuando precisamente no conocemos las acciones de los otros agentes (pero sí como opera el ambiente).

Los agentes tienen una experiencia en el ambiente dada por la secuencia de acciones que ejecutan y los estados que van transitando. El escenario es **episódico** cuando es posible determinar secuencias finitas de acciones que tienen un estado de inicio y un estado final, y sobre todo cada episodio es totalmente independiente de los episodios anteriores y futuros. Cuando esta independencia no se presenta, se dice que el proceso es **secuencial**.

Los ambientes **estáticos** son aquellos donde el estado no cambia mientras el agente no ejecuta sus acciones, y es **dinámico** cuando esto sí ocurre. Por otro lado, si lo que cambia es la calificación, la

evaluación de la función objetivo, el ambiente es **semidinámico**.

4.2 La importancia de la búsqueda

Dado un adecuado modelado, en un sistema digital con precisión finita, cualquier problema de inteligencia artificial, se podría resolver, optimizar en esa arquitectura, por fuerza bruta buscando todas las opciones posibles, y evaluar la función de costo, ejecutandolo sobre el sistema y verificar cuál es la mejor. Por ejemplo, en un juego de Ajedrez, o cualquier otro juego de mesa. El punto crítico, es que precisamente no se tiene infinito poder de cómputo posible, y queremos hacer una búsqueda que sea realizable computacionalmente.

4.3 Métodos de Búsqueda

La resolución de problemas basados en búsquedas arranca de la base del modelo del agente, donde hay diferentes componente que plantean un problema bien formado, donde existen:

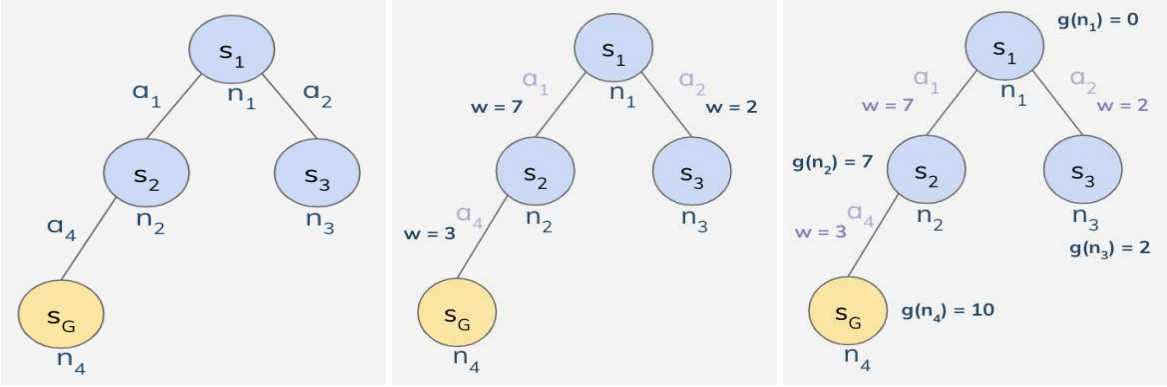
- **Estados**: diferentes estados s que modelan las diferentes configuraciones en las que puede estar el problema.
- **Estado inicial**: configuración inicial s_0 donde el problema no está resuelto.
- **Acciones**: acciones a.
- **Modelo de Transición T**: describe el efecto de aplicar la acción a estando en s, que deriva en un estado s', llamado sucesor.
- **Función de costo**: Determina el costo $g(T(s,a)) = g(s,a)$ para el modelo de transición T.
- **Condición de solución**: Determina si un estado s particular es o no solución del problema.

Utilizando los estados, las acciones y las transiciones, y sobre todo la evaluación que podemos hacer de la solución en cada estado, es posible plantear un árbol de búsqueda que partiendo de un estado inicial s_0, transiciona por los estados aplicando acciones y evaluando cada uno de los caminos para intentar encontrar el óptimo: aquel que tenga el menor costo acumulado. Cada nodo n_i del árbol corresponde a una secuencia específica de transiciones. Cada nodo entonces, puede tener un costo asociado $G(n) = \sum g(s_i, a_i)$ obtenido como la sumatoria de los costos de todas las transiciones de forma que partiendo del estado inicial sea posible alcanzar el estado final s'_{i+1}. Este árbol puede verse como un grafo si se consideran más de un nodo raíz.

Esto puede verse en la figura 4.2. Cada transición de estados $T(s,a) = s'$ tiene un costo asociado $g(s,a) = w$. Este costo se va acumulando a medida que se va transitando por el árbol de estados. Así por ejemplo en el nodo n_2 es $G(n_2) = 7$ y en en n_3 es $G(n_3) = 10$.

El aspecto más complejo está justamente en encontrar una representación adecuada del problema, en base a definir una arquitectura de estados, que representen todas las opciones posibles, manteniendo utilidad en las acciones para que sean acotadas, buscando la posibilidad de encontrar optimizaciones computacionales en las simetrías.

La definición de las acciones pasa también por definir meta-acciones que puedan ser un aglomerado de otras acciones más pequeñas y que aporte información directa al objetivo del método de búsqueda (es decir, no es necesario mantener siempre una correspondencia entre las acciones establecidas en el modelo con acciones concretas del problema en cuestión).



(a) Árbol de Estados (b) Árbol de Estados con pesos (c) Árbol de Estados con costos acumulados

Figura 4.2: Modelo base de estados.

δ	a_1	a_2	a_3	a_4
s_1	s_2	s_3	∅	∅
s_2	s_1	∅	∅	s_G
s_3	∅	∅	∅	s_1
s_G	∅	∅	∅	∅

(a)

Figura 4.3: Tabla de estados posibles. El estado s_G es el estado objetivo.

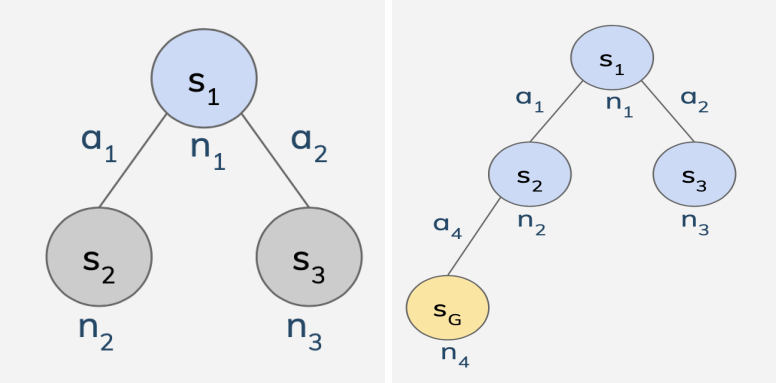
4.3.1 Eficiencia

Al realizar una búsqueda se requiere tener una medida de la eficiencia en el procedimiento. Lo básico es pragmatismo puro, **encontrar una solución**. La segunda es en base a **optimizar el costo** de la solución, que puede ser en base a un costo máximo, mínimo o promedio. Se puede medir en base a la **cantidad de acciones a realizar**, o incluso en base al propio **costo computacional** en función del uso de memoria o de tiempo de procesamiento.

La búsqueda puede ser a su vez **forward** hacia adelante, arrancando desde un estado inicial del problema hacia una solución, o puede ser **backward** o hacia atrás donde la raíz es una solución (y de ahí hacia el o los estados iniciales).

4.3.2 Estrategias de Búsqueda

La búsqueda es **completa** cuando alcanza y encuentra la solución (de existir y ser alcanzable). Es **óptima** cuando encuentra la solución alcanzable que tiene menor costo según algún criterio.



(a) Nodos No Explorados (b) Nodos Explorados

Figura 4.4: Árboles de Representaciones.

4.3.3 Representación del Problema

En el árbol de estados *Tr*, dado un nodo n, los nodos hijos que se obtienen al expandir el nodo n, se representan como $s(n_i) = \{n_{i+1}, n_{i+2}, ..., n_{i+3}\}$ (**s**ucesores). El conjunto formado por todos los nodos expandidos posibles es el conjunto frontera *Fr*.

Un estado es **expandido** cuando dentro del método de búsqueda se le han aplicado todas las acciones aplicables. Los estados expandidos conforman el grupo de los estados explorados *Exp*.

Un nodo no es un estado en sí, pero se encuentra siempre asociado a uno en particular. Por eso, el mismo estado puede estar vinculado a diferentes nodos. Siendo $G(n)$ el costo acumulado del nodo (la suma del arco desde n_0 hasta n), podemos encontrar dos nodos n_1 y n_2 que pueden estar asociados al mismo estado, de forma que

$$G(n_1) \neq G(n_2).$$

Las diferentes variantes de los métodos, tanto desinformados como informados, exploran diferentes maneras de aplicar y evaluar el algoritmo 1 a continuación. Esta variante presentada mantiene una lista de nodos expandidos para evitar ciclos en el grafo.

Algoritmo 1 Algoritmo General de Búqueda (GRAPH-SEARCH)

Árbol de búsqueda Tr
Conjunto frontera Fr
Conjunto de Nodos explorados Exp
Crear Tr,Fr,Exp vacíos.
Insertar nodo inicial $n_0 \rightarrow Tr, Fr$
while $Fr \neq \emptyset$ **do**
    Extraer primer nodo de $Fr \rightarrow n$
    if n Goal **then**
        Solución formada por el arco entre raíz $n_0 \rightarrow n \in Tr$. Termina
    end if
    Expandir el nodo n generando los sucesores $\{n_{+1}\}$
    $\{n_{+1}\} \rightarrow Fr$
    $\{n_{+1}\} \rightarrow Tr$, Como sucesores de n
    $n \rightarrow Exp$
    Reordenar Fr según criterio (dependiente del método)
end while
if Solucion vacía **then**
    No existe solución alcanzable
end if

4.4 Métodos de Búsqueda desinformada

Los métodos de búsqueda desinformados, sólo tienen la información que surge del propio modelo del problema, sin distinguir de los estados otra cosa que no sea si son o no **goal**, es decir los estados objetivo que el propio proceso de búsqueda intenta encontrar. Son poco eficientes en problemas donde podrían hacerse estimaciones.

Los métodos más comunes de búsquedas desinformadas son:

- BFS
- DFS
- IDDFS
- Costo Uniforme

4.4.1 Breadth First Search

BFS expande los nodos de menor profundidad primero. Es decir selecciona el primer nodo a expandir y recursívamente va expandiendo hasta alcanzar la profundidad máxima. Para que tenga sentido, el chequeo de si un estado es o no final se realiza cuando el nodo es agregado a la frontera. Es completa si el factor de ramificación es finito, encontrando la solución con menor profundidad. Cuando todas las acciones tienen el mismo costo, la solución es óptima. La contracara de BFS es que tiene complejidad $\mathcal{O}(b^d)$ siendo b la cantidad de nodos de un mismo nivel y d la profundidad. Esto no escala.

4.4.2 Uniform Cost Search

UCS, ó Búqueda uniforme, expande el nodo con menor costo acumulado. Por supuesto, cuando el costo es uniforme, es exactamente igual a BFS. La búsqueda es óptima cuando se cumple que

$G(s(n)) \geq G(n)\ \forall n$, o equivalente que $g(s,a) \geq 0\ \forall a,s$.

4.4.3 Depth First Search

DFS expande el nodo de mayor profundidad. No es óptimo, pero suele tener una mucho menor complejidad espacial, sobre todo en su version sin estados repetidos, ya que con estados repetidos puede quedar en bucles infinitos. Cuando existen muchas soluciones suele ser más rápido que BFS. Desde la implementación, BFS utiliza una cola FIFO para *Fr* en tanto que DFS utiliza LIFO.

Para evitar el problema de la búsqueda infinita de estados, se puede limitar la profundidad de la búsqueda con un hiperparámetro valor arbitrario que se denomina **Depth Limited Search**, generando una búsqueda que no es completa ni óptima. Para justamente no tener que arbitrariamente setear este parámetro, IDDFS **Iterative Deepening Depth First Search** es DLS donde la profundidad puede variar según cierto criterio. Se puede implementar manteniendo los nodos hoja que no fueron expandidos por cota de profundidad entre cada iteración. Con un valor de profundidad infinito es precisamente vanilla DFS.

4.4.4 Estados repetidos y Backtracking

La motivación detrás del uso de estados repetidos es no regresar a estados ancestro, ni crear rutas que tengan ciclos que rompen la estructura del árbol. Tampoco volver a expandir estados que ya fueron expandidos previamente. Es un compromiso entre el costo de almacenar y chequear estados.

Backtracking se refiere a poder generar sucesores y agregarlos a la frontera uno a la vez, y luego de verificar que el camino elegido no conlleva al óptimo, poder deshacer todos los cambios que se hayan hecho sobre las estructuras de soporte del algoritmo, y elegir otro nodo individual, y evaluarlo.

4.5 Métodos de Búsqueda Informados

Los métodos informados por el contrario, cuentan con más información para comparar entre diferentes estados. Con eso pueden hacer una estimación para calcular cuánto falta para resolver el problema, mediante el cálculo de una **heurística**. Con esa información también pueden calcular el costo del problema. Este modelo, más refinado, permite abordar justamente problemas que son intrínsecamente más complejos e implementar mecanismos de búsqueda que a su vez son más eficientes. Los dos principales exponentes son:

- Greedy
- A*

Estos métodos se basan en la definición de una **heurística** (def 4.5.1) que es una estimación *heurística* del costo asociado a cada estado, que sea fácil de calcular [2].

Definición 4.5.1 — Heurística. Función heurística $h(s)$ representa el costo estimado de la ruta más barata desde el estado s hacia un estado meta.
Si s Goal $\rightarrow h(s) = 0$
Si s no Goal $g(s,a) > 0\ \forall a$: Accion $\rightarrow h(s) > 0$
Si s no Goal $g(s,a) \geq 0\ \forall a$: Accion $\rightarrow h(s) \geq 0$

[2] De la física, existe un método similar denominado *Método de Fermi*, que es una técnica rápida para dado un problema poder entender cuál es el orden de magnitud de la respuesta búscada sobre todo cuando los datos no se tiene a mano o se requiere una respuesta rápida [56]

Se puede abusar la notación planteando

$$h(n) = h(s),$$

indicando que s es el estado representado en el nodo n. El valor de la heurística depende del estado.

Proposición 4.5.1 — Heurística Admisible. Una heurística es admisible si nunca sobrestima el costo real. Es perfecta si estima exactamente el costo real $h^*(n)$.

Las heurística se pueden encontrar dividiendo al problema en sub-problemas, relajando las propias limitaciones o las reglas del problema. También la heurística puede surgir como el máximo de otras métricas o pueden surgir de mezclar heurísticas, contemplando la valuación de cada una de ellas, sobre todo cuando atacan diferentes sub-problemas.

4.5.1 Local Greedy Search

Que sea **Greedy** hace referencia a que el algoritmo busca la mejor opción que tiene a mano, sin ir más allá en relación a los pasos siguientes. Esta variante informada arranca desde el nodo raíz y se expande, y sobre los expandidos estima h. Luego toma el nodo con menor h; este es el paso greedy. Si el conjunto recién expandido es vacío, necesita hacer **backtracking**, ir para atrás en el algoritmo y explorar otra opción.

Esta solución no es óptima ni completa para el caso que no evalúe repetición de estados. Como en todos los algoritmos informados, la clave suele estar en la selección de una muy buena función heurística, que va a determinar la complejidad temporal y espacial del algoritmo.

Si el algoritmo utilizado es 1 pero usando $h(n)$ para establecer el ordenamiento de Fr en el paso 15, el algoritmo se denomina **Global Greedy Search**. En este caso, a pesar de tener un costo mayor que LGS, el backtracking es más efectivo al poder evaluar más opciones.

4.5.2 A*

Es el algoritmo **estrella** de métodos de búsqueda y uno de los algoritmos más utilizados del mundo. Se basa en la función

$$f(n) = G(n) + h(n),$$

y ordena a los nodos en Fr en base a esta función en el paso 15, desempatando en base a $h(n)$ cuando $f(n)$ son iguales. Es idéntico a costo uniforme, sólo que usa $f(n)$ para estimar el menor costo.

El algoritmo garantiza que si $h(n)$ es admisible, entonces $f(n)$ nunca sobreestima el costo real de la mejor solución que pasa por el nodo n. La búsqueda es completa si hay una ramificación finita y el costo es mayor que un $\varepsilon > 0$. Por otro lado, es óptima cuando se cumple:

Teorema 4.5.2 A* es óptima (se encuentra el camino óptimo a la solución si existe) cuando:

- Cada nodo del grafo debe tener un número finito de sucesores.
- El costo de cada arco debe ser mayor que un $\varepsilon > 0$.
- La heurística debe ser admisible $h(n) \leq h^*(n)$

Se define

$$f^*(n) = G(n) + h^*(n)$$

.

Por supuesto requiere memoria y procesamiento para el cálculo de las heurísticas.

Proposición 4.5.3 — Lema 1: Heurística Admisible. En cualquier momento dado en la búsqueda de A*, existe un nodo $n \in Fr$ que cumple

1. n está en el camino óptimo al objetivo
2. $f(n) \leq f^*(n)$

Demostración. Prueba por Inducción.

Caso Base

Para el nodo inicial n_0. Todos los caminos comienzan con n_0 y por existir una solución, n_0 está en el camino óptimo al objetivo.

$$f(n_0) = G(n_0) + h(n_0) = h(n_0) \leq h^*(n_0) = f^*(n_o) \tag{4.1}$$

Paso inductivo

La frontera en el paso k contenía a un nodo que estaba en el camino óptimo. Entonces se elige nodo n^e para expandir.

Si el nodo n^e no es un nodo del óptimo, la frontera va a seguir teniendo a dicho nodo, entonces se cumple (1) y n' es dicho nodo.

Si el nodo n^e es tal nodo, A* lo expande, y alguno de sus hijos debería estar en el camino óptimo, por lo tanto se cumple (1) y $n' = n^e$

$$f(n') = G(n') + h(n') \leq G(n') + h^*(n') = f^*(n') \tag{4.2}$$

■

Teorema 4.5.4 — Terminación de A*. Partiendo de que la ramificación es finita, y el costo de las reglas/acciones son siempre mayor o igual a un número mayor a 0. Entonces, si A* puede no terminar, eventualmente llega a un punto donde

$$f(n) > f*(n),\ \forall n \in Fr$$

.

Definición 4.5.2 — Dominancia de Métodos de búsqueda. Se dice que M_1 domina a M_2 si para cada nodo expandido por M_1 también es expandido por M_2. Se dice que M_1 domina **estrictamente** a M_2 si

- M_1 domina a M_2
- M_2 **no** domina a M_1

Eficiencia de heurísticas

Suponiendo h_1 y h_2 heurísticas admisibles, si h_2 domina a $h_1 \rightarrow A_2^*$ expandirá menos nodos que A_1^*

Esto es porque existirán nodos que A_1^* expandió (y no llevan al camino óptimo) y A_2^* no, por lo que, para esos nodos:

$$G(n) + h_1(n) \leq f^*(n) \leq G(n) + h_2(n)$$

De esta forma, como regla general conviene tomar heurístcas admisibles que den el mayor valor posible.

Combinación de heurísticas

Dado un conjunto de heurísticas admisibles $h_1, ..., h_m$ se puede definir como una nueva heurística h' a la combinación de ellas, definida de la siguiente forma:

$$h' = \text{máx}(h_1, ..., h_m)$$

Esta nueva heurística tiene como propiedades:

1. Es admisible
2. Domina a todas las heurísticas que al conforman

Consistencia de heurísticas

Si una heurística es consistente, cada vez que A* expanda un nodo, habrá encontrado un camino óptimo al mismo.

Si una heurística es admisible, no necesariamente.

Completitud de A*

A* podría no terminar nunca.

Algunas de las situaciones donde puede suceder esto son:

- La ramificación es infinita para eal menos un estado.
- Existen arcos con costos ≤ 0.
- Los arcos tienen costos > 0, pero son asintóticamente decrecientes (Paradoja de Zenon)

4.5.3 Iterative Deepening A*

Concepto inspirado en IDDFS, se realiza un corte iterativo con un límite. También aquí se utiliza DFS. Esta búsqueda es completa y óptima bajo las mismas condiciones que A*. Requiere menos memoria que A*, pero puede expandir más nodos e incluso expandir muchas veces el mismo nodo (mismo problema que IDDFS).

El límite es un threshold que se impone sobre $f(n)$.

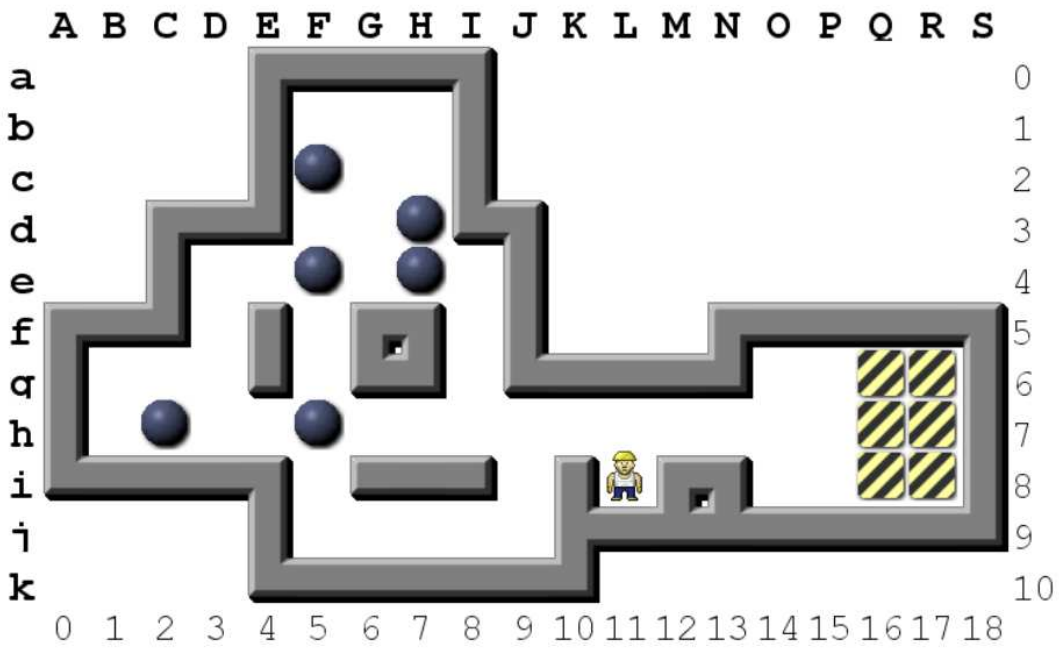
1. Inicialmente se toma $Lim = f(n_0)$
2. Mientras que no se encuentre solución, se realiza DFS hasta Lim.
3. Si no se encontró una solución, se toma n' el nodo de frontera con menor valor de f, y se toma este valor como el nuevo límite.

4.5.4 Heuristic Path Planning

Best-First con

$$f(n) = (1 - \omega) * G(n) + \omega * h(n)$$

$$f(n) = \begin{cases} \omega = 0 & \text{Uniform Cost Search} \\ \omega = \frac{1}{2} & \text{A* Search} \\ \omega = 1 & \text{Global Greedy Search} \end{cases}$$



(a)

Figura 4.5: Imagen de un típico tablero de Sokoban, tomada de [57].

4.5.5 Ejemplo de juego: Sokoban

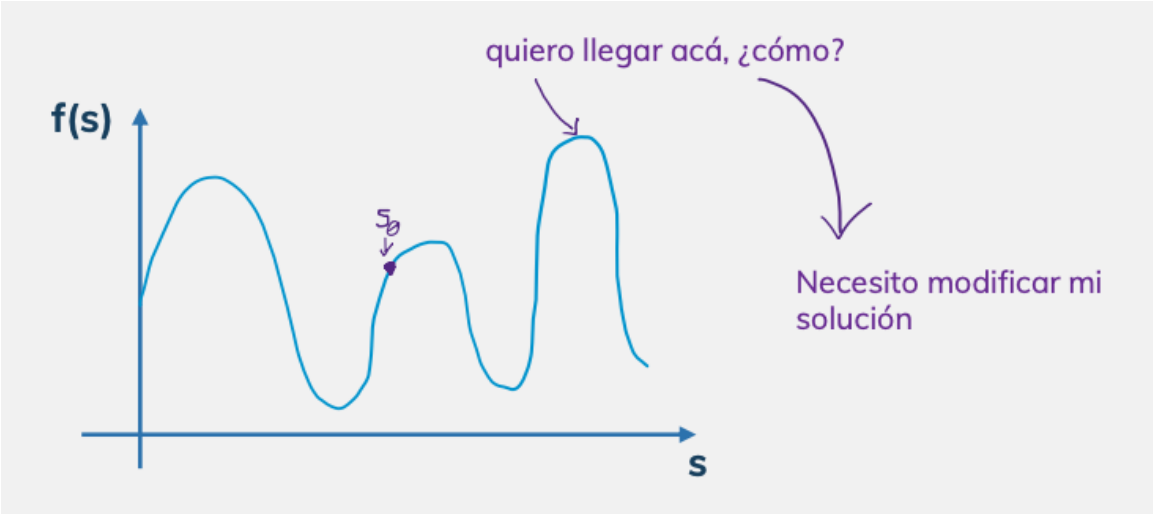
El Sokoban [57] es un juego clásico, caracterizado por un trabajador de un depósito que tiene en dos dimensiones que llevar un conjunto de cajas a sus destinos disponibles. *Soko* significa depósito y ban es un subfijo que representa el encargado o el trabajador. Fue diseñado en 1982 por Hiroyuki Imabayashi para las computadoras Spectrum y Commodore. El juego consiste en diferentes tableros con espacios disponibles donde el trabajador se puede mover, paredes infrancable, las locaciones donde están las cajas y los destinos donde hay que depositarlas. Para mover cada caja, el trabajador se coloca detrás de la misma y la puede mover hacia uno de los cuatro destinos cardinales. Este juego es excelente como plataforma para implementar y probar algoritmos de búsqueda. La razón de esto es porque el problema es NP-Hard y a su vez PSPACE-complete. Tableros simples requieren una gran cantidad de capacidad de procesamiento y de memoria para resolverse. Además tiene situaciones de deadlocks que no pueden resolverse, configuraciones de los tableros donde no hay forma de mover alguna de las cajas o de llevarla a alguno de los destinos.

4.6 Algoritmos de Mejoramiento Iterativo

Los algoritmos de búsqueda con mejoramiento iterativo, se basan en partir de una solución y mejorarla, optimizar el funcional, mediante perturbaciones a esa solución.

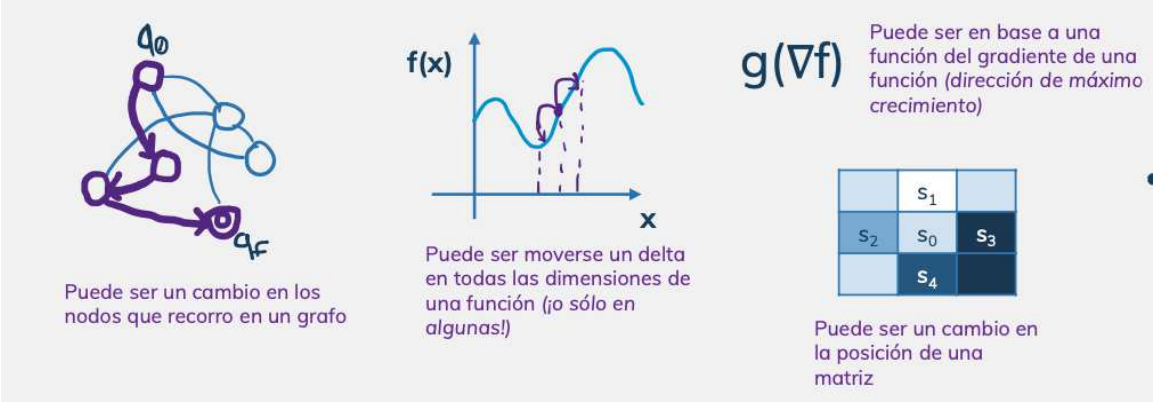
Definición 4.6.1 — Mejoramiento Iterativo. Sea $f(S)$ función de optimización que evalúa que tan buena es una solución S particular. Los algoritmos iterativos se basan en encontrar $S*$ tal que

$$S* / \forall S \;\; f(S*) \geq f(S)$$



(a)

Figura 4.6: Función de optimización y mejoramiento iterativo.



(a)

Figura 4.7: Alterantivas para implementar el mejoramiento iterativo.

Las diferentes opciones giran alrededor de la idea de variar de manera azarosa la solución encontrada, moviéndose por las posibles direcciones de crecimiento o decrecimiento (ver Optimizaciones en el capítulo 6).

Travelling Salesman Problem

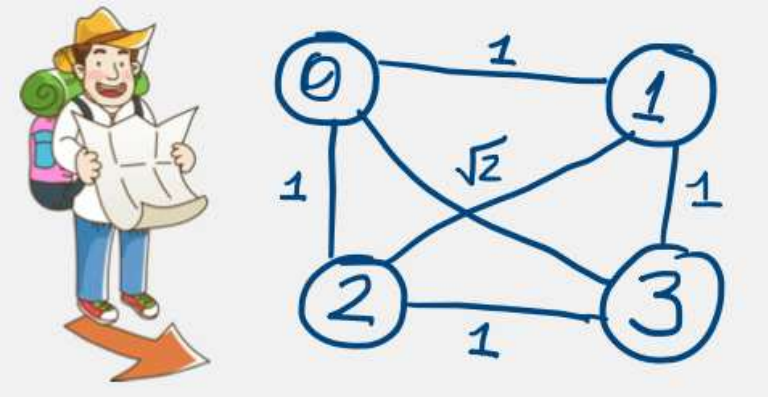
El problema del viajante de comercio (TSA) es un clásico NP-Hard. Es un problema de recorrido de grafos donde hay n ciudades y la distancia entre cada una de ellas. El planteo del problema pasa por encontrar la ruta más corta, visitando cada ciudad exactamente una vez y al final regresar a la ciudad de la que se parte en el principio.

La entrada del problema es el número de ciudades n y los n pares ordenados con las coordenadas de cada ciudad (x, y). El output del problema tiene que ser la lista de ciudades según su índice que hay que recorrer o eventualmente los waypoints que corresponden con la sucesión de puntos que va a reflejar por donde tiene que ir el salesman (tocando y pasando por todas las ciudades y completando el

circuito).

Solución por métodos iterativos

Consideremos el problema de TSA de la figure 4.8, con las distancias dados por los rótulos en las aristas.



(a)

Figura 4.8: Problema básico de Travel Salesman Problem

Para resolver este problema iteratívamente, se puede hacer:

1. Tomo una solución al azar, por ejemplo: $sol = \{0-3-1-2(-0)\}$
2. Evalúa la solución en la función objetivo: $f(sol) = 4.83$.
3. Se realizan cambios azarosos en la solucion sol. Por ejemplo, se elige una ciudad y se permuta su ubicación, $sol_{a1} = \{3-0-1-2\}, sol_{a2} = \{0-1-3-2\}, sol_{a3} = \{0-1-2-3\}$
4. Se selecciona una en base a su valuación: $f(sol_{a2}) = 4$

4.6.1 Hill Climbing

En este caso las variaciones azarosas se hacen en la dirección de crecimiento de la función, marcada por el gradiente, de forma que se suba a la cumbre (de ahí el nombre). Es un algoritmo de heurística local, donde a cada paso se genera una nueva frontera Fr (que no es la misma frontera de los métodos de búsquedas que se usa para identificar los nodos a expandir). El problema de este algoritmo es que suele estancarse muy fácilmente en máximos locales.

Algoritmo 2 Algoritmo Hill Climbing

```
Conjunto frontera Fr
while Fr ≠ ∅ do
    Moverse un paso arbitrario en ambas direcciones de cada una de las dimensiones.
    Tomar cada punto solución s_i → Fr (agregarlo en la frontera).
    Evaluar los puntos de la frontera f(s_i) ∀s_i ∈ Fr
    Seleccionar el valor máximo de la frontera como nueva solución argmax_{s_i} f(s_i)
end while
```

Simple Hill Climbing: Elige la primer solución de la frontera con mejor valuación que el estado actual.

Steepest Hill Climbing: Siempre elige la solución de la frontera con mejor valuación que el estado actual.

Stochastic Hill Climbing: Elige al azar una solución de la frontera y, en base a que tan buena es, decide si moverse o elegir otra.

4.6.2 Simulated Annealing

El algoritmo de recocido simulado, un clásico de la física computacional. Este algoritmo surge de modelar el comportamiento de ferrometales donde se los somete a un campo magnético, inicialmente a mucha temperatura, y donde la temperatura se va reduciendo gradualmente, lo que da origen a que el material se vuelve ferromagnético, ya que el movimiento térmico inicial le da espacio a las moléculas de poder acomodarse, y la tendencia en la que se acomodan a medida que se reduce la temperatura es la influenciada por el campo magnético [4].

El algoritmo es parecido a Hill Climbing, pero involucra un parámetro que regula la estocasticidad, el balance entre exploration/explotation, y que en este algoritmo representa la **temperatura**. A medida que van pasando la iteraciones, esta temperatura se va disminuyendo progresivamente según un perfil de progresión. El algoritmo procede similar a Hill Climbing, estableciendo un conjunto Frontera Fr, pero ignora la valuación de cada solución allí depositada con una probabilidad relativa a la temperatura (y opcionalmente, proporcional a su valuación). Es decir el algoritmo selecciona la mejor opción disponible y se mueve a esa solución según el propio perfil de la exponencial (eligiendo una opción donde $d < 0$, sin mejora, más al principio, y quedándose con las opciones mejores a medida que disminuye la temperatura). La temperatura puede reducirse iteración a iteración o de a bloques dependiendo un valor fijo o alguna condición.

Algoritmo 3 Simulated Annealing

```
1:  Tomar s = s_0 ; t = t_0
2:  while Pis False do
3:      while Repetir N_rep do
4:          Seleccionar s' ∈ E(s) (entorno de s)
5:          d ← f(s') − f(s)
6:          if d > 0 then
7:              Tomo s' (es una mejor solución)
8:          else
9:              u ← rand(0,1)
10:             if u < exp(−d/t) then
11:                 s = s'
12:             end if
13:         end if
14:     end while
15:     t = α(t) Reducción de la temperatura
16: end while
```

4.6.3 Beam Search

Similar a Hill Climbing pero comienza con k nodos iniciales[3]. En cada paso, se generan los sucesores de los k nodos, que pasarán a conformar la frontera. De todos los sucesores, se toman los

[3]El valor de k se conoce como el ancho del rayo o el beam width

mejores k nodos de la frontera y se vuelve a repetir. Es diferente a k hill climbings en paralelo, ya que compara todos los mejores en todos los caminos entre sí, vinculando la información entre todos, y por eso puede eliminar rápidamente los nodos iniciales malos para tomar otros caminos mejores. Puede converger mucho entre los k nodos, generando tan poca diversidad que se elabora un hill climbing mucho más costoso.

Stochastic Beam Search

Similar a Beam Search, pero elige los k nodos con una probabilidad proporcional a su valuación. De esta forma, los nodos **malos** también pueden llegar a elegirse (con menor probabilidad). Similar al proceso de selección natural (ver capitulo 5).



5. Algoritmos Genéticos

La biología juega un juego de optimización plasmado en la evolución y la selección natural. La lucha por la supervivencia y la perpetuidad imponen una presión sobre los seres vivos para que solo aquellos que logran **encontrar** la correcta combinación de genes para cada ambiente específico, sobreviven y perpetúan esa combinación.

Esta idea, fue capturada por varios investigadores que la plasmaron en un algoritmo de optimización, en realidad una suite de **Algoritmos Genéticos** que ofrecen un marco muy potente para orientar una búsqueda [5].

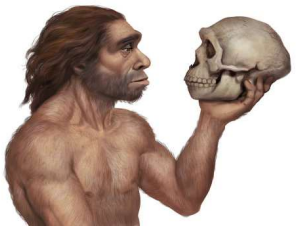


Figura 5.1: Los neanderthales y nosotros, una mezcla complicada.

5.1 Definiciones Evolutivas

La propuesta ofrecida en este escrito es la basada en Holland [58]. Es un algoritmo probabilístico de optimización/adaptación estructurado según un modelo simplificado de la mecánica de la selección natural y la teoría de la evolución. Tiene una enorme ventaja de que es altamente paralelizable (por ser cada individuo independiente) y tiene múltiples hiperparámetros y variables.

La idea general de los algoritmos genéticos (GA) consiste en generar una población de individuos, donde los parámetros libres a optimizar se codifican en una especie de gen. Cada individuo representa una combinación completa de todos los parámetros. A su vez, esos parámetros son justamente las variables de una función de optimización, que en GA se denomina *aptitud*, y que generan un resultado numérico. Así la maximización/minimización de esa función de costo depende de la combinación de valores de los parámetros plasmada en el código genético de cada individuo. Mediante una simulación de la evolución de la población de individuos, el algoritmo intenta encontrar aquellos que precisamente

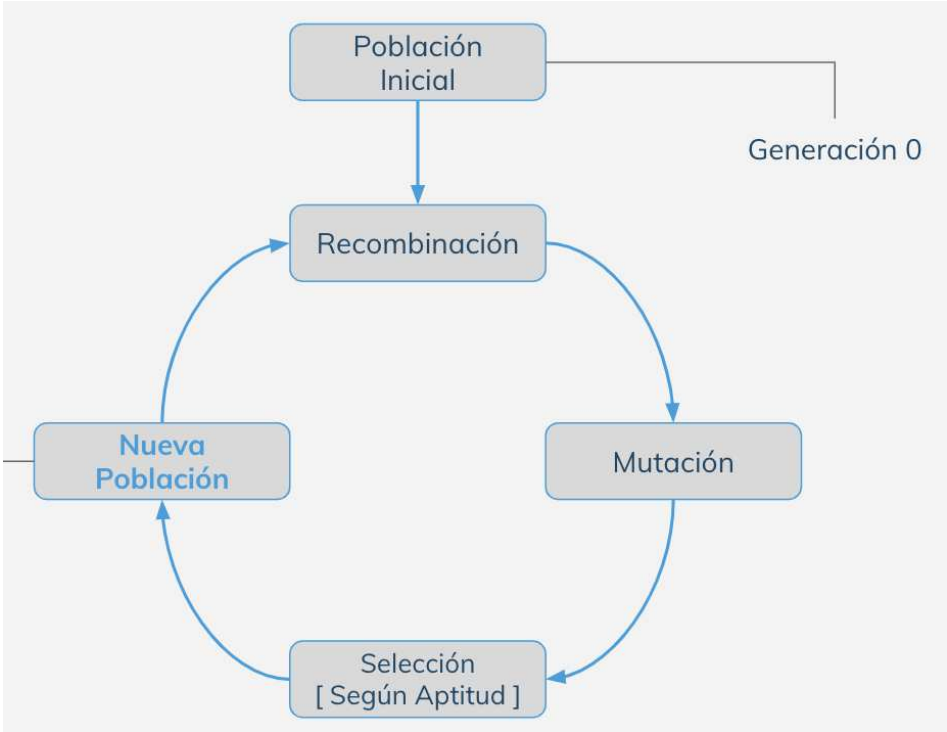


Figura 5.2: Esquema de bloques del patrón básico de un algoritmo genético

alcancen el óptimo a buscar.

Los componentes básicos son:

- Genotipo: estructura/arquitectura
- Población inicial
- Función de aptitud
- Método de selección de padres
- Método de cruza
- Método de mutación
- Método de selección de nueva generación
- Condición de corte.

Definición 5.1.1 — Genotipo. Establece la forma particular del gen, la manera en la que se codificara a nivel de bits, bytes, estructura de datos.

Definición 5.1.2 — Locus. Es la posición de un gen particular dentro del propio genoma.

Definición 5.1.3 — Cromosoma. El cromosoma es el conjunto completo de los genes. Por ejemplo, según el ejemplo de la tabla 5.1 sería [1][0][1][0][1].

Definición 5.1.4 — Fenotipo. Son las características observables afectadas por algún gen.

Amarillo Patito	Triangulo
[1][0]	[1][0][1]

Cuadro 5.1: Table caption

Definición 5.1.5 — Alelos. Las variantes posibles de un gen, representadas por la codificación que se establezca para el gen. A la variedad de alelos representados en la población se le llama **diversidad**.

Definición 5.1.6 — Generación 0. La Generación 0 es la población inicial de individuos de tamaño N.

Definición 5.1.7 — Aptitud. Es la función de costo $f(i)$ de la optimización que el algoritmo intenta resolver. Es evaluado para cada individuo i. También documentado como *adaptabilidad* o *fitness*.

Definición 5.1.8 — Aptitud Relativa. Normalizado al conjunto de individuos

$$p(i) = \frac{f(i)}{\sum_{j=1}^{N} f(j)} \tag{5.1}$$

Métodos de Selección

Los diferentes métodos de selección determinan alternativas para seleccionar los individuos de una generación a la siguiente.

⊕ **Muestreo Directo: Elite**

Elige un conjunto K, una elite, del conjunto de inviduos. Para ello se los ordena según la aptitud y se elige cada uno $n(i)$ veces según la 5.2.

$$n(i) = \lceil \frac{(K-i)}{N} \rceil \tag{5.2}$$

⊕ **Estocástico: Ruleta**

De las aptitudes relativas $p(j)$ se derivan las aptitudes acumuladas para cada individuo.

$$q(i) = \sum_{j=0}^{i} p(j) \tag{5.3}$$

Luego se generan K números aleatorios de una $r_j \leftarrow U[0,1)$ que cumplan con $q_{i-1} < r_j < q_i$. Es decir se distribuye el espacio muetral entre todo los individuo según su aptitud relativa (a mayor aptitud mayor oportunidad de salir seleccionado en la ruleta).

⊕ **Universal**

Similar a Ruleta pero cambiando la ecuacion de r_j en base a $r \leftarrow U[0,1)$.

$$r_j = \frac{r+j}{K}, j\varepsilon[0,(K-1)] \tag{5.4}$$

⊕ **Ranking**

Selección por Ranking deriva una función alternativa de pseudo-aptitud $f'(i)$ que se calcula en base a un ranking. El ranking $rank(i)\varepsilon[1,N]$ es es una permutación de asignación a los N individuos dada por la función de aptitud real $p(i)$. Con esto se calcula:

$$f'(i) = \frac{N - rank(i)}{N} \tag{5.5}$$

A partir de este valor, se realiza el cálculo según ruleta utilizando la pseudo-aptitud relativa.

⊕ **Boltzmann**

Para este método también se define una función de pseudo-aptitud con la siguiente función:

$$f'(i,g,T) = \frac{e^{\frac{f(i)}{T}}}{< e^{\frac{f(i)}{T}} >_g} \tag{5.6}$$

donde i corresponde al individuo, T corresponde a la Temperatura del sistema, $f(i)$ es la fución de aptitud y $<>_g$ es el cálculo del promedio de la aptitud poblacional. Luego se utiliza esta función de pseudo-aptitud como la aptitud aplicando la selección por el método de Ruleta.

Los métodos basados en temperatura al igual que como ocurre con los métodos de mejoramiento iterativo, aplican una función de decrecimiento de temperatur exponencial que tiene el efecto de balancear exploration/explotaition. Esto favorece la búsqueda de variantes al principio del algoritmo, y la identificación de los más eficientes a medida que el algoritmo avanza.

⊕ **Torneos Probabilístico y Determinísticos** Los métodos de selección basada en torneos se

basan en justamente forzar una competencia, donde se selecciona un subconjunto de los individuos menor a al población total. La ventaja que tiene es que tiene una mayor velocidad de ejecución, ya que no se procesa toda la población, ni requiere variables intermedias, utilizando la función de aptitud de manera directa. Además de una facilidad en la propia implementación. La desventaja por otro lado es que imprimen una presión de selección muy alta, con lo que esto conlleva.

Los métodos de selección determinísticos se basan en:

- De la población original de tamaño N, se eligen M individuos al azar.
- De los M individuos, se elige el mejor.
- Se repite el proceso hasta conseguir los K individuos que se precisan.

Por otro lado, la variante de torneos probabilística, introduce un componente extra de estocasticidad para la selección de los individuos:

- Se elige un Threshold de $[0.5, 1]$.
- De la población original de tamaño N, se eligen solo 2 individuos al azar.
- Se toma un valor r al azar uniformemente entre $r \leftarrow U[0,1)$.
- Si $r < Threshold$ se selecciona el más apto.
- Caso contrario, se selecciona el **menos** apto.
- Se repite el proceso hasta conseguir los K individuos que se precisan.

Los torneos probabilistícos dan más oportunidades a aquellos individuos para que mantengan variabilidad en la población genética.

Convergencia Prematura

La convergencia prematura ocurre cuando la población converge aun subconjunto con muy poca variedad y sin haber llegado a una aptitud aceptable. Es decir, se reduce la variabilidad necesaria y suficiente para mantener la exploración en el conjunto de posibilidades.

Esto provoca que el método de optimización dado por el algorimto genético arroja un subóptimo, a menos que dicho punto de convergencia sea el máximo global o que la población escape de dicho máximo local por mutacion.

Las causas principales son una presión de selección **muy alta**, una probabilidad de mutación **muy baja** o un tamaño de la población muy escaso[1].

5.2 Crossover

Una vez que se seleccionan los individuos se los apárea y se realiza la recombinación de genes. Se suele plantear un parámetro que regula la probabilidad de recombinar, P_c que define si hay o no recombinación en la generación de los hijos. Para el caso de que no haya sólo se deja el impacto de la mutación y los genes de padres a hijos simplemente se copian.

Existen muchas variantes para la recombinación basadas en una reproducción de uno o más individuos involucrados. A continuación algunas de ellas que toman dos individuos y obtienen dos hijos:

5.2.1 Cruce de un punto

Directo, se elige un locus al azar y se intercambian los alelos a partir de ese locus. Es decir

$$P = [0, S]$$

con S siendo la cantidad de genes.

5.2.2 Cruce de dos puntos

En este caso se eligen dos locus al azar y se intercambian los alelos entre ellos.

$$P_1, P_2 \in [0, S]; P_1 \leq P_2$$

[1] Se cree que hace 900000 años el homo heidelbergensis casi se extingue generando una convergencia prematura, donde sólo 1300 individuos humanos vivos existían en la Tierra en ese momento, dejando una huella en nuestros genes de esa enorme reducción en la diversidad genética[59].

5.2.3 Cruce anular

Primero se considera al gen como un anilo, donde el último conecta con el primero circularmente. Luego se elige un locus al azar P y una longitud L. Finalmente se intercambia el segmento de longitud L a partir de P teniendo en cuenta la estructura anular. En este caso $P = [0, S-1]$ con $L = [0, \lceil \frac{S}{2} \rceil]$.

5.2.4 Cruce uniforme

Se produce un intercambio de alelos en cada gen con probabilidad P, que normalmente es 0.5.

5.3 Mutación

La mutación se define como una variación en la información genética que se almacena y codifica en el cromosama. Para que la mutación sea más eficiente, una correcta arquitectura y separación de genes es necesaria. El objetivo algorítmico de la mutación es enriquecer la diversidad genética (explorar el espacio de soluciones) y evitar los máximos locales (y escaparlos).

Las variantes son:

- **Mutación de gen:** se altera un solo gen con una probabilidad P_m.
- **Mutación multigen limitada:** se selecciona una cantidad $[1, M]$ al azar de genes a mutar, con probabilidad P_m.
- **Mutación multigen uniforme:** cada gen tiene una probabilidad P_m de ser mutado.
- **Mutación completa:** con una probabilidad P_m se mutan todos los genes del individuo, acorde a la función de mutación definida para cada gen.

La mutación o cambio del gen puede no sólo implicar cambiar un valor (1 por 0) sino que dependiendo de la codificación requiere una definición precisa. Por ejemplo, puede implicar aplicar un delta al gen, en algún sentido y con alguna distribución.

5.3.1 Generando generaciones

De una perspectiva pragmática, existen diferentes alternativas también para construir una nueva generación.

Supervivencia Aditiva

Generar K hijos de K padres. En este caso la nueva generación se formará seleccionando N individuos del conjunto $[N+K]$ siendo N los individuos de la generación actual y K los hijos.

Supervivencia Exclusiva

Generar K hijos de K padres.

- Si $K > N$ la nueva generación se genera seleccionando N de los K hijos unicamente.
- Si $K <= N$ la nueva generación se conformará por los K hijos generados $+(N-K)$ individuos seleccionados de la generación actual.

Brecha generacional

Determina la cantidad de individuos que sobreviven de una generación a la otra. Esto se captura en un indicador $G\varepsilon[0,1]$.

- $G = 1$: toda la población es reemplazada.
- $G = 0$: ningún individuo es reemplazado.
- G determina la suma convexa entre mantenimiento y reemplazo, con $(1-G)*N$ individuos de la generación actual y $G*N$ los nuevos individuos de reemplazo.

Criterios de Corte

El último punto a clarificar es la identificación de los posibles criterios de corte: ¿cuándo parar?.

- Tiempo
- Cantidad de generaciones
- Solución aceptable (cota sobre la función de aptitud).
- Estructura: cuando una parte relevante de la población no cambia por una cantidad de generaciones (estancamiento poblacional).
- Contenido: cuando la mejor aptitud no cambia por una cantidad de generaciones (se alcanza los valores óptimos en la función).

Esquemas

Los esquemas sirven para agrupar diferentes genes utilizando wildcards. Por ejemplo, el esquema $*01001*01$ representa $001001001, 001001101, 101001001, 101001101$.

Teorema 5.3.1 — Teorema de los esquemas. Aquellos esquemas con una aptitud media superior a la media de la población, de longitud pequeña y con un orden bajo aumentarán su presencia de manera exponencial en las sucesivas generaciones.

El orden del esquema determina la cantidad de genes con información que un preserva y se calcula como

$$o(S) = Len - wildcards$$

siendo *Len* la longitud del gen, y *wildcards* representa la cantidad de wildcards del esquema. Por ejemplo, $o(*01*) = 2$, $o(**0*1***1) = 3$. Mientras más alto el orden del esquema, mayor es la probabilidad de perderlo en una mutación.

El uso de esquemas permite componer **bloques constructivos**. Esquemas cortos, de bajo orden pero con una aptitud alta pueden ser elegidos, recombinados y re-elegidos nuevamente para ir formando cromosomas con mayor aptitud. Esto permite reducir la complejidad del problema, dividiendolo en subproblemas: se optimizan los esquemas, se encuentran nuevas esquemas o eventualmente se agrupan.

III

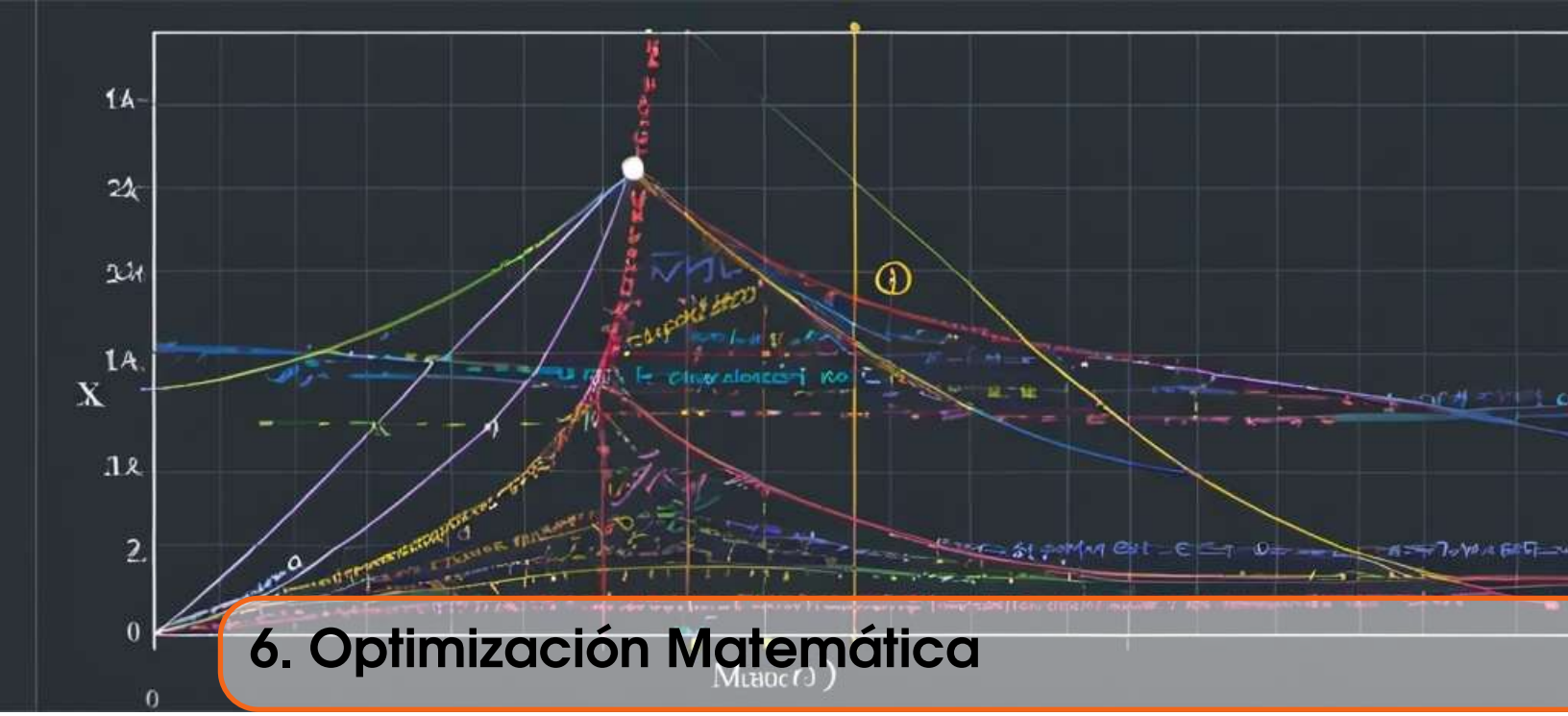
Perceptrón

6 Optimización Matemática 59
6.1 La latita de Coca
6.2 El problema de optimización
6.3 El método simplex
6.4 Optimización no lineal sin restricciones

7 El modelo de Neurona 69
7.1 La bioinspiración
7.2 Separabilidad lineal

8 La era del Coneccionismo 81
8.1 Perceptrón Multicapa
8.2 Algoritmo del Perceptrón Multicapa
8.3 Variantes de optimización
8.4 Arquitectura del Perceptrón Multicapa

9 Métricas de Evaluación 95
9.1 Identificar el patrón de manera correcta
9.2 Métricas estándar



6. Optimización Matemática

El área de optimización matemática [10] puede considerarse "media matemática"[1]. Surge a partir de los primeros trabajos de Cauchy en 1847 y la propuesta del método del gradiente descendente. Luego, en la era de la posguerra tiene un auge importante impulsado por el desarrollo cuantitativo de la industrialización en masa, con los algoritmos de programación lineal, y la variante estocástica del algoritmo de gradiente descendente. En los 80's surge la teoría convexa y el marco general de optimización. Finalmente, en la primera década del siglo XXI, las ideas de optimización comienzan a aplicarse masivamente a problemas de aprendizaje automático. A partir del 2012 comienza un nuevo empuje que encuentra en las redes neuronales una aplicación directa. Muchos profesores y estudiantes dedicaron su vida a esta temática sacrificando su felicidad con horas de estudio en un salón oscuro y polvoriento. Ahora nos toca a nosotros [60].

6.1 La latita de Coca

Imaginarse una lata de coca cola. Una característica interesante que puede buscarse es encontrar cuales tienen que ser las dimensiones de una lata cilíndrica que contenga un litro, para minimizar su superficie y así la cantidad de aluminio que hay que utilizar.

Este es un típico problema que se estudia en cursos de Cálculo o Análisis Matemático alrededor del mundo y se puede resolver exactamente como un problema de optimización. La idea es partir de la base de definir el problema según la geometría de la lata:

$$V = \pi r^2 h \tag{6.1}$$



Figura 6.1: De nada Coca Cola Corporation por resolverte la vida.

[1] Se puede demostrar que toda area de matemática es media matemática, pero no tenemos lugar acá para hacerlo.

donde r es el radio del cilindro y h su altura. Luego la superficie de la lata, la que usará su aluminio, suponiendo un grosor despreciable viene dada por

$$S = 2\,\pi\,r^2 + 2\,\pi\,r\,h \tag{6.2}$$

De aquí surge el planteo como problema de optimización

$$S(r,h) = 2\,\pi\,r^2 + 2\,\pi\,r\,h \qquad\qquad g(r,h) = \pi\,r^2\,h = 1000\,\text{cm}^3 \tag{6.3}$$

Es decir se busca optimizar $S(r,h)$ (encontrar los r y h que hacen que sea más chiquito) sujeto a la restricción dada en $g(r,h)$ a un volumen fijo. La solución pasa por despejar primero h de la segunda ecuación, reemplazarla en la primera, y resolver directamente utilizando la propiedad de que la derivada de un función es cero en sus extremos.

$$g(r,h) = \pi\,r^2\,h = 1000\,\text{cm}^3$$
$$h = \frac{1000\,\text{cm}^3}{\pi\,r^2}$$

Reemplazando en 6.2

$$S(r) = 2\,\pi\,r^2 + 2\,\pi\,r\,(\frac{1000\,\text{cm}^3}{\pi\,r^2})$$
$$S(r) = 2\,\pi\,r^2 + \frac{2000\,\text{cm}^3}{r}$$

El punto extremo ocurre cuando la derivada de $S(r)$ respecto a r es cero. Esto es

$$S'(r) = 4\,\pi\,r - \frac{2000\,\text{cm}^3}{r^2} = 0$$
$$4\,\pi\,r = \frac{2000\,\text{cm}^3}{r^2}$$
$$4\,\pi\,r^3 = 2000\,\text{cm}^3$$
$$r^3 = \frac{500\,\text{cm}^3}{\pi}$$
$$r = \sqrt[3]{\frac{500\,\text{cm}^3}{\pi}}$$

Como el punto que se desea encontrar es el mínimo, esto ocurre cuando la derivada pasa de negativa a positiva, y eso es justamente en la solución positiva de $r = 5.41\ cm$, lo que provoca un valor de $h = 58\ cm$ para la altura[2]. Esto es un típico problema de optimización no lineal con restricciones y como se ve el cálculo de la derivada tiene un valor crucial para su solución.

[2]Fijaos que es una lata de medio metro, pero de un litro!

6.2 El problema de optimización

Optimización puede servir para minimizar los costos de producción de una empresa, resolver problemas de dinámica de transporte, aplicarse en teoría de juegos, hallar los parámetros óptimos de una función de distribución, etc. Existen numerosos algoritmos y áreas de optimización, entre las cuales podemos encontrar:

- Optimización Discreta: Programación entera y combinatoria
- Optimización Discreta: Ecuación diofántica
- Optimización Lineal: Programación Lineal
- Optimización No lineal sin restricciones convexa: garantía de extremos globales
- Optimización no convexa: métodos de primer orden: GD,SGD,Adam
- Optimización no convexa: métodos de segundo orden: Newton
- Optimización no convexa: cero orden: Bayesiana, Powel, Sim Optimistic Optimization

Para bosquejar el proceso de optimización se presenta el método Simplex y luego nos vamos a concentrar específicamente en la optimización no lineal sin restricciones, dada su relevancia para el entrenamiento de las redes neuronales.

Teorema 6.2.1 — Mínimo de función monotónica. El problema básico de optimización puede resumirse en la ecuación siguiente

$$\min_{x\in\Re^n} f(x),\ f:\Re^n\to\Re, \vec{x}=[x_1,x_2,...,x_n] \tag{6.4}$$

$$g_i(\vec{x}) <= 0 \tag{6.5}$$

$$h_j(\vec{x}) = 0 \tag{6.6}$$

donde $\vec{x}$ son las variables de diseño, $f(\vec{x})$ es la función objetivo/costo, en tanto que g_i y h_j son restricciones. El objetivo es encontrar $\vec{x}^*$ que resuelve el problema para $f(\vec{x}^*)$.

6.3 El método simplex

Este es un método tradicional de programación lineal, muy común en el área de Investigación Operativa (2.2.11). Fue desarrollado primero durante la segunda guerra mundial por George Dantzig, y fue muy importante para los aliados para resolver problemas de logística y de producción. Luego fue reinventado independientemente en la Rusia soviética por Leonid Khachian, creyendo que era revolucionario pero que al final no fue otra cosas que el mismo método. Un aspecto interesante es que muestra como el proceso de optimización conlleva en sí mismo una búsqueda de una solución, y esto se ve de manera directa en este método donde lo que se hace es identificar la región de validez de la solución, las intersecciones, y luego se prueba una por una cada intersección, eligiendo la que impactará mejor en la función de costo, hasta que se encuentra la óptima (o se determina que no es posible resolver) [61, 62].

Este método intenta encontrar la solución del problema de optimización dado por una multiplicación vectorial **cx** según las restricciones dadas por la solución de un sistema de ecuaciones, según la definición 6.3.1.

Definición 6.3.1 — Simplex. Se intenta resolver el sistema lineal
$\min f(\vec{x}) = \mathbf{cx}$
condicionado a $Ax = b,\ \mathbf{x} > 0$

El método garantiza que cualquier problema de optimización lineal puede llevarse a la forma estándar mediante tres reglas.

1. $\max(\mathbf{cx} : A\mathbf{x} = \mathbf{b},\ \mathbf{x}) = -\min(-\mathbf{cx} : A\mathbf{x} = \mathbf{b},\ \mathbf{x})$
2. $x \geq 0 \rightarrow x = x_1^+ - x_2^-$
3. Slack: $z = \mathbf{b} - A\mathbf{x} > 0$

La primera establece la equivalencia entre que maximizar algo es equivalente a encontrar el inverso aditivo de la minimización del inverso aditivo de la función de costo.

La segunda regla permite mediante el agregado de una variable, representar variables que pueden tomar valores negativos, como la suma de dos variables, donde ambas son positivas pero la segunda resta la primera.

La tercer regla es para establecer variables de slack[3]. Dada una inecuación, puedo agregar una variable de slack dependiente que representa la diferencia entre el valor de un miembro versus el valor del otro, y reemplazar la inecuación por una igualdad.

6.4 Optimización no lineal sin restricciones

La optimización no lineal sin restricciones permite encontrar los valores que optimizan funciones no-lineales, sin ningún tipo de restricción adicional [63, 64].

$$\min_{x \in \Re^n} f(x),\ f : \Re^n \rightarrow \Re, \tag{6.7}$$

Donde f es una función no lineal, por ejemplo:
1. $f(x_1, x_2) = x_1^2 + sen(x_1 * x_2) - x_1 * x_2$
2. $f(x_1, \ldots, x_n) = \sum_{i=1}^{n} (x_i - cos(x_i))^2$
3. $f(x_1, \ldots, x_n) = \sum_{i=1}^{n} (x_i - g(x_i, \xi_i))^2$

Esto es particularmente interesante en el caso de las redes neuronales, cuando la función a optimizar es directamente una función llamada función de error, $E(w)$, que depende de los pesos sinápticos de la red.

$$\min_{w \in \Re^n} E(w),\ E : \Re^n \rightarrow \Re, \tag{6.8}$$

donde

- $E(w_1, \ldots, w_n) = \frac{1}{2} \sum_{\mu=1}^{p} (\zeta^\mu - g(\sum_{i=1}^{n} w_i x_i^\mu))^2$ es la función de error.
- $w = (w_1, \ldots, w_n)$ es el vector de pesos sinápticos que queremos ajustar.

[3] Muy famosas en este método.

- n es la cantidad de pesos sinápticos o parámetros libres de la red.
- $g(\cdot)$ es una función no lineal de activación.
- $\mathbf{x}$ son las μ muestras del conjunto de datos, en tanto que ζ son las etiquetas de salida asociadas a cada muestra.

Llamar a esta función una función de error (o costo) alude a que se puede considerar a $g(\sum_{i=1}^{n} w_i x_i^{\mu})$ como un mecanismo, una transformación de una entrada x_i^{μ} (un dato de entrada) en una salida ζ^{μ}, mediada por parámetros libres w_i, que son los que se quieren ajustar, para que cada entrada produzca la salida. La función dada mide el error entre lo que devuelve el mecanismo y lo que **debería** salir (el valor deseado).

Existen diferentes alternativas para solucionar estos problemas, y gran parte de la teoría de optimización cubre estas alternativas.

- Métodos exactos: Calculan una fórmula cerrada para la solución.
- Métodos de aproximación de la solución:
  - Métodos basados en las derivadas primeras, o en el gradiente.
  - Métodos basados en las derivadas segundas, o en el hessiano.
  - Métodos sin derivadas.
  - Métodos Estocásticos (estiman la dirección).

6.4.1 Definiciones preliminares

Las siguientes definiciones y teoremas ofrecen las condiciones necesarias y suficientes para establecer optimalidad.

Definición 6.4.1 — Matriz semi-definida positiva. Una matriz A es definida semi-positiva si

- Si $x^t Ax > 0,\ x \neq 0$.
- Si todos sus autovalores son positivos
- Hay otras características que definen a las matrices definidas positivas, por ejemplo los valores de la diagonal deben ser mayores que la suma de los otros elementos de la fila.
- Si $x^t Ax \geq 0,\ x \neq 0$, entonces se dice que la matriz es **semi definida positiva**

Teorema 6.4.1 — Existencia de mínimos locales y globales. Sea la función f diferenciable con primera y segunda derivadas continuas, entonces x^*...

- Es mínimo global si $\forall x,\ f(x^*) \leq f(x)$
- Es mínimo local si $f(x^*) \leq f(x)\ \forall x,\ \|x - x^*\| < \varepsilon$

Teorema 6.4.2 — Condiciones necesarias de optimalidad. Las condiciones son:

- **Condición necesaria de primer orden**
  Si x^* es un mínimo local de f entonces $\nabla f(x^*) = 0$.
- **Condición necesaria de segundo orden**
  Si x^* es un mínimo local de f entonces $\nabla f(x^*) = 0$ y $H_f(x^*)$ (el hessiano) es una matriz semidefinida positiva.

Teorema 6.4.3 — Condición suficiente de optimalidad. Si x^* es tal que $\nabla f(x^*) = 0$ y $H_f(x^*)$ es definida positiva, entonces es un mínimo local de f.

El Hessiano determina si la función es concava o convexa:

- Función convexa $\rightarrow$ Hessiano matriz definida positiva.
- Función cóncava $\rightarrow$ Hessiano matriz definida negativa.

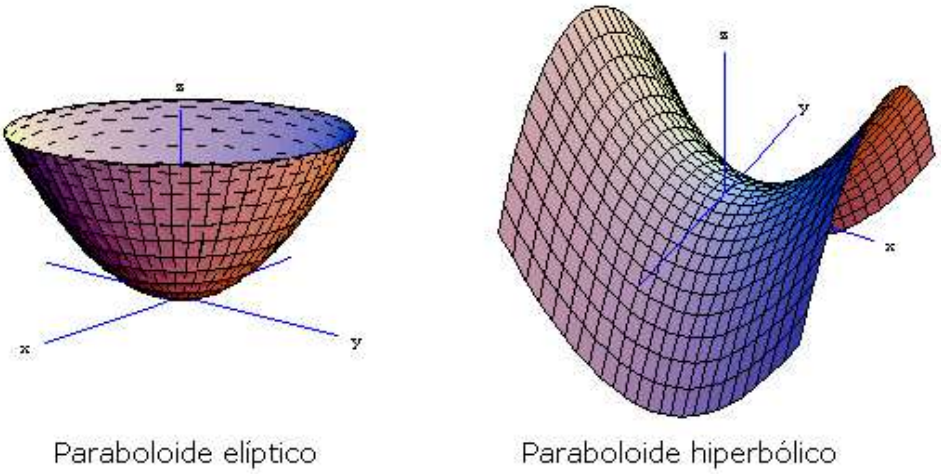


Figura 6.2: Hessiano definido positivo Hessiano NO definido positivo

Hete aquí lectores, que encontrarán siempre detrás de cualquier tipo de algoritmo que ose llamarse de IA, el siguiente procedimiento subyacente en el entrenamiento de sus parámetros libres, de alguna u otra manera.

Algoritmo 4 Procedimiento general de optimización

1: Punto inicial x_k, dato de entrada.
2: Iterativamente para cada paso k.
3: **while** no converge **do**
4: Buscar una dirección de movimiento d_k.
5: Calcular o decidir la longitud de paso α_k.
6: Actualizar al nuevo punto $x_{k+1} = x_k + \alpha_k d_k$.
7: **end while**
8: Salida: Puntos x^* idealmente óptimo global.

La idea detrás del Algoritmo 4 es encontrar dos cosas. Una es el largo del paso de iteración y la segunda es la dirección para ir encontrando puntos que cumplan $f(x_{k+1}) < f(x_k)$. La dirección de movimiento debe ser descendiente, o sea $d_k^t \nabla f(x_k) < 0$ a sabiendas de que:

- El gradiente es la dirección de máximo crecimiento de una función.
- Cualquier dirección contraria a la del gradiente, es una dirección de decrecimiento de la función.

y el paso debe ser tal que permita que el algoritmo converja en un tiempo acotado y que a su vez permita en un número de pasos finito siempre acercarse más a la solución.

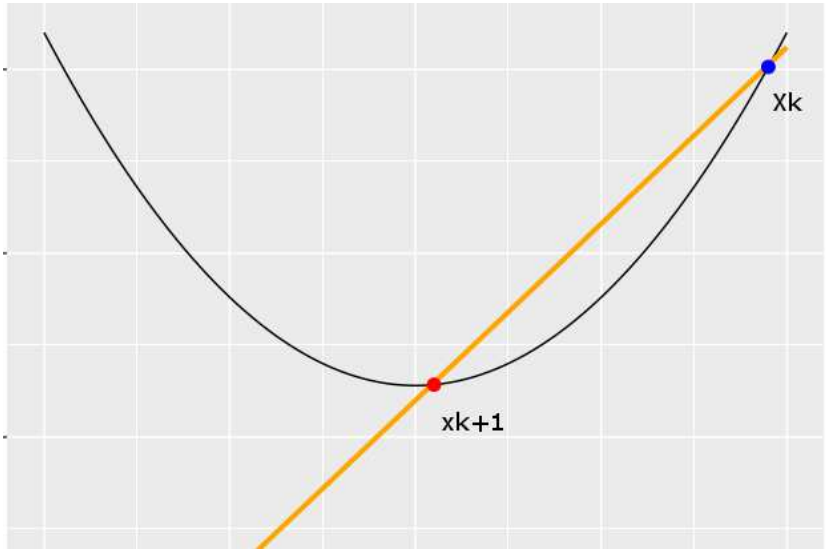


Figura 6.3: El objetivo del procedimiento general de optimización es encontrar la dirección y el paso para pasar de X_k a un X_{k+1} que produzca un valor más pequeño en la función de costo.

6.4.2 Gradiente Descendente

Este famoso método se basa justamente en utilizar la dirección de máximo decrecimiento de la función a optimizar que es la correspondiente al gradiente.

$$d_k = -\nabla f(x_k) \tag{6.9}$$

Si bien tiene convergencia lenta, tiene la ventaja de no requerir información de la segunda derivada.

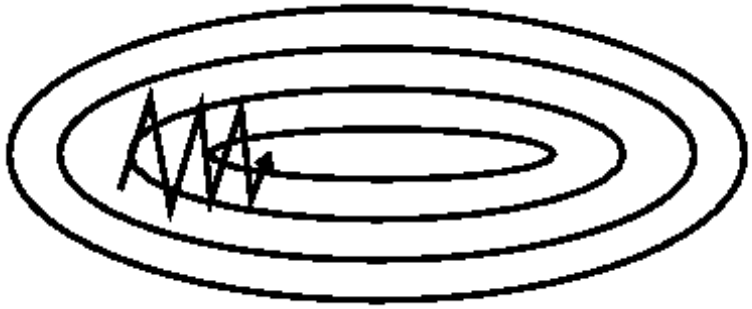


Figura 6.4: Gradiente descendente avanza en zig-zag.

La Figura 6.4 muestra las curvas de nivel de la función escalar de error, el centro sería el valor mínimo. El gradiente sigue la dirección indicada, lo cuál se visualiza inmediatamente que genera un problema al aproximarse a la solución en zig zag, lo que no es un buen balance entre velocidad de acercamiento y precisión. Se puede suavizar un poco más utilizando el concepto de **Momentum**, que como veremos más adelante se basa en la idea de tomar la dirección de descenso como una combinación lineal de direcciones de descenso calculadas en pasos anteriores.

$$x_{k+1} = x_k - \alpha_k \nabla f(x_k) - \beta \alpha_{k-1} \nabla f(x_{k-1}) \tag{6.10}$$

Este promedio ponderado entre direcciones suaviza el zig zagueo del método del gradiente con $0 < \beta < 1$. Esto es porque la nueva dirección es un promedio ponderado por beta de las dos direcciones anteriores.

6.4.3 Método de Newton

Como se vio en el ejemplo de la latita de Coca, la derivada segunda aporta información. Lo que entonces podemos hacer es aproximar el gradiente en sí mismo por el polinomio de Taylor de segundo orden,

$$\nabla f(x_{k+1}) = \nabla f(x_k + \alpha_k d_k) = \nabla f(x_k) + \alpha_k H(x_k) d_k \tag{6.11}$$

Entonces, si x_{k+1} fuera el mínimo, $\nabla f(x_{k+1}) = 0$ y por lo tanto

$$\nabla f(x_k) + \alpha_k H(x_k) d_k = 0$$

de donde resulta

$$d_k = -H^{-1}(x_k) \nabla f(x_k)$$

Este caso parecería ser la solución definitiva, ya que se puede encontrar el punto exacto donde ocurre el mínimo (sin tener que iterar). Sin embargo, el problema aparece en tener que estimar el Hessiano y/o su inversa para cualquier función que haya que optimizar.

6.4.4 Métodos Cuasi Newton

Estos métodos se basan en intentar disminuir el costo computacional asociado al cálculo del Hessiano y particularmente su inversa. Se basan en realizar una aproximación de $H^{-1}(x_k)$ y reemplazarla por una matriz de aproximación definida positiva B_k. Las variantes de estos métodos justamente difieren en como realizar la aproximación de esta matriz.

Sin embargo, aún con la mejora en el rendimiento, los métodos cuasi Newton tienen la desventaja de poseer un alto costo computacional para aplicaciones de entrenamiento de redes neuronales.

En su lugar, se puede utilizar el método **L-BFGS (limited memory Broyden Fletcher Goldfarb Shanno (BFGS))** [65]. Este algoritmo, siguiendo con la idea del huevo y la gallina, implementa una búsqueda iterativa para estimar el Hessiano $B_{k+1} \sim B_k$.

6.4.5 Métodos de Direcciones Conjugadas

Estos métodos buscan las direcciones por las cuales buscar la solución, explotando el espacio de direcciones que se establecen mediante la idea de direcciones conjugadas con una matriz. Tienen la ventaja de ofrecer una garantía de convergencia y una cota en la cantidad de pasos cuando la forma de la función de costo es específica.

Teorema 6.4.4 — Vectores A-conjugados. Sea el conjunto de direcciones (vectores) $d_1, \ldots, d_n$ y A una matriz simétrica definida positiva, entonces:

- Si $d_i^t A d_j = 0$, $\forall i \neq j$ entonces se dice que $d_1, \ldots, d_n$ son direcciones A-conjugadas.

- Si un conjunto de vectores es A-conjugado, con A simétrica, definida positiva entonces es también un conjunto linealmente independiente.

Teorema 6.4.5 — Cota en la convergencia de los Métodos de Direcciones Conjugadas. Dada una función cuadrática $f : \Re^n \to \Re$ $f(x) = x^t Hx + b^t x$ Si un método de minimización no lineal realiza las búsquedas unidimensionales sobre direcciones H-conjugadas, entonces el método converge en n pasos.

Por supuesto no siempre es posible que la función de costo tenga la forma que establece el teorema, pero sirven como un punto de partida de otros métodos que intentan aproximar la idea con funciones mucho más complejas.

6.4.6 Métodos de Gradientes Conjugados 1952

Esté método se basa en que para cada paso k, se calcula una nueva dirección d_{k+1} que es $H(x_k)$-conjugada con todas las direcciones anteriores $d_1, \ldots, d_k$, basada en el $\nabla f(x_k)$.

Exige sólo identificar el gradiente en cada paso.

6.4.7 El método de Powell 1964

Este método está basado en la determinación de direcciones conjugadas sin requerir el uso de derivadas parciales (orden cero). El método genera en cada paso k un conjunto de direcciones conjugadas

$$d_1^k, \ldots d_n^k.$$

Comienza con un conjunto de direcciones linealmente independientes $d_1^0, \ldots d_n^0$ y un punto inicial x_0. Luego en el paso k, saca del conjunto la dirección d_k^k y agrega una dirección conjugada con $d_1^{k-1}, \ldots d_{k-1}^{k-1}$ (las de los pasos anteriores). Este método es en el fondo, una búsqueda lineal.

6.4.8 Gradiente Descendente Estocástico

El método más utilizado en redes neuronales es el **SGD**, Stochastic Gradient Descend, y se basa en una extensión al método de gradiente descendente pensada sobre todo para funciones de error.

Se tienen p observaciones $\mathbf{x}^\mu = \{x_1, \ldots x_n\}$, $\mu = 1, \ldots, p$ de dimensión n del conjunto de entrenamiento (X) junto con sus p salidas ζ^μ de dimensión 1 y $\mathbf{w} = (w_1, \ldots, w_n)$ el vector de pesos sinápticos (parámetros libres). Entonces, queremos hallar $\mathbf{w}$ que minimice $E(\mathbf{w})$:

$$E(\mathbf{w}) = \frac{1}{2} \sum_{\mu=1}^{p} (\zeta^\mu - \sum_{i=1}^{n} w_i x_i^\mu) \tag{6.12}$$

$$E(\mathbf{w}) = \frac{1}{p} \sum_{\mu=1}^{p} E^\mu(x^\mu, \mathbf{w}) \tag{6.13}$$

$$\mathbf{w}^{t+1} = \mathbf{w}^t - \eta_t \frac{1}{p} \sum_{\mu=1}^{p} \nabla_{\mathbf{w}} E^\mu(x^\mu, \mathbf{w}^t)$$

Considerando que $\frac{1}{p}\sum_{\mu=1}^{p}\nabla_w E(x^\mu, \mathbf{w}^t)$ es un estimador de la esperanza del gradiente, dado un conjunto de entrenamiento,... **¿Por qué no usar cualquier otro estimador?**.

Por ejemplo: un

$$\nabla_w E(x^\nu, \mathbf{w}^t),$$

para algún ν arbitrario o un subconjunto aleatorio $\sum_{\mu=1}^{k}\nabla_w E(x^\mu, \mathbf{w}^t)$, $k << p$.

Esta es la esencia del método de gradiente descendente estocástico. Donde solamente una parte de los datos se utiliza para calcular la dirección de descenso en cada paso. El nombre original de este método fue ADALINE, o Adapter Linear Element.

Los autores del método, demuestran en su libro [66] que el método converge, pero no siempre va descendiendo.

ADAGrad, 2011 (Adaptative Gradient)

Es un método que mejora ADALINE que tiene como principal objetivo adaptar el valor de la tasa de aprendizaje en cada paso y la actualización del vector $\mathbf{w}$ se realiza coordenada a coordenada. Sea $g_t = \nabla E(\mathbf{w}_{t-1})$ entonces $(g_t)_i$ es la i-ésima coordenada $g_{1:t} = \{g_1, \ldots, g_t\}$ son todos los gradientes anteriores hasta el paso t.

Mientras que en SGD se hace

$$\mathbf{w}^{t+1} = \mathbf{w}^t - \eta_t \nabla_{\mathbf{w}} E^\mu(x^\mu, \mathbf{w}^t)$$

En ADAGRAD se calcula

$$\mathbf{w}_i^{t+1} = \mathbf{w}_i^t - \frac{\eta_t}{\sqrt{G_{ii}^t + \varepsilon}}\nabla_{\mathbf{w_i}} E^\mu(x^\mu, \mathbf{w}^t)$$

con $G^t = \sum_{\tau=1}^{t} g_\tau * g_\tau^t$.

La ventaja de este método es que cada coordenada tiene su propia actualización.



7. El modelo de Neurona

Dado el éxito que representan las redes neuronales y, en particular, el área de *Deep Learning* dentro del campo de inteligencia artificial, es importante entender el origen de las mismas y su importancia histórica. Para ello, ahondaremos en este capítulo las bases que sentaron el modelo primitivo detrás de las redes neuronales artificiales.

Frank Rosenblatt (1928-1971), de formación psicólogo experimental, desarrolló un mecanismo denominado Perceptrón en la Universidad de Cornell, en el marco de un proyecto para la marina norteamericana[1]. Ideó el algoritmo y su implementación en hardware *Mark I*. Tenía 400 sensores de luz que actuaban como una retina, proveyendo información a 1000 neuronas dispuestas en una sola capa. Estas neuronas realizaban el procesamiento y producían una única salida.

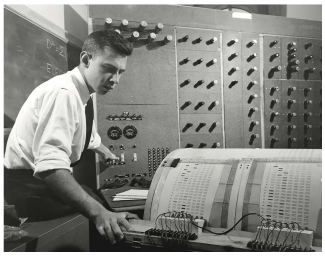


Figura 7.1: Frank Rosenblatt junto a la máquina Mark I (implementación física del algoritmo del Perceptrón).

En 1958, el New York Times declaraba que *la máquina será el primer dispositivo en pensar como lo hace el cerebro humano* [50, 67]. La motivación e inspiración de Rosenblatt radicaba en su formación y tendencia a la interdisciplinaridad, característica esencial del campo de redes neuronales. Frank buscaba *entender la cantidad mínima de elementos que un cerebro debía tener físicamente, para capturar las cosas maravillosas que es capaz de hacer.* El algoritmo propuesto se basa en una combinación lineal de entradas, umbralizada para generar una respuesta binaria, similar a los impulsos eléctricos que transmiten las neuronas del cerebro. Rosenblatt fue capaz de probar que, con una sola capa y una regla simple de actualización, podría alcanzar el cálculo computacional que necesitaba; en pocas palabras, la gran

[1] Gran parte de las líneas de investigación en inteligencia artificial tienen raíces que arrancan en la Segunda Guerra Mundial o en el período posguerra en el marco de la carrera armamentista entre la Unión Soviética y EEUU. Interesante relato sobre el tema en "A History of Warfare", Mariscal Montgomery, "Engineers of Victory: The Problem Solvers Who Turned the Tide in the Second World War" de Paul Kennedy, o "Accesory to War" de Neil deGrasse Tyson.

innovación fue su capacidad de establecer un esquema probado de *aprendizaje*. El algoritmo del Perceptrón es una piedra fundamental de la disciplina que busca artificializar la inteligencia, y fue el primer trabajo que impulsó el **hype** en torno al área, una característica constante que podemos experimentar a día de hoy.

7.1 La bioinspiración

La idea del Perceptrón simple tiene su origen en el modelo de neurona creado por McCulloch y Pitts, en 1943 [13]. Es un modelo simplificado, basado en los conocimientos que existían en la época en relación al funcionamiento de las neuronas y la necesidad de encontrar un diagrama simple que explicara las características principales. Establece que una neurona posee un conjunto de estímulos que llegan al soma (también llamada cuerpo) a través de sus dendritas. En el soma se produce una integración de la información, la cual se traduce en una excitación o inhibición de la neurona. Si el valor luego del procesamiento supera cierto umbral, se produce una salida binarizada de la neurona, correspondiente a una salida activa. Se puede encontrar una representación esquemática de lo explicado previamente en la figura 7.2.

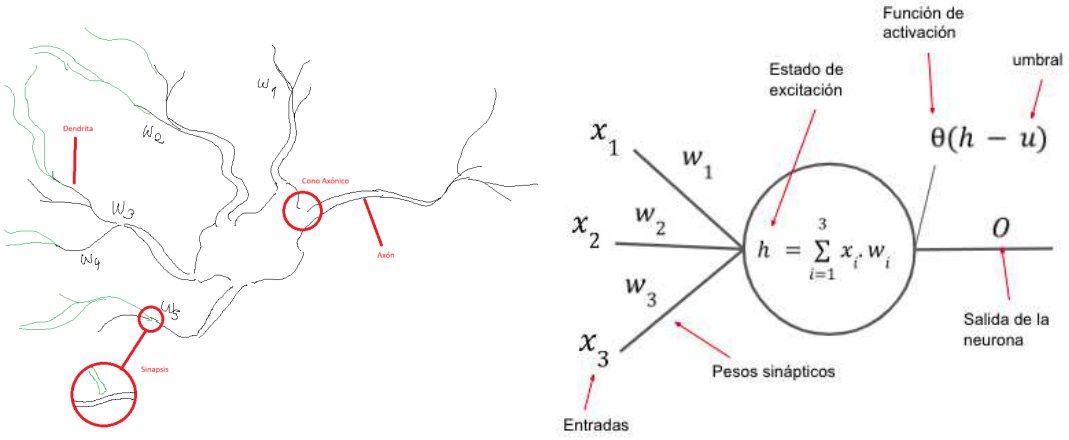


Figura 7.2: A la izquierda, se encuentra el diagrama simplificado de una neurona, de acuerdo a los conocimientos de la época. A la derecha, se traducen los componentes biológicos a conceptos matemáticos. Los x_i son las entradas, los w_i son los pesos sinápticos y u es el umbral. En este caso particular, se cuenta con tres entradas, por lo que se tienen tres pesos sinápticos y un umbral. La integración o procesamiento de la información está dada por el producto interno entre el vector de pesos sinápticos y el vector de entradas, representado por la variable h. La función ϕ es la función de activación, la cual para las primeras versiones del Perceptrón se circunscribía a la función escalón o signo.

Matemáticamente, este modelo corresponde a:

$$O = \phi(\sum_{i=1}^{N} w_i x_i - u) \tag{7.1}$$

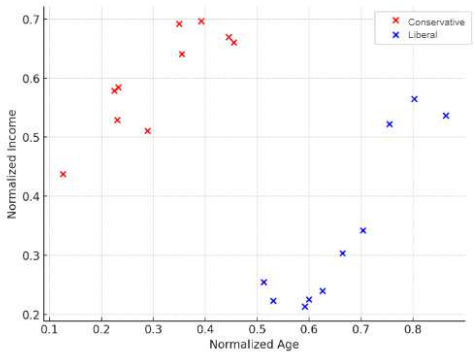
donde O es la salida del modelo de neurona, x_i son las entradas (valores provenientes de un conjunto de características), w_i son los denominados pesos sinápticos (parámetros libres), u es el umbral, que

representa el borde de decisión entre el estado activo y el estado inactivo de la neurona y ϕ es la función de activación. Para las primeras versiones del perceptrón, la función de activación se circunscribía a la función escalón o signo:

$$\phi(x) = \begin{cases} 1 & \text{if } x >= 0, \\ 0 & \text{if } x < 0. \end{cases} \tag{7.2}$$

7.2 Separabilidad lineal

Consideremos un conjunto de datos en dos dimensiones, donde cada eje corresponde a una característica determinada y donde los puntos pueden ser clasificados en dos clases. En el ejemplo de la figura 7.3, el conjunto de puntos corresponde a individuos que pueden ser clasificados según su orientación política, de acuerdo a las características de edad normalizada e ingresos económicos normalizados (valores entre 0 y 1).



Age	Income	Party
0.445	0.669	1.0
0.349	0.692	1.0
0.232	0.585	1.0
0.125	0.438	1.0
0.224	0.579	1.0
0.23	0.529	1.0
0.392	0.696	1.0
0.355	0.641	1.0
0.455	0.66	1.0
0.289	0.51	1.0
0.513	0.255	-1.0
0.755	0.522	-1.0
0.626	0.24	-1.0
0.703	0.342	-1.0
0.863	0.536	-1.0
0.6	0.225	-1.0
0.664	0.303	-1.0
0.802	0.565	-1.0
0.592	0.213	-1.0
0.531	0.223	-1.0

Figura 7.3: A la izquierda, se encuentra graficado el conjunto de puntos en dos dimensiones, cada uno perteneciente a una clase diferente. Los puntos de la clase 1 están representados en azul y los de la clase -1 en rojo. A la derecha, se puede ver la tabla de datos, con los valores de las características de edad e ingresos económicos para cada uno de los puntos.

En base a la información provista, ¿es posible encontrar una recta que permita separar con perfecta precisión los puntos en dos grupos? La figura 7.4 demuestra que esta recta existe y puede modelarse según la clásica ecuación $y = mx + b$.

La existencia de la recta permite esbozar una generalización: dado un punto nuevo, puedo determinar a qué clase pertenece, dependiendo de su posición con respecto a la recta de separación. Para determinar a qué clase pertenece este dato, es posible proyectar el nuevo punto sobre la recta y evaluar el signo de esa función.

Para evaluar el punto sobre la recta, recurrimos a la ecuación de la recta, la cual podemos reescribir de la siguiente manera:

$$0 = y - mx - b \tag{7.3}$$

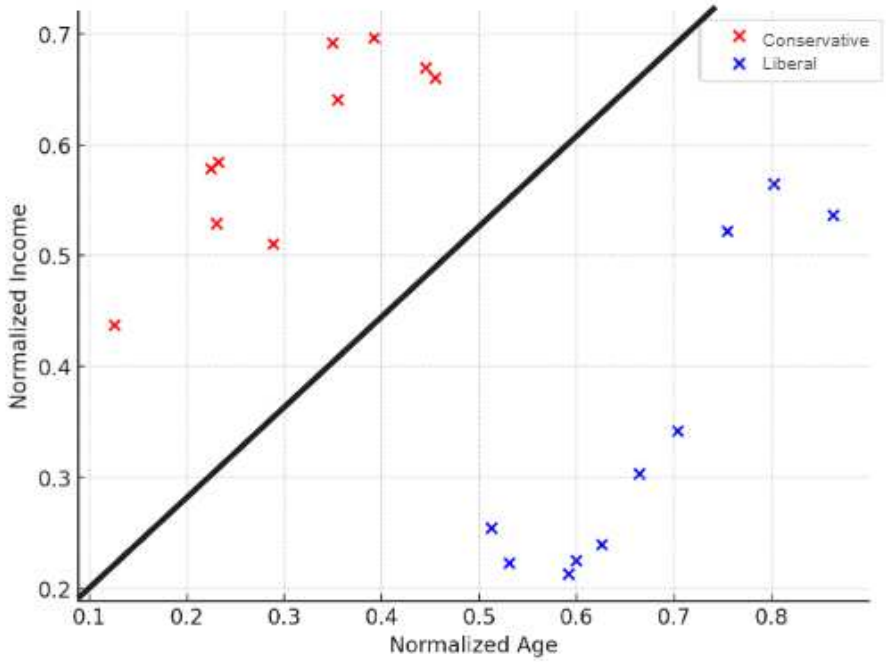


Figura 7.4: Recta de separación para el conjunto de datos provisto. La misma permite una perfecta separación entre las dos clases.

Si reemplazamos y por w_2x_2, $-mx$ por $-w_1x_1$ y $-b$ por $-w_0$, obtenemos:

$$0 = w_2x_2 - w_1x_1 - w_0 \tag{7.4}$$

Ahora, reemplazando w_2, w_1 y w_0 por los valores que se corresponden con la recta en 7.4 (la recta identidad) obtenemos:

$$0 = x_2 - x_1 - 0 \tag{7.5}$$

¿Qué valor toman los puntos correspondientes a la clase 1? Por ejemplo, para el punto $(0.445, 0.669)$, el valor de la función es:

$$0.669 - 0.445 = 0.224 \tag{7.6}$$

Por otro lado, ¿qué valor toma el punto $(0.703, 0.342)$, correspondiente a la clase -1? El valor de la función es:

$$0.342 - 0.703 = -0.361 \tag{7.7}$$

Si tomamos la magnitud de la proyección de cada uno de los puntos y evaluamos el signo de la misma, podemos determinar a qué clase pertenece el punto. Para ello, utilizamos la función signo:

$$sign(x) = \begin{cases} 1 & \text{if } x >= 0, \\ -1 & \text{if } x < 0. \end{cases} \tag{7.8}$$

Aplicando esta función a los puntos obtenemos:

$$sign(0.224) = 1 \tag{7.9}$$

$$sign(-0.361) = -1 \tag{7.10}$$

Podemos generalizar este proceso para un punto nuevo, h, de la siguiente manera:

$$sign(h) = \begin{cases} 1 & \text{if } h >= 0, \\ -1 & \text{if } h < 0. \end{cases} \tag{7.11}$$

donde h se encuentra definido como:

$$h = \sum_{i=1}^{N} w_i x_i + w_0 \tag{7.12}$$

donde N es la dimensión del espacio. Si h es positivo, identificamos a ese punto como perteneciente a la clase 1 en tanto que si es negativo, lo identificaremos como perteneciente a la clase -1. Por otro lado, notemos que w_0 es el umbral de la neurona, que representa el borde de decisión entre el estado activo y el estado inactivo de la neurona, antes denotado como u.

En este ejemplo, los datos se encuentran representados en dos dimensiones, con lo cual debemos encontrar una recta. Sin embargo, en el caso general donde los datos se encuentran representados en N dimensiones, la recta se generaliza a un hiperplano de dimensión $N-1$.

¿Qué significa que el hiperplano de separabilidad lineal tenga la misma estructura que la fórmula propuesta del perceptrón simple de la sección 7.1? Que la herramienta propuesta por Frank Rosenblatt es capaz de resolver problemas linealmente separables de clasificación binaria. Esto no era una novedad para la época, ya que existen otros algoritmos que permiten resolver este tipo de problemas, provenientes del campo de Aprendizaje Automático (*Machine Learning*). La innovación radica en lo que veremos a continuación: la capacidad de proponer un algoritmo bioinspirado para encontrar una solución a un problema determinado, mediante un proceso iterativo, análogo al proceso de aprendizaje (esto es una sobresimplificación y antropomorfización del algoritmo, pero es útil para entender el concepto).

7.2.1 Aprendizaje Hebbiano

El modelo McCulloch-Pitts permite resolver un modelo simple de clasificación binaria. En lo explicado previamente, hemos logrado implementar conceptos relacionados con la neurona biológica, como soma, axones y dendritas, a una simplificación matemática para obtener este clasificador.

La pieza faltante del modelo está centrada en el concepto de sinapsis, del cual no hemos hecho mención. El concepto de sinapsis explica la fortaleza de la conexión entre neuronas en el cerebro y, de acuerdo a esta fortaleza, las neuronas son capaces de excitar o inhibir otras neuronas. La sinapsis se vuelve más fuerte mediante procesos de aprendizaje. Entonces, ¿cómo se puede modelar este proceso de *aprendizaje* en nuestro sistema, emulando lo que ocurre en sistemas biológicos reales?. Donald Hebb [14] entra en escena, con su conjetura homónima: aquellas conexiones (sinapsis) en las cuales las neuronas intervinientes poseen estados de actividad correlacionados a lo largo del tiempo, son reforzadas y este refuerzo depende de la intensidad y período de tiempo en que esto ha ocurrido. Este planteo es conocido por su famosa frase:



Figura 7.5: Donald Hebb (1904-1985), psicólogo canadiense conocido por la conjetura de Hebb sobre el aprendizaje sináptico.

"Neurons that fire together, wire together"

Esta conjetura permitió dar explicación a diferentes fenómenos observados en psicología conductual. Por ejemplo, el aprendizaje condicionado de Pavlov [68], donde un perro aprende a salivar cuando se le presenta un estímulo (la campana) en conjunto con un alimento (el alimento).

Guiándonos por la conjetura de Hebb, podemos establecer que cada vez que la neurona recibe un nuevo estímulo, es posible aplicar una regla de aprendizaje que permita variar cada uno de los pesos sinápticos. Dicha variación se hará de acuerdo a la información provista por el estímulo.

$$w_i^{nuevo} = w_i^{viejo} + \Delta w_i \tag{7.13}$$

donde el valor Δw_i dependerá de la activación de la entrada y también de la salida, a la vez.

$$\Delta w_i = 2\eta x^\mu \zeta^\mu \tag{7.14}$$

con η actuando como una constante de proporcionalidad que regula el aprendizaje. Esta constante es conocida como *tasa de aprendizaje* y es un hiperparámetro del algoritmo. Los hiperparámetros son valores que se pueden especificar en el algoritmo y que afectan el proceso de aprendizaje, pero no son actualizados por el propio proceso de aprendizaje (parámetros libres).

Con esto se puede derivar una regla simple de actualización. Cuando se presenta la entrada x^μ y el estado de activación O^μ coincide con la salida deseada ζ^μ no hay que hacer nada. Pero, si no coincide, hay que actualizar w_i. Esto equivale a:

$$\Delta w_i = \begin{cases} 2\eta x^\mu \zeta^\mu & \text{si } O^\mu \neq \zeta^\mu, \\ 0 & \text{otro caso} \end{cases} \tag{7.15}$$

Asumiendo que O^μ y ζ^μ toman valores 1 y -1, la función por partes se puede escribir como:

$$\Delta w_i = \eta(\zeta^\mu - O^\mu)x^\mu \tag{7.16}$$

Esta formulación permite una interpretación desde la perspectiva de la representación geométrica del problema de separabilidad lineal. Cuando $\zeta^\mu > O^\mu$, la ecuación 7.16 representa mover un poco al vector de pesos hacia el patrón de entrada x^μ. Con esto, la próxima vez que calculamos el producto interno $\sum_{i=1}^{N} w_i x_i^\mu$, será mayor y más probable que ζ^μ y O^μ se parezcan.

Desde el punto de vista geométrico, la regla de actualización hebbiana está alterando el plano de separación para dividir de manera adecuada a los elementos en sus clases. Esta conexión que suena trivial, es muy potente, porque la base de la regla proviene de un modelo mecanístico experimental del comportamiento de la estructura sináptica de las neuronas.

7.2.2 Algoritmo del Perceptrón Simple

Señoras, Señores, agárrense de sus butacas, afirmen el mate y el termo, porque estamos en posición de desplegar el algoritmo del Perceptrón simple.

Definición 7.2.1 — Época. Presentación completa del conjunto de datos de entrenamiento durante el proceso de aprendizaje, dado por la ecuación 7.16.

Definición 7.2.2 — Hiperparámetros. Son aquellos valores que se pueden especificar en el algoritmo y que afectan el proceso de aprendizaje, quedando fijos durante la duración del mismo. Encontrar la correcta configuración de hiperparámetros es una tarea crucial para el éxito del algoritmo pero NO son actualizados por el propio proceso de aprendizaje, a diferencia de los parámetros libres.

Definición 7.2.3 — Pesos sinápticos. Son los elementos del vector $\mathbf{w}$ que determinan el hiperplano de separación. De acuerdo al modelo de neurona, representan la fuerza de conexión sináptica. Estos son los parámetros libres a ajustar mediante el aprendizaje (incluyendo el umbral, también conocido como bias).

Definición 7.2.4 — Conjunto de entrenamiento. El conjunto de entrenamiento $x[\cdot]$, donde cada entrada tiene dimensión N. Es posible añadir una coordenada extra (valor constante para todos los patrones) que sirva para el umbral. Hay p entradas.

Definición 7.2.5 — Salidas esperadas. Las salidas esperadas quedan plasmadas en $\zeta[\cdot]$.

Definición 7.2.6 — Función de activación. La función de activación determina la salida de la neurona y restringe los valores que puede tomar la misma. En el caso del perceptrón simple, dicha función está dada por $sign(\cdot)$.

Algoritmo 5 Algoritmo del Perceptrón Simple

Inicializar $epoch = 0$
Inicializar $w = zeros(N, 1)$
Inicializar $error = 1$
Inicializar $error_{min} = \infty$
Especificar hiperparámetro MAX_EPOCHS, η.
while $error > EPSILON \wedge epoch < MAX_EPOCHS$ **do**
    Elegir un patrón x^μ al azar entre 1 y p
    Calcular la excitación $h = x^\mu \cdot w$
    Calcular la activación $O^\mu = sign(h)$
    Calcular $\Delta w = \eta * (\zeta^\mu - O^\mu) * x^\mu$
    Actualizar $w = w + \Delta w$
    Calcular el error $error = \sum_{\mu=1}^{p} (\zeta^\mu - O^\mu)^2$
    if $error < error_{min}$ **then**
        $error_{min} = error$
        $w_{min} = w$
    end if
    Incrementar $epoch$
end while
Retornar w obtenido para el dataset X.

Definición 7.2.7 — Inicialización de los pesos sinápticos. El esquema de inicialización de los pesos es un hiperparámetro más a especificar. Una estrategia para el perceptrón simple es inicializarlos de manera aleatoria con $w = rand(N, 1) * 2 - 1$. La elección de inicialización puede ser modificada según las necesidades del problema a resolver, pero no se ahondará en este tema en el presente documento. Se debe tener en cuenta que, en la medida en que los algoritmos basados en redes neuronales sean más complejos, esta estrategia puede no ser la más adecuada.

7.2.3 Perceptrón Lineal

Como fue mencionado previamente, la función de activación del perceptrón simple restringe los valores que puede tomar la neurona con lo cual, restringe la clase de problemas que puede resolver o modelar. Si se reemplaza la función de activación escalón por la función identidad, el Perceptrón se convierte en lineal, donde la salida sólo depende del producto interno entre los patrones de entrada x y los pesos w.

$$O^\mu = \sum_{i=1}^{N} w_i x_i^\mu \tag{7.17}$$

Los valores son continuos, en el espacio de los reales, y no se encuentran umbralizados entre $\{-1, 1\}$ o $\{0, 1\}$, como ocurría con la función de activación escalón. De este nuevo planteo, se puede definir una función de costo, derivable, que adoptará un mínimo absoluto cuando el valor del vector de pesos w permita que la salida $O^\mu = \zeta^\mu$, es decir, que genere la salida exacta para todos los patrones del conjunto de entrenamiento.

$$E(w) = \frac{1}{2}\sum_{\mu=1}^{p}(\zeta^{\mu} - O^{\mu})^2$$
$$= \frac{1}{2}\sum_{\mu=1}^{p}(\zeta^{\mu} - \sum_{i=1}^{N} w_i x_i^{\mu})^2$$

La función de costo planteada está basada en la suma de los errores al cuadrado (SSE, Sum of Squared Errors) y tiene un factor arbitrario de $\frac{1}{2}$ que utilizaremos más adelante.

Es importante notar que $E(\cdot)$ depende de los pesos w_i. Entonces, si se logra mover w en la dirección correcta podríamos acercarnos al valor mínimo de $E(\cdot)$ que, por ser una función de error, idealmente debería ser cero. Es importante tener en cuenta que, para la mayoría de los problemas reales, este valor no será cero ya que es poco probable que los datos modelen una relación perfectamente lineal.

Una posibilidad para encontrar el valor de w que minimiza $E(\cdot)$, si retornamos a nuestra caja de herramientas de optimización, es usar el gradiente, ya que representa la dirección de mayor crecimiento de la función que estamos buscando minimizar. Con esto en mente, moveremos a los pesos sinápticos en la dirección **opuesta a la de mayor crecimiento**:

$$\Delta w_i = -\eta \frac{\partial E}{\partial w_i} \tag{7.18}$$

para lo cual calculamos la derivada parcial. Aplicando la regla de la cadena:

$$\frac{\partial E}{\partial w_i} = \frac{\partial}{\partial w_i}\left(\frac{1}{2}\sum_{\mu=1}^{p}(\zeta^{\mu} - O^{\mu}(w_i))^2\right)$$
$$= \sum_{\mu=1}^{p}(\zeta^{\mu} - O^{\mu}(w_i)) \cdot (-1) \cdot \frac{\partial O^{\mu}}{\partial w_i}$$
$$= \sum_{\mu=1}^{p}(\zeta^{\mu} - \sum_{i=1}^{N} w_i x_i^{\mu}) x_i^{\mu} \tag{7.19}$$

A continuación, reemplazamos en 8.6:

$$\Delta w_i = -\eta \frac{\partial E}{\partial w_i}$$
$$= \eta \sum_{\mu=1}^{p}(\zeta^{\mu} - O^{\mu}) x_i^{\mu}$$

Con esto se modifica cada peso puntual evaluando todos los patrones. Si se quiere hacer el ajuste evaluando sólo un patrón nos queda

$$\Delta w_i = \eta(\zeta^{\mu} - O^{\mu}) x_i^{\mu} \tag{7.20}$$

Notemos que ésta es exactamente la misma regla de actualización del Perceptrón con la función de activación escalón. **Algo que planteamos inspirados en un modelo biológico experimental coincide**

con el planteo, desde el punto de vista matemático, de minimizar el valor de una función A esta regla llamada **delta rule** también se la conoce como la regla de Widrow-Hoff (1960), y estos autores la bautizaron como ADALINE, o Adapter Linear Element. [2].

Aprendizaje en el Perceptrón lineal

El Perceptrón simple escalón resuelve problemas linealmente separables. Por otro lado, el Perceptrón lineal simple resuelve sistemas lineales independientes[3]. Es decir encuentra una aproximación del w que resuelve el siguiente sistema:

$$\mathbf{xw} = \zeta,$$

donde $\mathbf{x}$ es la matriz de datos de entrada, w es el vector de pesos sinápticos y ζ es el vector de salidas deseadas. Esta fórmula puede expandirse de la siguiente forma:

$$\begin{aligned} w_1 x_1^1 + w_2 x_2^1 + \ldots + w_N x_N^1 &= \zeta^1 \\ w_1 x_1^\mu + w_2 x_2^\mu + \ldots + w_N x_N^\mu &= \zeta^\mu \\ w_1 x_1^p + w_2 x_2^p + \ldots + w_N x_N^p &= \zeta^p \end{aligned}$$

La convergencia de este aprendizaje depende fuertemente del valor de η, recordando que hay que actualizar $w_i^{nuevo} = w_i^{viejo} + \Delta w_i$ en base a $\Delta w_i = \eta(\zeta^\mu - O^\mu)x^\mu$ por lo que

- Si η no es lo suficientemente pequeña, el método puede diverger.
- Si η es demasiado pequeña puede demorar la convergencia.

7.2.4 Aprendizaje en el Perceptrón Simple No Lineal

La idea del aprendizaje basado en la optimización del gradiente, puede extenderse fácilmente a una función de activación no lineal. El estado de activación del Perceptrón es

$$O^\mu = g(\sum_{i=1}^{N} w_i x_i^\mu) \tag{7.21}$$

donde $g(\cdot)$ puede ser una función sigmoidea tal como $g(x) = tanh(\beta x)$ o $g(x) = \frac{1}{1+e^{-2\beta x}}$. Una lista más exhaustiva de variantes de funciones de activación puede encontrarse en 23.2. Por otro lado, en la tabla 7.1 se encuentran definidas las fórmulas de las funciones de activación tangente hiperbólica y logística.

nombre	$g(h)$	$g'(h)$
$tanh(\beta h)$	$tanh(\beta h)$	$\beta(1-g(h)^2)$
$logistic(h)$	$\frac{1}{1+e^{-2\beta h}}$	$2\,\beta g(h)(1-g(h))$

Cuadro 7.1: Funciones de activación tangente hiperbólica y logística. Se encuentra definida la fórmula y sus derivadas

[2]Ted Hoff fue estudiante de Bernard Widrow y más adelante, en 1968, se convirtiría en el co-inventor del predecesor del microprocesador (un conjunto de 12 circuitos diferentes en un sólo chip que cambiaba su funcionalidad de acuerdo al requerimiento) [69]

[3]La demostración es trivial y queda como ejercicio para el lector.

La función de costo que modela la regla de actualización de los pesos sinápticos en este escenario queda definida exactamente igual que con el Perceptrón lineal

$$E(w) = \frac{1}{2}\sum_{\mu=1}^{p}(\zeta^{\mu} - O^{\mu})^2$$
$$= \frac{1}{2}\sum_{\mu=1}^{p}(\zeta^{\mu} - g(\sum_{i=1}^{N} w_i x_i^{\mu}))^2.$$

$$\Delta w_i = -\eta \frac{\partial E}{\partial w_i}$$

pero usamos **la regla de la cadena**[4] para propagar la derivada también de g dentro del cálculo

$$\Delta w_i = \eta \sum_{\mu=1}^{p}(\zeta^{\mu} - g(h^{\mu}))g'(h^{\mu})x_i^{\mu}$$

donde usamos que $h^{\mu} = \sum_{i=1}^{N} w_i x_i^{\mu}$.

Nuevamente, considerando tomar sólo una única entrada en particular.

$$\Delta w_i = \eta(\zeta^{\mu} - O^{\mu})g'(h^{\mu})x_i^{\mu} \tag{7.22}$$

La ventaja de la función logística o la tangente hiperbólica es que las derivadas se pueden expresar como funciones de ellas mismas.

Hemos demostrado que el perceptrón simple es un modelo simplificado de la neurona biológica, sobre el cual se propone un algoritmo de aprendizaje que permite encontrar los pesos w que minimizan una función de costo $E(\cdot)$. Con dicha estrategia se pueden resolver problemas de clasificación, de regresión lineal y no lineal, cambiando la función de activación y ajustando los cálculos de la regla de actualización de acuerdo a dicho cambio.

Si bien esta propuesta aparentaba enorme potencial para la época, se publicó en 1969 un influyente artículo de Minsky y Papert que planteó un problema clave: esta mecánica (que auguraba decodificar nada más y nada menos que el cerebro humano) fallaba para resolver una función booleana trivial como el XOR. Este suceso dio pie al inicio del primer **invierno** de la inteligencia artificial.

[4] Esto veremos que es fundamental para el capítulo siguiente donde expandimos esto a esquemas de muchas capas.

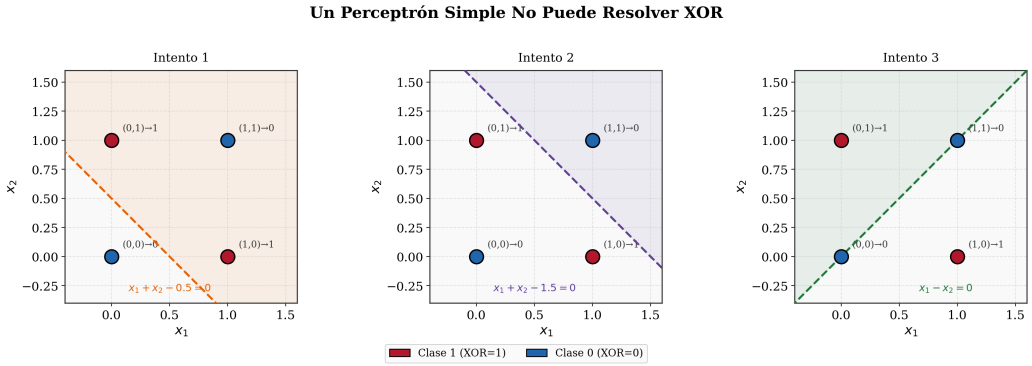


Figura 7.6: Múltiples intentos fallidos para encontrar un hiperplano que permita separar las clases del problema XOR.



8. La era del Coneccionismo

8.1 Perceptrón Multicapa

El problema del XOR del paper de Minsky y Papert [31] tenía una resolución, planteada directa y explícitamente en ese mismo trabajo: la utilización de una estructura *jerárquica aglomerativa*, lo cual implica la distribución de múltiples neuronas en capas sucesivas. Esta idea tiene un correlato en neurociencia, donde se verifica que la misma estructura se encuentra presente en los sistemas nerviosos de muchos seres vivos [70].

¿A qué nos referemos cuando utilizamos la expresión estructura *jerárquica aglomerativa*? En el capítulo Perceptrón, se introdujo el modelo de neurona junto a las fórmulas y procesos que permiten hacer uso de dicha estructura para clasificación y regresión. Lo que se propone en este capítulo es utilizar las salidas de perceptrones individuales como entradas de otros perceptrones, ubicados en una capa siguiente.

Desde la perspectiva de los datos, se puede pensar que cada capa intermedia, a la cual denominamos capa oculta, está creando nuevas características para los datos originales, es decir, modifica su representación. El objetivo es que las características aprendidas en cada capa sean útiles para capas sucesivas. Este esquema jerárquico soluciona la función booleana XOR, porque permite agregar un plano de separación adicional, como puede observarse en la figura 8.1. El modelo correspondiente que permite resolver el problema se muestra en la figura 8.2.

A la estructura jerárquica aglomerativa, aquella que tiene al menos una capa oculta, se le llama *Perceptrón multicapa, MLP o Multilayered Perceptron* y se corresponde con la arquitectura base de redes neuronales artificiales. Se puede observar un ejemplo de perceptrón multicapa en 8.3.

8.1.1 La promesa detrás de las redes neuronales

¿Por qué las redes neuronales se volvieron tan famosas? Si bien no hemos hablado aún del término *Deep Learning*, lo cual haremos más adelante en el libro, la arquitectura base y sus variantes han

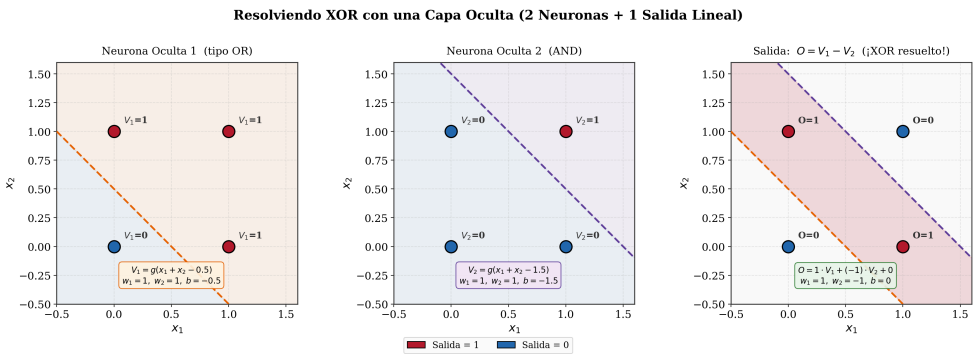


Figura 8.1: Solución al problema XOR utilizando un Perceptrón multicapa. A la izquierda se muestra el hiperplano de separación para la neurona oculta V_1, y en la imagen central se muestra el hiperplano de separación para la segunda neurona oculta V_2. A la derecha se muestra el criterio de clasificación de los elementos de entrada, dado por la salida de la neurona O. La función de activación escalón es la que se utiliza en este escenario, denotada por θ.

conquistado exitosamente el terreno para resolver gran variedad de problemas: clasificación de objetos, reconocimiento facial, análisis de series de tiempo, etc.

Como hemos visto al comenzar el capítulo, el perceptrón multicapa se plantea como una arquitectura flexible, pero ¿por qué? ¿Qué garantía tenemos de que esta arquitectura es una buena estrategia para aproximar los problemas que queremos resolver? No perdamos de vista nuestro norte: encontrar una aproximación a una función f determinada. Por ejemplo, quiero una función que prediga la probabilidad de que una imagen pertenezca a una determinada clase y, lo cual implica encontrar $p(y|x)$, donde x pueden ser imágenes de objetos de resolución arbitraria e y es un vector del tamaño de los objetos que quiero clasificar.

Cada área de inteligencia artificial y, en particular, de aprendizaje automático (sección 10.1) presenta diferencias en los algoritmos propuestos, pero existe una estructura común al momento de plantear un problema: necesitamos un conjunto de datos, un modelo, una función de costo y un proceso de optimización.

El conjunto de datos puede interpretarse como una muestra aleatoria de un proceso de generación de datos, al cual no tenemos acceso y, es de dicha fuente de la cual queremos aprender nuestra representación o función. El modelo, es aquella herramienta que seleccionamos para aproximar la función y luego utilizar para analizar nuevas muestras. En este caso, el modelo es un perceptrón multicapa. La función de costo es la forma de indicar al modelo que se está equivocando. Por ejemplo, en el capítulo del Perceptrón utilizamos la diferencia entre salida esperada y salida del perceptrón elevada al cuadrado (MSE). Por último, el proceso de optimización es aquél que permite ajustar los parámetros libres del modelo para hallar una mejor aproximación. Tanto en el capítulo anterior como en este capítulo nos enfocamos en métodos basados en gradiente descendente, que fueron introducidos en la sección 6.4.2.

Si bien no nos adentraremos en los detalles matemáticos, haremos nuestro mejor esfuerzo para explicar en palabras el impacto que presenta la interacción de tres conceptos teóricos: el teorema de aproximación universal, el principio de máxima verosimilitud y la divergencia de Kullback-Leibler. Lo que sigue a continuación no es esencial para entender el perceptrón multicapa y su implementación, pero da un fundamento teórico sobre la "promesa" de las redes neuronales.

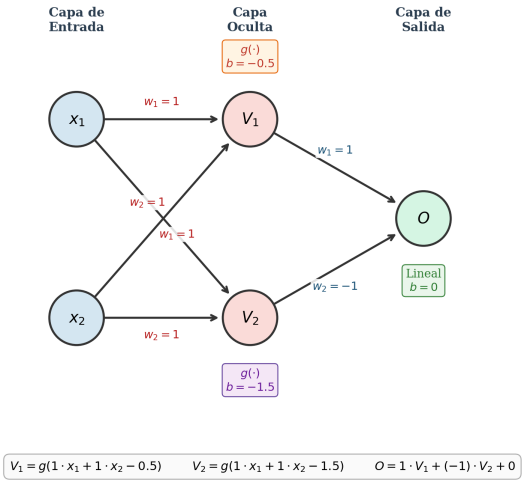


Figura 8.2: Arquitectura de un Perceptrón multicapa para resolver el problema XOR. El esquema corresponde a la red cuya arquitectura da el comportamiento mostrado en la figura 8.1. La función de activación utilizada es la función escalón, denotada por θ.

El teorema de aproximación universal, propuesto en 1989 [71] y extendido en 1991 [72], plantea que dado un perceptrón multicapa con suficientes neuronas ocultas y funciones de activación no lineales, es posible aproximar cualquier función continua. Nótese que este enunciado nada dice sobre la cantidad de neuronas que debe ser utilizada ni sobre la factibilidad de aproximar la función deseada, problemas importantes que siguen investigándose hoy en día pero sobre los cuales se ha progresado enormemente. Lo que debemos capturar de este teorema es el poder de expresividad que da a la arquitectura que hemos elegido: si asumimos que aquello que queremos encontrar es una función continua (o lo es aproximadamente), un perceptrón multicapa es un algoritmo con buenas chances de poder representarlo, al menos desde el punto de vista teórico.

Por otro lado, el principio de máxima verosimilitud, publicado en 1922 por Fisher [73], nos dice que, si asumimos que existe un único valor de parámetros que aproxima la verdadera distribución de los datos, podemos derivar una función de costo utilizando la fórmula de verosimilitud (en su versión logarítmica) que permite aproximar este conjunto de parámetros. Este principio es deseable por sus propiedades de consistencia y eficiencia [6]. Consistencia porque al tener infinitos datos, la solución encontrada se corresponde con el verdadero valor de los parámetros, y eficiencia porque requiere menos muestras que otros principios para obtener un error de generalización apropiado. La conclusión importante que nos da el principio es que tenemos que basar la función de costo en la distribución de probabilidad a la cual corresponden las salidas esperadas, dadas por el conjunto de datos.

Por último, para aquellos que se adentren en la matemática del principio de máxima verosimilitud, podrán comprobar que la función de costo derivada del mismo, la entropía cruzada, es equivalente a la divergencia de Kullback-Leibler [74] (también llamada divergencia KL) de dos distribuciones P y Q. Lo importante es que la divergencia KL nos da una métrica de "similitud" entre dos distribuciones. En este caso P y Q se corresponden a la distribución aproximada por nuestro modelo (Q) y a la distribución de los datos en nuestro conjunto (P). No es una medida de distancia, ya que no es simétrica, pero nos da una interpretación importante sobre las distribuciones que modelamos al modificar los parámetros

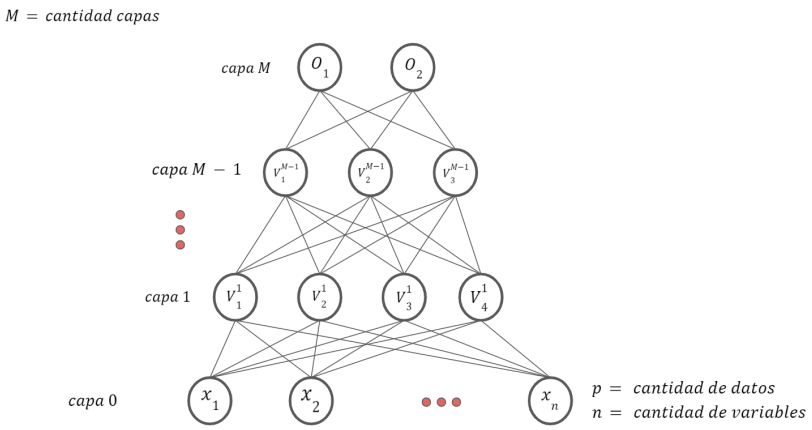


Figura 8.3: Perceptrón multicapa con estructura jerárquica aglomerativa arbitraria. La capa 0 corresponde a la capa de entrada, la capa M corresponde a la capa de salida, y las capas intermedias $1, 2, ..., m-1$ corresponden a las capas ocultas. La letra M indica el número de capas. Cada capa puede tener un número arbitrario de neuronas. Los datos de entrada se representan como un vector x de dimensión n (cantidad de variables o características). El tamaño del conjunto de entrenamiento es p. Para representar las neuronas, se utiliza la letra V para las neuronas de la capa oculta y O para las neuronas de la capa de salida. Para simplificar el esquema, no se incluye el bias, en el cual se hará hincapié más adelante.

libres. Nos dice que, a medida que minimizamos la función de costo, estamos acercando la distribución aprendida a la distribución que queremos hallar. Nótese la diferencia entre el principio de máxima verosimilitud y la divergencia KL. En el caso anterior, hablábamos sobre la garantía de encontrar un conjunto de parámetros pero no teníamos conocimiento sobre cómo interpretar la distribución que la aproximación estaba modelando. Analizar la situación desde la perspectiva de la divergencia de Kullback-Leibler nos permite entender qué ocurre con la probabilidad que dicho conjunto de parámetros está modelando, lo cual es de máximo interés para nosotros.

Estos tres conceptos explican, de manera teórica, por qué esta herramienta es tan poderosa. Tenemos un principio que nos dice qué optimizar (el de máxima verosimilitud), una interpretación que nos indica qué es lo que ocurre mientras estamos optimizando y un teorema que provee una familia de modelos lo suficientemente expresiva para una amplia variedad de problemas. Es importante destacar que cada planteo tiene sus limitaciones y suposiciones matemáticas en las cuales no nos hemos adentrado, así como la dificultad de llevar estas ideas al ámbito práctico computacional.

8.1.2 Propagación hacia adelante

¿Cómo es entonces la evolución de las entradas hacia adelante en nuestra red? Dicha evolución se denomina propagación hacia adelante, comúnmente conocida como *forward propagation*. Consideremos la red planteada en la figura **??**. Los pesos sinápticos w_{jk} corresponden a los pesos de la capa oculta, en tanto que W_{ij} corresponde a los pesos de la capa de salida. Las activaciones de la capa oculta las llamamos V_j.

- j, i son los índices para referenciar las nueronas de la capa oculta, y las neuronas de la salida, respectivamente.
- El subíndice k referencia una característica de la muestra. N es la cantidad de características de las muestras. Aquí se hará un pequeño abuso de notación, ya que tanto k como N serán utilizados

no sólo para referenciar a los datos de entrada, sino también para referenciar a las neuronas de una capas oculta cuando estemos calculando la activación de neuronas en una capa oculta posterior.

- El supraíndice m referencia la capa oculta en la cual se encuentra la neurona o a la cual se encuentra conectado un peso sináptico. La cantidad de capas en una red neuronal se denota por M.
- El supraíndice μ referencia una muestra particular, con p la cantidad de muestras.

Al presentar la muestra μ a la red, en la primera capa oculta se calcula la excitación de la neurona j como

$$h_j^{1\mu} = \sum_{k=1}^{N} w_{jk}^1 x_k^\mu \tag{8.1}$$

lo que permite generar una activación en la capa oculta, dada por la función de activación g, como

$$V_j^{1\mu} = g(h_j^{1\mu}) = g(\sum_{k=1}^{N} w_{jk}^1 x_k^\mu) \tag{8.2}$$

Para las capas ocultas posteriores, tomamos el índice k como aquel que referencia la característica de la neurona de la capa oculta anterior, y calculamos la activación de la neurona j como

$$V_j^{m\mu} = g(h_j^{m\mu}) = g(\sum_{k=1}^{N} w_{jk}^m V_k^{m-1\mu}) \tag{8.3}$$

Finalmente, podemos calcular la salida de una neurona de la capa de salida, como

$$O_i^\mu = g(\sum_{j=1}^{N} W_{ij} V_j^{(M-1)\mu}) \tag{8.4}$$

Esto determina todas las salidas de la red hacia adelante. Dependiendo de la cantidad de neuronas en la capa de salida, se tendrá un escalar o un vector como resultado final. Esto se corresponde con una decisión de diseño, fuertemente relacionada con el problema a resolver. El proceso de propagación es simple de entender una vez que se entiende el caso del Perceptrón, ya que las neuronas en capas ocultas y la capa de salida pueden observarse individualmente como perceptrones simples, ya sea del tipo lineal o no lineal. Nótese que no hemos forzado una función de activación, sino que la misma es determinada por quien implementa la arquitectura.

8.1.3 Propagación hacia atrás

En el Perceptrón el aprendizaje está dado por el proceso de actualizar los pesos sinápticos cuando la salida es distinta al valor esperado. En el caso del Perceptrón multicapa, podemos tomar un enfoque similar con respecto a las neuronas en la capa de salida, ya sean un escalar o un vector. Esto permite tener la información necesaria para actualizar los pesos conectados a la capa de salida: de una proporción del error se corrigen directamente los pesos de las conexiones que unen directamente las entradas con las salidas.

Ahora, ¿cómo proceder en el caso de una arquitectura con capas ocultas? ¿Cómo llevamos para atrás esa señal de error de la cual disponemos? ¿Qué estado de activación deseado tienen que tener las neuronas de las capas intermedias para poder establecer un error y, de acuerdo al mismo, corregir los pesos de las conexiones que unen las entradas con esas neuronas ocultas intermedias? La respuesta se encuentra en la rama matemática del cálculo diferencial: podemos utilizar la regla de la cadena para calcular la derivada parcial de la función de costo con respecto a los pesos de las neuronas de la capa oculta.

Tomemos como proceso de optimización el gradiente descendente, el cual fue utilizado para el entrenamiento del Perceptrón en el capítulo 7. Procedemos a adaptar la actualización de los pesos para el perceptrón multicapa:

$$\Delta W_{ij} = -\eta \frac{\partial E}{\partial W_{ij}} \tag{8.5}$$

$$\Delta w^m_{jk} = -\eta \frac{\partial E}{\partial w^m_{jk}} \tag{8.6}$$

Ambas ecuaciones pueden calcularse, como mencionamos previamente, utilizando la regla de la cadena de las derivadas parciales. Supongamos que tenemos una arquitectura como la que puede observarse en la figura **??**. Podemos ver algunos ejemplos de la derivada parcial de la función de costo con respecto a los distintos pesos sinápticos

$$\frac{\partial E}{\partial W_{12}} = \frac{\partial E}{\partial O_1}\frac{\partial O_1}{\partial W_{12}}$$
$$\frac{\partial E}{\partial w^1_{11}} = \sum_{i=1}^{2} \frac{\partial E}{\partial O_i}\frac{\partial O_i}{\partial V^1_1}\frac{\partial V^1_1}{\partial w^1_{11}}$$
$$\frac{\partial E}{\partial w^1_{21}} = \sum_{i=1}^{2} \frac{\partial E}{\partial O_i}\frac{\partial O_i}{\partial V^1_2}\frac{\partial V^1_2}{\partial w^1_{21}}$$

Es importante observar que los pesos de capas ocultas toman la sumatoria sobre todas las neuronas de salida. Esto sucede porque podemos calcular el error del valor esperado con respecto a cada neurona de salida, la cual contribuye en cierta proporción a la activación de la neurona de la capa oculta en cuestión.

En las pocas ecuaciones que hemos desarrollado, es posible observar que muchos términos se repiten: la derivada del error con respecto a las neuronas en la capa de salida aparece en todos las las ecuaciones. Cuantas más capas ocultas y neuronas en ellas, más calculos repetidos tendremos. Recordemos que el desarrollo de redes neuronales tomó lugar entre 1950 y 1980, donde el hardware era limitado para operaciones computacionalmente intensivas.

Fue en dicho contexto que en 1986, Rumerlhart, Hinton y Williams presentan el algoritmo de propagación hacia atrás o *Backpropagation*, en el contexto de su trabajo de Procesamiento Paralelo Distributivo [30], propuesta para solucionar las dificultades computacionales. Es imperativo destacar la importancia de este algoritmo en el campo de la inteligencia artificial en su conjunto. Inclusive al día de hoy, es la piedra fundamental para el cómputo de gradientes de las distintas variantes de redes

neuronales y fue uno de los elementos cruciales para llevar el campo de redes neuronales a la era del *Deep Learning*, como veremos más adelante.

En pocas palabras, ¿qué hace el algoritmo de propagación hacia atrás? Realiza un cómputo eficiente de la derivada parcial para los pesos sinápticos en capas ocultas y de salida. Esto implica calcular una magnitud δ_j^m, asociada a cada neurona. Se puede interpretar la misma como la señal del error en dicha neurona. Este paso intermedio permite que la cantidad de cálculos crezca linealmente con respecto a la cantidad de parámetros que debemos ajustar en la red neuronal.

Procederemos a continuación a realizar el desarrollo de las fórmulas que nos den una regla de actualización para pesos en la capa de salida y pesos en la capa oculta. Quienes deseen saltear esta sección (aunque recomendamos no hacerlo), pueden ver un resumen del resultado final en la figura 8.4. Para todo el resto, es momento de agarrar el mate y armarse de un poco de paciencia. Procedamos.

Tomaremos como función de costo la diferencia entre salida esperada y salida de la red neuronal elevada al cuadrado, como se muestra a continuación. Simplificaremos todos los cálculos para una muestra particular μ. Recordemos, del capítulo 7, que ζ^μ es la salida esperada para la muestra μ

$$E(w) = \frac{1}{2}\sum_{\mu,i}(\zeta_i^\mu - O_i^\mu)^2 \tag{8.7}$$

$$O_i = g(h_i^\mu) \tag{8.8}$$

$$h_i^\mu = \sum_j W_{ij} V_j^{(M-1)\mu} \tag{8.9}$$

Como el nombre del algoritmo lo indica, iremos de atrás hacia adelante, comenzando por calcular la expresión de actualización para los pesos que conectan con las neuronas en la capa de salida, W_{ij}. Para ello necesitamos la derivada de la función de costo con respecto a dichos pesos. Una pequeña aclaración sobre la función de costo: el término $\frac{1}{2}$ tiene como propósito la simplificación de la expresión más adelante.

$$\begin{aligned}
\Delta W_{ij} &= -\eta \frac{\partial E}{\partial W_{ij}} \\
&= -\eta \frac{\partial E}{\partial O_i}\frac{\partial O_i}{\partial h_i^\mu}\frac{\partial h_i^\mu}{\partial W_{ij}} \\
&= -\eta \frac{\partial (\frac{1}{2}\sum_{\mu,i}(\zeta_i^\mu - O_i^\mu)^2)}{\partial O_i}\frac{\partial g(h_i^\mu)}{\partial h_i^\mu}\frac{\partial \sum_j W_{ij} V_j^{(M-1)\mu}}{\partial W_{ij}} \\
&= -\eta(\zeta_i^\mu - O_i^\mu)(-1)g'(h_i^\mu)V_j^{(M-1)\mu} \\
&= \eta(\zeta_i^\mu - O_i^\mu)g'(h_i^\mu)V_j^{(M-1)\mu} \\
&= \eta\,\delta_i^\mu V_j^{(M-1)\mu}
\end{aligned}$$

donde definimos la siguiente mangnitud:

$$\delta_i^\mu = (\zeta_i^\mu - O_i^\mu) g'(h_i^\mu) \tag{8.10}$$

Ahora, vamos a calcular la expresión de actualización para los pesos que conectan con las neuronas en la capa oculta, w_{jk}^m.

$$\begin{aligned}
\Delta w_{jk}^m &= -\eta \frac{\partial E}{\partial w_{jk}^m} \\
&= -\eta \frac{\partial E}{\partial O^\mu} \frac{\partial O^\mu}{\partial h^\mu} \frac{\partial h^\mu}{\partial V_j^{m\mu}} \frac{\partial V_j^{m\mu}}{\partial h_j^{m\mu}} \frac{\partial h_j^{m\mu}}{\partial w_{jk}^m} \\
&= -\eta \sum_i (-1(\zeta_i^\mu - O_i^\mu) g'(h_i^\mu) W_{ij}) \frac{\partial V_j^{m\mu}}{\partial h_j^{m\mu}} \frac{\partial h_j^{m\mu}}{\partial w_{jk}^m} \\
&= \eta \sum_i (\zeta_i^\mu - O_i^\mu g'(h_i^\mu) W_{ij}) \frac{\partial V_j^{m\mu}}{\partial h_j^{m\mu}} \frac{\partial h_j^{m\mu}}{\partial w_{jk}^m} \\
&= \eta \sum_i \delta_i^\mu W_{ij} \frac{\partial V_j^{m\mu}}{\partial h_j^{m\mu}} \frac{\partial h_j^{m\mu}}{\partial w_{jk}^m} \\
&= \eta \sum_i \delta_i^\mu W_{ij} g'(h_j^{m\mu}) V_k^{(m-1)\mu} \\
&= \eta \delta_j^{m\mu} V_k^{(m-1)\mu}
\end{aligned}$$

donde definimos la siguiente mangnitud:

$$\delta_j^{m\mu} = \sum_i \delta_i^\mu W_{ij} g'(h_j^{m\mu}) \tag{8.11}$$

Hemos llegado a las expresiones que necesitamos para actualizar todos los pesos de una red neuronal. Podemos calcular en una primera instancia las magnitudes δ asociadas a cada neurona y luego, calcular Δ_w. Si bien los cálculos fueron realizados para un dato del conjunto y un peso sináptico particular, es posible expresar las fórmulas de manera vectorial, lo cual es importante al momento de implementar el algoritmo. Por otro lado, modificar la función de costo tiene como consecuencia que las expresiones aquí presentadas también se modifican. Por último, notar que la expresión es genérica para cualquier función de activación g. Un buen ejercicio para aquellos interesados implica tomar otra función de costo (por ejemplo, la entropía cruzada) y realizar las derivaciones necesarias.

8.2 Algoritmo del Perceptrón Multicapa

A conitnuación se presenta un algoritmo base que permite entrenar una red neuronal utilizado el algoritmo de *Backpropagation*. Este algoritmo puede ser modificado para incluir diferentes variantes de inicialización de parámetros libres, de mecanismos de optimización y de modos de entrenamiento: incremental, 8.2.1, por lotes, 8.2.2, etc.

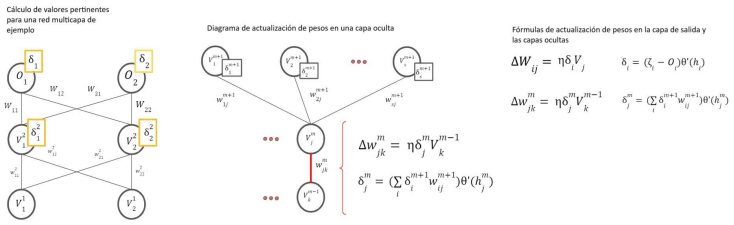


Figura 8.4: Esquema de retropropagación del error en un Perceptrón multicapa.

Algoritmo 6 Algoritmo del Perceptrón Multicapa

Inicializar $epoch = 0$
Inicializar $w = initializeWeights()$
Inicializar $error = 1$
Inicializar $error_{min} = \infty$
Especificar hiperparámetro MAX_EPOCHS, η.
while $error > EPSILON \wedge epoch < MAX_EPOCHS$ **do**
    Elegir un patrón x^{μ} al azar entre 1 y p
    $O^{\mu} = forwardPropagation(\mu)$
    $\Delta w = backPropagation()$
    $w = w + \Delta w$
    $error = computeError(data, w)$;
    if $error < error_{min}$ **then**
        $error_{min} = error$
        $w_{min} = w$
    end if
end while
Retornar w obtenido para el dataset X.

Con la definición de las dos funciones para calcular la propagación adelante y hacia atrás.

Algoritmo 7 forwardPropagation()

$V_k^{0\mu} = x_k^{\mu}$
$V_j^{m\mu} = g(h_j^{m\mu}) = g(\sum_k w_{jk}^{m\mu} V_k^{(m-1)\mu})$
Retorna $O^{\mu} = V^{M\mu}$ cuando M es la última capa de la red.

Algoritmo 8 backPropagation()

$\delta_i^{M\mu} = (\zeta_i^{\mu} - V_i^{M\mu}) g'(h_i^{M\mu})$ para la última capa.
$\delta_j^{m\mu} = \sum_i \delta_i^{(m+1)\mu} w_{ij}^{m+1} g'(h_j^{m\mu})$ para todo m entre 1 y $M-1$.
Retorna $\Delta w_{jk}^{m\mu} = \eta \delta_j^{m\mu} V_k^{(m-1)\mu}$ para todo m entre 1 y M.

El perceptrón multicapa, junto con el algoritmo de *Backpropagation*, plantea un método **bioinspirado** y eficiente de aprendizaje automático, definido en la sección 10.1.1. Permite, de manera supervisada, el ajuste de parámetros libres para optimizar una función de error de manera tal que es posible modelar las salidas esperadas para un conjunto de datos determinado. Esto es muy poderoso e integra multiples disciplinas como ciencias de la computación, matemáticas y neurociencia.

8.2.1 Aprendizaje Incremental

En la forma incremental de entrenamiento, aquella utilizada en el algoritmo base presentado anteriormente, el proceso de propagación hacia adelante y hacia atrás se ejecuta cada vez que una entrada x^{μ} es presentada a la red. Como puede verse en el algoritmo, se propaga dicha entrada hasta obtener los estados de activación de la capa de salida. Luego, se retropropaga el error obteniendo todos los δ correspondientes para cada unidad de cada capa. Con toda esa información se puede disparar una actualización (en paralelo) de todos los pesos. Cuando se completan todos los datos del conjunto de entrenamiento, se ha completado una **época**.

8.2.2 Aprendizaje por lotes

El aprendizaje por lotes o *batches*, implica acumular la información de actualización para todos los elementos del conjunto de datos, antes de realizar actualización de pesos. Cada vez que una entrada es presentada a la red, se propaga dicha entrada hasta obtener los estados de activación de la capa de salida. Luego, se retropropagan el error obteniendo los δ para cada neurona de cada capa pero ahora los Δw se acumulan y se repite este proceso para todas las entradas del conjunto. Al términar la época, los valores acumulados se impactan en los pesos.

Existe una variante intermedia entre el aprendizaje incremental y el aprendizaje por lotes, que es el aprendizaje por mini-lotes. En este caso, se acumula la información de actualización para un subconjunto de datos del conjunto de entrenamiento, antes de realizar actualización de pesos. Este subconjunto se denomina mini-lote o *mini-batch* y su tamaño es un hiperparámetro que debe ser especificado por el usuario.

8.3 Variantes de optimización

El proceso de optimización en la red depende del problema[1] y el esquema planteado en la versión inicial del Perceptrón multicapa puede ser insuficiente para alcanzar convergencia en un tiempo razonable. Para superar aquellas dificultades podemos hacer uso de lo visto en el capítulo 6 ya que es posible expandir los métodos allí planteados con el objetivo específico de la aplicación en redes neuronales.

8.3.1 Momentum

Esta variante a la regla de actualización de los pesos mitiga el inconveniente de la oscilación en torno al mínimo o la divergencia que suele ocurrir cuando W está cerca de un mínimo local o global.

$$\Delta w_{ij}(t+1) = -\eta \frac{\partial E}{\partial w_{ij}} + \alpha \Delta w_{ij}(t) \tag{8.12}$$

[1] Acá todo depende siempre del problema.

con $\alpha = 0.8$ o $\alpha = 0.9$. Si la función de costo $E(\cdot)$ está en una región plana, $\Delta w_{ij}(t)$ será similar a $\frac{\partial E}{\partial w_{ij}}$ y por lo tanto $\Delta w_{ij}(t+1)$ se incrementará en módulo. Por el contrario, si la función $E(\cdot)$ está en un valle que determinaría un comportamiento más oscilatorio, $\Delta w_{ij}(t+1)$ tenderá a compensarse para evitar oscilaciones.

8.3.2 Tasa de aprendizaje adaptativa

En el algoritmo de gradiente descendente, el valor de η determina el paso, el salto entre un valor y el siguiente. No es bueno que este valor sea constante porque la forma de la función de costo $E(\cdot)$ en un momento determinado implica diferentes expectativas sobre los pasos para alcanzar la convergencia. Si la misma decrece de manera consistente (por ejemplo durante k iteraciones seguidas), quizás sería interesante incrementar la tasa de aprendizaje para que el proceso de optimización "vaya más rápido".Por otro lado, si $E(\cdot)$ comienza a incrementarse en un momento del aprendizaje, sería conveniente disminuir η para evitar la divergencia.

$$\Delta\eta = \begin{cases} +a & \text{si } \Delta E < 0 \text{ en forma consistente,} \\ -b\eta & \text{if } \Delta E > 0 \text{ empieza a ser consistente} \\ 0 & \text{en cualquier otro caso} \end{cases} \tag{8.13}$$

También es posible actualizar el η por neurona particular (para los pesos de las conexiones que llegan a esa neurona) o por conexión (un η para cada w_{jk}^m).

8.3.3 RMSProp

Este algoritmo fue propuesto por Hinton en 2012 [75]. Si bien su uso no está tan extendido como el ya popularizado Adam, es importante para entender este último. El objetivo es ajustar las tasas de aprendizaje por cada uno de los pesos, ya que cada dirección puede tener comportamientos diferentes. Realizar este tratamiento individualizado permite acelerar la convergencia. El algoritmo RMSProp define las siguientes ecuaciones 8.14 y 8.15:

$$g_t = \frac{\partial E}{\partial w_{ij}} \tag{8.14}$$

$$S_t = \gamma S_{t-1} + (1-\gamma) g_t^2 \tag{8.15}$$

El término g_t^2 en la ecuación 8.15 corresponde a la operación de cuadrados aplicada miembro a miembro del vector g_t.

$$\Delta w_{ij} = -\frac{\eta}{\sqrt{S_t + \varepsilon}} \times g_t \tag{8.16}$$

El parámetro ε es agregado para evitar dividir por cero. La idea del algoritmo es ajustar el learning rate de acuerdo al root mean square (RMS). Teniendo presente que este es un proceso de optimización, el learning rate es el paso, el salto que hay que dar. En este caso al promediar el learning rate se está tratando de encontrar un tamaño de paso crítico que sirva para todo el proceso y que balancee

performance versus convergencia. Por ejemplo, un learning bajo, se amortiguan mejor los cambios, se reducen las oscilaciones, en tanto que un learning rate alto, aprende más rapido.

El valor del RMS viene dado por el cálculo iterativo del promedio pesado de los g_t anteriores, según

$$\begin{aligned}
S_t &= (1-\gamma)g_t^2 + \gamma S_{t-1} \\
&= (1-\gamma)g_t^2 + \gamma((1-\gamma)g_{t-1}^2 + \gamma S_{t-2}) \\
&= (1-\gamma)g_t^2 + (1-\gamma)\gamma g_{t-1}^2 + \gamma^2 S_{t-2} \\
&= (1-\gamma)g_t^2 + (1-\gamma)\gamma g_{t-1}^2 + \gamma^2((1-\gamma)g_{t-2}^2 + \gamma S_{t-3}) \\
&= (1-\gamma)g_t^2 + (1-\gamma)\gamma g_{t-1}^2 + \gamma^2(1-\gamma)g_{t-2}^2 + \gamma^3 S_{t-3} \\
&= (1-\gamma)(g_t^2 + \gamma\, g_{t-1}^2 + \gamma^2 g_{t-2} + \ldots).
\end{aligned}$$

8.3.4 Adam

En 2014, se propuso una actualización al algoritmo de RMSProp que justamente combinaba las características de este approach pero incluyendo momento. Este algoritmo, bautizado Adam [76], es actualmente el más utilizado en Deep Learning porque justamente balancea eficiencia y convergencia.[2]

El núcleo del algoritmo viene dado por los pasos establecidos en el Algoritmo 9:

$$\begin{aligned}
m_t &\leftarrow \beta_1 m_{t-1} + (1-\beta_1)g_t \\
v_t &\leftarrow \beta_2 v_{t-1} + (1-\beta_2)g_t^2 \\
\hat{m}_t &\leftarrow \frac{m_t}{1-\beta_1^t} \\
\hat{v}_t &\leftarrow \frac{v_t}{1-\beta_2^t} \\
\theta_t &\leftarrow \theta_{t-1} - \alpha\frac{\hat{m}_t}{\sqrt{\hat{v}_t}+\varepsilon}
\end{aligned}$$

La corrección del sesgo se realiza para compensar los cálculos realizadas en las primeras iteraciones, que vienen por el cálculo de los estimadores de los momentos. Este algoritmo tiene una asignación un tanto arbitraria pero con una prueba empírica en el propio paper, de hiperparámetros que se conoce que son buenos para las implementaciones donde la función objetivo es una función de error. Estos son $\alpha = 0.001, \beta_1 = 0.9, \beta_2 = 0.999$ y $\varepsilon = 10^{-8}$.

Resumiendo, este algoritmo se encuentra **dentro** del propio proceso de optimización y aprendizaje (no hay ningún *loop* interno dentro de Adam en sí). Así como ocurre con RMSProp, Adam realiza la actualización peso a peso (las operaciones son vectoriales).

8.4 Arquitectura del Perceptrón Multicapa

A partir de que es posible aglomerar las neuronas, aparece la idea y el concepto de arquitectura de la red neuronal: ¿Cuál es la arquitectura más adecuada para resolver un problema en particular, o una familia de problemas? Esto es un arte más que una ciencia[3], sin una receta clara.

[2] El paper de Kingma tiene casi 200000 citas, un número exorbitante que muestra la efectividad de este algoritmo.

[3] Este es un popular eufemismo *cliché* común también en otras disciplinas para no decir que en realidad no tenemos ni idea cómo se hace.

Algoritmo 9 Adam Optimization Algorithm

Inicializar parámetros: θ, tasa de aprendizaje α, tasas decaimiento β_1, β_2, y la constante ε
Inicializar el primero y el segundo momento: $m_0 = 0$, $v_0 = 0$
$f(\theta)$: Función estocástica de parámetros θ
Inicializar $t = 0$
while no converge **do**
    $t \leftarrow t + 1$
    Calcular el gradient: $g_t \leftarrow \nabla_\theta f(\theta_{t-1})$
    Calcular el estimador sesgado del primer momento: $m_t \leftarrow \beta_1 m_{t-1} + (1-\beta_1) g_t$
    Calcular el estimador sesgado del segundo momento: $v_t \leftarrow \beta_2 v_{t-1} + (1-\beta_2) g_t^2$
    Recalcular el primer momento: $\hat{m}_t \leftarrow \frac{m_t}{1-\beta_1^t}$
    Recalcular el segundo momento: $\hat{v}_t \leftarrow \frac{v_t}{1-\beta_2^t}$
    Actualizar los parámetros: $\theta_t \leftarrow \theta_{t-1} - \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t}+\varepsilon}$
end while
θ_t Resultado parámetro

En redes neuronales la arquitectura suele representarse en base a la cantidad de capas, y la cantidad de neuronas en cada capa. Un buen tip es considerar como capa aquellos lugares en la red donde se pueden ubicar pesos. Por ejemplo, una red con una sola capa oculta tiene dos capas: la primera corresponde a la capa oculta que recibe las entradas directamente, y la segunda toma las salidas de esa capa intermedia, las pasa por pesos de una segunda capa y finaliza generando la salida uni- o multidimensional. A este esquema hay que agregarle, para completar la arquitectura, la dimensión de los datos de entrada. Luego se pueden sumar todos los pesos y con eso se determina la cantidad de parámetros libres de la red, inducido por la arquitectura.

La arquitectura se transforma en un hiperparámetro extra a definir. La arquitectura de la red es un componente clave: de seleccionar un esquema adecuado, aumentan las chances de alcanzar la convergencia de los pesos y obtener adecuado rendimiento en la generalización del problema. El problema es que encontrar esa arquitectura no es un proceso sencillo ni tecnificado.

De lo explicado previamente se hace evidente la importancia de los estudios de ablación [77]. De ese "menjunje" al que se llega de arquitectura (más otros hiperparmetros), los estudios de ablación nos permite definir cuáles podemos eliminar, reducir, y mantener para alcanzar el mismo nivel de eficiencia en la red. En el medio del *hype* del área actual, este es un tema crítico porque permite eliminar la complejidad extra agregada sin justificación. En lugar de definir una red compleja con una gran arquitectura, puede que un simple perceptrón multicapa sea suficiente para solucionar el problema. En definitiva, es el proceso de prueba y error el que nos lleva a adaptar la familia de redes neuronales a cada uno de los problemas particulares en los que se usa esta herramienta.

9. Métricas de Evaluación

9.1 Identificar el patrón de manera correcta

Las redes MLP del capítulo 8 pueden actuar como un **Clasificador**. Esto es, la red fue expuesta a un conjunto de datos, que conformaron el conjunto de aprendizaje, y pudo en algún punto identificar un patrón subyacente, que le permite al ver datos nuevos, pero que son del mismo dataset, inferir de manera adecuada la etiqueta correspondiente a cada muestra.

Surge la pregunta ¿qué es clasificar correctamente? y ¿cuántos elementos eventualmente clasifica correctamente?

De acá surge la necesidad de establecer métodos estándar para evaluar el desempeño de un clasificador. Esto permite, comparar diferentes clasificadores, comparar el mismo clasificador sobre diferentes conjuntos de datos, y evaluar los cambios en los hiperparámetros en relación a la performance[1].

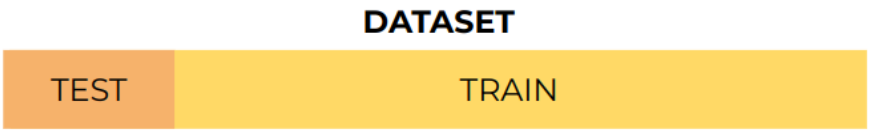


Figura 9.1: El set de datos, la matriz X se puede al menos partir en dos partes, entrenamiento y test.

9.2 Métricas estándar

Lo que se busca de mínima es que el clasificador **generalice** al set de datos. Para esto el dataset (Fig. 9.1) se divide en dos: una parte se utiliza para **converger** los parámetros con el conjunto de

[1] Una excelente exposición introductoria puede verse en el capítulo 5 de [78]

entrenamiento (*train set*), y luego se usa el grupo remanente (*test set*) para evaluar cuán bien el clasificador pudo capturar el patrón y obtener buenos resultados de reconocimiento.[2]

Esto suscita dos problemas. El primero es cómo evaluar si esta división es o no la adecuada. El segundo problema es cómo evaluar la capacidad de generalización de la red.

9.2.1 Matriz de Confusión

La matriz de confusión sirve para establecer una métrica de evaluación de la performance de un clasificador binario/multiclases y se construye en base a listar en las columnas las clases predichas en tanto que en las filas van las clases verdaderas. Luego se enumeran los elementos del grupo de datos de **test** según su rótulo verdadero y el rótulo predicho por el clasificador.

Predicciones	**Positivo?**	**Negativo?**
Positivo	TP	FN
Negativo	FP	TN

Cuadro 9.1: Matriz de Confusión

Esta terminología surge del ambiente médico, por eso se hace referencia a positivo y negativo como el resultado de un test bioquímico. Son TP, True Positive, aquellos ejemplos que el predictor marcó como POSITIVO, y que resultaron ser verdaderamente positivos. Es decir los casos donde acertó en detectar, llamemosle, la patología. Por ejemplo en este caso, este valor queremos que sea lo más alto posible, para que el predictor no se pierda nigún positivo, que al menos los identifique a todos los que hay. El segundo valor que queremos mantener alto es TN, True Negative, los resultados que verdaderamente eran negativos y que a su vez el predictor los identificó como tal. Este valor representa la capacidad del predictor de separar la paja del trigo. Los otros dos valores, FP y FN son los falsos positivos y los falsos negativos, que son aquellos POSITIVOS y NEGATIVOS que marcó MAL el predictor. Por supuesto, estos queremos que sean lo más bajo posible.

De esto surge entonces, la medida más popular de Accuracy, o Performance.

Definición 9.2.1 — Accuracy. La performance o desempeño de un clasificador se puede obtener mediante

$$Acc = \frac{TP+TN}{TP+TN+FP+FN} \tag{9.1}$$

En el caso de que la matriz de confusión es diagonal, el clasificador es perfecto y alcanza una performance de 1. Es importante tener en cuenta, que la peor performance, cuando el clasificador es binario, no es 0, sino $\frac{1}{2}$ que es la que corresponde a un clasificador que simplemente tira una moneda para determinar la clase.

La matriz de confusión suele generalizarse a clasificación multiclase, donde se suelen usar heatmaps

9.2.2 Métricas

A Accuracy se suman otras métricas estandarizadas que aportan información sobre el desempeño de un clasificador desde diferentes perspectivas.

[2] Si el dataset es lo suficientemente grande, se puede dividir en tres partes, agregando un *validation set* para evaluar el desempeño del modelo en diferentes configuraciones de hiperparámetros antes de su evaluación final en el *test set*.

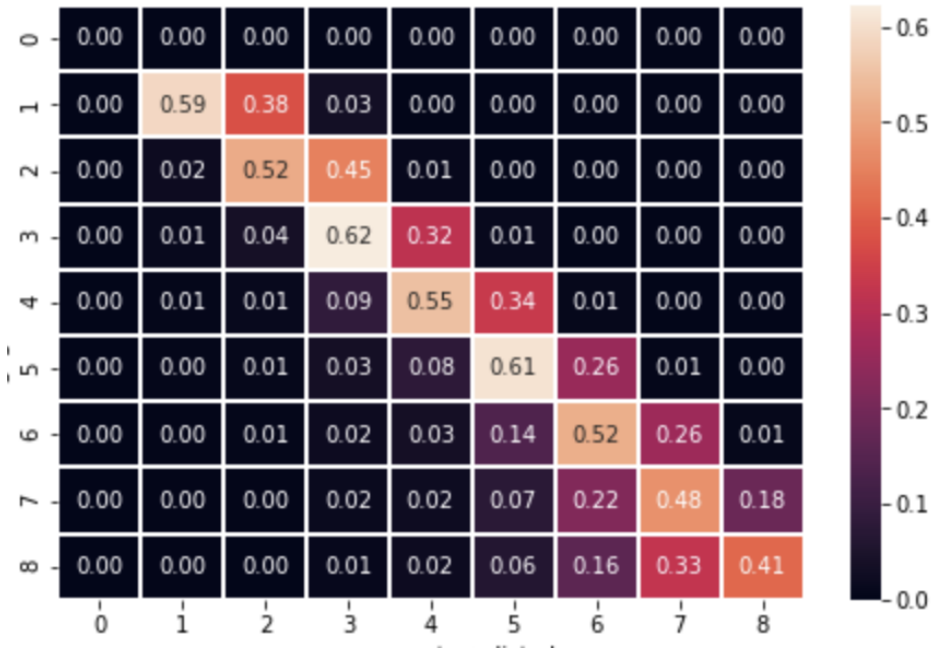


Figura 9.2: Ejemplo de una matriz de confusión multiclases.

Definición 9.2.2 — Precision. Representa cuan bien acertó los que acertó en relación al problema.

$$Precision = \frac{TP}{TP+FP} \tag{9.2}$$

Definición 9.2.3 — Recall. Cuán bien el predictor atrapó a todos los que en definitiva eran positivos .

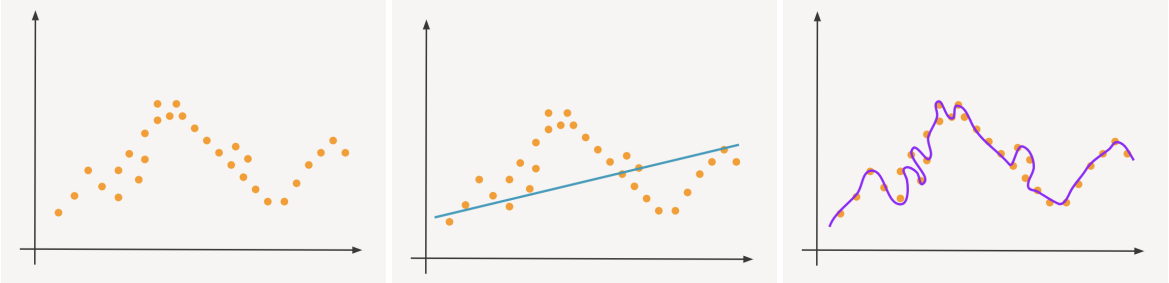
$$Recall = \frac{TP}{TP+FN} \tag{9.3}$$

Definición 9.2.4 — F1-Score. La performance o desempeño de un clasificador se puede obtener mediante

$$F1-Score = \frac{2 \times Precision \times Recall}{Precision + Recall} \tag{9.4}$$

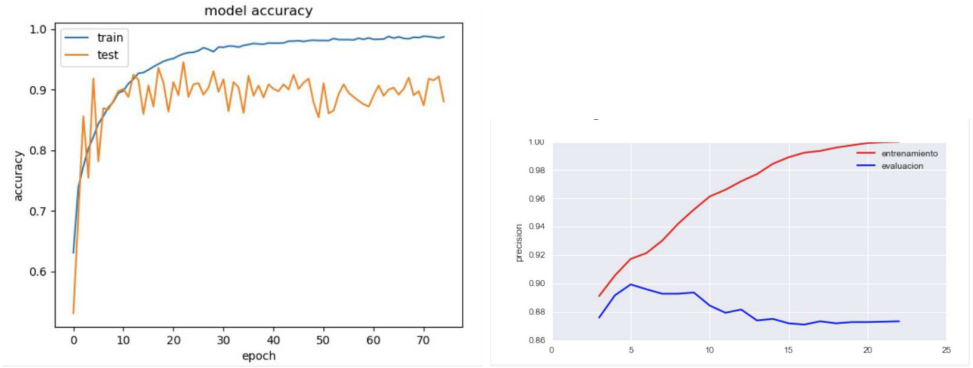
9.2.3 Ajuste, subajuste y sobreajuste

Al entrenar el clasificador, se busca **converger**, es decir utilizar de manera adecuada los datos de entrada para **ajustar** los parámetros libres de forma que la funcion de costo/optimización alcance el mínimo (fig. 9.3). Cuando esto no ocurre, es decir se está en algún mínimo local, o ni siquiera, el clasificador se dice que está **underfitted** o subajustado. Por el contrario, cuando el clasificador está sobreentrenado, se presenta la situación de **overfitting** donde el mínimo encontrado sigue la curva del ruído en los datos, por lo que se ajusta de manera excesiva a las variaciones del conjunto de



(a) Puntos a Ajustar (b) Underfitting (c) Overfitting

Figura 9.3: Niveles de ajuste sobre un conjunto de puntos. Underfitting se da cuando la curva de ajuste no capture bien el patrón inherente en los puntos (en este caso una curva). Por el contrario, cuando el ajuste es milimétrico, la curva captura el ruído natural en los datos y pierde fuerza de generalización.



(a) Undertraining (b) Overtraininng

Figura 9.4: Curvas de desempeño por época para MLPs que incurren en subentrenamiento o sobreentrenamiento.

entrenamiento y es incapaz de identificar verdaderamente el patrón: falla en **generalizar**[3].

Normalmente en un MLP (o similar) que actúa como clasificador, la situación de subajuste se visualiza porque el desempeño de la red es alto para el conjunto de entrenamiento y hay una tendencia en el entrenamiento que a medida que aumenta el entrenamiento el desempeño sobre los datos de test aumenta también, como se ve en la Figura 9.4a. Por otro lado, al incurrir en una situación de sobreajuste ocurre lo contrario y a mayor entrenamiento el desempeño sobre los datos de test se reduce hasta perder toda fuerza predictiva. El sobreajuste es un gran **cuco** porque puede darse por problemas en los datos, como balanceo de clases, datos con excesivo ruido, o incluso insuficiente cantidad de datos para discernir el patrón subyacente.

9.2.4 Convalidación Cruzada

La partición sugerida implica la percepción de que los datos son altamente homogéneos, es decir que la elección de cómo etablecer esa partición no impactará en los resultados. Esto es raramente así, por lo que existen métodos de validación que apuntan a eliminar el sesgo en la selección arbitraria de

[3]Tomaremos esta definición, aún cuando el concepto de overfitting es muy abusado en la jerga de aprendizaje automático, ya que suele ser una consideración posthoc cuando el método falla en la generalización [79].

como partir los datos en entrenamiento y test.

El método mas utilizado para esto, es **k-Fold Cross Validation**

k-Fold Cross Validation

Este método es muy simple y lo que se hace es:

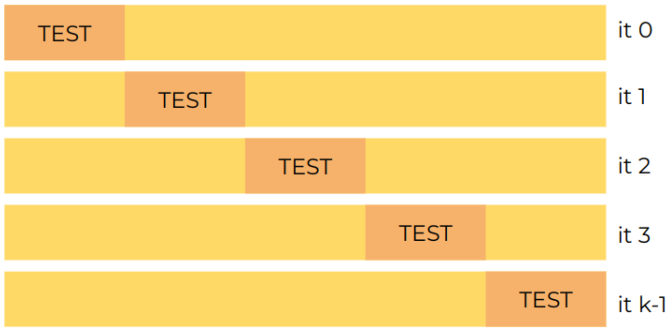


Figura 9.5: En este caso, el dataset se divide e 5 partes donde el rol de test se va alternando en cada una.

1. Iterar T veces (normalmente $T = k$)
2. Dividir el dataset en k partes iguales
3. Tomar $k-1$ partes y usarlas para entrenar el clasificador
4. Usar como grupo test la parte remanente que **no** se uso para entrenar
5. Evlauar la performance sobre esa parte
6. Iterar

Finalmente se obtienen parámetros de desempeño del promedio de las T iteraciones. Este método ayuda a evitar cualquier sesgo que pueda presentarse en la división del dataset. Adicionalmente, si se evalua el desvío de las métricas se puede generar una métrica de la homogeneidad en los propios datos.

IV

Automatizando el Aprendizaje

10 Aprendizaje Automático 103
10.1 Redefiniendo Aprendizaje
10.2 Métodos de Aprendizaje
10.3 Pipeline de Machine Learning

11 Transformaciones en los datos 109
11.1 Estructura interna de los datos

12 Análisis de Componentes Principales 113
12.1 Preliminares

13 Red de Kohonen 119
13.1 Mapa Autoorganizado

14 Red de Hopfield 123
14.1 Hopfield 1982

15 Modelo de Oja y Sanger 129
15.1 La búsqueda del poder de síntesis
15.2 Red de Oja
15.3 Red de Sanger



10. Aprendizaje Automático

10.1 Redefiniendo Aprendizaje

El área de redes neuronales cae dentro de la frontera de lo que se denomina **Aprendizaje Automático** o *Machine Learning*. Puede definirse como:

> **Definición 10.1.1 — Aprendizaje Automático.** Técnicas para identificar relaciones entre datos (X) o mapeos ($X \rightarrow Y$) basadas en algoritmos libres de modelo que dada una serie de parámetros libres que estructuran esas relaciones o mapeos, pueden ajustarse mediante algún proceso de optimización matemática [35].

Para darle encuadre a esta definición, es importante remarcar el aspecto **libre de modelo** de la definición. Esto quiere decir, que el esquema algorítmico a implementar no asume ningún lineamiento específico en relación a la existencia e interacción de los componentes del problema. Es decir, no hay establecido ningún modelo, que generalmente es cuantitativo y matemático, sobre cómo los diferentes elementos del sistema interactúan entre si.

Esto que parece una huevada, es un bomba, y es realmente un cambio radical como metodología general para abordar problemas del mundo real. Por ejemplo, tenemos un cañón del cuál tenemos solo un parámetro libre que es el ángulo de tiro. Diferentes ángulos van a provocar diferentes distancias donde va a caer la bala. Y por supuesto, lo que queremos hacer es predecir donde va a caer la bala, dado un ángulo particular del cañon. La física Newtoniana básica nos da una herramienta, un modelo de la mecánica y dinámica de ese movimiento, el tiro oblicuo, que podemos utilizar para resolver cuantitivamente el problema: vamos a poner un ángulo, hacemos la cuenta, y nos da donde va a caer. Sin embargo, el abordaje utilizando machine learning, tira por la borda ese modelo, y se limita a disparar el cañón muchas veces, medir la distancia, repetir ese proceso varias veces, construir un dataset, y con eso entrenar un modelo de ML[1] que nos va a permitir resolver el mismo problema: predecir.

[1] Fijense que usamos la misma palabra para dos cosas distintas: modelo del mundo que mecanísticamente simplifica su

La definición 10.1.1 es la más abarcativa y la que vamos a adoptar en este texto. De aquí surge entonces el concepto de **Aprendizaje** (que queremos decir cuando decimos que una computadora aprende): aprende cuando se ejecuta lo establecido en la definición 10.1.1.

10.2 Métodos de Aprendizaje

Siendo cruel, banal e imperdonable con el área, una manera pragmática de describirla es como una bolsa de gatos de métodos. Pero todos caen en la definición 10.1.1, es decir tienen algún tipo de clavija libre que puede ajustarse para optimizar algún funcional [78]. La figura 10.2 muestra las diferentes ramas principales en las que se agrupan los métodos.



Figura 10.1: Bolsa de gatos de métodos variopintos donde todos tienen parámetros libres a ajustar.

La primera división tiene que ver con los métodos bioinspirados/biomiméticos y los que no lo son. Hoy (circa 2024) esta división coincide con los métodos basados en redes neuronales multicapas profundas y los que no lo son, que por yuxtaposición pueden llamarse *Shallow* o superficiales. En estos, que son los métodos tradicionales de aprendizaje automático, se pueden incluir cuatro categorías principales que son los frecuentistas, los bayesianos, los `if++`, geométricos y los transformacionales. Los métodos frecuentistas se basan en estadística matemática y son los herederos del análisis estadístico: La regresión lineal, regresión logística (logit), LDA **L**inear **D**iscriminant **A**nalysis. Luego, los enemigos cantados de los primeros, los bayesianos, se basan en la fórmula de Bayes y la posibilidad de realizar inferencia causal utilizando la capacidad de poder iterar esta ecuación: desde **Naive Bayes** hasta **EM**. Luego están aquellos métodos que utilizan condicionales con umbrales libres que actúan de parámetros como los **Decision Trees** (árboles de decisión) y **Random Forest**. Le siguen los métodos basados en conceptos de álgebra lineal y su geometría como k-NN **K**-**N**ear **N**eighbours (vecinos cercanos), el gran SVM **S**upport **V**ector **M**achine, **DBSCAN**, **k-means** y sus variantes. Esta humilde descripción taxonómica culmina con los transformacionales que son métodos que buscan generar una transformación de los datos optimizando alguna función de costo que busque alguna reorganización de los datos y su agrupamiento, como PCA (capítulo 12), SVD (capítulo 18), y otros métodos de reducción de dimensionalidad como t-SNE.

10.3 Pipeline de Machine Learning

De acá surge el modelo de la figura 10.3. Este modelo es pervasivo a cualquier implementación de un sistema que entre dentro de la categoría de aprendizaje automático. Primero, del mundo real, de la realidad, se realiza una medición. Esta medición puede involucrar un proceso manual o incluso un proceso automático (Automatización). Luego hay un proceso de adquisición de una señal, que involucra también la propia digitalización de la información. Esto es convertir la información fenomenológica en datos digitales en una computadora. A partir de ese punto comienza lo que se llama el **pipeline** de machine learning. El primer punto es una tárea de preprocesamiento que es ajustar la información, corregirla, recuperar datos perdidos o faltantes. Y sobre todo ajustar los datos para lo que viene después. El segundo paso del pipeline está concentrado en el armado de una sintesis, una compresión de los datos que capture numéricamente el patrón en un vector que luego se quiere detectar y generalizar.

accionar, y también usamos modelo para describir un esquema de ajuste de parámetros libres optimizando una función de costo en base a un conjunto de datos. Alguién debería inventar una nueva palabra.

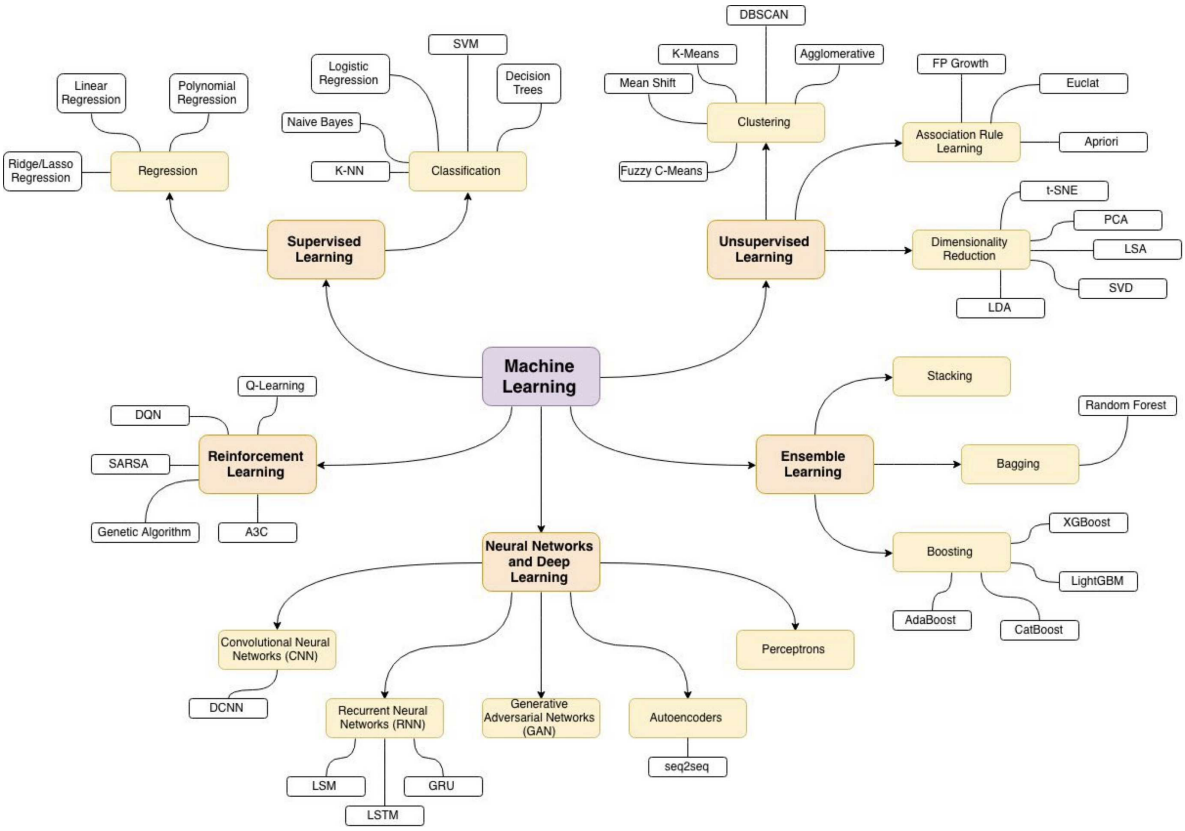


Figura 10.2: Aprendizaje Automático y las áreas asociadas.

Este aspecto es central en aprendizaje automático y es donde se concentra lo que se denomina **feature engineering**. El paso final tiene que ver con el fin de todo el modelo que puede ser una de los cuatro objetivos como clasificar, regresionar, optimizar y generalizar. Luego del pipeline, el insight generado por el proceso es utilizado para un sistema de aplicación, que puede afectar ese mismo ambiente de estudio en un ciclo cerrado.

10.3.1 El problema Inverso

La figura 10.4a representa la estructura del problema inverso de ingeniería, y es un punto crítico a tener en cuenta en cualquier implementación de aprendizaje automático o inteligencia artificial. El fenómeno natural que queremos estudiar tiene una representación intrínseca en L señales continuas analógicas. Esto puede ser cualquier cosa, pero son las fuentes que nos interesa entender para poder identificar algún patrón y luego poder generalizar. Ese fenómeno es capturado por N transductores que reciben una representación de las fuentes que no es exactamente la misma información, sino que está transformada por la función vectorial $x^N(t) = f(s^L(t))$. Este proceso tiene además un inexorable error en la propia transducción con lo que la señal que realmente se genera es $\hat{x}^N(t) = f(s^L(t)) + \xi$.

Estas señales son digitalizadas por un ADC conformando la matriz de datos $x^N[n]$ que es verdaderamente lo que ingresa al modelo. A partir de este punto, el modelo deriva de esa información una salida $y^K[t]$ que representa el output, la salida, pudiendo conformar un sistema MIMO, Multiple Input Multiple Output.

Paralelamente, se podría intentar recuperar una estimación de las fuentes originales del propio

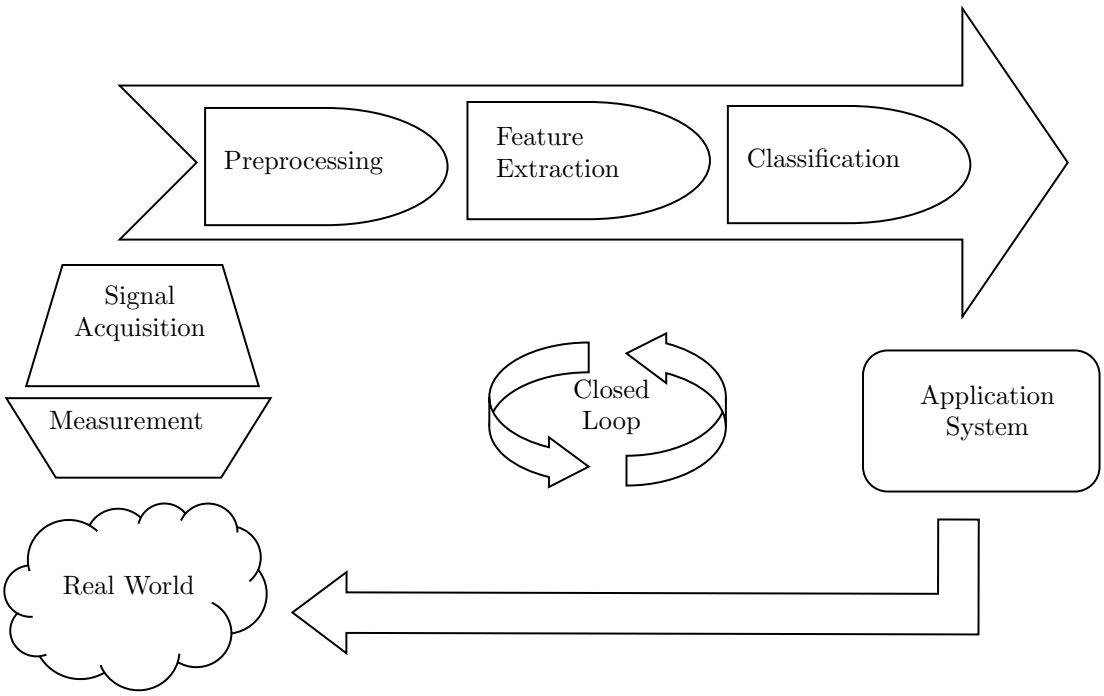
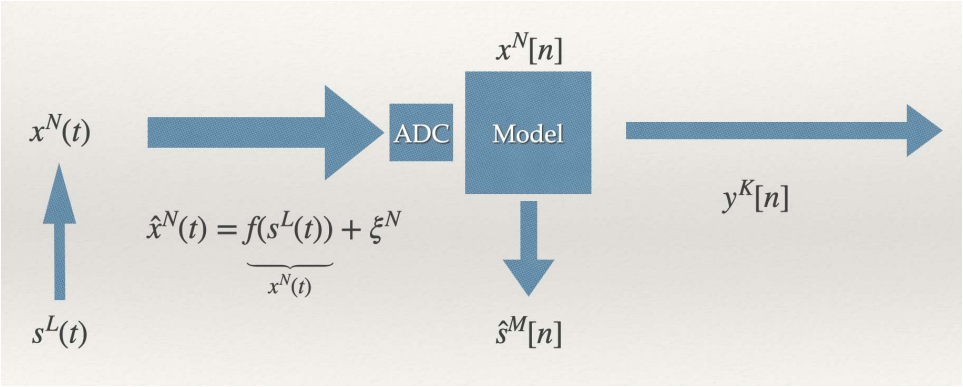
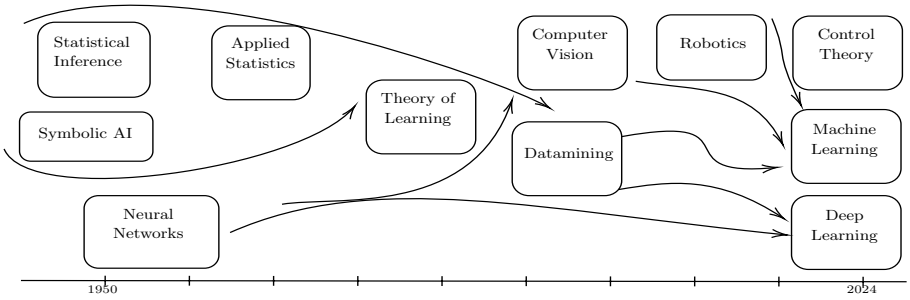


Figura 10.3: Arquitectura General de un Sistema de Aprendizaje Automático

fenómeno físico, que se utilizan para derivar todo el proceso, las $\hat{s}^M[n]$. Existen un conjunto de técnicas transoformacionales que entran se denominan de separación de fuentes, o **B**lind **S**ource **S**eparation, que entran dentro del paraguas de aprendizaje automático [80]. Esto siempre es una estimación porque la información de las fuentes originales, con precisión absoluta, es siempre inaccesible. Esto es independiente de la propia capacidad que tenga el modelo, (AGI, superinteligente o lo que sea).



(a)



(b)

Figura 10.4: Problema inverso y evolución histórica de las técnicas de aprendizaje automático.



11. Transformaciones en los datos

El aprendizaje no supervisado se concentra en aprender patrones y relaciones existentes entre las propias muestras y sus variables, y no en aprender un mapeo específico entre las variables independientes y la variable respuesta, el *label*. Están fuertemente relacionados con la idea de transformar los datos, con buscar representaciones alternativas que ayuden precisamente a capturar esa información. Estos algoritmos suelen utilizarse de manera independiente, pero también como parte del propio pipeline de procesamiento, ya que generalmente esa nueva transformación puede ser útil para el caso supervisado.

11.1 Estructura interna de los datos

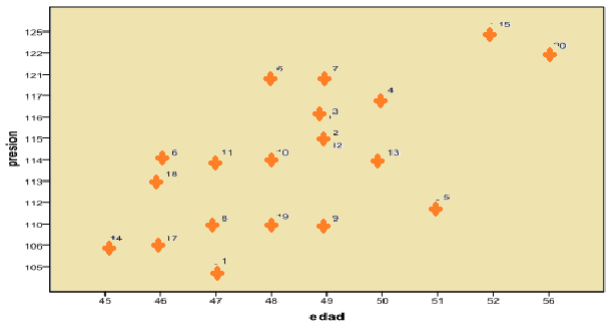
Lo que se busca es la identificación de la propia estructura de la información. Se busca como la información de las variables interactúa entre entre ellas y como eso lo puedo utilizar para identificar patrones subyacentes.

■ **Ejemplo 11.1** Por ejemplo, Para una investigación se seleccionan al azar 20 pacientes hipertensos sobre los que se midieron las siguientes variables:

- X_1: Presión arterial media (mm Hg)
- X_2: Edad (años)
- X_3: Peso (Kg).
- X_4: Superficie corporal (m^2)
- X_5: Duración de la Hipertensión (años)
- X_6: Pulso (pulsaciones/minuto)
- X_7: Medida del stress.

Esto implica que la dimensión de la matriz de muestras es 7. Podemos intentar reducirla a dos variables, arbitrariamente: Presión vs Edad.

Figura 11.1: Presión vs Edad



Del gráfico surge que dos individuos que se encuentran *cerca* podrían tener una característica similar de estas dos variables combinadas. Es decir, dos invidivuos que están cerca en este gráfico, deberían representar situaciones del mundo real parecidas. Esta es la idea subyacente en los métodos de aprendizaje no supervisado.

■

Para identificar esa estructura interna se apela a:

1. Agrupamiento o Clustering.
2. Asociaciones.
3. Reducción de dimensionalidad.

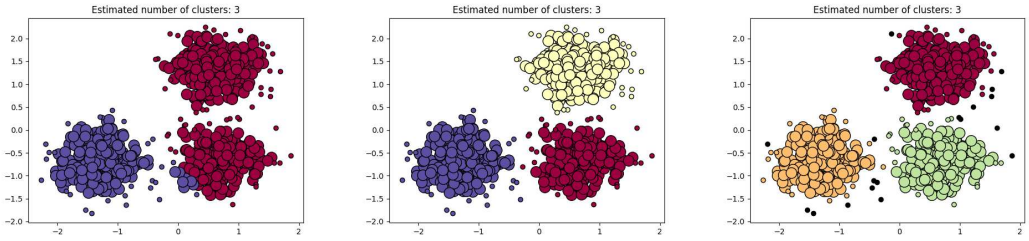
Los métodos de aprendizaje no supervisado hacen uso de las estructuras geométricas de los espacios donde viven las muestras X, y establecen sobre los mismos medidas de similitud. Cuando un elemento x_i es parecido a otro x_j. Para esto se usa todo mayoritariamente ideas de algebra lineal, pero se expande rapidamente a un abánico de técnicas más amplio.

Particularmente en este texto, nos concentraremos en los métodos que implementan estas asociaciones basados en estructuras biomiméticas de redes neuronales. Sin embargo, se ofrece una pequeña introducción que servirá para comprender los conceptos generales.

11.1.1 Clustering y Agrupamientos

Basado en establecer esas medidas de similaridad entre los datos para verificar si pertenecen a subgrupos internos claramente identificables: buscar agrupamientos.

La figura 11.2 muestra por ejemplo tres variantes que reflejan los problemas más comunes en los agrupamientos. El primero es el agrupamiento dado por *k-means* [5]. Este algoritmo simple identifica en base a un hiperparámetro que especifica la cantidad de clusters a identificar, y luego asigna cada elemento al cluster ya asignado para los k vecinos más cercanos. Para la figura 11.2a la elección de $k = 2$ parecería no ser la mejor ya que graficamente se ven que los clusters son 3 lo cual se representa en el segundo gráfico 11.2b. Por otro lado el tercer gráfico 11.2c utiliza un algoritmo diferentes, *dbscan* [81], que tiene en consideración la densidad de elementos, y que permite identificar como algunos elementos como no pertenecientes a ninguno de los tres clusters en particular (en negro en el gráfico).



(a) k-Means con $k = 2$ (b) k-Means con $k = 3$ (c) DBSCAN con k libre

Figura 11.2: Alternativas de Clustering



12. Análisis de Componentes Principales

Hacemos aquí un interludio para presentar un método de transformación de datos que enfatiza un aspecto de la inteligencia que es la capacidad para sumarizar información, para transformarla a un formato que capture cierta escencia de la información. Uno de estos métodos es el método de análisis de componentes principales.

12.1 Preliminares

- Media Muestral: Dado un conjunto de datos de una variable $\mathbf{X} = \{\mathbf{x_1}, , \mathbf{x_2}, \ldots, \mathbf{x_n}\}$

$$\bar{x} = \frac{1}{n}\sum_{i=1}^{n} x_i$$

- Varianza Muestral:

$$s^2 = \frac{1}{n-1}\sum_{i=1}^{n}(x_i - \bar{x})^2$$

  es una medida de dispersión.
- Desviación estándar muestral: Es otra medida de dispersión

$$s_{dv} = \sqrt{s^2}$$

- Covarianza muestral: es una medida de asociación lineal entre los datos de dos variables. Dado el conjunto de datos:

$$\left[\begin{array}{c|c|c|c} \mathbf{X_1} & \mathbf{X_2} & \dots & \mathbf{X_n} \\ \hline x_{11} & x_{12} & \dots & x_{1n} \\ x_{21} & x_{22} & \dots & x_{2n} \\ \vdots & & & \\ x_{m1} & x_{m2} & \dots & x_{mn} \end{array}\right]$$

$$s_{ik} = \frac{1}{n}\sum_{j=1}^{m}(x_{ji} - \bar{x_i}) * (x_{jk} - \bar{x_k})$$

Matriz de Covarianzas:Las covarianzas forman una matriz simétrica definida positiva

$$S = \begin{bmatrix} s_{11} & s_{12} & \dots & s_{1n} \\ s_{21} & s_{22} & \dots & s_{2n} \\ \vdots & & & \\ s_{m1} & s_{m2} & \dots & s_{mn} \end{bmatrix}$$

Interpretación de la Covarianza muestral

- $s_{ik} > 0$ indica una asociación lineal positiva entre los datos de las variables.
- $sik < 0$ indica una asociación lineal negativa entre los datos de las variables.
- $s_{ik} = 0$ indica que no hay una asociación lineal entre los datos de las variables.

R La varianza muestral es la covarianza muestral entre los datos de la i-ésima variable con ella misma, algunas veces se denota como s_{ii}

Variables estandarizadas:

$$\tilde{\mathbf{X}}_\mathbf{1} = \frac{\mathbf{x_1} - \bar{\mathbf{x}}_\mathbf{1}}{\mathbf{s_1}}$$

A cada variable le resto su propia media y divido por su propia desviación estándar.
Correlación muestral: Calcular las covarianzas con las variables estandarizadas:
Es otra medida de asociación lineal. Para los datos de la i-ésima y k-ésima variable se define como:

$$r_{ik} = \frac{s_{ik}}{\sqrt{s_{ii}}\sqrt{s_{kk}}}$$

La correlación está acotada entre -1 y 1.

12.1.1 Transformación PCA

Si las variables están muy correlacionadas, entonces poseen información redundante. La idea del método es justamente eliminar la redundancia. Transformar el conjunto original de variables en otro conjunto de nuevas variables que sean combinaciones lineales de las anteriores pero que no estén correlacionadas entre sí, llamado **conjunto de componentes principales**. La técnica de componentes principales es debida a Hotelling (1933), aunque sus orígenes se encuentran en los ajustes ortogonales por mínimos cuadrados introducidos por K. Pearson (1901).

Dadas p variables originales, se buscan $q < p$ variables que sean combinaciones lineales de las p originales, recogiendo la mayor parte de la información o variabilidad de los datos.

Si las variables originales no están correlacionadas, entonces no tiene sentido realizar un análisis de componentes principales.

La característica que maximiza la variabilidad.

Es un buen factor para diferenciar objetos de un conjunto de datos. Por ejemplo, Conjunto de datos con información de vehículos

- Característica 1: Cantidad de ruedas.
- Característica 2: Longitud del vehículo. (Mayor variabilidad)

Utilizando la segunda nos daríamos cuenta si el registro corresponde a un auto o un colectivo.

Supongamos que se dispone de los valores de p-variables en n elementos de una población dispuestos en una matriz X de dimensiones $n \times p$, donde las columnas contienen las variables y las filas contienen los elementos.

Variables:

$$\{x_1, \ldots, x_p\}$$

PCA realiza una transformación del conjunto de datos X que consiste de una traslación y una rotación, definidas de manera tal que la varianza del nuevo conjunto de variables sea máxima.

La Primera componente

$$y_1 = \sum_{j=1}^{p} a_{1j}(x_j - \bar{x}_j) = a_{11}(x_1 - \bar{x}_1) + \ldots + a_{1p}(x_p - \bar{x}_p)$$

con

$$\vec{a_1} = (a_{11}, a_{12}, \ldots, a_{1p}) \in \mathbb{R}^p$$

El conjunto de Componentes principales es una combinación lineal de las variables originales.

Buscamos que $\vec{a_1} / \|\vec{a_1}\| = 1$ y que la $Var(y_1)$ resulte máxima en el conjunto de las combinaciones lineales posibles de las variables originales. Cargas:

Los coeficientes $a_{ji},\ i = 1, \ldots, p, j = 1, \ldots, p$ se denominan cargas (o loadings).

¿Cómo hallar los valores de las cargas?

Son los autovectores de la matriz de covarianzas

Este problema se resuelve buscando los $v1, \ldots, v_n$ y los $\lambda_1, \ldots, \lambda_n$

$$det(S_X - \lambda_i I) = 0$$

y los v_i tal que

$$S_X v_i = \lambda_i v_i$$

Entonces

$$y_1 = v_{11}x_1 + \cdots + v_{n1}x_n$$

$$y_2 = v_{12}x_1 + \cdots + v_{n2}x_n$$

y así con todos.

Además, el autovalor λ_i es la varianza de la componente i.

Ordenando los autovalores de S_X de mayor a menor, $\lambda_1 \geq \lambda_2 \geq \ldots \geq \lambda_n$ podemos reducir la dimensionalidad tomando los autovectores correspondientes a los primeros q autovalores, que son los que proveen mayor información (en términos de variabilidad).

El porcentaje de variabilidad que poseen las primeras q componentes es

$$Variabilidad = \frac{\sum_{i=1}^{q} \lambda_i}{\sum_{i=1}^{n} \lambda_i}$$

Entonces, para reducir la dimensionalidad de un data set considerando sus características más importantes, deberíamos requerir que la proyección cubra, por ejemplo, el 95 % de las varianzas, o sea que

$$Variabilidad \geq 0.95$$

1. Paso 1:Tomar un conjunto de Datos X. Las variables deben estar en las columnas.
2. Paso 2: Restar la media de cada conjunto de variables. Calcular $X - \bar{X}$. Obtener un conjunto de datos con media cero.
3. Paso 3: Calcular la matriz de Covarianzas.
4. Paso 4: Calcular autovalores y autovectores de la mariz de covarianzas y ordenar los autovalores de mayor a menor.
5. Paso 5: Formar la matriz E tomando los autovectores correspondientes a los mayores autovalores.
6. Paso 6: Calcular las nuevas variables $Y = E(X - \bar{X})$

Matriz Covarianzas vs. Matriz de Correlación Si alguna de las variables, por ejemplo la primera, tiene valores mayores que las demás, la manera de aumentar la varianza es hacer tan grande como podamos la coordenada asociada a esta variable. Por ejemplo: pasamos de medir en km. a medir en metros, el peso de esa variable en el análisis aumentará.

Entonces, cuando las escalas de medida de las variables son muy distintas, la maximización de la varianza dependerá decisivamente de estas escalas de medida y las variables con valores más grandes tendrán más peso en el análisis.

Si queremos evitar este problema conviene utilizar las variables estandarizadas para calcular las componentes principales, de manera que las magnitudes de los valores numéricos de las variables originales sean similares. Lo cual es lo mismo que pplicar el análisis de componentes principales utilizando la matriz de correlaciones en lugar de la matriz de covarianzas.

Cuando las variables tienen las mismas unidades, ambas alternativas son posibles.

Ejemplo:

Sea la matriz de covarianzas

$$S = \begin{pmatrix} 3 & 1 & 1 \\ 1 & 3 & 1 \\ 1 & 1 & 5 \end{pmatrix}$$

correspondiente a las variables aleatorias $(X_1, X_2, X3)$ con media cero.

1. Escribir las variables (Y_1, Y_2, Y_3) de componentes principales y calcular qué proporción de la varianza total explica cada componente.

Autovalores: $\lambda_1 = 6$, $\lambda_2 = 3$, $\lambda_3 = 2$

Autovectores:

$$E = \left(\begin{array}{ccc} V_1 & V_2 & V_3 \\ \hline -0.40 & -0.57 & 0.70 \\ -0.40 & -0.57 & -0.70 \\ -0.81 & 0.57 & 0 \end{array}\right)$$

$$Y_1 = -0.40X_1 + (-0.40)X_2 + (-0.81)X_3$$
$$Y_2 = -0.57X_1 + (-0.57)X_2 + 0.57X_3$$
$$Y_3 = 0.70X_1 + (-0.70)X_2$$

Proporción de varianza de cada componente: Autovalores: $\lambda_1 = 6\ , \lambda_2 = 3\ , \lambda_3 = 2$

$$de\ Y_1 = \frac{6}{11}$$
$$de\ Y_2 = \frac{3}{11}$$
$$de\ Y_3 = \frac{2}{11}$$

¿Qué información aportan las cargas o *loadings*?

- Si la carga (coeficiente o *loading*) de una variable en la componente principal es positiva, significa que la variable y la componente tienen una correlación positiva.
- Si por el contrario, la carga es negativa, este hecho indica que dicha variable se correlaciona en forma negativa con la primera componente.

Así la primera componente representa un índice (o una característica) por el cual se pueden ordenar los registros.



13. Red de Kohonen

La red de Kohonen, también llamada SOM, Self Organized Map, mapa autoorganizado, es una red bioinspirada que permite detectar regularidades en los datos de entrada. Su autor fue Teuvo Kohonen, investigador Finlandés que publicó su idea por primera vez en 1982, y fue una de las razones de la segunda ola de la inteligencia artificial [82, 83].



Figura 13.1: Teuvo Kohonen contento por su gran trabajo.

13.1 Mapa Autoorganizado

La arquitectura de la red se compone de dos capas, donde en la primera corresponde a las entradas, y la segunda a una capa donde las neuronas están conectadas con sí mismas positivamente y tienen conexiones inhibitorias con las neuronas vecinas. Este sistema produce a lo largo del tiempo, que alguna de las unidades tomen un nivel de activación mayor mientras el nivel de las demás unidades se anula.

Como se observa en la figura 13.2, la capa central es una grilla o mapa que puede a su vez tener diferentes layout topológicos.

13.1.1 Aprendizaje Competitivo

La idea de Kohonen fue implementar un mecanismo biomimético de aprendizaje competitivo. Dada una unidad de entrada x, la neurona que tenga vector de pesos w más parecido, más similar, bajo alguna medida de similitud, será la ganadora. Esto es un esquema de agrupamiento, porque el resultado que genera es que entradas similares van a activar la misma neurona de la grilla, agrupandose. Es competitivo porque las neuronas terminan compitiendo unas con otras con el objetivo de que finalmente sólo una de las neuronas de salida se active y las demas sean forzadas a valores de respuestas minimos. Este mecanismo se denomina **winner take all**.

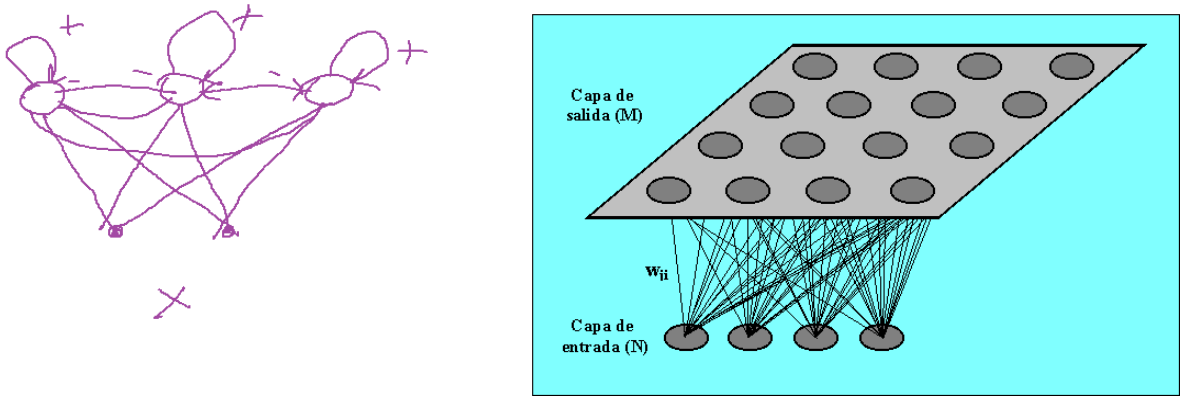


Figura 13.2: Mapa autoorganizado, estructura de conexiones de la red y grilla de salida.

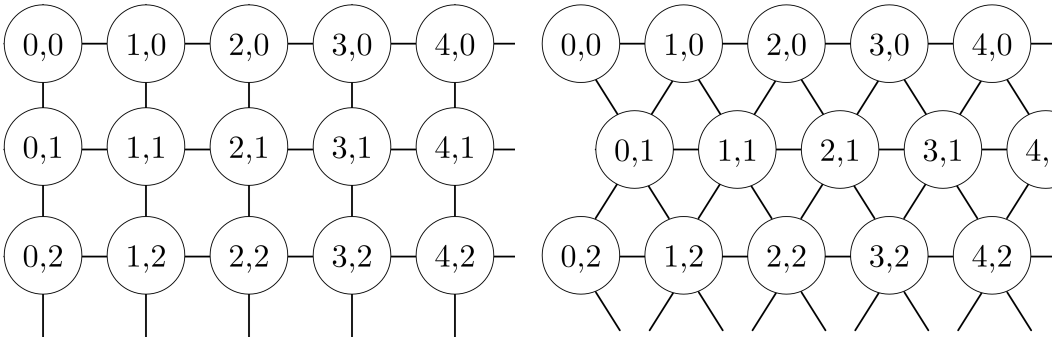


Figura 13.3: Estructuras posibles para la grilla de la red.

13.1.2 Arquitectura

La red de Kohonen tienen una sola capa de neuronas (más una de entradas) que forma una grilla bidimensional (*kxk*) y en la que cada neurona está conectada a todas las componentes de un vector de entrada n-dimensional. Los datos así de entrada que son multidimensionales se proyectan en un espacio bidimensional. La grilla puede ser rectangular o hexagonal.

Sobre la grilla se define un **vecindario** de 4-vecinos con radio $R = 1$ ú 8-vecinos con radio $R = \sqrt{2}$. Cada vez que se coloca un vector en la entrada, se selecciona una neurona ganadora y se aplica una regla de actualización a esa neurona y su vecindario.

Las características de la red son:

- Se establece un tamaño particular para la grilla de $k \times k$, con una topología, rectangular o hexagonal.
- Cada neurona de salida $j \in \{1,...,k^2\}$ tiene asociado un vector de pesos $W_j = (w_{j1},...,w_{jn})$.
- Los pesos W_j de cada neurona de salida, tienen la misma dimensión que los datos de entrada.

13.1.3 Algoritmo

El algoritmo es bastante directo y se basa en una actualización iterativa de los pesos de la red a medida que se van procesando los ejemplos del conjunto de entrenamiento.

Algoritmo 10 Red de Kohonen

1: $X^p = \{x_1^p, \ldots, x_n^p\},\ p = 1, \ldots, P$ son los registros de entrada.
2: Definir la cantidad de neuronas de salida: $k \times k$.
3: Inicializar los pesos $W_j,\ j = 1, \ldots, k^2$, cada $W_j = (w_{j1}, \ldots, w_{jn})$:
4: Con valores aleatorios con distribución uniforme.
5: Con ejemplos al azar del conjunto de entrenamiento.
6: Seleccionar un tamaño de entorno inicial $R(0)$.
7: Seleccionar la tasa de aprendizaje inicial $\eta(0) < 1$.
8: **while** iteración t **do**
9: Seleccionar un registro de entrada X^p.
10: Encontrar $\hat{k}$ ganadora con vector de pesos $W_{\hat{k}}$ más cercano a X^p:
11: Definir una medida de similitud d y luego calcular:

$$W_{\hat{k}} = \arg \min_{1 \leq j \leq N} \{d(X^p - W_j)\}$$

12: Actualizar los pesos de las neuronas vecinas según la regla de Kohonen.
13: **end while**

Regla de Kohonen

Partiendo de la activación (la selección) de la neurona $\hat{k}$, que es la neurona ganadora y utilizando la definición del radio del vecindario que es $R(t)$, entonces el conjunto vecindario es:

$$N_{\hat{k}}(t) = \{neu / ||neu - neu_{\hat{k}}|| < R(t)\}$$

donde $R(0)$ es un dato de entrada y $R(t) \to 1$ cuando $t \to \infty$. Esto implica un esquema de actualización del radio aunque también puede permanecer constante durante todo el proceso.

Una vez que el conjunto de neuronas a activar está seleccionado, se pasa a actualizar los pesos de todas estas neuronas vecinas de $\hat{k}$ utilizando la regla de de Kohonen:

- Si $j \in N_{\hat{k}}(t) \to W_j^{t+1} = W_j^t + \eta(t) * (X^p - W_j^t)$
- Si $j \notin N_{\hat{k}}(t) \to W_j^{t+1} = W_j^t$

El valor de $\eta(t)$ actúa como un factor de descuento que se va reduciendo en cada iteración, $\eta(t) \to 0$. Por ejemplo, se puede utilizar $\eta(t) = \frac{1}{t}$.

Este factor es clave para que la red de Kohonen converja: que los pesos de la red se parezcan a los datos de entrada y capturen su estructura de similitud.

Demostración. Asumimos una iteración de un paso t a un paso $t+1$:
$W_{\hat{k}}^{t+1} - X^p = W_{\hat{k}}^t + \eta(t)(X^p - W_{\hat{k}}^t) - X^p = (1 - \eta(t))(W_{\hat{k}}^t - X^p)$
Entonces...

$$||W_{\hat{k}}^{t+1} - X^p|| \leq ||W_{\hat{k}}^t - X^p||$$

con lo que la diferencia entre el peso y la entrada se va reduciendo. Esto muestra que el nuevo vector de pesos está más cerca del patrón X que el anterior. ■

Medidas de similitud (o funciones de propagación)

- Distancia Euclídea:

$$W_{\hat{k}} = \arg \min_{1 \leq j \leq N} \{\left\|X^p - W_j\right\|\}$$

- Distancia Exponencial:

$$W_{\hat{k}} = \arg \max_{1 \leq j \leq N} \{e^{-\left\|X^p - W_j\right\|^2}\}$$

  (normalizando todos los vectores)

Inicialización

- Los pesos se pueden inicializar con valores aleatorios: Puede tener el problema de que algunas unidades queden lejos de los valores iniciales y entonces nunca ganen. Se dice que son unidades muertas.
- Para evitar eso se puede inicializar los pesos con muestras de los datos de entrada.
- La cantidad total de iteraciones conviene elegirla en función de la cantidad de neuronas de entrada, por ejemplo $500 * n$.
- El valor de $R(0)$ puede ser el tamaño total de la red y va decreciendo hasta llegar a $R_f = 1$, donde solamente se actualizan las neuronas vecinas pegadas.

Preprocesamiento de los datos

La red requiere estandarizar (ver 23.2.1) las variables del conjunto de entrenamiento y prueba:

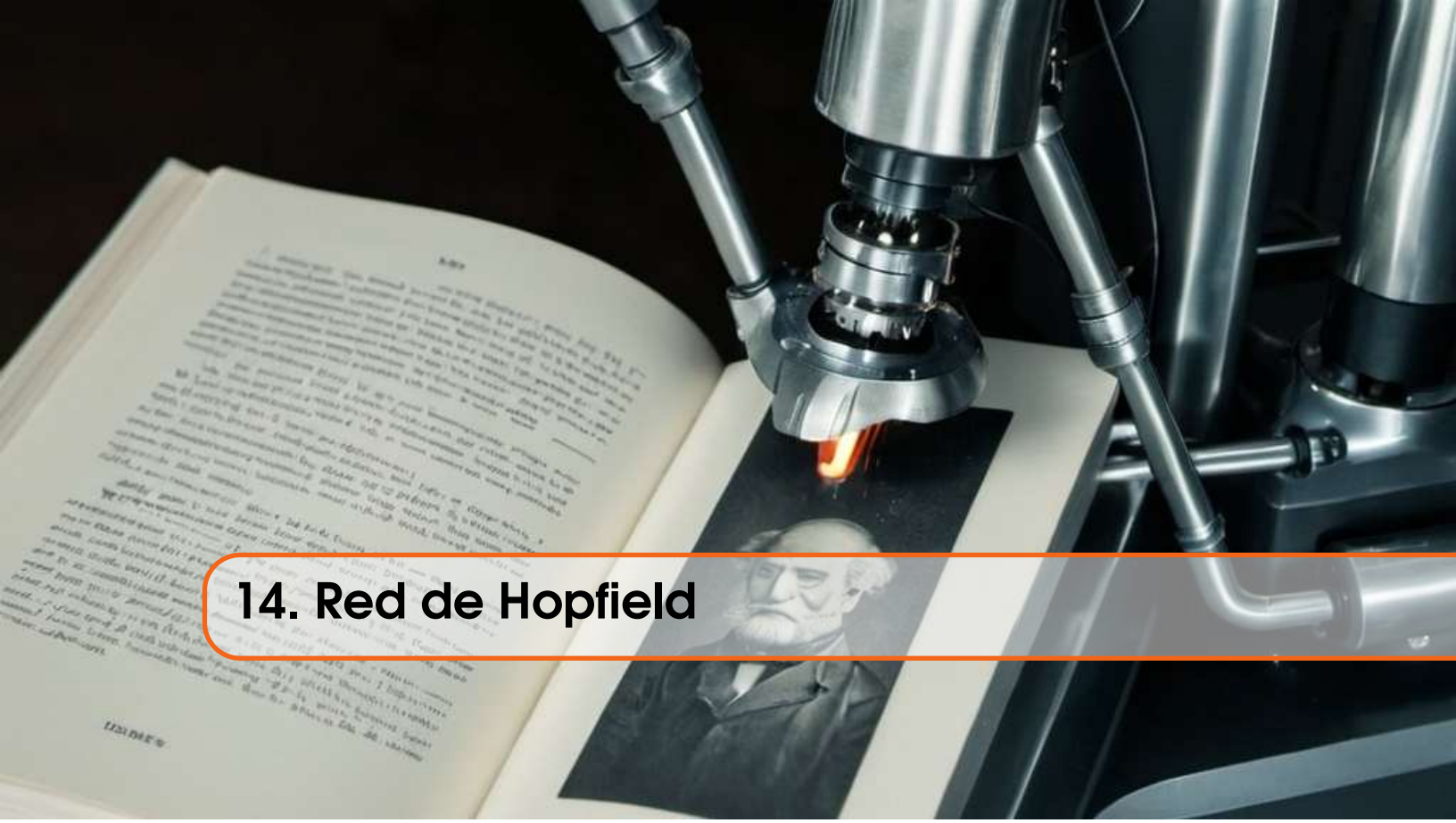
$$\tilde{X}_i = \frac{X_i - \bar{X}_i}{s}$$

donde $\bar{X}_i$ es la media y s_i es la desviación estándar de X_i.

Reducción de la dimensionalidad

La red de Kohonen es un algoritmo bioinspirado de MDS, Multidimensional Scaling [84], simple y directo para establecer clusters bidimensionales. Al usar esquemas bidimensionales, es posible además visualizar el resultado del entrenamiento de la red y apreciar como las neuronas de salida forman una matriz, y verificar en qué coordenadas se encuentra la neurona asociada a cada ejemplo de entrenamiento.

- La red de Kohonen puede ser más rápida que el Perceptrón multicapa.
- Al ser no supervisado puede aplicarse en casos donde el conjunto de datos solo tiene valores de entrada sin etiquetar.
- Si el conjunto de variables es muy grande puede ser difícil asociarlo con un conjunto bidimensional.
- Solo puede realizarse con variables numéricas.
- Hay que decidir el tamaño de la grilla desde el principio y no hay un criterio demostrado para hacerlo.



14. Red de Hopfield

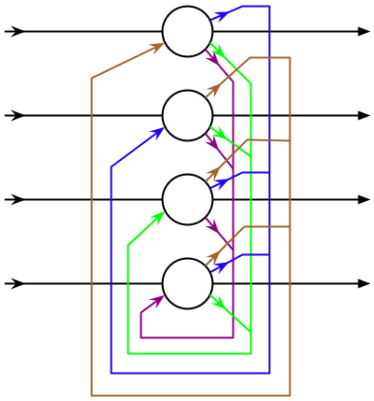
La red de Hopfield es un modelo de redes neuronales inspirada en modelos de la física computacional que tiene una propiedad biomimética de memoria asociativa. Dado un conjunto de p patrones almacenados, cuando se presenta a la red un nuevo patrón A, permite responder con uno de los patrones almacenados B, de forma tal que sea el más parecido a A. Al patrón A se lo denomina **estímulo** en tanto que B se denomina **respuesta**. Este interesante comportamiento, con muchos paralelismos con sistemas biológicos, se logra mediante el establecimiento de una red recurrente donde la temporalidad se hace presente y que manifiesta una estructura de un sistema dinámico, que evoluciona con el tiempo, y que a su vez tiene una energía asociada al sistema. Hopfield realiza su propuesta desde la perspectiva de la física computacional, inspirándose en el modelo de Ising y Simmulated Annealing [4]. Esto fue muy inteligentemente manipulado para generar el efecto de una memoria asociativa, que asocia estímulos de entrada, con respuestas memorizadas. La red de Hopfield fue sustancial en impulsar la segunda era de la inteligencia artificial, basada justamente en el modelo conexionista, lo cual quedó plasmado en la adjudicación de nada más y nada menos que el premio Nobel 2024 de Física [85].



Figura 14.1: John Hopfield muy contento después de crear su red homónima.

14.1 Hopfield 1982

La red de Hopfield de 1982 es una Red Recurrente discreta. Las redes recurrentes conforman sistemas dinámicos donde las salidas de cada neurona corresponden a las entradas en un paso de tiempo posterior. Se le llama versión del ’82 a la versión discreta, en tanto que la versión ’84 corresponde a la versión continua.



La red de Hopfield tiene las siguientes característcas:

- Todas las neuronas están conectadas entre sí. Todas contra todas en una configuración estrella.
- Ninguna neurona está conectada con sí misma.
- El conjunto permitido de valores de entrada y salida es $\{-1,1\}$, es un conjunto binario. De esta manera todas las neuronas en una Red Hopfield son binarias, tomando solamente uno de los dos estados posibles: activo o inactivo.
- Todas las neuronas están en una sola capa de entrada y salida.
- El modelo de Hopfield puede representar un sistema físico donde el estado está dado por la salida de todas las neuronas al mismo tiempo, y a su vez la función de costo del sistema refleja la energía del sistema, donde los mínimos de esa función corresponden a los estados estables que es donde están los patrones almacenados.

Las redes de Hopfield actúan como memorias asociativas. En el tiempo $t = 0$ un patrón es colocado en todas las entradas de la red (que corresponden a las entradas de todas las neuronas). Luego se calculan todas las salidas que serán las entradas del sistema para el tiempo $t + 1$. El sistema se deja evolucionar de esta manera, hasta que se alcanza un punto estable donde no se producen más cambios. Los valores de la salida en ese momento, **serán** justamente uno de los patrones almacenados en la red.

Estos patrones pueden ser imágenes (Figura 14.2), conformando un patrón de entrada visual. Cada neurona tiene entonces 2 estados posibles:

- $S_i = +1$
- $S_i = -1$

Para una red de N neuronas, el estado queda representado por el vector de estados $S = [S_1, S_2, ..., S_N]^T$.

14.1.1 Memoria Asociativa

Supongamos que tenemos un conjunto de patrones almacenados $\xi^\mu, \mu = 1, ..., p$, ya aprendidos, en una red de Hopfield. Presentamos un nuevo patrón ζ con el objetivo de encontrar el patrón almacenado más cercano a ζ. Para eso necesitamos que la red almacene pesos sinápticos w_{ij}, $i = 1, \ldots, N$, $j = 1, \ldots, N$ tal que la red nos devuelva el patrón ξ^μ más cercano a ζ.

Las neuronas de la red tienen las siguientes características:

- Cada neurona i es un Perceptrón simple con la función de activación escalón (1, -1).

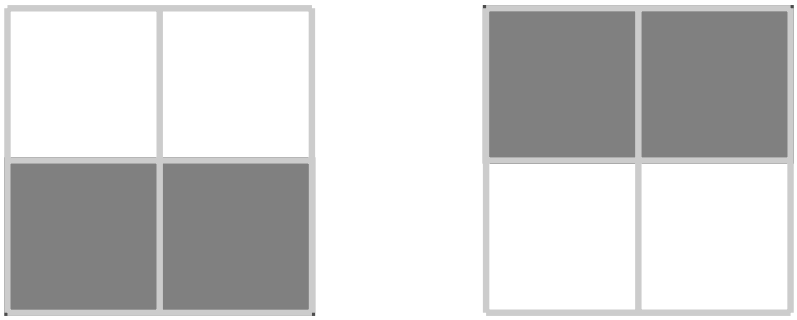


Figura 14.2: Patrones de imágenes binarias. Asumiendo blanco=1 y gris=-1, el primer patrón es $\xi_1 = (1,1,-1,-1)$ en tanto que el segundo es $(-1,-1,1,1)$.

- Cada par de neuronas (i,j) se conectan por el peso sináptico w_{ij}.

Comenzamos con una configuración $S_i = \zeta_i,\ i = 1,\ldots,n$, y queremos ver si hay un conjunto de pesos w_{ij} que hagan a la red alcanzar el estado $S_i = \xi_i^{\mu_0}$, donde ξ^{μ_0} es un patrón almacenado, en tanto que $\xi_i^{\mu_0},\ i = 1,\ldots,n$ es el estado del patrón almacenado más parecido a $\zeta_i,\ i = 1,\ldots,n$. La neurona i modifica su estado S_i de acuerdo a la regla:

$$h_i = \sum_{j=1}^{N} w_{ij} S_j,\ i \neq j$$

$$S_i = \begin{cases} +1 & si\ h_i > 0 \\ -1 & si\ h_i < 0 \end{cases}$$

En el caso de que $h_i = 0$ entonces la neurona i permanece en el estado previo. Esto por supuesto es equivalente a:

$$S_i = sgn(h_i)$$

Este paso anterior representa la actualización del estado del sistema, donde se actualizan uno a uno todos los valores de salida, dado el estado completo del sistema en el paso de tiempo anterior. Así, la red evoluciona hasta que S_i no se modifica más $\forall i = 1,\ldots n$.

Si tomásemos sólo un patrón en el conjunto, ($\xi_i = 1$ ó $\xi_i = -1$), $i = 1,\ldots N$, asumiendo que tenemos N nodos en la red, podemos elegir los pesos sinápticos de la siguiente forma:

$$w_{ij} = \frac{1}{N}\xi_i\xi_j,\ i,j = 1,\ldots N$$

y así evolucionar la red haciendo

$$S_i = sgn\left(\sum_{j=1}^{N} w_{ij}\xi_j\right) = sgn(\xi_i) = \xi_i \qquad \forall i.$$

Esto es debido a que $\xi_j^2 = 1 \qquad \forall j = 1,\ldots N$ y por lo tanto se produce la convergencia: alcanza el patrón almacenado.

Esto se extiende para el caso de muchos patrones, haciendo uso de la regla de Cooper [86] para el cálculo cerrado de los pesos:

$$w_{ij} = \frac{1}{N} \sum_{\mu=1}^{p} \xi_i^\mu \xi_j^\mu,\ i \neq j$$

donde p es el número total de patrones almacenados. Esta ecuación asigna con un valor elevado los pesos entre dos nodos i y j cuando ambos están activados o ambos desactivados.

Como la arquitectura de la red no admite conexiones directas de cada neurona con sí misma y la conexión es simétrica, se toma también $w_{ii} = 0$ y $w_{ij} = w_{ji}$. Todo esto matricialmente queda

$$W = \tfrac{1}{N} K\, K^T$$
$$S(t+1) = sign(W\, S(t))$$

.

El algoritmo 11 muestra el paso a paso del algoritmo completo.

Algoritmo 11 Modelo de Hopfield

1: Almacenamiento: Se tiene p patrones $\xi^1, \xi^2, ..., \xi^p$ de dimensión N, binarios.
2: Los pesos sinápticos w_{ij} conectan la neurona i con la j.
3:

$$w_{ij} = \begin{cases} \frac{1}{N} \sum_{\mu=1}^{p} \xi_i^\mu \xi_j^\mu & j \neq i \\ 0 & j = 1 \end{cases}$$

4: Sea ζ un vector de consulta $N-$dimensional desconocido, presentado a la red.
5: $S_i(0) = \zeta_i$ es el estado de la neurona i en el tiempo $t = 0$
6: **while** S NO permanezca estable **do**
7: $\quad S_i(t+1) = sgn\left(\sum_{j=1}^{N} w_{ij} S_j(t)\right), j \neq i$
8: $\quad t++$
9: **end while**
10: Return $S(t_{final})$

Remarcamos que los pesos sinápticos se precalculan sólo al inicio (la cuenta esa sería hacer el aprendizaje, que en este caso es directo e instantáneo, porque tiene una fórmula cerrada). En el Perceptrón multicapa o en cualquier otra red, esto ocurriría si pudiesemos resolver directamente el proceso de miimización, y no requeriría iterar.

14.1.2 Estabilidad

Sea ξ_i^ν el elemento i del patrón ν. La condición de estabilidad es:

$$h_i^\nu = \sum_j w_{ij} \xi_j^\nu = \frac{1}{N} \sum_j \sum_\mu \xi_i^\mu \xi_j^\mu \xi_j^\nu \tag{14.1}$$

lo cual representa

$$h_i^\nu = \frac{1}{N}\sum_j \sum_\mu \xi_i^\mu \xi_j^\mu \xi_j^\nu$$
$$h_i^\nu = \frac{1}{N}\sum_j \sum_{\mu\neq\nu} \xi_i^\mu \xi_j^\mu \xi_j^\nu + \frac{1}{N}\sum_j \xi_i^\nu \xi_j^\nu \xi_j^\nu$$
$$h_i^\nu = \xi_i^\nu + \frac{1}{N}\sum_j \sum_{\mu\neq\nu} \xi_i^\mu \xi_j^\mu \xi_j^\nu .$$

El sistema entonces es estable si: (1) el segundo término, el **crosstalk** es cero, ó (2) cuando los patrones son ortogonales: esta es una de las limitaciones de esta red.

14.1.3 Convergencia

Hopfield demostró que la red está asociada a una función de Energía, dada en su forma de Lyapunov [4]:

$$H(w) = -\frac{1}{2}\sum_{i,j} w_{ij} S_i S_j \tag{14.2}$$

y que los mínimos locales de esta función son precisamente las localizaciones donde están los patrones.

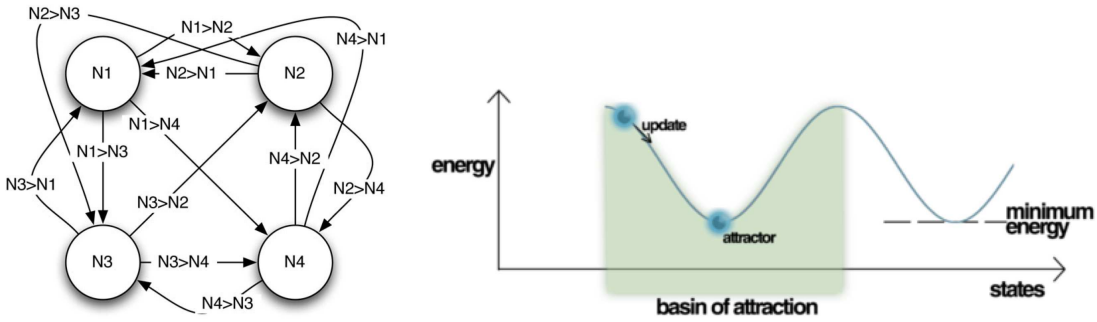


Figura 14.3: Red de Hopfield donde todas las salidas son las entradas de las otras neuronas. El estado de la red puede capturarse con una función de costo/energía que busca minimizarse ante cualquier perturbación (la entrada) y en los valles están los patrones aprendidos.

Las funciones de energía siempre decrecen o permanecen constante cuando el sistema evoluciona. Como los pesos son simétricos y $w_{ii} = 0$ entonces puedo sumar solamente la parte superior de la matriz y multiplicar por 2.

Así mismo, sea S_i' el nuevo valor de $S_i \rightarrow S_i' = sgn(\sum_{j\neq i} w_{ij} S_j)$:

- Si $S_i' = S_i$ entonces la energía no se modifica.
- Si $S_i' = -S_i$ entonces

$$
\begin{aligned}
\Delta H &= H' - H \\
&= -\sum_{j>i} w_{ij} S'_i S_j + \sum_{j>i} w_{ij} S_i S_j \\
&= 2S_i \sum_{j>i} w_{ij} S_j \\
&= 2S_i \underbrace{\sum_{j} w_i j S_j}_{S'_i} < 0
\end{aligned}
$$

porque estamos considerando los S'_i con signo contrario a S_i, y entonces la energía decrece cada vez que un S_i cambia.

14.1.4 Limitaciones

- El número máximo de patrones que puede almacenar es igual al 15 % del número de neuronas de la red. O sea que $p \leq 0.15 * N$, siendo N la dimensión de los patrones.
- Los patrones deben ser más o menos ortogonales (vectorialmente) o puede utilizarse la distancia de Hamming como un evaluador de la similitud/ortogonalidad de los patrones.

La segunda limitación es la existencia de **Estados Espúreos**: como los patrones son atractores, la función de energía H puede tener otros mínimos locales que no son los patrones almacenados. Justamente estos mínimos son estables espúreos que no corresponden particularmente a alguno de los patrones almacenados [3].



15. Modelo de Oja y Sanger

15.1 La búsqueda del poder de síntesis



Figura 15.1: Erkki Oja contento porque está en Aalto.

¿Es la inteligencia la capacidad de sintetizar, de comprimir [87] información? La transformación de los datos por el método de análisis de componentes principales en el capítulo 12 puede verse desde una perspectiva de teoría de la información. Desde ese ángulo la variabilidad intrínseca de los datos puede representar el medio por el cuál se transmite información. Más variación, más información, más importancia y desde esa perspectiva entender cuál es la contribución de cada dato en relación a cuánta variabilidad aporta.

A finales del Siglo 21 varios investigadores, entre los que se encontraron Erkki Oja y Terence Sanger, encontraron que algunos esquemas de redes neuronales simples, con reglas de actualización biomiméticas de los pesos, similares a la regla de actualización Hebbiana, convergían a una representación similar a la obtenida en la descomposición de componentes principales. Es decir, podían encontrar una nueva representación de los datos, donde los ejes estaban determinados según cuánta variabilidad inherente existían en ellos.

¿Será esta una forma de codificar la información apropiada para capturar su esencia, generar una abstracción y de allí un mecanismo de inteligencia?

15.2 Red de Oja

Retomando la ecuación del Perceptrón lineal simple:

$$O^{\mu} = \sum_{i=1}^{n} x_i^{\mu} w_i \tag{15.1}$$

La actualización de los pesos sinápticos surge de

$$\Delta w = \eta(\zeta^{\mu} - O^{\mu})x^{\mu} \tag{15.2}$$

con η como la tasa de aprendizaje, ζ la salida deseada, O la salida real obtenida, y la regla de actualización iterativa sobre los pesos viene de

$$w^{n+1} = w^{n} + \Delta w. \tag{15.3}$$

Esta regla de actualización es supervisada. ¿Hay alguna manera de hacerla no supervisada con algún criterio de optimalidad? La respuesta es que sí, y eso es lo que plantea Oja.

Teorema 15.2.1 — Regla de Aprendizaje Hebbiano para Aprendizaje No Supervisado. Dados m datos de entrada $x^1, ..., x^m$, con $x^{\mu} \in \mathbb{R}^N \forall \mu$, un Perceptrón lineal simple calcula la salida hacia adelante como:

$$O^{\mu} = \sum_{i=1}^{N} w_i x_i^{\mu} \tag{15.4}$$

donde $w \in \mathbb{R}^N \; \forall \; \mu$. La actualización no supervisada de los pesos sinápticos puede hacerse como

$$\Delta w_j = \eta O^{\mu} x_j^{\mu} \tag{15.5}$$

$$w_j^{n+1} = w_j^{n} + \Delta w_j. \tag{15.6}$$

Oja [88] demostró que si un Perceptrón lineal con esta función de actualización converge, entonces el vector de pesos resultante $w(n_{final})$ es una proyección sobre la dirección de máxima variación de los datos, es decir sobre el eje de la **primera componente principal**.

Sin embargo esta regla de actualización Hebbiana diverge. Por ello, Oja propone una actualización donde mantiene a raya a los pesos normalizando cada actualización en relación a la norma del paso anterior:

$$w_j^{n+1} = \frac{w_j^{n} + \eta O^{n} x^{n}}{||w^{n}||} \tag{15.7}$$

teniendo en cuenta que el patrón μ es presentado en el paso n, y con

$$||w^{n}|| = \sum_{j=1}^{N} (w_j^{n} + \eta O^{\mu} x_j^{\mu})^2)^{\frac{1}{2}}. \tag{15.8}$$

Oja estima esta actualización mediante el cálculo del polinomio de Taylor en η

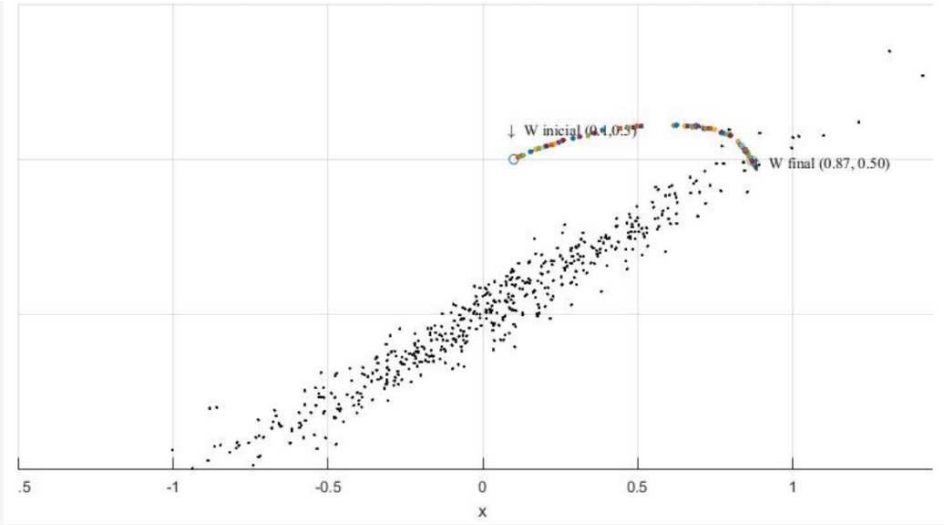


Figura 15.2: Simulación de una neurona lineal con 2 entradas y actualizando w según la regla de Oja. Se ve como el peso se desplaza hacia el eje correspondiente a la mayor variabilidad de los datos.

$$w_j^{n+1} = \frac{w_j^n}{||\mathbf{w}||} + \eta \left(\frac{y(\mathbf{x^n})\mathbf{x_j^n}}{||\mathbf{w}||} - \frac{w_j \sum_{j=1}^{N} y(\mathbf{x^n})\mathbf{x_j^n}\mathbf{w_j}}{||\mathbf{w}||^{\mathbf{3}}} \right)$$

$$\frac{w_j^n}{||\mathbf{w}||} + \eta \left(\frac{y(\mathbf{x^n})\mathbf{x_j}}{||\mathbf{w}||} - \frac{w_j^n y(\mathbf{x^n}) \sum_{\mathbf{j=1}}^{\mathbf{N}} \mathbf{x_j^n}\mathbf{w_j^n}}{||\mathbf{w}||^{\mathbf{3}}} \right)$$

convergiendo a :

$$\Delta w_j^{n+1} = \eta (O\, x_j^n - O^2\, w_j^n) \tag{15.9}$$

Con esta regla, el valor de los pesos se mantiene acotado, no diverge, e incluso converge luego de varias iteraciones al autovector correspondiente al mayor autovalor de la matriz de correlaciones de los datos de entrada, es decir al primer componente principal de la transformación de PCA.

Oja también demostró que para asegurar convergencia se requería $\eta = \frac{1}{1.2\lambda_1}$, siendo λ_1 el primer autovalor principal. Como este valor no se conoce, se puede aproximar con $\eta_0 <= 0.5$ para comenzar, y luego para facilitar la convergencia ir reduciendo este valor, dividiendolo por el paso de iteración del algoritmo (tampoco es buena idea reducirlo drásticamente; la suma de los η utilizados durante todos los pasos tiene que diverger).

15.2.1 Implementación

La arquitectura es un Perceptrón simple con una sola capa de salida. La actualización de los pesos sigue la propia relga de Oja y la inicialización de los pesos se hace con valores uniformes entre 0 y 1. Hiperparámetros adecuados para esta red son $\eta(0) = 0.5$ y

Algoritmo 12 Regla de Actualización de Oja

1: Entrada: N datos con media 0, η y w.
2: **while** época in épocas **do**
3: **while** i iterando de 1 a N (las dimensiones) **do**
4: $s = \tilde{x} \cdot \tilde{w}$
5: $w_i += \eta . s.(x_i - s.w)$
6: **end while**
7: **end while**
8: w Resultado

Así, este algoritmo encuentra los pesos que representan la primer componente principal en base a una regla de actualización biomimética.

15.3 Red de Sanger

Terence Sanger propuso una extensión a la regla de Oja, conformando la Red Hebbiana de Sanger [89], que mediante el aprendizaje Hebbiano converge los pesos a la matriz completa de autovectores de la matriz de covarianza de los datos, por lo que permite encontrar toda la descomposición en componentes principales. Este algoritmo se llama *Algoritmo Hebbiano Generalizado*:

Teorema 15.3.1 — GHA, Generalized Hebbian Algorithm. Asumamos una salida vectorial de la red neuronal con

$$O_j^\mu = \sum_{i=1}^{N} w_{ji} x_i^\mu . \tag{15.10}$$

La regla generalizada hebbiana de Sanger extiende la regla de Oja con

$$w_{il}^{n+1} = w_{il}^n + \eta O_i^n (x_l - \sum_{j=1}^{i} O_j^n w_{jl}^n) \tag{15.11}$$

con $1 \leq i \leq k,\ 1 \leq k \leq N,\ 1 \leq l \leq N,\ \eta \geq 0$

Lo que la regla hace es mantener *a raya* los valores de los pesos restandole todas las componentes encontradas en los pasos anteriores, en todas las coordenadas. Esta regla de actualización se puede aplicar de manera directa en el Algoritmo 12 y permite encontrar la matriz completa de los autovectores principales, que permite encontrar todas las cargas para todos los ejes de la transformación de PCA [89, 86].

V

Deep Learning

16 Deep Brain 135
16.1 Biomimesis neuronal
16.2 La Neurona Recargada

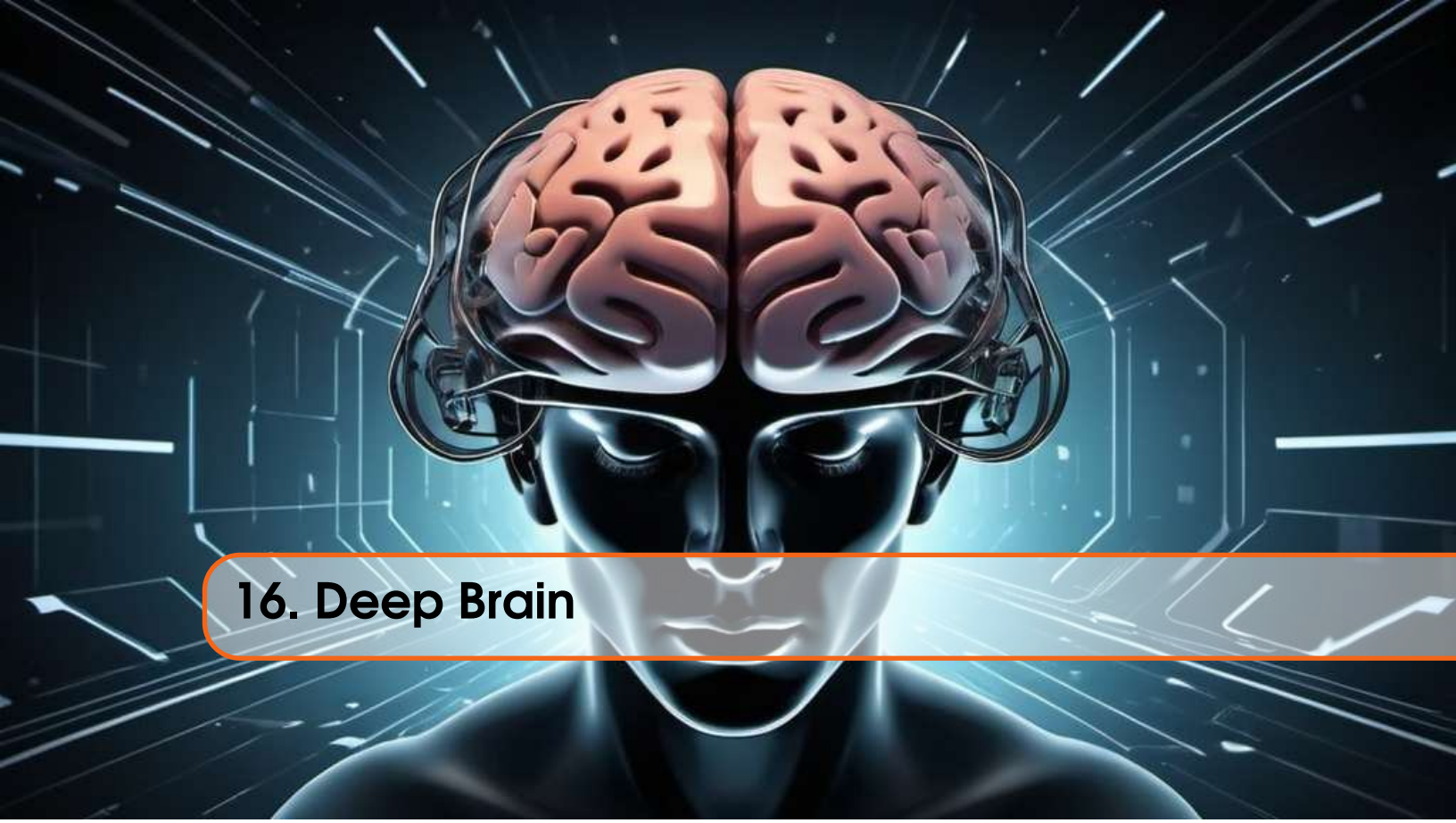
17 Aprendizaje Profundo 145
17.1 Guía para el tío y la tía que pregunta
17.2 Los cuatro pilares de Deep Learning

18 Autoencoders 153
18.1 Autoasociadores
18.2 Autoencoder Generativo
18.3 VAE - Variational Autoencoder
18.4 Apéndice

19 Convolutional Neural Networks 173
19.1 La operación de Convolución
19.2 Convolución en Imágenes
19.3 Neocognitrón
19.4 Capa Convolucional
19.5 La capa de Pooling
19.6 Full Layer y Softmax
19.7 Backpropagation
19.8 ¿Por qué funcionan?
19.9 Inteligibilidad
19.10 Arquitecturas Reusables
19.11 Más allá de las imágenes
19.12 Quiero más

20 Generative Adversarial Networks 187
20.1 Generando gatitos
20.2 Juego MinMax
20.3 Colapso Modal
20.4 El gran simulador

21 Transformers 193
21.1 Las Redes Recurrentes
21.2 Arquitectura de un Transformer
21.3 Attention is all you need
21.4 Embedding
21.5 Entrenamiento



16. Deep Brain

16.1 Biomimesis neuronal

Como se mencionó en el capítulo 5 sobre algoritmos genéticos, la naturaleza implementa un poderoso algoritmo de optimización aplicado sobre la vida en la Tierra, separando lo que sirve para sobrevivir y perpetuarse de lo que no sirve para cada una de las eras particulares. En algún momento, ese algoritmo convergió elitistamente a preservar y promover organismos que fueron desarrollando la capacidad de transmitir y procesar información de manera cada vez más sofisticada mediante los sistemas nerviosos, provocando así el desarrollo de lo que llamamos inteligencia.

Los organismos vertebrados primates, como el ser humano, tienen un sistema nervioso que se divide en dos partes. El CNS, Central Nervous System, el sistema nervioso central y el PNS Peripherial Nervous System, o sistema nervioso periférico. El CNS se encuentra dentro del cráneo en tanto que el periférico se extiende por la columna vertebral y cubre todo el cuerpo, culminando en terminaciones nerviosas que se encargan de ejecutar acción mediante los músculos así como también sensar que es lo que está ocurriendo a nuestro alrededor conformando nuestros sentidos. Con la adición de identificar aspectos posturales mediante la identificación de la elongación y posición relativa de nuestro cuerpo mediante la propriocepción.

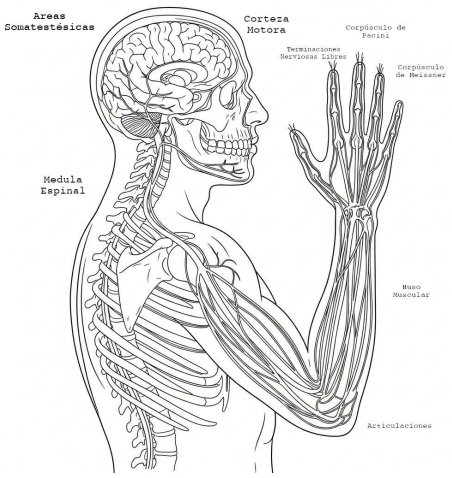


Figura 16.1: El Sistema Nervioso Central y el Sistema Nervioso Periférico.

El sistema conformado se encuentra así dividido en dos subsistemas autonómicos *que llevan la vida adelante* simpático y parasimpático. El primero autonomiza la respuesta de *fight or flight*, pelear o luchar en tanto que el segundo

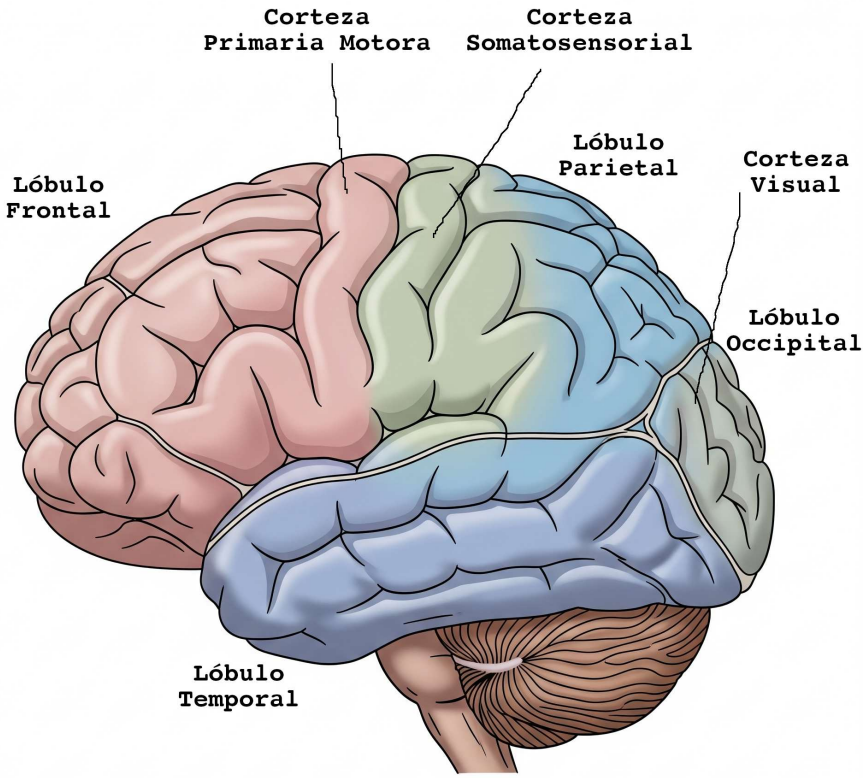


Figura 16.2: Diferente regiones anatómicas y funcionales del cerebro.

regula la homeostasis y el metabolismo resumido en *rest and digest* [90, 91].

16.1.1 Grasa inteligente

El cerebro es físicamente una colección de tejido con alto contenido de ácidos grasos compuesto de materia gris en la corteza, y materia blanca que corresponden a los axones neuronales. Se encuentra embebido en el líquido cerebroespinal, contenido dentro del cráneo. Este líquido es generado en tres ventrículos que se encuentran en el centro del cerebro y es reabsorbido en las meninges, la capa externa al cerebro, en contacto con la corteza, y con la dura-mater, justo en el inicio del cráneo [92].

El cerebro a su vez se divide en diferentes áreas. En la capa exterior está la corteza, el área más nueva del cerebro, más desarrollada comparativamente en los seres humanos. Aquí está la base del comportamiento abstracto. La figura 16.2 muestra las diferentes divisiones de la corteza, muchas de ellas asociadas a rasgos funcionales de cada área [93].

Diferentes hendiduras surcan la anatomía del cerebro, donde se destaca la hendidura central que divide al cerebro en dos hemisferios. El cuerpo calloso es literalmente un manojo de fibras que conectan ambas partes y su grosor es un aspecto distintivo del ser humano, no tan presente ni marcado ni siquiera

en otros primates [94].

De la divisiones en hemisferios y de las divisiones en diferentes secciones funcionales surge el principio del cerebro rígido conexionista, donde cada parte realiza una tarea específica funcional. Un daño en una parte implica, la perdida o compromiso de su rol funcional. Esta idea dominó la neurociencia por muchos años, muy influenciada por el hecho de que el propio estudio y descubrimiento surgía de casos concretos donde una persona tenía comprometida o dañada, por enfermedad o traumatismo, una parte del cerebro, afectando algún aspecto comportamental. Esa correlación sugería que esa zona era la encargada de ese aspecto comprometido. Claros ejemplos de esto son el propio y famoso Phineas Gage y Luis Victor Leborgne [95].

16.1.2 Una neuro-computadora

Podemos hacer una analogía metafórica entre el cerebro y una computadora actual [96] y describir sus diferentes partes desde esa perspectiva.

Firmware

Estructuras internas en el cerebro, más primitivas e histológicamente diferentes, son conformadas por el cerebelo y el pons o puente de Varolio. Estas son las regiones donde se regula principalmente los sistemas autonómicos. Es el **firmware** del cerebro.

GPUs, Graphical Processor Units

La corteza occipital, ubicada justo arriba de la nuca, es la encargada de procesar la información visual, tal como una placa gráfica en un ordenador. Se encuentra conectada a los ojos por el nervio óptico que en el camino atraviesa áreas subcorticales como el tálamo visual, donde actúan mecanismos del sistema parasimpático que disparan por ejemplo reacciones defensivas (esta es la razón por lo que sin pensar cuando nos aplauden al lado pestañamos o realizamos un gesto evasivo, una reacción).

El área occipital es muy relevante desde el lado biomimético porque de su estudio se derivaron muchas de las ideas que son actualmente las bases de diversos mecanismos aplicados en redes neuronales, típicamente las redes convolucionales que se detallan en el capítulo 19.

NLP, Natural Language Processing

Uno de los impulsos más grandes que recibió la neurociencia moderna fue con el descubrimiento de Paul Broca de la corteza lingüística. Esto surgió en base a observaciones de cómo lesiones en esa área afectaban particularmente el funcionamiento normal en el procesamiento y generación del lenguaje [95]. Así entonces, el *área de Broca* así como también el *área de Wernicke* están relacionadas a la capacidad humana de procesamiento y generación del lenguaje natural. Son precisamente por ello, los centros donde se localiza el NLP humano[1].

CPU, Central Processor Unit

La corteza prefrontal es también un área característica evolutiva y comparativamente altamente desarrollada en el ser humano, y concentra el control ejecutivo, social, el centro de las decisiones. Es dónde el **yo** está depositado. Se realiza el control consciente del pensamiento y las acciones de acuerdo a un objetivo consciente interno.

[1] Si bien actualmente las computadoras no tienen un componente físico de NLP, seguramente en los años venideros veremos componentes de NLP embebidos directamente en el hardware, con algún motor de lenguajes interno incorporado, esto como la extensión natural de los TPU.

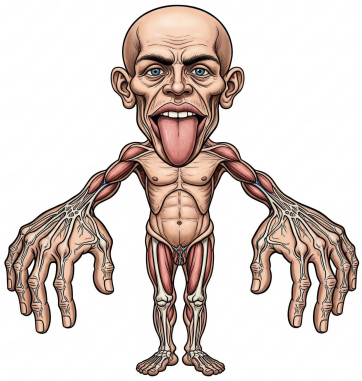


Figura 16.3: Homunculus

El motherboard

Las áreas que se encargan de sensar el tacto de nuestras extremidades, el área somatosensorial, y las regiones encargadas de coordinar nuestros movimientos, las áreas premotoras y motoras, se encuentran una al lado de la otra en la zona temporal y central.

Desde el área central, en la propia hendidura, las diferentes zonas en la corteza de estas tres áreas van mapeando las diferentes partes del cuerpo, en relación a lo que sensan y lo que controlan. Si representamos otra vez en el cuerpo humano, el porcentaje de estas zonas que está destinado a mapear cada una de las partes del cuerpo, podemos visualizar el famoso *homunculus* (Figura 16.3) donde se ve cuanto de cada área de estas zonas se le dedica a cada parte. Por ejemplo, las manos son grandes porque hay una proporción mayor de regiones en estas áreas que se encarga de percibir y controlar las manos.

La RAM

El hipocampo, central en el cerebro, es un área clave en el sueño y en el almacenamiento y la generación de la memoria. Es donde se produce la neurogénesis [97], el nacimiento y especialización de nuevas neuronas.

La fusión sensorial - Tempo Parietal Junction

La unión de la información sensorial se coordina en la unión tempo parietal, el TPJ, que coordina la información recibida por diferentes modalidades: proprioceptiva, visual, vestibular, y las combina para dar una única e integrada idea de la realidad. Esto es equivalente a las técnicas de fusión de sensores [98] que comúnmente se buscan en robótica.

16.2 La Neurona Recargada

Retomando la presentación del modelo de neurona de los capítulos 7 y 8 en este se ofrecen más detalles sobre su complejidad y estructura, que muestran que no es tan simple como allí estaba planteado [99].

16.2.1 ¿Cómo dispara una neurona?

El disparo de la neurona es mucho más complejo que el planteado hasta el momento[100]. Uno

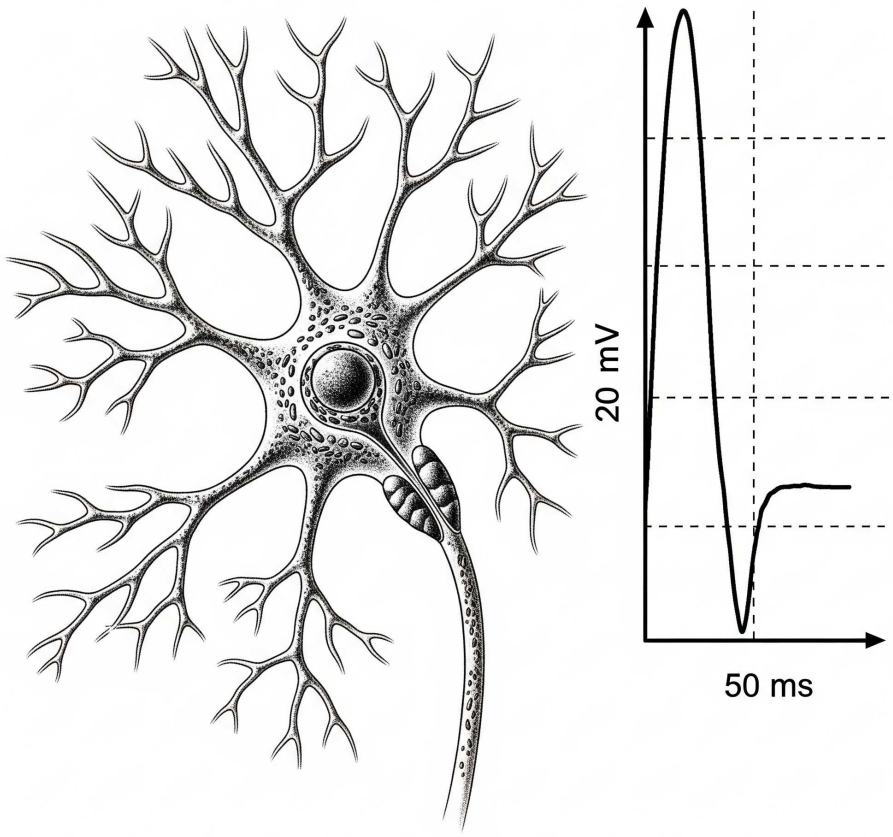


Figura 16.4: Neurona pistolera disparando.

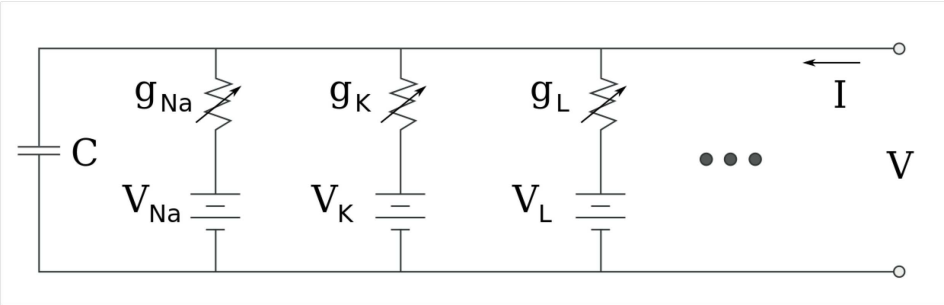


Figura 16.5: Diagrama de Kirchhoff que representa el modelo de Hodgkin-Huxley.

de los modelos más establecidos que intenta capturar esa complejidad fue el que postularon Hodgkin-Huxley [101]. Ellos estudiaron como se comportaban los disparos neuronales en el axón gigante del calamar, y establecieron un modelo dinámico que explicaba el comportamiento como un potencial de acción de equilibrio inestable que es coordinado, de manera activa, por corrientes iónicas.

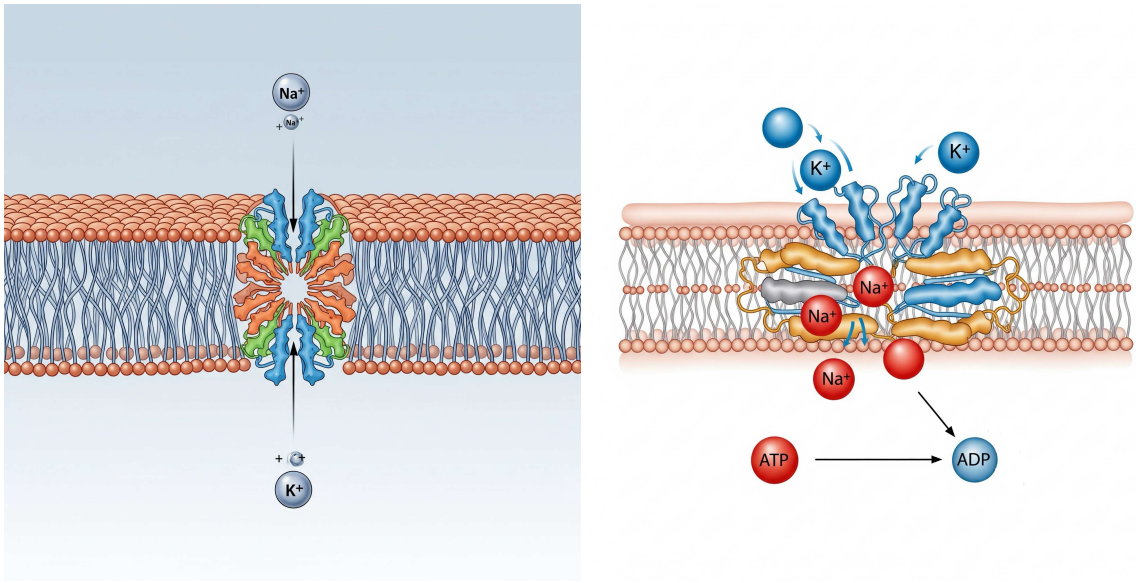
$$I = -(C_m \frac{dV}{dt}) + \frac{Ie}{A}$$
$$I = C_m \frac{dV_m}{dt} + g_K(V_m - V_K) + g_{Na}(V_m - V_{Na}) + g_l(V_m - V_l)$$

Lo que esta ecuación diferencial establece que es que la corriente que circula de un lado al otro de la membrana celular, y la que en definitiva determina la aparición del potencial de acción súbito, es proporcional a la tasa de variación del voltaje en el tiempo, y a la diferencia de voltaje existente entre el voltaje actual y voltaje de referencia de tres referencias distintas. Cada una de estas mediadas por conductancias, el reciproco de la resistencia eléctrica. Experimentalmente, H & H encontraron que cada uno de estos estaba mediado por concentraciones de iones de potasio (K), iones de sodio (Na) y iones de fuga, también llamados *leakage*. Ellos descubrieron que la dinámica del disparo del potencial de acción, estaba regulada por estos iones, átomos cargados eléctricamente por un desbalance de electrones, que se desplazaban de manera transversal a la dirección de propagación del potencial, y que su modelo explicaba el comportamiento todo-nada, generado por la propia dinámica súbita del intercambio de iones.

16.2.2 Una Computadora Nanotecnológica

La neurona mantiene pasiva y activamente una diferencia de potencial entre el interior y el exterior de la célula de alrededor de -90 mV. Esta diferencia viene dada justamente por el filtrado selectivo que hace la membrana plasmática por la cuál los diferentes iones no pueden pasar directamente. Existen canales que son proteínas de membrana (fig 16.6) que actúan como mediadores para mantener este potencial. Los canales de fuga permiten que el sodio+ ingrese en la célula y que el potasio+ salga al exterior, actuando de manera pasiva simplemente por afinidad electroquímica. Por otro lado, la bomba de sodio-potasio, positiviza el interior de la célula, sacando 3 sodio+ e ingresando sólo 2 potasio+, pero lo hace de manera activa consumiendo ATP, la *moneda* de energía biológica.

Estas proteínas de membrana tienen la importante característica que la sensibilidad electroquímica a los iones es dependiente del voltaje. Este aspecto es el que permite que ante un pequeño cambio en el potencial eléctrico, cambia la conformación de estas proteínas, cambia entonces la conductancia y



(a) Canales de fuga (b) Bomba de sodio-potasio

Figura 16.6: Proteínas de membrana que mantienen pasiva y activamente el potencial de membrana.

esto genera una rápida corriente de iones, hacia adentro y hacia afuera de la neurona, que es lo que propaga el potencial de acción, la espiga. Esto se visualiza en la figura 16.8, donde se ve la linea de tiempo de izquierda a derecha, y el comportamiento sucesivo de los dos canales. En el inicio, un pequeño cambio en el potencial de acción de equilibrio en *reposo* (-90 mV) es suficiente para generar una apertura de una compuerta del canal para que comience a ingresar masivamente cationes de sodio que positivizan el interior de la neurona llevando el potencial hacia el cero. Este flujo rápido, provoca la *despolarización*, donde se produce una sobreestimulación que lleva el potencial de membrana a $+40$, lo cual corresponde al pico de la espiga. En paralelo, en los canales de fuga, el cambio en el voltaje cambia la conformación del canal, lo cual abre una compuerta para que comiencen a salir cationes de potasio que negativizan el interior, pero lo hacen con un retardo, son más lentos. Esta acción retardada, es lo que permite que el potencial de acción comience a retornar al valor de equilibrio de -90, que se manifiesta finalmente con el cierre nuevamente de las compuertas en los canales, que retornan toda la situación al estado de equilibrio donde se produce la *repolarización* [99].

16.2.3 Conducción a Saltos

El tamaño de una célula, considerando el soma y las dendritas es del orden de los $20\mu m$. Sin embargo los axones pueden ser largos, muy muy largos, de varios metros. Por ejemplo, salen del cerebro donde vive la neurona, viajan por la médula espina, se conectan a una segunda neurona y llegan a una terminación nerviosa. Es una gran distancia por la que debe viajar la espiga del disparo neuronal. Sin embargo, el potencial de acción que se propaga por la membrana plasmática tiene una baja velocidad de propagación 0.25 $\frac{m}{s}$ y además tiene mucha perdida. Entonces para que los axones de las neuronas puedan propagarse las grandes distancias del cuerpo humano se requiere de un mecanismo para permitir transmitir las espigas. Esto se logra mediante un grupo de células, las células de Schwann o vainas de mielina, que recubren el axón, favorecen su aislamiento, de la misma forma que en ingeniería

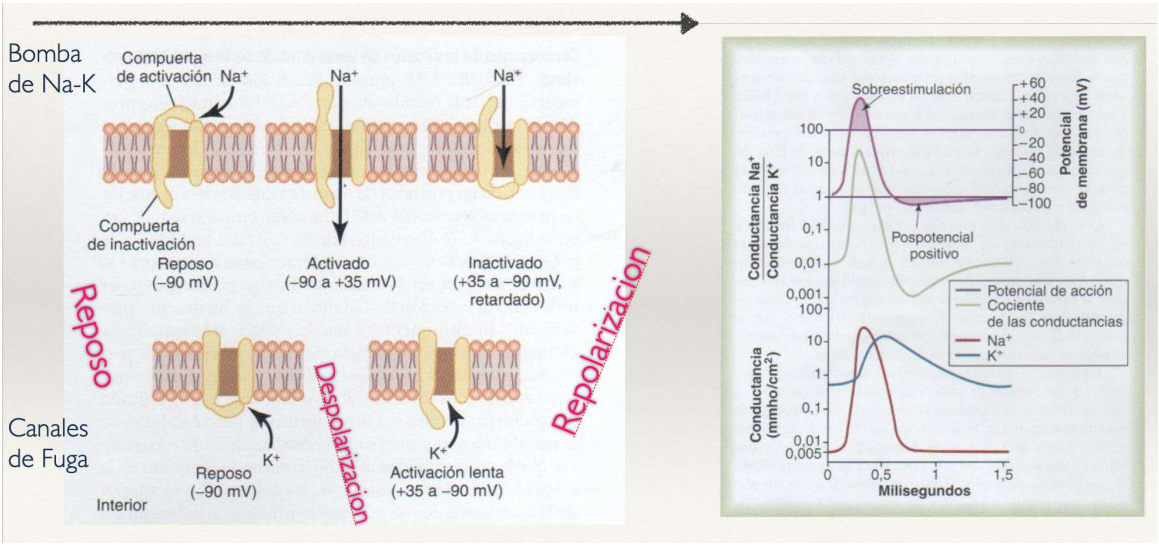
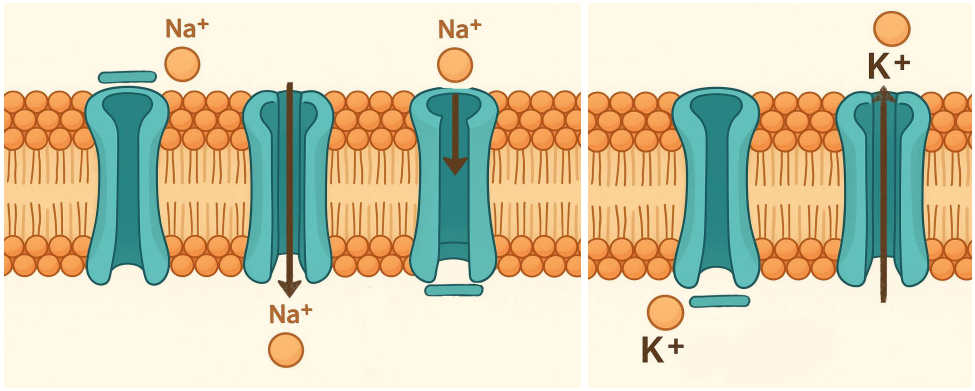


Figura 16.7: Secuencia de intercambio de iones a través de las proteínas de membrana que generan la espiga del disparo neuronal.



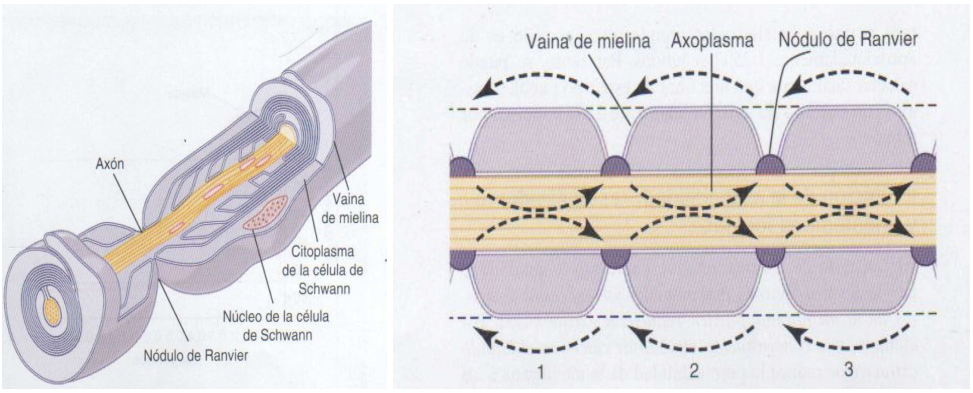
(a) Canales de Sodio (b) Canales de Potasio

Figura 16.8: Secuencia de intercambio de iones a través de las proteínas de membrana que generan la espiga del disparo neuronal.

desarrollamos aislantes plásticos para los cables. Adicionalmente, estas células se acomodan sobre el axón de forma que dejan lugar para unas estructuras bastante equiespaciadas, denominadas Nodos de Ranvier, que actúan como repetidores para regenerar la la espiga con mayor amplitud y así permitir propagarla por mayor distancia.

16.2.4 Sinapsis

Las neuronas buscan conectarse con otras neuronas todo el tiempo. Una neurona que no se conecta con ninguna otra tiende a morirse. El punto de esa conexión es la sinapsis neuronal (Fig. 16.10), donde una neurona presináptica se conecta con una neurona postsináptica.



(a) Células de Schwann (b) Nódulos de Ranvier

Figura 16.9: Conducción Saltatoria mediada por la aislación de las vainas de mielina y por los *repetidores* en los nódulos de Ranvier.

El potencial de acción viaja por el axón hasta el terminal presináptico, donde hay vesículas transmisoras que contienen un neurotransmisor (una proteína). Cuando el potencial llega a la hendidura presináptica, proteínas de membrana desarman las vesículas, liberando el neurotransmisor en la hendidura sináptica (de 200 a 300 angstroms). Hay más de 40 diferentes tipos de neurotransmisores, que al flotar libremente por la hendidura sináptica por afinidad electroquímica se ligan a canales por ligando que están ubicados sobre la membrana de la neurona postsináptica. Al ligarse con el neurotransmisor, estos canales se abren, generan un intercambio iónico y disparan un nuevo potencial de acción que ahora viaja por la neurona postsináptica, propagandolo, o fuerzan un cierre de los canales iónicos provocando un efecto inhibitorio. Los neurotransmisores, con su conformación en tres dimensiones, son gran parte del abanico famacológica de la psiquiatría que se centra en producirlos o producir drogas similares que también pueden operar sobre la sinapsis mediante afinidad electroquímica, cual tetris, con los canales por ligandos. Así en la sinapsis, el mecanismo Hebbiano de aprendizaje se manifiesta mecanísticamente en muchos niveles. De base, el disparo coordinado de la neurona presináptica y postsináptica refuerza la sinapsis, mediante el reclutamiento de más vesículas, mayor actividad mitocondrial (y por ende de producción energética para mantener el potencial de acción) y también más canales por ligando en la neurona postsináptica. Pero no solo se limita a estos cambios directos, sino también a lo que se denomina activaciones por segundo mensajero, que son cambios enzimáticos, activación y transcripción de nuevos genes, o incluso cambios estructurales. Intentar capturar y reducir toda esta complejidad maravillosa en un sólo número es al menos una suposición audaz.

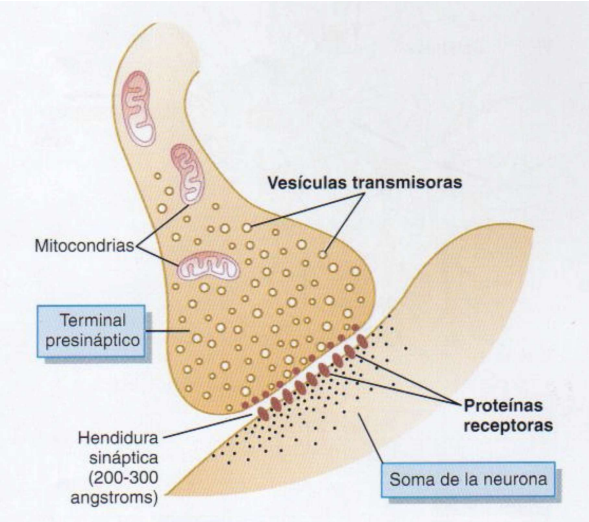


Figura 16.10: La sinápsis neuronal, lo que puede fallar en el final.



17. Aprendizaje Profundo

¿Por qué Deep? ¿Por qué Profundo? La verdadera razón es marketing. Como ocurrió, ocurre y ocurrirá, la tecnología es una piedra angular del mundo actual y necesita tener estrategias de comercialización. Muchas veces, viejas tecnologías reciben nuevos nombres, *buzzwords* o neologísmos, para identificarla, distinguirla de otras, estructurar un mensaje sobre esa palabra, hacerla moda, y vender más en el fondo [102]. A las redes neuronales, una disciplina muy académica, sumida en cierto ostracismo en el inicio del siglo 21[1] le ocurrió exactamente eso y recibió un nuevo nombre, muy marketinero, que combinaba una característica cierta de las redes neuronales artificiales, el esquema jerárquico algomerativo, con un significado fuerte a caballo de una estrategia comercial.

17.1 Guía para el tío y la tía que pregunta

¿Qué es Deep Learning? Es un desarrollo de ciencias de la computación, pero a la vez muy interdisciplinario, con bases biomiméticas que utiliza una estructura jerárquica aglomerativa.

Los siguientes son mitos, extractos literales tomados de diferentes medios, y hasta incluso libros, de conceptos errados en relación a lo que es la Inteligencia Artificial, y en específico Deep Learning:

Los algoritmos de IA no se programan: mito influenciado probablemente por la frase de Arthur Samuel en 1959, ”[Machine learning is the] field of study that gives computers the ability to learn without being explicitly programmed“ [103]. Es decir no se programa el paso a paso de cómo resolver la tárea, sino que se implementa un algoritmo genérico parametrizado, que a su vez incorpora un procedimiento para ajustar esos parámetros. Todo eso, por supuesto, hay que programarlo.

[1] Los ganadores del premio Turing de 2019, Yann LeCun, Geoffrey Hinton y Yoshua Bengio, mencionan que ellos se esforzaban por publicar sus trabajos de redes neuronales disimulados en revistas de Computer Vision, la estrella de esa era, justamente para poder acceder a factores de impacto más alto asociados con las revistas científicas que publicaban esos temas.

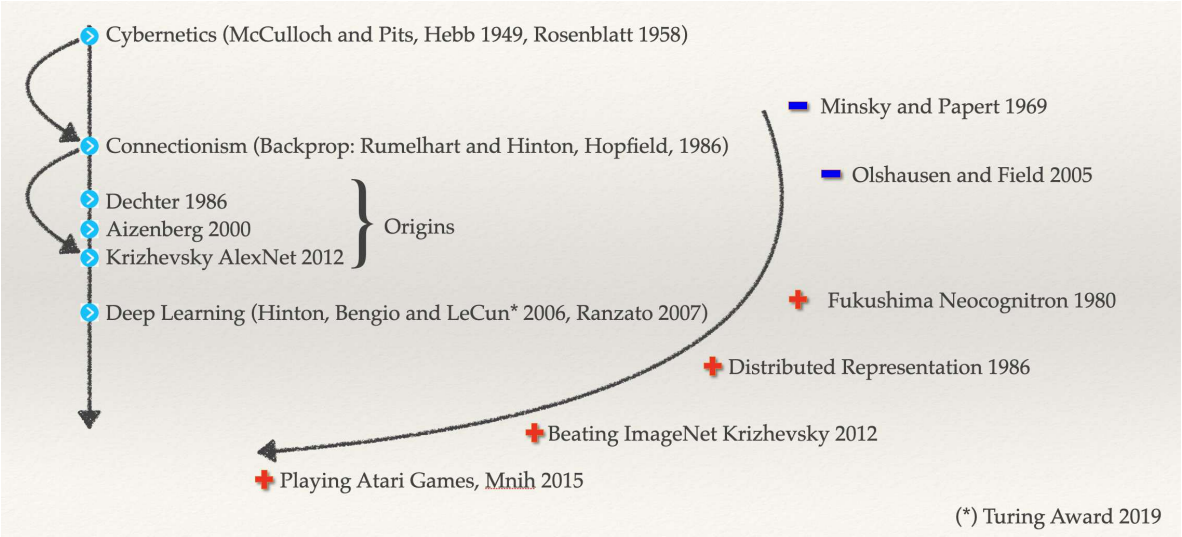


Figura 17.1: Los trabajos que determinaron los distintos veranos de la inteligencia artificial (+) y aquellos que cimentaron los dos anteriores inviernos de la IA (-). La primer era fue la cibernética, la segunda la conexionista, y podemos afirmar que esta tercer es la era de las redes profundas.

Los robots vienen a reemplazarnos: mito por dos razones. La primera es que los avances actuales en IA no implican automáticamente avances similares en robótica; es muy complejo enfrentarse al mundo natural, al mundo físico. La segunda es que la idea del reemplazo es un concepto más filosófico-sociológico y no un concepto tecnológico en sí (ha ocurrido en el pasado y seguirá ocurriendo esta visión de que una tecnología en particular reemplazará a los seres humanos).

La tecnología empezó a reemplazarnos, y más nos vale sacarle provecho que quedarnos amargados por la herida que nos provoque: en línea con el anterior, es el mismo mito del reemplazo, pero este agrega una situación de culpa a los seres humanos, sin sentido alguno.

Los algoritmos tienen además la ventaja que no se cansan y pueden interactuar con el contexto a velocidades supersónicas en comparación con las personas: este mito, además de las imprecisiones que conlleva, es tal porque intenta inútilmente comparar una persona, un ser humano, con un algoritmo. Esto es un sinsentido, con un alto grado de trivialidad y lleva a un debate erróneo.

¿Es está tecnología el advenimiento de las máquinas inteligentes o de los robots que todo lo controlan, o peor, los robots que inician el apocalípsis robot? No, estamos lejísimos aún de esa situación [104]. Generar inteligencia como la que nosotros identificamos en otros seres o en los seres humanos es excesivamente difícil [105].

En concreto es una tecnología revolucionaria que hoy está en el núcleo y centro de la frontera digital y que se caracteriza por haber logrado avance en las siguientes áreas [106]:

- Computer Vision:
- Natural Language Processing:
- Speech Recognition:

Los tremendos avances en esta área están dominando la escena actual, y todo lo que hoy llamamos

AI se centra exclusivamente en aplicaciones en estas áreas, como por ejemplo:

- Control magnético del plasma de un reactor de fusión tokomak mediante aprendizaje por refuerzo profundo [107].
- Descubrimiento de los restos de un barco antiguo hundido que no podía encontrarse [108].
- Desarrollo de nuevas formas más eficientes de multiplicación de matrices [109].
- Alphafold: la red de deep learning que permite predecir la estructura tridimensional, y por ende su funcionalidad, en base a la secuencia de aminoácidos [110].

17.2 Los cuatro pilares de Deep Learning

Puntualizando entonces, el área actual de Deep Learning está caracterizada por cuatro pilares, cuatro componentes claros que en verdad la caracterizan. Estos cuatro pilares de Deep Learning son:

- Deep, Profundidad: Múltiples Niveles de Composición
- Feature Learning: Aprendizaje de la Representación
- Datasets masivos y estandarizados
- GPUs, Graphic Processing Units, hardware especializado para el procesamiento paralelo de multiplicación de matrices.

17.2.1 Profundidad

Los problemas del Shallow Learning

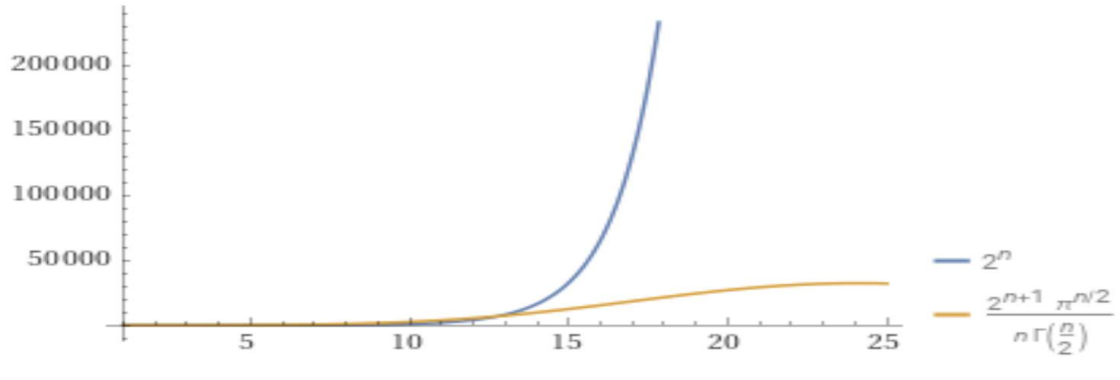
Durante los 90s y entrando en los 2000s, el foco de la inteligencia artificial estaba centralizado en Computer Vision y en algoritmos orientados a datos basados fuertemente en estadística. Esta es el área tradicional de **Aprendizaje Automático**.

Las estrellas de esos años fueron complejos algoritmos que intentaban entender y decodificar de qué se trataba el procesamiento de imágenes y la visión por computadora. Esquemas muy sofisticados se desarrollaron que fueron muy exitosos para problemas pequeños y controlados pero fallaban cuando el problema escalaba o el nivel de ruido crecía [111]. Lo mismo ocurrió inicialmente para resolver el problema del ajedrez, o el procesamiento del habla. Se buscaba entender las particularidades de cada caso, intentando plasmar algorítmicamente cómo los seres humanos creemos que entendemos estos problemas. Sin embargo, este abordaje sólo era válido para demostraciones acotadas y fallaban al escalar o ante problemas más realistas.

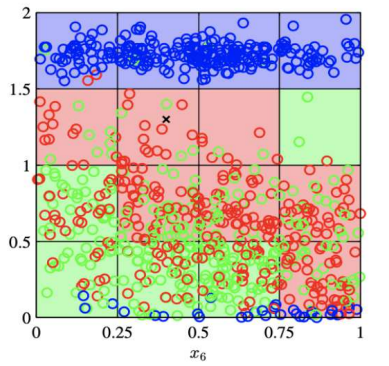
Una de las razones por las cuales *estos métodos tuvieron un éxito limitado* está relacionada con el problema de la alta dimensionalidad. Estos métodos suelen basarse en composiciones de features (ver 3.1) y estos features tienen dimensiones muy altas. Y esto **hackea** nuestra intuición.

La maldición de la alta dimensionalidad

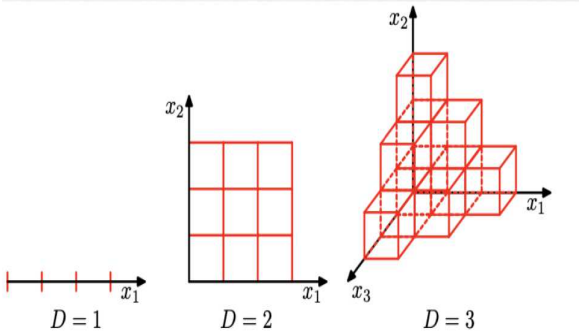
Supongamos que tenemos un circulo circunscripto en un cuadrado. El diametro del circulo coincide con el lado del cuadrado. Si medimos la superficie cubierta por ambos, **intuitivamente**, tendremos la percepción de que son practicamente iguales, y que sólo en los bordes pequeños existe una zona que queda por afuera del círculo pero que es cubierta por el cuadrado. Si lo extendemos a una esfera circunscripta en un cubo, esta percepción se mantiene. Si seguimos extendiendo esto a más dimensiones, vemos que la tendencia es en realidad otra. Esto se puede ver en la figura 17.2a donde se



(a) Hiperesfera



(b) K-NN en 2D



(c) K-NN en varias dimensiones

Figura 17.2: La maldición de la alta dimensionalidad.

grafica el hipervolumen cubierto por el cubo, netamente exponencial, y el hipervolumen cubierto por la hiperesfera, hipergeométrico, ambos en función de la cantidad de dimensiones. No solo se ve que hipervolumen del cubo crece **exponencialmente** de manera absoluta y comparativamente contra el de la hiperesfera sino que además la hiperesfera comienza a **decrecer!** alrededor de la dimensión 21. Esto es un delirio al sentido común, y muestra que la hiperesfera circunscripta en altas dimensiones ocupa una parte infinitesimal del volumen disponible [112].

Asumir que la consistencia local se mantiene en todo el espacio disponible puede no ser cierta en altas dimensiones, ya que los datos existentes son esparsos, pueden estar distribuidos en un hiperespacio totalmente vacío, y generan poca estructura de la cual agarrarse. Esto se ve bien en la figura 17.2b: esto es el típico algoritmo de knn [113]. Cada cuadro en el gráfico corresponde a un color dado por el voto mayoritario de los elementos que ya están ubicados en cada uno. Sin embargo, al extender esta idea a más dimensiones (fig 17.2c), comienzan a construirse hipercubos que generan más y más espacio disponible y en el punto extremo quizás haya sólo uno elemento por cubo o incluso ninguno en absoluto. Esto además enfatiza con claridad que la necesidad de datos para generar la estructura que revela los patrones inherentes crece exponencialmente con la dimensionalidad del feature, imponiendo una enorme limitante a la escalabilidad de estos métodos.

El Problema Fundamental de las Redes Neuronales

Uno de los resultados más fundamentales de redes neuronales, está dado en el teorema 17.2.1. Este teorema probado por Cybenko en 1989 [71] establece que:

Teorema 17.2.1 — UAT - Universal Approximation Theorem. Un MLP con una única capa oculta que contiene un número finito de neuronas puede aproximar a cualquier función continua en un subconjunto compacto de $\mathbb{R}$ dadas funciones de activación apropiadas.

Sea $C(X,\mathbb{R})$ un conjunto de funciones continuas de $\mathbb{R}^n$ a $\mathbb{R}^m$. Sea $\sigma \in C(\mathbb{R},\mathbb{R})$, con $(\sigma \circ x)_i = \sigma(x_i)$ representando la función de activación.
Entonces, para cada $n \in \mathbb{N}, m \in \mathbb{N}$, un conjunto compacto $K \subseteq \mathbb{R}^n, f \in C(K,\mathbb{R}^m), \varepsilon > 0$ existe $k \in \mathbb{N}, A \in \mathbb{R}^{kxn}, b \in \mathbb{R}^k, C \in \mathbb{R}^{mxk}$ de forma tal que

$$\sup_{x \in K} ||f(x) - g(x)|| < \varepsilon \tag{17.1}$$

con $g(x) = C.(\sigma \circ (A.x + b))$

Este resultado implica que eventualmente basta con una red de una sola capa oculta para representar cualquier tipo de función computable[114]. Es un aproximador universal. Esto tiene la ventaja que no requiere aplicar ningún tipo de intuición sobre las particularidades del problema y esquiva, un poco, la problemática de la alta dimensionalidad de los features. Simplemente, con entrenamiento suficiente, debería ser capaz de encontrar los valores de los pesos adecuados y aproximar cualquier cosa.

Pero, si con una red neuronal de una sola capa, puedo aproximar cualquier cosa, ¿qué gano poniendo muchas capas? La ventaja de la profundidad viene dada porque la estructura composicional, jerárquica aglomerativa, ofrece dos beneficios: la invarianza a las traslaciones y la localidad a las diferentes escalas. Esto se visualiza muy bien con las redes convolucionales. La invarianza a las traslaciones representa que sin importar en que lugar de la entrada exista una característica importante en los datos, la naturaleza composicional y aglomerativa de una red que tiene muchas capas, permitirá que esa característica se capture en algún lugar de la jerarquía. El segundo punto, la localidad a diferentes escalas, permite ver de manera indistinta eventos que ocurren a diferentes escalas y capturar sin importar la escala: son invariantes a ella. Esto es **anillo al dedo** para muchos eventos naturales, particularmente en imagenes:

R Many natural questions on images correspond to algorithms that are compositional

Es decir, existen muchos problemas en la naturaleza que presentan la necesidad de tener invariantes en el desplazamiento (en el tiempo o en el espacio) e invariantes en la escala. Un ejemplo de este último son los mapas de las costas, donde sin importar la escala, las costas lucen muy similares.

Adicionalmente, hay una perspectiva más práctica que se presenta y es que las redes neuronales más profundas son en general más fáciles de entrenar de manera efectiva, más eficientes en encontrar los óptimos (más fáciles de optimizar) y generalizan mejor el problema. Es decir, son razones muy pragmáticas [115].

Pero esto no es nuevo. Se sabía que las redes neuronales tenían que ser adecuadas para resolver muchos de los problemas tradicionales del mundo natural, pero algo no cerraba. Puntualmente existía lo que se llamo **El problema fundamental de las redes neuronales** que consistía en lo siguiente: cuando se aplica el algoritmo de backpropagation en muchas capas, los errores que se propagan pueden ser tan chicos que caen por fuera de la resolución computacional, y eso provoca que eventualmente los gradientes se vuelvan cero o exploten. Es decir aparece un problema numérico [116]. Una serie de

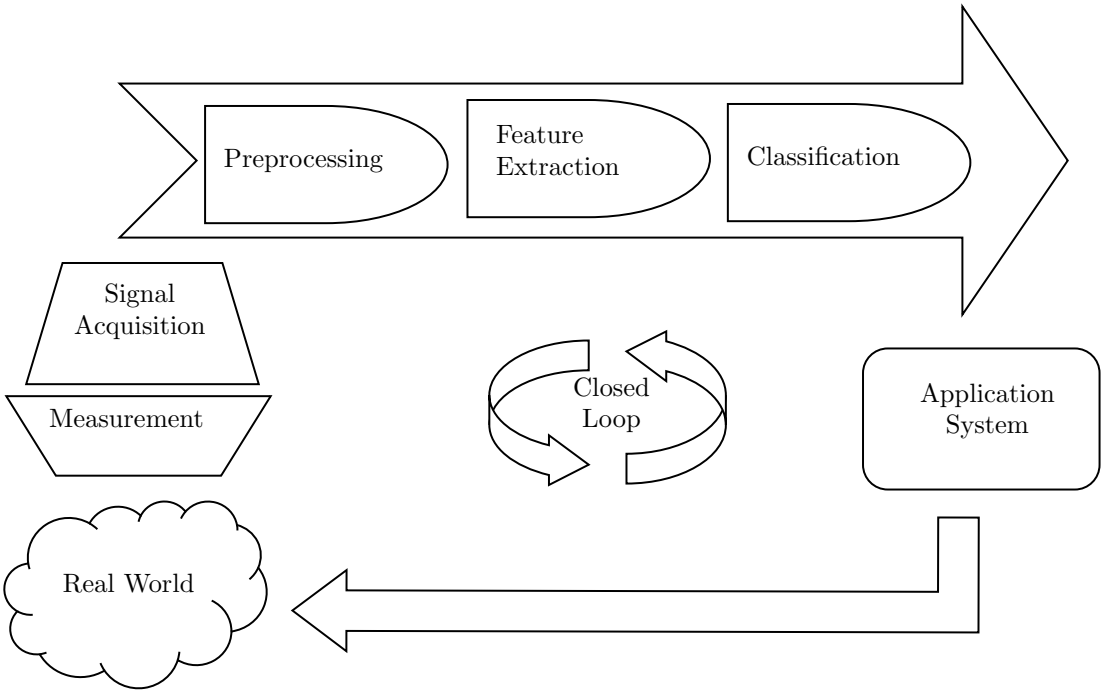


Figura 17.3: Arquitectura de un modelo de procesamiento basado en aprendizaje automático

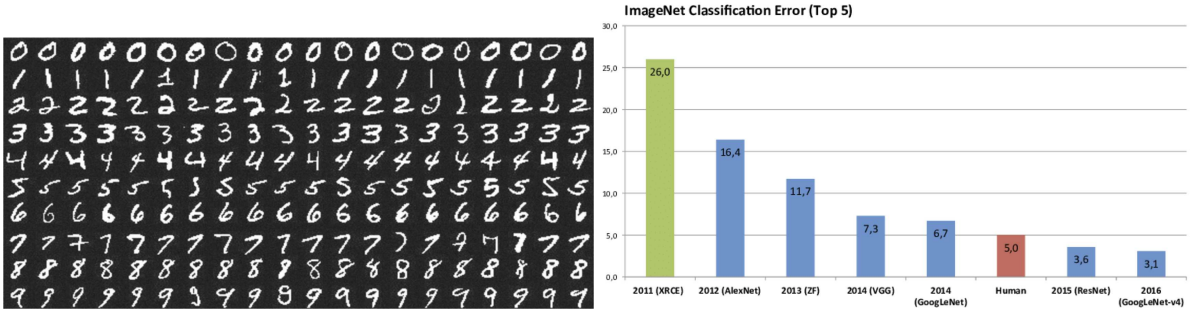
trabajos abordó esta problemática entre los que se encuentra la idea de Unsupervised Pretraining [117] que es donde se acuña el nombre **Deep** que se subió al caballo del hype y que renombró totalmente el área.

Sin embargo, todos estos avances tenían por detrás un problema mayor y era que el hardware necesario para realizar implementaciones escalables y efectivas de redes neuronales profundas simplemente no existía [118].

17.2.2 Aprendizaje de la Representación

Este segundo pilar de Deep Learning se visualiza cuando consideramos el esquema arquitectónico de la figura 17.3 que detalla cómo operan los algoritmos de aprendizaje automático.

Una parte importante de este proceso es la ingeniería de la característica o **feature engineering**. Esto es el diseño del vector que codifica y sintetiza la información más importante que precisamente se necesita para capturar el patrón. Las redes de deep learning tienen la característica que delegan la identificación de cómo tiene que ser ese feature, directamente al propio proceso de aprendizaje, forzando aprender pesos sinápticos que justamente son los que determinan qué feature utilizar y cómo es el mismo. Esto genera, que las redes de deep learning, reciben como entrada directamente la información con algún preprocesamiento pero mayoritariamente cruda, con las mismas variables y dimensiones que naturalmente tiene. La red asume, dentro de todo el proceso de aprendizaje, sintetizar de alguna manera la información guiada por el propio objetivo plasmado en la función de costo. Este esquema se visualiza claramente en las redes convolucionales (capítulo 19).



(a) Dataset de MNIST (b) Tasas de éxito en el challenge de ImageNet

Figura 17.4: Los datasets de MNIST e ImageNet se utilizaron durante muchos años para verificar y probar la calidad de los algoritmos de clasificación.

17.2.3 Datasets

El tercer pilar de Deep Learning tiene que ver con la masiva acumulación de datos provocada por la transformación digital de toda la sociedad. Esta fue iniciada con las computadoras personales, las fiebres de la .COM, el auge de los celulares, las tablets, las apps y ahora la IA. Tenemos datos masivos que prácticamente involucran a todas las sociedades humanas y potencialmente cada persona del mundo, y muchísimos procesos del mundo, son generadores de datos.

Adicionalmente, la existencia de datasets estandarizados, ayudo y promovió la colaboración en el área y el poder trabajar comparando diferentes métodos sobre una métrica común dada por los propios datos.

17.2.4 GPUs

Los procesadores actuales, los CPU, Central Processor Units, están basados en desarrollos y avances de muchos años, ordenados por la ley de Moore. Esta profecía autocumplida planteaba que cada dos años, se iban a duplicar la cantidad de transistores contenidos en un circuito integrado, y con eso se iban a ganar incrementos en la performance y capacidad de los procesadores. Ese crecimiento llegó a su fin, porque se alcanzaron los límites físicos de la miniaturización. En paralelo, la industria de los videojuegos, muy lucrativa, comenzó a necesitar una masiva necesidad de computo en paralelo, y para ello se desarrollaron los GPU, Graphic Processor Unit, con una arquitectura diferente, más simple, pero enfatizando el paralelismo masivo (Figura 17.5). Esto permitía realizar los cálculos de procesamiento gráfico, que requieren multiplicación de matrices en paralelo para mover y rotar poligónos, así como también para las operaciones de geometría proyectiva (más multiplicación de matrices).

En 2012 Hinton se dio cuenta que las operaciones de las redes neuronales podían aprovecharse del hardware de las GPUs que básicamente estaba diseñado para el mismo tipo de operación computacional, multiplicacion paralela de matrices, y decidió junto con su estudiante de doctorado, Alex Krizhevsky usar una playstation para implementar una red neuronal convolucional, AlexNet, e intentar resolver el challenge de ImageNet [119]. Su éxito fue tan rotundo, que el año siguiente todos los participantes del challenge hicieron propuestas basados en el mismo esquema.

Goodfellow lo describe y el texto es excelente en relación a esto [120]:

> R **La industria de los videojuegos financio el hardware necesario para que Deep Learning se convierta hoy en la estrella y motor de la revolución de la inteligencia artificial.**

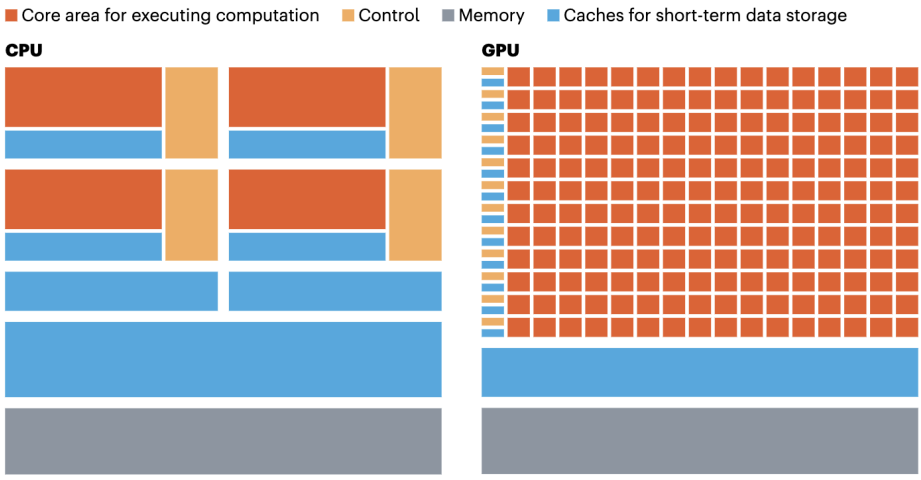


Figura 17.5: Arquitecturas de los CPUs, más complejo y especializada, versús los GPUs más simple y más paralela [119]

Esto se debe a que los GPUs fueron optimizados para realizar multiplicación de matrices de manera paralela y esto es exactamente lo que en definitiva se hace en las redes neuronales. Es decir, las redes neuronales actuales, ganaron la lotería del hardware (o quizás esperaron suficiente) [118].

La contracara de esto es que no debe minimizarse que esta revolución actual existe, porque se le está tirando hardware al problema. Por ejemplo, para entrenar GPT-3, con 175 millones de parámetros, OpenAI generó 10^{23} flops en 1024 GPUs por un mes entero, con un costo de varios millones de dolares.

18. Autoencoders

Las primeras implementaciones de los autoencoders surgieron con el nombre de **Autoasociadores**[121] durante la década del 90. Los autoencoders tienen una arquitectura de red neuronal no supervisada, que se puede utilizar para reducir la dimensionalidad de los datos, hacen compresión, y sobre todo son la base de algunos modelos de redes neuronales generativas. Junto con las GANs son las primeras redes que empiezan a jugar con el concepto de la generación: como **generar** muestras nuevas de un conjunto de datos X.

Surgen a partir de la siguiente premisa: ¿Qué pasa si a la salida final de un MLP es la entrada para otro MLP invertido?

18.1 Autoasociadores

18.1.1 Arquitectura

La arquitectura base esta conformada por dos redes neuronales artificiales de perceptrones multicapas, donde la salida de la primera red se conecta con la entrada de la segunda. La segunda red tiene la distribución invertida de neuronas en las diferentes capas y como salida tiene la misma dimensión que la entrada de la primera red. Lo que se busca en el autoencoder que dada una entrada particular que se presenta en la primera red, se establezcan la combinación de pesos correcta para que la salida que se obtiene de la segunda red sea exactamente igual a la entrada.

Esto es:

$$Z = f(X) = h(XW + b)$$
$$X' = g(Z) = h(ZV^T + p)$$

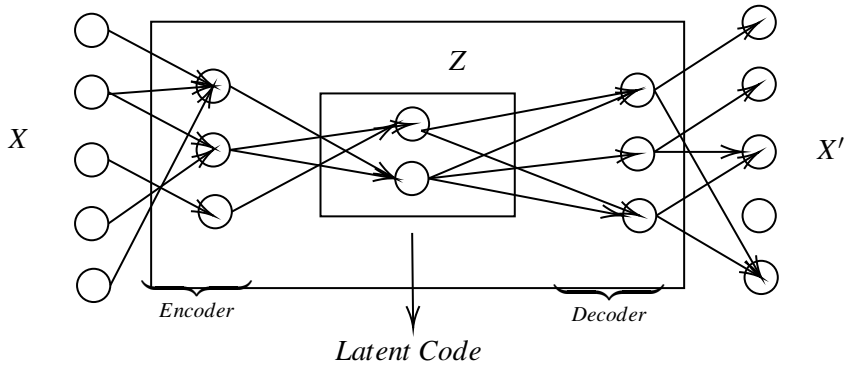


Figura 18.1: Diferentes partes de un Autoencoder.

y esto debe satisfacerse de manera que $L(X,X') = ||X - X'||^2$.

El entrenamiento entonces está dado por cualquier método de optimización que minimice la diferencia entre la salida X' y X, y que converja a los pesos sinápticos que logran mapear eso.

La figura 18.1 muestra la representación de estas dos redes. La primera parte se denomina **Encoder**, en tanto que la segunda es **Decoder**. El punto del medio, donde se encuentra la salida de la primera red con la entrada de la segunda, se denomina **Latent Code**. Una vez entrenado, un dato X en la entrada de la primera red, el codificador, se transforma en un código nuevo en el espacio latente (de la dimensión que este sea), para luego ese valor, entrar en la segunda red, el decodificador, y volver a ser el valor que se presento en la entrada. Esto es, se genera un nuevo código para todos los datos del conjunto X que queda plasmado en el espacio latente.

18.1.2 Autoencoder Lineal

Primero, cabe repasar dos definiciones importantes

Definición 18.1.1 — Decomposición Espectral. Sea X una matriz invertible, con autovalores reales, entonces la matriz puede ser diagonlizada y factorizarse como:

$$X = ELE^T \tag{18.1}$$

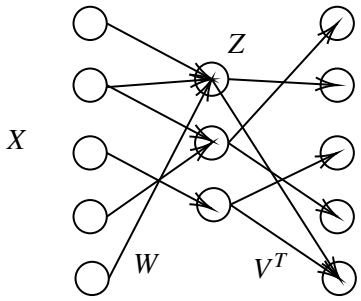
donde L es la matriz diagonal formada por los autovalores y E es la matriz de filas formada por sus autovectores.

Definición 18.1.2 — Decomposición en Valores Singulares. Para el caso de matrices no cuadradas, existe una operación similar pero más general que es la descomposición en valores singulares, SVD. Dada una matriz X de $n \times d$,

$$SVD(X) = \tilde{Z}\Sigma V^T \tag{18.2}$$

donde $\tilde{Z}$ son los vectores columnas singulares de izquierda, Σ es una matriz diagonal positiva y V son los vectores singulares derechos. El cálculo de SVD se realiza minimizando $||X - \tilde{Z}\Sigma V^T||$.

Podemos tomar la definición de Autoencoder pero aplicar solamente funciones de activación identidad (fig **??**). De ahí que la operación de minimización del autoencoder se reduce a la expresada en la ecuación **??**.



Así una vez que el autoencoder aprende minimiza

$$J = ||X - ZV^T|| \tag{18.3}$$

$$X \approx ZV^T \tag{18.4}$$

donde,

- X: La matriz de datos de $n \times d$. Hay n datos de dimensión d.
- Z: La salida de la capa interna del autoencoder, del código de $n \times k$.
- V^T: La matriz de pesos sinápticos asociada al Decodificador de $k \times d$.

Esto entonces nos permite llegar al siguiente teorema[121].

Teorema 18.1.1 — Autoencoder Lineal Converge a PCA. Las salida del código interno Z del autoencoder lineal son las salidas de las proyecciones de los datos en los componentes principales.

$$T_{PCA}(X) = X\,E = Z \tag{18.5}$$

Demostración

Si V^T, que tiene los pesos del decoder del Autoencoder, es ortonormal, entonces $X \approx ZV^T$ del Autoencoder puede verse como la descomposición en valores singulares $X \approx SVD(X) = \tilde{Z}\,\Sigma\,V^T$ con $\tilde{Z}\Sigma = Z$, ya que la descomposición de SVD surge de resolver la minimización de la Ecuación 18.3. A su vez, por la descomposición espectral, con los autovectores de la matriz de covarianza $\frac{X^TX}{n-1}$, se puede obtener $\frac{X^TX}{n-1} = E\,L\,E^T$ donde E es la matriz de transformación de PCA con los autovectores.

Entonces, reemplazando X como la factorización por $X \approx SVD(X) = \tilde{Z}\,\Sigma\,V^T$:

$$\frac{X^TX}{n-1} = (\tilde{Z}\,\Sigma\,V^T)^T(\tilde{Z}\,\Sigma\,V^T)\frac{1}{n-1}$$
$$\frac{X^TX}{n-1} = (V^T)^T\Sigma^T\tilde{Z}^T\tilde{Z}\,\Sigma V^T\,\frac{1}{n-1}$$
$$\frac{X^TX}{n-1} = V\Sigma^T\Sigma V^T\frac{1}{n-1} = V\Sigma^2V^T\,\frac{1}{n-1}$$

donde X es la matriz de covarianza.

$$\frac{X^T X}{n-1} = V\Sigma^2 V^T \; \frac{1}{n-1} = V\left(\frac{\Sigma^2}{n-1}\right) V^T$$

Pero por la descomposición espectral, esto es también igual a $E\ (L)\ E^T = \frac{X^T X}{n-1} = V\left(\frac{\Sigma^2}{n-1}\right) V^T$ por lo que si X tiene media cero,

- $\lambda_i = S_i^2/n-1$, con S_i los valores singulares de Σ.
- $V = E$

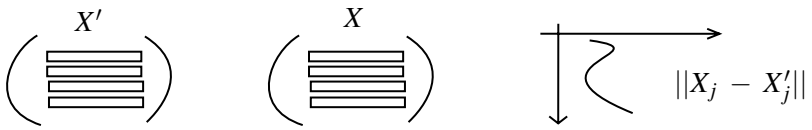
Entonces al transformar los datos con PCA $T_{PCA}(X) = X\ E = (\tilde{Z}\ \Sigma\ \underbrace{V^T)\ E}_{I} = \tilde{Z}\ \Sigma = Z$ que corresponde a la salida del espacio latente.

Esto no es una manera muy eficiente de obtener las proyecciones en los componentes principales, pero lo que permite es pensar al autoencoder como una extensión no lineal de la descomposición en componentes principales (cuando se usan funciones de activación no lineales). Es una asumpción fuerte. El otro punto, es que muestra que la descomposición en valores principales, es aparentemente una forma **canónica** de sintetizar información, donde estructuras que buscan sintetizar la información llegan a la misma solución (ver 15).

18.1.3 Detección de Outliers

Los autoencoders generan una nueva codificación para el conjunto de datos, una codificación más eficiente que intente capturar la escencia de la propia estructura de los patrones internos contenidos en ese conjunto. Por eso son eficientes para detectar outliers, elementos particulares en el conjunto de datos que se salen de la norma.

Supongamos que tenemos un autoencoder entrenado con un conjunto X el cual ya aprendió. Si sometemos como entrada a ese autoencoder nuevos elementos x que no pertenecían al conjunto inicial, se obtiene una salida x', y la diferencia entre ambos $||X - X'||$ debería ser mínima si x tiene una estructura similar a los datos de entrada, y crecer a medida que ese nuevo valor de x difere del resto de los elementos del conjunto. Así, evaluando la diferencia entre la entrada y la salida, es posible tener una métrica que representa cuán *outlier* es cada uno de los nuevos elementos x.



El procedimiento de detección de outliers consiste en:

- Se entrena al autoencoder con el conjunto de entrenamiento.
- Se mide con alguna medida la diferencia entre cada valor de entrada del autoencoder X y los valores obtenidos en la salida X'.

18.1.4 Denoising Autoencoder

El anglicanismo **Denoising** hace referencia a la eliminación del ruído. Como el autoencoder genera una aproximación de la matriz de datos X en base a $X \approx X' = Z\ V^T$ entonces esto permite utilizarse para dos táreas.

Ruido se le llama a perturbaciones sobre los datos que contaminan el contenido de información que se quiere recuperar. Hay muchos tipos diferentes de ruidos y son inherentes a cualquier proceso de obtención de información. Es el mal de todos, imposible de eliminar por completo.

Los Denoising AutoEncoder (DAE) pueden utilizarse para eliminar ruido ya que la estructura interna del Encoder/Decoder intenta preservar el contenido de información más relevante en el conjunto de datos.

La idea entonces es una vez aprendido un conjunto de datos, cuando aparecen nuevas muestras que pueden estar contaminadas con ruido, reemplazar la muestra ruidosa por la muestra obtenida en la salida del Decodificador.

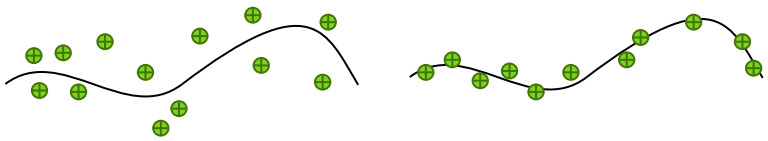
Para esto, en vez de entrenar con los elementos obtenidos directamente del conjunto de datos, se modela el ruido, es decir, se lo caracteriza numéricamente (e.g. salt-and-pepper) o en general con una función de distribución de probabilidad (e.g. Gaussiano, Rayleigh) y se agrega a los datos X generando una nueva instancia $\tilde{X}$. Sin embargo, la salida que se pone en la red corresponde al dato X sin alterar.

18.1.5 Contractive Autoencoder

Los CAE agregan un término regularizador a las funciones de costo, que le resta sensibilidad a la entrada, intentando aprender representaciones más simples y crudas de los datos. El término regularizador en los CAE es la norma de Frobenius de la matriz del Jacobiano.

$$||J_h(X)||_F^2 = \sum_{ij}\left(\frac{\partial h_j(X)}{\partial X_i}^2\right) \tag{18.6}$$

De esa manera por ejemplo, datos en dos dimensiones pueden reducirse a una dimensión, como una representación de una función de dos dimensiones, que pasa a una representación paramétrica (con un solo parámetro t).



18.1.6 SAE - Sparse Autoencoders

Los SAE por otro lado, utilizan una arquitectura donde el espacio latente tiene más dimensiones de la entrada. Para lograr imponer restricciones que permitan adquirir información de la estructura de los datos, imponen un término regularizador que favorece la presencia de esparcicidad, esto es, que un número de neuronas del espacio latente estén en 0 anuladas.

Los SAE se pueden utilizar para realizar **Feature Learning**, es decir para identificar caracterizaciones o transformaciones de los datos que son útiles para discriminarlos.

La Esparcicidad se impone mediante una restricción umbralizada, que de no superarse desactiva todas las neuronas involucradas.

Por ejemplo, los SAE se pueden utilizar para identificar de manera unívoca cuántas codificaciones distintas están siendo capturadas por un autoencoders con códigos binarios que sólo activan una neurona (esto es una técnica utilizada por ejemplo actualmente para entender la codificación que se hace en ChatGPT).

18.2 Autoencoder Generativo

Con un conjunto de datos, se pueden establecer dos tipos de modelos, en relación a qué se quiere hacer con ellos.

- **Modelo Discriminativo**: es crudo en los datos. Por ejemplo, una ANN busca la hiper-sábana que separa los datos en dos clases sin importarle cómo se generan (o como se podrían) generar esos datos.
- **Modelo Generativo**: establece cierta relación causal. El modelo hipotetiza cómo se generan los datos en sí. Busca tener una especie de motor de generación de esos datos.

El modelo discriminativo es el que mayoritariamente utilizamos hasta ahora. Por otro lado, el modelo generativo, plantea la pregunta de dado un conjunto de muestras existentes, cómo puedo hacer para generar nuevas muestras que sean inindistinguibles de las ya existentes.

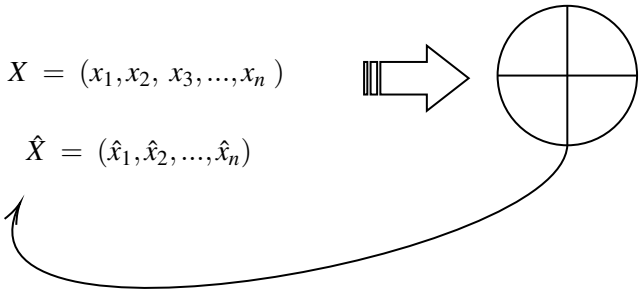


Figura 18.2: El modelo generativo busca entender el proceso de como generar nuevas muestras, inindistinguibles de las originales.

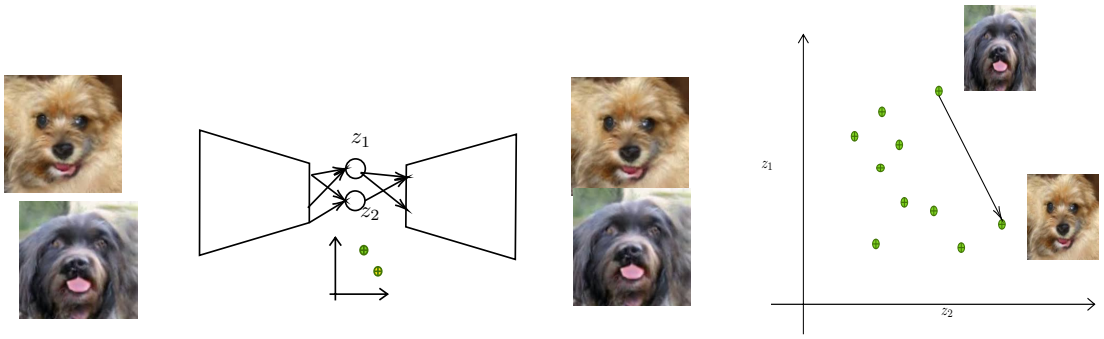
La pregunta que surge entonces es si es posible utilizando una red neuronal, implementar un mecanismo generativo causal, que me permita obtener muestras que comparten características representativas de un conjunto de datos. La respuesta, o una respuesta, es el Autoencoder. Y para eso, podemos hacer uso de la nueva codificación que se establece en el espacio latente Z, y el uso del decodificador, la segunda parte del autoencoder.

Como se ve en la figura 18.3, dos imágenes de perros son codificadas por un autoencoder ya entrenado, generando dos puntos, dos códigos, en el espacio latente Z.

La idea es justamente que al explotar la codificación que se construye en el propio espacio latente, es posible, obtener representaciones nuevas de los elementos X mediante la selección de valores Z que no fueron mapeados de manera directa por ningún X del conjunto de entrenamiento.

El procedimiento entonces para implementar un autoencoder generativo viene dado por:

- Utilizamos un Autoencoder para codificar en el Espacio Latente todos los patrones.
- Luego dejamos de lado el Encoder, para movernos dentro de un vector de representación en el espacio latente de Z.
- Generamos muestras sucesivas especificando de manera directa, por ejemplo en dos dimensiones, los valores de z_1 y z_2.
- Para cada vector de valores de z_i obtenemos una nueva muestra generada por el Decodificador.



(a) Espacio Latente (b) Concept Vector

Figura 18.3: Cada punto representa una correspondencia en el espacio latente con un dato concreto del dataset X. Así viajando de un punto a otro por los valores intermedios, puedo generar nuevos códigos Z intermedios y obtener para cada uno de ellos representaciones de imágenes X' que podrían compartir la estructura de las originales. Estaría generando caras de perritos.

18.2.1 La estructura del espacio latente

Si el autoencoder generativo es totalmente determinístico, solo le es posible mapear estrictamente todos los elementos del conjunto de datos X y en general, de no mediar ningún otro proceso que permite explotar las codificaciones del espacio latente, no tendrá estructura y representaciones intermedias en él, no representaran nada concreto en el conjunto X. Por esta razón, se requiere que el espacio latente tenga una estructura. La figura 18.4 muestra dos situaciones de espacios latentes. Primero uno disperso donde muchos valores intermedios no tienen ninguna representación. El de la derecha por otro lado, muestra una estructura más cohesiva representando que los valores intermedios tienen una representación de los datos.

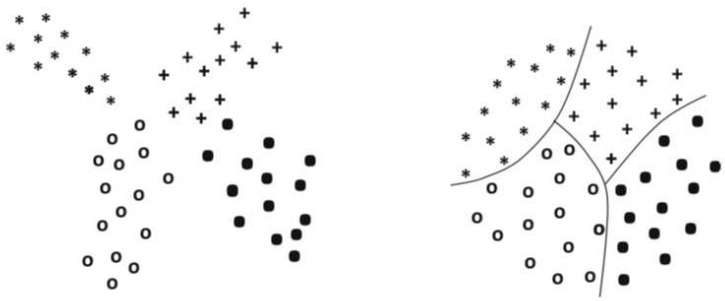


Figura 18.4: A la izquierda un espacio latente sin ningún tipo de estructura donde puntos intermedios no tienen ninguna representación de codificación. A la derecha un espacio latente compacto más cohesivo donde la mayor cantidad de puntos tienen una representación continua.

Necesitamos entonces algo más que nos permita **capturar** lo que pasa en el espacio latente, para poder generar nuevas muestras **válidas** a medida que nos movemos en ese espacio latente.

Una manera de resolver esto, es darle una estructura estadística al espacio latente. Es decir en vez de ver la salida que se obtiene para un único punto de Z, ver las salidas que se obtienen para un conjunto de puntos que son sampleados alrededor de cada una de las representaciones en el espacio latente de los valores de X.

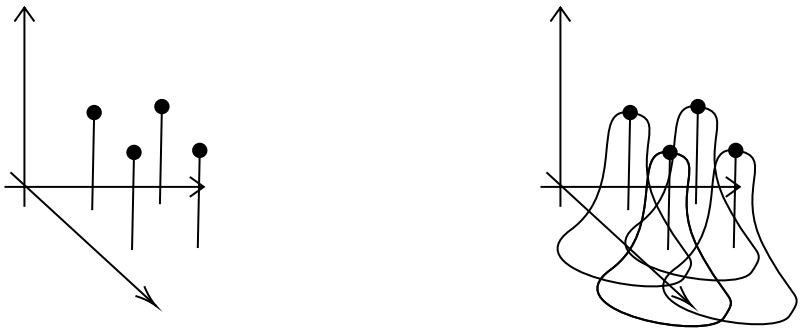


Figura 18.5: En el autoencoder generativo (izq.) cada valor de X tiene una única representación en el espacio latente. En cambio en el autoencoder variacional, se samplean sucesivamente valores alrededor de cada uno de los puntos combinandolos entre ellos, lo que permite que cualquier punto Z tenga alguna representación posible de los valores de X.

18.3 VAE - Variational Autoencoder

Los VAE, los autoencoders variacionales [122] son una convergencia de dos ideas, donde una vez más se verifica que la clave de la innovación está en conectar hechos que solo en apariencia parecen ser inconexos:

- Una propuesta de una red neuronal estocástica, un Perceptrón multicapa estocástico dividido en un Encoder y un Decoder, que conforman un Autoencoder.
- Una estimación variacional de un modelo generativo[123, 124, 125, 126].

Desde un punto de vista más práctica, y programático, lo que vamos a hacer con el VAE, es dar la posibilidad que al encontrarnos un X que usamos como entrada, mapea de manera estocástica a valores de Z distintos, y de esa manera forzar al Decoder a generar una decodificación en X' para ese Z que se parezca al X, lo más posible. Es decir, vamos a estar mapeando los X, que tienen una cardinalidad dada, a un espacio completo de Z, de una cardinalidad mucho mayor.

18.3.1 Redes Neuronales Estocásticas

Una red de una capa oculta es un aproximador universal[127] que puede representar cualquier función computable (17.2.1). Entre otras cosas, puede representar una función estocástica, regida por una p.d.f.

Definición 18.3.1 — Funciones Estocásticas. Es una función donde la asignación de x a y es

estocástica y está regida por una p.d.f. condicional [128].

$$f(x) = L(x) + \varepsilon(x)$$

$y = f(x)$ tal que, dado x, $y = f(x)$ según $p(y/x)$.

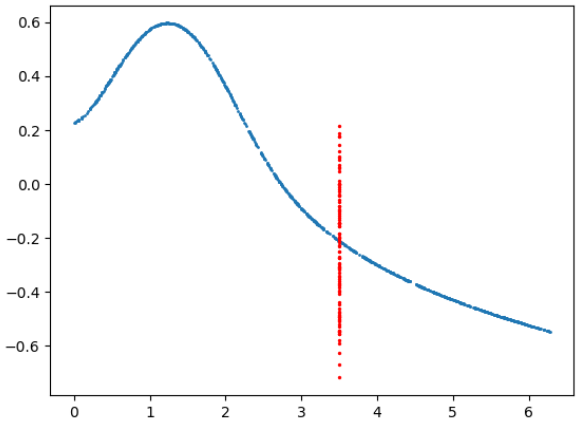


Figura 18.6: Una función seno sin$(\cdot)$ estocástica. En azul se muestran el promedio de los valores generados por la función. En 3.5 en rojo se muestran todas las instancias particulares generadas para ese punto.

Las funciones estocásticas hacen mapeos no determinísticos, como se puede ver en la figura 18.6. Una red neuronal que permite representar una función estocástica se denomina una **Red Neuronal Estocástica** [127]. Para implementarlas se agrega un nodo, una capa donde se genera un número aleatorio pero de una forma que la red pueda seguir retropropagando el error, y para eso se usa el truco de la reparametrización.

El Truco de la Reparametrización

La clave está en el **reparametrization trick** haciendo que el valor de z se calcula en función de $\bar{\mu}$ y de $\bar{\Sigma}$.

$$z = h(\bar{X}) = \varepsilon \odot \bar{\Sigma}(\bar{X}) + \bar{\mu}(\bar{X}) \tag{18.7}$$

El nodo estocástico recibe entonces los parámetros de una distribución de probabilidad, como por ejemplo, la media y el desvío estandard para una normal, para luego con esos parámetros, samplear un valor r que es utilizado con el truco de la reparametrización, para muestrar con esa distribución que tiene esos parámetros. El efecto final, es que la red se convierte en un mapeo estocástico, entre x e y (fig 18.7).

18.3.2 Inferencia Variacional

La segunda idea que confluye en el autoencoder variacional es la inferencia variacional. La teoría variacional sirve para resolver problemas de sistemas complejos en física, intentando determinar una función de distribución de probabilidad que normalmente no tiene una definición cerrada o no es computable hacerlo. Suele resolverse mediante sampleo (Monte-Carlo), o explorando datos del fenómeno.

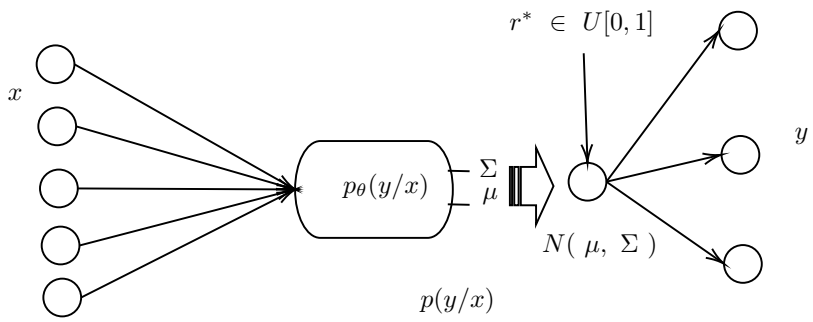


Figura 18.7: Una función $\sin(\cdot)$ estocástica

La conexión es que inferir una p.d.f. tiene cierta semejanza al proceso de aprendizaje automático (Peterson, Anderson 1987 con Mean Field Theory [129], Michael Jordan[1] en 1999 [130] y Hinton, Van Camp en 1993 [131]). Es decir hay una conexión entre aprender una distribución de probabilidad, paramétricamente, que tiene justamente parámetros libres, y la idea de ajustar esos parámetros en base a datos para identificar patrones.

Así en el 2014 Kingma et al [122] plantea el autoencoder variacional, reviviendo los autoasociadores, y planteando por primera vez (hasta donde sabemos) la idea **generativa**. La herramienta central esta dada en la definición 18.3.2.

Definición 18.3.2 — Inferencia Variacional. La inferencia variacional intenta resolver el problema de estimar una función de densidad de probabilidad (p.d.f.) aproximandola con funciones conocidas, y optimizando los parámetros de esa función mediante la solución de un problema de optimización.

Justamente detrás de la inferencia variacional, hay un proceso de optimización y eso se puede resolver mediante la convergencia de la red neuronal.

18.3.3 Optimizar el Autoencoder Variacional

Podemos ahora upgradear el Autoencoder determinístico, utilizando la red estocástica, donde el Encoder de la red lo vamos a usar para representar una función estocástica, $p(z/x)$ para el cuál dado un x de la entrada, la función nos va a generar **algún** z en el espacio latente siguiendo esa p.d.f.. Por otro lado, el del Decoder, vamos a tomar **algún** z del espacio latente, y lo vamos a mandar probabilisticamente devuelta al mundo de x con la función (conjugada a priori) $p(x/z)$.

¿Cuál es la forma de $p(x/z)$ o $p(z/x)$? No sabemos, ni podemos saber nunca, porque dependen de la estructura intrínseca de los datos x. Pero ahora, con la inferencia variacional en el bolsillo, podemos usar el autoencoder para que optimizando la aproximemos por funciones de distribución que sí conocemos, que planteamos nosotros, paramétricas.

Los ingredientes que tenemos, van a ser:

- $p(z/x)$: es la p.d.f. de los datos que desconocemos, y que la vamos a aproximar por $q(z/x)$
- $q(z/x)$: es la p.d.f. que vamos a **plantear** (a gusto) para aproximar $p(z/x)$.
- $q(z)$: vamos a asumir que los datos tienen una distribución normal alrededor de cada z.

[1] No el basquetbolista, el otro.

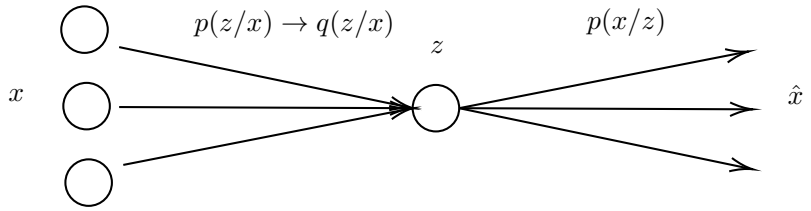


Figura 18.8: El autoencoder variacional y las p.d.f. que mapea.

- $p(z)$: vamos a asumir que efectivamente probabilisticamente cada x mapea multidimensionalmente normal a z. Que sea **normal** es una restricción sobre la $p(\cdot)$. Podría ser otra.

Es necesario sí determinar cuál será la función de optimización. Para eso, podemos arrancar con el objetivo del problema variacional, estimar $p(z/x)$ con $q(z)$ usando la divergencia KL:

$$\text{mín}\, KL(s||t) = -\sum t(x) \log \frac{s(x)}{t(x)} \tag{18.8}$$

$$\begin{aligned} KL(q(z)||p(z/x)) &= -\sum q(z) \log \frac{p(z/x)}{q(z)} \\ &= -\sum q(z) \log \frac{\frac{p(z,x)}{p(x)}}{\frac{q(z)}{1}} \\ &= -\sum q(z) \log \frac{p(z,x)}{q(z)} \frac{1}{p(x)} \\ &= -\sum q(z) \{\log \frac{p(z,x)}{q(z)} - \log p(x)\} \end{aligned}$$

$$\begin{aligned} &KL(q(z)||p(z/x)) \\ &\qquad = -\sum q(z) \{\log \frac{p(z,x)}{q(z)} - \log p(x)\} \\ &\qquad = -\sum_z q(z) \log \frac{p(x,z)}{q(z)} + \sum_z q(z) \log p(x) \\ &\qquad = -\sum_z q(z) \log \frac{p(x,z)}{q(z)} + \log p(x) \underbrace{\sum q(z)}_{1} \end{aligned}$$

Con esto llegamos a que

$$KL(q(z)||p(z/x)) = -\sum_z q(z) \log \frac{p(x,z)}{q(z)} + \log p(x) \tag{18.9}$$

Pero estos términos podemos rearreglarlos pasando la sumatoria al primer miembro y espejando la igualdad.

$$\log p(x) = KL(q(z)||p(z/x)) + \sum_z q(z) \log \frac{p(x,z)}{q(z)} \tag{18.10}$$

Desde esta ecuación, el valor $\log p(x)$ es fijo porque depende solamente de los datos, por lo que no puede ajustarse. Como la divergencia queremos que sea lo más chica posible, esto es equivalente a maximizar el segundo térmimo.

$$\underbrace{\log p(x)}_{\text{fijo para x}} = \underbrace{KL(q(z)||p(z/x))}_{\text{lo más pequeño posible}} + \underbrace{\sum_z q(z) \log \frac{p(x,z)}{q(z)}}_{\mathscr{L}\text{, lo más grande posible}} \tag{18.11}$$

$$\mathscr{L} \leq \log p(x) \tag{18.12}$$

Ese término $\mathscr{L}$ se llama *Variational Lower Bound*, y tiene que ser lo más grande posible y cuando $KL \to 0$ entonces es exactamente el $\log p(x)$. Por lo tanto, minimizar originalmente 18.9 es equivalente a maximizar $\mathscr{L}$.

$$\begin{aligned}
\mathscr{L} &= \sum q(z) \log \frac{p(x,z)}{q(z)} \\
&= \sum q(z) \log \frac{p(x/z)p(z)}{q(z)} \\
&= \sum q(z) \{\log p(x/z) + \log \frac{p(z)}{q(z)}\} \\
&= \sum q(z) \log p(x/z) + \sum q(z) \log \frac{p(z)}{q(z)}
\end{aligned}$$

$$\begin{aligned}
\mathscr{L} &= \sum q(z) \log p(x/z) && + \sum q(z) \log \frac{p(z)}{q(z)} \\
\mathscr{L} &= \mathbb{E}_{q(z)} \log p(x/z) && - KL(q(z)||p(z))
\end{aligned}$$

Esta es la expresión final a la que arribamos como función de optimización que nos va a permitir resolver el problema de optimización, y que justo queda en dos términos, la base más el término regularizador.

Recapitulando
Vamos a usar $p_\theta(z)$, $q_\phi(z)$ para aproximar las características de las muestras del dataset, haciendo

$$\theta^*, \phi^* = \arg\max \mathscr{L}(\theta, \phi, x).$$

De forma que podamos asumir una distribución para $p_\theta(z)$ simple, y elegir otra distribución para $q_\phi(z)$, que determinará la estructura del espacio latente. Con esto aproximamos $p_\theta(z/x)$ con $q_\theta(z/x)$. Al optimizar $\mathscr{L}$ se encontraran los valores para los parámetros de esas distribuciones, ϕ^*, θ^*.

Con esto, podemos asumir que:
- $p_\theta(z) = \mathscr{N}(0, \mathscr{I})$
- $q_\phi(z) = \mathscr{N}(\mu(x), \Sigma(x))$, con $\Sigma(x)$ diagonal.

La primera parte que es el Error de Reconstrucción equivale a:

$$p_\theta(x/z) = \frac{1}{\sqrt{(2\pi)^k}\sqrt{\Sigma(z)}} exp\{(x-\mu(z))^T \Sigma(z)^{-1}(x-\mu(z))\}$$

El *log* de esto es proporcional a la diferencia entre el X de entrada y el $\hat{X}$ de salida.

$$\log p_\theta(x/z) \;\; \alpha \;\; (x - \underbrace{\mu(z)}_{\hat{x}})^T \Sigma(z)^{-1} (x - \underbrace{\mu(z)}_{\hat{x}})$$

Y en relación al término regularizador con $p_\theta(z) = \mathscr{N}(0, \mathscr{I})$,

$$KL(q_\phi(z)) = \mathscr{N}(\mu(x), \Sigma(x)) || \mathscr{N}(0, \mathscr{I}),$$

este luego de varios pasos[2], lleva a la siguiente expresión:

$$\begin{aligned} KL &= \frac{1}{2}\{trace(\Sigma(x)) + \mu^T\mu(x) - k - \log(|\Sigma(x)|)\} \\ &= \frac{1}{2}\{\sum_k \Sigma(x) + \sum_k (\mu(x))^2 - \sum_k 1 - \log(\Pi_k \Sigma(x))\} \\ &= \frac{1}{2}\{\sum_k \Sigma(x) + \sum_k (\mu(x))^2 - \sum_k 1 - \sum_k (\log \Sigma(x))\} \\ &= \frac{1}{2}\sum_k (\Sigma(x) + (\mu(x))^2 - 1 - \log \Sigma(x)). \end{aligned}$$

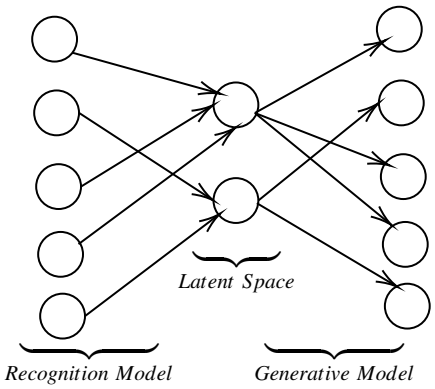
donde por razones numéricas, se reemplaza $\Sigma(x) = \exp(\Sigma(x))$, quedando

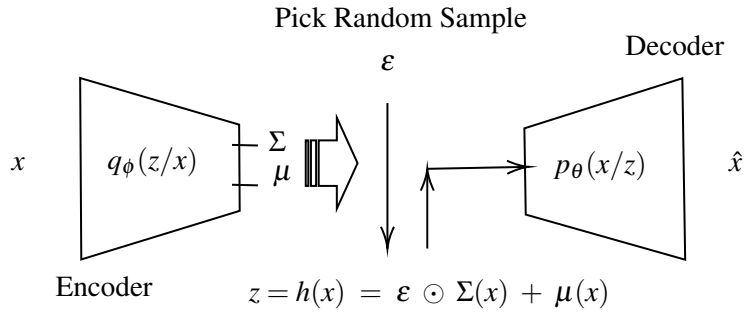
$$KL = \frac{1}{2}\sum_k (\exp \Sigma(x) + (\mu(x))^2 - 1 - \Sigma(x)).$$

Asi la formula final de la función a optimizar es

$$J = L + \lambda R \tag{18.13}$$

[2]En el apéndice 18.4 de este capítulo está el paso a paso.





$$\text{máx}\,\mathscr{L} = \mathbb{E}_{q(z)} \log p(x/z) - KL(q(z)||p(z))$$
$$\text{mín}\,\mathscr{L} = -\mathbb{E}_{q(z)} \log p(x/z) + KL(q(z)||p(z))$$
$$\text{mín}\,\mathscr{L} = ||\bar{X} - \bar{X}'|| + \frac{1}{2}\sum_k (\exp\Sigma(x) + (\mu(x))^2 - 1 - \Sigma(x))$$
$$\text{mín}\,\mathscr{L} = ||\bar{X} - \bar{X}'|| - \frac{1}{2}\sum_k (1 + \Sigma(x) - (\mu(x))^2 - \exp\Sigma(x))$$

Resumiendo, el algoritmo del autoencoder variacional es:

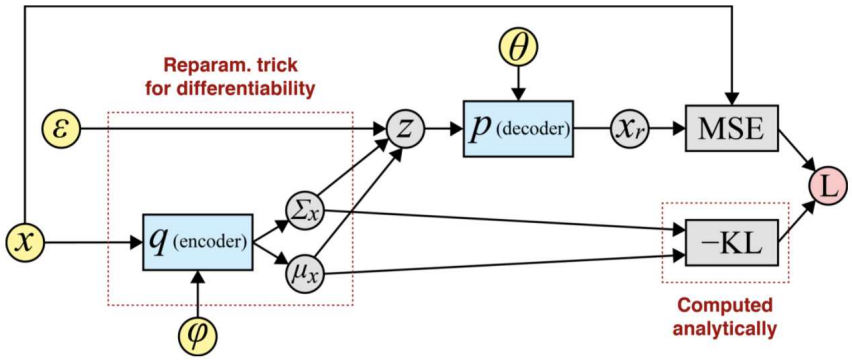
Tomamos un x y lo ponemos en la entrada de la red. Con ese x vemos que valores de Σ y μ obtenemos en la salida del Encoder. Luego sampleamos un z de $p(z) = \mathscr{N}(0, \mathscr{I})$ (esto es equivalente a obtener un ε multiplicarlo por la varianza p.a.p. y sumarle la media).

Con ese z lo pasamos por el Decoder y obtenemos un $\hat{x}$. Entrenamos al Autoencoder ahora para que minimice $\mathscr{L}$ actualizando primeros los pesos del Decoder, luego pasando por la función $h(x)$ y de ahí retropropagando el error obtenido en μ y Σ hacia los pesos del Encoder.

El *Parametrization Trick* de la capa estocástica de la red, permite poder retropropagar el error manejando esa capa que tiene una entrada extra que es una variable aleatoria [132]:

Fijense que el término regularizador, NO depende de la salida del decoder ($\bar{X}$). Depende de variables internas (Σ y μ) que llegan a la capa latente.

¿Cómo propagar[133] el error por el nodo estocástico?



Supongamos que queremos actualizar un peso particular del encoder ω_e en base a la función de costo:

$$J = L + \lambda R$$

Lo que queremos calcular es $\frac{\partial J}{\partial \omega_e}$ para poder usarlo en la

$$\omega_e^{t+1} = \omega_e^t - \eta \frac{\partial J}{\partial \omega_e}$$

$$\begin{aligned}\frac{\partial J}{\partial \omega_e} &= +(\frac{\partial L}{\partial z}\frac{\partial z}{\partial \mu}\frac{\partial \mu}{\partial \omega_e} + \lambda \frac{\partial R}{\partial \mu}\frac{\partial \mu}{\partial \omega_e})... \\ &= +(\frac{\partial L}{\partial z}\frac{\partial z}{\partial \sigma}\frac{\partial \sigma}{\partial \omega_e} + \lambda \frac{\partial R}{\partial \sigma}\frac{\partial \sigma}{\partial \omega_e})\end{aligned}$$

La $\frac{\partial L}{\partial z}$ corresponde a la retropropagación del error en el decoder, en tanto que $\frac{\partial z}{\partial \mu}$ es 1, y $\frac{\partial \mu}{\partial \omega_e}$ es la retropropagación hacia el encoder. $\frac{\partial R}{\partial \mu}$ corresponde a la derivada de Eq. 18.13.

Por otro lado el segundo término corresponde a los cambios x el lado del μ, donde la $\frac{\partial z}{\partial \sigma}$ es ε, la $\frac{\partial R}{\partial \sigma}$ corresponde a la derivada de Eq. 18.13 (ahora con respecto a σ), y la $\frac{\partial \sigma}{\partial \omega_e}$ es la retropropagación hacia el encoder.

Los autoencoder variacionales son hoy centrales en muchos algoritmos y soluciones que buscan generar nuevos datos, generar nuevas muestras y son utilizados como herramienta de **data augmentation**[134]. Se puede encontrar más información sobre Inferencia Variacional [135], Metodos Variacionales [122], VAE [136] y Autoencoders [137, 138, 139].

18.4 Apéndice

Probabilidad y Estadística

$$N(x/\mu,\sigma) = \frac{1}{\sqrt{2\pi\sigma^2}} exp(-\frac{(x-\mu)^2}{2\sigma^2}) \tag{18.14}$$

$$N(\mathbf{x}/\vec{\mu},\Sigma) = \frac{1}{\sqrt{|2\pi\Sigma|}} exp(-\frac{1}{2}(\mathbf{x}-\vec{\mu})^T\Sigma^{-1}(\mathbf{x}-\vec{\mu})) \tag{18.15}$$

Teoría de la Información

Repaso cortito y al pie de las definiciones más importantes de teoría de información.

Variable aleatoria: $X = x_i$

Información: $I = -\log p(x_i)$

Entropía: Se puede interpretar como una medida en la cantidad de información que tienen todos los estados posibles que puede tomar x. Es un promedio ponderado.

$H = \mathbb{E}_{p(x_i)} I(x_i) = -\sum p(x_i)\log p(x_i)$

Información Mutua: $I(x,y) = H(y) - H(y/x) = \sum_x \sum_y p(x,y) log \frac{p(x,y)}{p(x)p(y)}$

$H(x) = I(x,x)$, y se tiene que,

- $I = 0$ implica que $p(x) = 1$ Certeza !
- $I > 0$ algo de información
- $I(x_k) > I(x_i) \Rightarrow P_k < P_i$, lo más raro (menos probable) aporta más información.

La entropía es mínima, cero, cuando es una situación de total certeza (teniendo en cuenta que $\lim_{p\to 0^+} p\log p = 0$), y es máxima, cuando todos los eventos son equiprobables (probabilidad uniforme). Los valores que puede tomar son entonces $0 <= H(x) <= log(2k+1)$.

Entropía condicional: $H(x/y) = H(x,y) - H(y)$

Entropía conjunta: $H(x,y) = \sum_{i=1}^{m}\sum_{j=1}^{m} p(x_i,y_i)\log\frac{1}{p(x_i,y_i)}$

Definición 18.4.1 — Cross Entropy.

$$H_p(q) = -\sum_{k=1} q(y_k)\log p(y_k) \tag{18.16}$$

La definición 18.4.1 se usa cuando hay dos distribuciones en juego y donde la que se usa para determinar la información individual aportada por cada evento es diferente de la que se usa para la ponderación. Esto plantea una comparación desde la perspectiva de la codificación de los datos. Esta definición puede simplificarse para el caso donde hay dos eventos posibles.

Definición 18.4.2 — Binary Cross Entropy.

$$H_p(q) = -(y_i)\log p(y_i) + (1-y_i)\log(1-p(y_i)) \tag{18.17}$$

En el contexto de aprendizaje automático, el promedio de la entropía binaria cruzada se puede utilizar como una función de costo para un problema de clasificación supervisado binario. Ahí justamente aparecen las dos distribuciones, la real que proviene de los datos y es la etiqueta que representa la clase verdadera, y la que produce el clasificador.

Definición 18.4.3 — Average Binary Cross Entropy.

$$H_p(q) = -\frac{1}{N}\sum_{i=1}^{N}\{(y_i)\log p(y_i) + (1-y_i)\log(1-p(y_i))\} \tag{18.18}$$

Aquí, $q(y_i) = y_i = 0|1$, y representa el label, $p(y_i), 1..N$ es la probabilidad asignada a ese label para la muestra k. *Binary Cross Entropy*[3] sirve como una función para estimar cuál es la efectividad de un clasificador que asigna el label y_k a cada elemento $1..N$ con probabilidad $p(y_k)$. Un clasificador perfecto que estima y_k con probabilidad 1, genera un valor de binary cross entropy de cero.

Definición 18.4.4 KL: Kullback-Leibler Divergencia ó Entropía relativa Es una norma (en rigor no) que permite ver la distancia entre distribuciones de probabilidad, i.e. si tiende a cero implica que las distribuciones son parecidas. Una medida de similaridad.

$$KL(q||p) = -\sum_{x_i} q(x) log \frac{q(x)}{p(x)} \tag{18.19}$$

donde $p(x)$ es la referencia a la que se compara $q(x)$.

Propiedades:

- $KL(q||p) \neq KL(p||q)$
- $KL(q||q) = 0$
- $KL(q||p) \geq 0$
- $KL(q||p) = H_p(q) - H(q) \geq 0$

Estimador por Máxima Verosimilitud

MLE, Maximum Likelihood Estimator. Dadas muestras $(x_1, x_2, ..., x_n)$ y una p.d.f. $p(x)$. La pregunta que se intenta abordar es, con las muestras x_i, ¿Para qué valor de ϕ es más probable que la muestra observada x haya ocurrido?

Entonces $L(x_i; \phi) = p(X = x_1)p(X = x_2)...p(X = x_n)$. Se busca entonces maximizar $L(\phi)$ mediante:

$$\frac{dL}{d\phi} = 0 \text{ o } \frac{dlnL}{d\phi} \tag{18.20}$$

Teorema 18.4.1 — Máximo de función monotónica.

$$\hat{\phi} \text{ es MLE de } \phi \Rightarrow f(\hat{\phi}) \text{ es MLE de } f(\phi) \tag{18.21}$$

Regularización

La Regularización reduce los errores de generalización. Es una manera de agregar restricciones a un funcional de optimización para forzar al aprendizaje de algo útil. Se traduce como un término de penalización que se busca que se mantenga pequeño al buscar minimizar una función de costo. Ayuda a no considerar las soluciones complejas o extremas en el propio problema de optimización.

Este método, proviene de la teoría de optimización y se lo conoce como *Tikhonov Regularizacion*. En ML, puede ser L^2-regularized o L^1-regularized. Por ejemplo, en cuadrados mínimos,

$$\min_{\phi} \frac{1}{N} ||y - x\phi||(+\lambda||\phi^2||) \tag{18.22}$$

El método de MLE se transforma en MAP cuando se considera un adicional de regularización. Early Stopping Dropout o Weight decay ($w_{t+1} = w_t - \alpha\Delta_w - \lambda w_t$) son métodos que intrinsecamente involucran una regularización.

[3] Cross Entropy Loss también puede llamarse *Logistic Loss* o *Multinomial Logistic Loss*.

Derivación del Métodos de Cuadrados Mínimos en base a modelos gráficos

Y depende de X y de W.

El objetivo es minimizar $L(w) = \sum_{i=1}^{N} (W^T x_i - y_i)^2$ para encontrar $\hat{w} = \arg\min_w L(w)$

$$P(y,w,x) = P(y/w,x)P(w)P(x) \tag{18.23}$$

Para eso, la idea sería encontrar los valores de w que maximizan la conjunta de tener los y que tenemos pero dados los x que observamos, por lo que a la condicional tenemos que condicionarlo a esos x que podemos observar (los valores x para los cuales queremos encontrar la resolución por cuadrados mínimos).

$$\begin{aligned} P(y,w,x) &= P(y/w,x)P(w)P(x) \\ P(y,w,x/x) &= P(y/w,x,x)P(w/x)P(x/x) \\ P(y,w/x) &= P(y/w,x)P(w)1 \end{aligned} \tag{18.24}$$

Teniendo en cuenta que $P(x/x)$ is 1. Para especificar este modelo, podemos asumir dos cosas, primero que los valores de y dados x y w están todos alrededor de una normal con media, más o menos, $w^T x$, y desvío $\sigma^2 I$ (es decir solo la diagonal de la matriz de covarianza).

Además podemos asumir que los valores de w tienen media cero, y también un desvío estandar diagonal $\omega^2 I$.

Esto es,

$$\begin{aligned} P(y/w,x) &= N(y/w^T x, \sigma^2 I) \\ P(w) &= N(w/0, \gamma^2 I) \end{aligned} \tag{18.25}$$

Asumiendo esto, entonces, $P(y,w/x)$ es

$$\begin{aligned} P(y,w/x) &= P(y/w,x)P(w) \\ &= log(P(y/w,x)P(w)) \\ &= log(P(y/w,x)) + log(P(w)) \\ &= log(exp(-\frac{1}{2}(y-w^T x)^T (\sigma^2)^{-1}(y-w^T x))) + log(exp(-\frac{1}{2}w^T(\gamma^2)^{-1}w^T)) \\ &= -\frac{1}{2\sigma^2}(y-w^T x)^T (y-w^T x) - \frac{1}{2\gamma^2}w^T w \\ &= (-1)[\alpha||y-w^T x||^2 + \delta||w||^2] \end{aligned} \tag{18.26}$$

Maximizar eso es lo mismo que minimizar directamente $||y-w^T x||^2 + \delta||w||^2$. El primer término es entonces el resultado esperado que corresponde a la solución del método de cuadrados minimos para encontrar una aproximación lineal de la relación entre x e y. Pero en este caso, adicionalmente, nos está dando un término regularizador, lo cual determina que de todas las soluciones posibles, aquella donde w es lo más pequeña mejor, parecería ser la que genera la solución más probable (dadas las distribuciones que asumimos).

Inverse Sampling Theorem

Cuando solo se tiene un generador de números pseudoaleatorios con una distribución uniforme $U[0,1]$, típico **rand**, se puede utilizar el teorema 18.4.2 para generar valores con cuál distribución arbitraria.

Teorema 18.4.2 Eligen un valor de Y $U[0,1]$, se fijan que les da en $X = F^{-1}(Y)$. La distribución de esos X va a corresponder con la pdf en cuestión.

$$Y\ U[0,1], F \text{ invertible y } F = CDF(pdf()),$$
$$X = F^{-1}(Y) \text{ tiene } CDF(X) = F$$

$$Y\ U[0,1], F = CDF(pdf()), X = \inf_x\{x : F(x) \geq Y\}$$

Tied Weights

a) Para un valor fijo de V, la matrix W óptima satisface $D^T D(W^T V^T V - V) = 0$.

$$\begin{aligned} DW^T V^T - D &= 0 \\ D(W^T V^T - I) &= 0 \\ D^T D(W^T V^T - I) &= D^T 0 \\ D^T D(W^T V^T - I)V &= D^T 0V \\ D^T D(W^T V^T V - V) &= 0 \end{aligned} \tag{18.27}$$

b) Si D tiene rango d, entonces, $D^T D$ es invertible y no puede ser cero. Por eso en 18.27, no queda otra que $W^T V^T V - V = 0$ lo cual implica que $W^T V^T V = V$.

c) Demostrar que si $W^T V^T V = V$ entonces, $W = (V^T V)^{-1} V^T$ siempre y cuando $V^T V$ sea invertible.

$$\begin{aligned} W^T V^T V &= V \\ W^T (V^T V) &= V \\ W^T (V^T V)(V^T V)^{-1} &= V(V^T V)^{-1} \text{invertivilidad de } V^T V \\ W^T &= V(V^T V)^{-1} \\ (W^T)^T &= (V(V^T V)^{-1})^T \\ W &= (V(V^T V)^{-1})^T \\ W &= ((V^T V)^{-1})^T V^T \\ W &= ((V^T V)^T)^{-1} V^T \text{La T de la inversa, es la inversa de la T} \\ W &= (V^T (V^T)^T)^{-1} V^T \\ W &= (V^T V)^{-1} V^T \textbf{ Listo} \end{aligned} \tag{18.28}$$

d) Ahora que pasa si los pesos del autoencoder son los mismos (a, b y c). Esto se da cuento $W = V^T$ donde las columnas de V son ortonormales.

a)

$$||DW^TV^T - D||||D(W^TW) - D|| \tag{18.29}$$

de ahí surge que

$$\begin{aligned} D^TD(W^TW - D) &= 0 \\ W^TW &= I \end{aligned} \tag{18.30}$$

b)

$$\begin{aligned} W^TV^TV &= V \\ W^TWV &= V \\ V &= V \end{aligned} \tag{18.31}$$

y c()

$$\begin{aligned} W^TV^TV &= V \\ W^T(W)V &= V \\ W^TWV &= V \\ V &= V \end{aligned} \tag{18.32}$$

Si los w_i son ortonormales,

$$w_i^Tw_j = \begin{cases} 0 & j \leq i \\ 1 & j \neq i \end{cases} \tag{18.33}$$

Desarrollo del Término Regularizador del VAE

Demostración. Vamos a pasar de

$$KL(q_\phi(z)) = \mathcal{N}(\mu(x), \Sigma(x))||\mathcal{N}(0, \mathcal{I}),$$

a

$$KL = \frac{1}{2}\{trace(\Sigma(x)) + \mu^T\mu(x) - k - \log(|\Sigma(x)|)\}$$

Sean $P(x) \sim \mathcal{N}(\mu_1, \Sigma_1)$ y $Q(x) \sim \mathcal{N}(\mu_2, \Sigma_2$ con $\mu 1, \mu 2 \in \mathbb{R}^k$ y $\Sigma_1, \Sigma_2 \in \mathbb{R}^{k\times k}$.

■



19. Convolutional Neural Networks

Las redes convolucionales tienen su origen en el trabajo de Hubel y Wiesel de 1962 [70] que les valió el premio Nobel de Medicina de 1981. Ellos detectaron que en la corteza visual de los gatos, había algunas capas donde la cantidad de veces que disparaban las neuronas de esa capa se correspondía con la inclinación de una barra que le mostraban al gato.

Detectaron un **neural coding**, una capa que era sensible a la inclinación de la barra en el campo visual del gato. Este fue uno de los primeros trabajos donde se pudo obtener información de este estilo, disparos neuronales que se correspondían con información perceptiva de una manera simple.

Este trabajo además, sirvió como tremenda inspiración para el área de Computer Vision. Si el cerebro usa un mecanismo que permite detectar inclinación en los objetos en el campo visual, quizás esa sea una buena manera de procesar la información visual y derivar de ella contenido. Esto es lo que captura Fukushima con su propuesta del Neocognitron, muy inspirado en este trabajo y que LeCun, en paralelo, formaliza muy bien desde el lado de procesamiento de señales en su propuesto de las redes convolucionales.

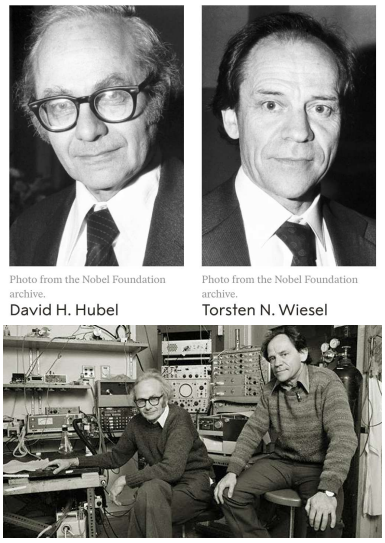


Figura 19.1: Hubel y Wiesel, premios nobel de Medicina de 1968

Las CNN, Convolutional Neural Networks, o redes neuronales convolucionales, o **ConvNet** desde el punto de vista más formal, están basadas en la operación de convolución. Han sido tremendamente exitosas y son uno de los motores detrás del auge de Deep Learning actual. Es más, han sido tan exitosas que nos estamos olvidando de que están, como ocurre con todas las muy buenas tecnologías.

19.1 La operación de Convolución

Para poder entender redes convolucionales es importante primero definir y entender la operación de convolución, que surge como la definición 19.1.1.

Definición 19.1.1 — Convolución. La operación de convolución es una operación lineal que se da sobre una señal, es decir una función unidimensional en el tiempo.

- Suponiendo que $x(t)$ es una señal unidimensional en función del tiempo y $w(t)$ es un **núcleo** de convolución,
- $s(t) = \int x(k)w(t-k)dk$
- $s(t) = (x*w)(t) =< x(k), w^*(t-k) >$, donde $*$ es el conjugado.
- (Correlación $s(t) = \int x(k)w(t+k)dk$ (+ positivo))
- Discreta: $s[n] = \sum_k x[k]w[n-k]$

La convolución matemática cumple un rol primordial en el procesamiento de señales digitales. Por ejemplo, cualquier filtro digital lineal puede representarse por una igualdad basada en convoluciones discretas.

Definición 19.1.2 CCDE - Constant Coefficient Difference Equation

$\sum_{k=0}^{N-1} w_1[k]y[n-k] = \sum_{k=0}^{M-1} w_2[k]x[n-k]$

- $Y(z) = H(z)X(z)$
- w_1, w_2 son kernels de convolución, x, y son señales discretas.
- X, Y son los estados del sistema y H es la función de transferencia.

19.1.1 Moving Average

Una manera interesante de entender la convolución es verla como una extensión del promedio ponderado en una señal. Imaginemos que hay una señal, una función donde el eje X es el tiempo t. Discretamente, para cada punto de t hay un valor en la señal, como podría ser el precio de bitcoin. Si queremos ver la tendencia de la información, sin perdernos en las variaciones diarias, lo que podemos hacer es para cada día, reemplazar el dato concreto del precio, por el promedio de los 5 días anteriores, hoy, y los 4 días poteriores. Eso va a suavizar la señal de precios, y nos va a permitir ver la tendencia sin importar tanto las variaciones diarias.

El siguiente código por ejemplo, es una implementación de esta operación, que no es ni más ni menos que una convolución de una señal con un núcleo de convolución que está compuesto por valores correspondientes a una distribución uniforme para generar el efecto del promedio.

```
windowlength = 10
avgeeg = np.convolve(signal,
     np.ones((windowlength,))/windowlength,
           mode='same')
```

El kernel de convolución puede tener cualquier tipo de longitud, y justamente representa cuántos valores alrededor del valor central, van a utilizarse para hacer el promedio (en este caso). Por supuesto, pueden usarse otros valores para el kernel que generan un efecto totalmente diferente.

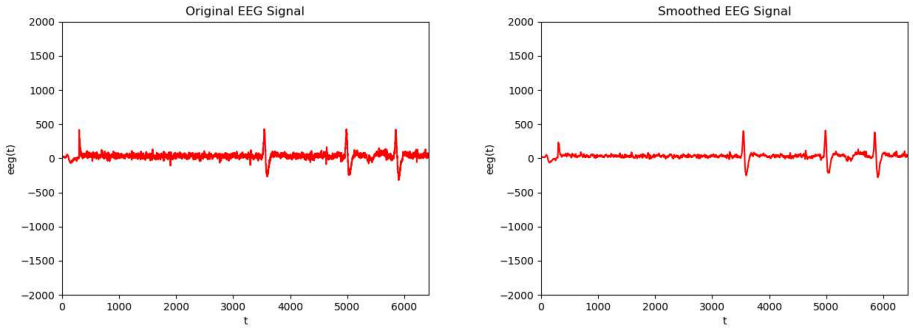


Figura 19.2: A la izquierda una señal de un sensor de electrofisiología que es suavizada con un filtro convolucional del moving average.

Esto por ejemplo, se puede utilizar para procesar una señal de otro tipo, como información de sensores, como se ve en la figura 19.2, donde la aplicación del moving average, genera un suavizado en la señal que permite visualizar mejor los eventos más significativo y ocultar el ruído.

19.1.2 Padding

El proceso de convolución requiere tener un cuidado especial con los bordes. Como el soporte de la señal es finito y requiere alinearse con el núcleo, hay que manejar que pasa en los borde, en relación a como se alinean la señal y el núcleo. Hay tres maneras de realizarlo. La primera es **valid** e implica alinear los valores del kernel con los valores existentes de la señal. Al estar haciendo un promedio, esto implica que el tamaño del resultado de esta operación, tiene menos cantidad de elementos (se reduce el tamaño del soporte en la salida). El segundo es **same** que corresponde a la situación donde se **paddea**, se agregan, a la señal original con dos ceros, uno de cada lado, y con eso permite que la señal de salida tenga exactamente la misma longitud que la de entrada, manteniendo el tamaño (por eso same). Finalmente, la tercera opción **full** implica agregar dos ceros de padding de cada lado, para que la señal generada incremente su tamaño.

Por ejemplo, con una señal discreta [1,2,-5,4,2,-1], con un kernel [-1,2,-1], se obtienen los siguientes resultados con cada esquema de padding:

- Valid: [8, -16, 11, 1]
- Same: [0, 8, -16, 11, 1, -4]
- Full: [-1,0, 8, -16, 11, 1, -4,1]

El esquema de padding determina entonces cómo es la reacción de la operación a las condiciones de borde, y el tamaño de la señal de salida.

19.2 Convolución en Imágenes

La operación de filtrado básica en Visión por Computadora es mediante la aplicación de una operación de convolución bidimensional con diferentes kernels de convolución. En el caso de una imagen hay dos índices por los que se mueve la operación, el alto y el ancho, pero intrínsecamente es la misma operación.

$$S(i,j) = (I*k)(i,j) = \sum_m \sum_n I(m,n)K(i-m, j-n)$$
$$= \sum_m \sum_n I(i-m, j-n)K(m,n)$$

En Visión por Computadora o Análisis y Tratamiento de imágenes, es necesario aplicar 'filtros' a las imágenes para resaltar ciertas características. Estos filtros se basan en la idea de componer, a mano, mediante **feature engineering**, un kernel de convolución que genere lo que se quiere resaltar y con eso podés identificar información en la imágen. Por ejemplo, para detectar bordes horizontales o verticales, como muestra la figura 19.3, se usan filtros que enfatizan los cambios repentinos en los valores de los píxeles en la dirección horizontal o vertical.

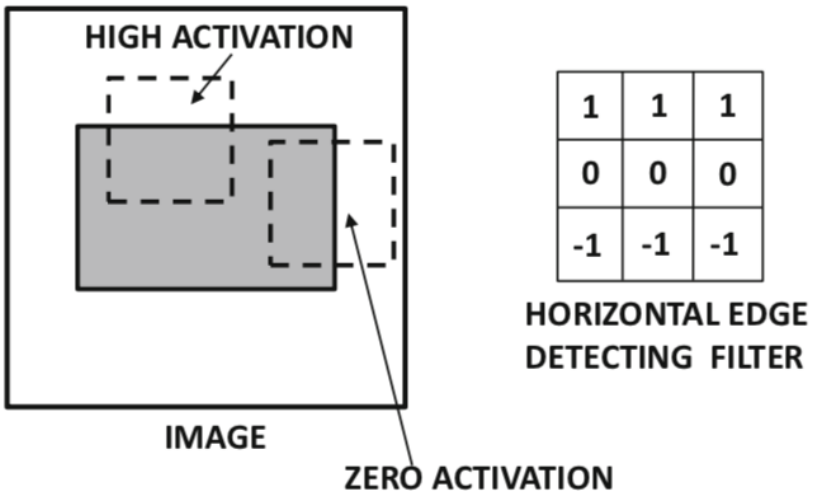


Figura 19.3: Filtros convolucionales para detectar bordes.

Los filtros de convolución de 3×3 que permiten detectar estos bordes son matrices como las siguientes:

$$S_x = \begin{pmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{pmatrix}, S_y = \begin{pmatrix} 1 & 1 & 1 \\ 0 & 0 & 0 \\ -1 & -1 & -1 \end{pmatrix}$$

Al aplicar la operación de convolución con este kernel a cada uno de los pixeles de una imagen (seguido por una operación de umbralización) se van obteniendo imágenes de salida donde se enfatizan los bordes en una dirección, en otra o en ambas.

Durante varios años, el área de Computer Vision se basaba en algoritmos, que con mucho conocimiento sobre la dinámica de la generación de imágenes visuales, proponían diferentes kernels de convolución, creados a mano, que generaban salidas específicas que servían para identificar diferentes patrones en las imágenes

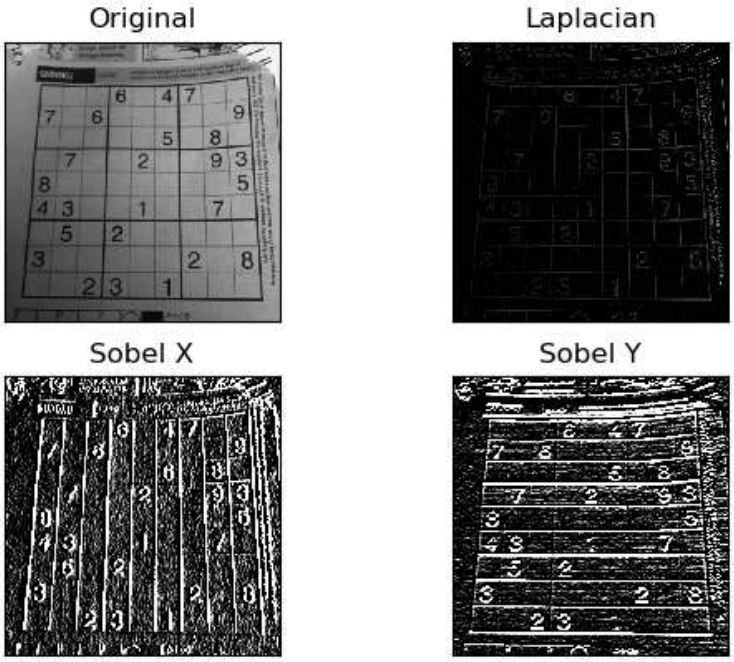


Figura 19.4: Resultado de aplicar la operación de convolución con diferentes filtros que provocan que se resalten los bordes horizontales o verticales. El Laplaciano es el resultado de sumar las derivadas segundas de los valores de la imagen, con lo cual se identifican al mismo tiempo tanto los bordes horizontales como los verticales.

19.3 Neocognitrón

Los sistemas nerviosos biológicos son excelentes identificando de manera rápida y en tiempo real patrones visuales.



Figura 19.5: Kunihiko Fukushima, el creador del Neocognitrón.

En el trabajo de Wiesel y Hubel, ellos descubrieron que la tasa de los disparos de las neuronas de diferentes capas de la corteza visual de los gatos, estaba correlacionada con la orientación, con el ángulo en el que inclinaban una barra que presentaban en el espacio visual del gato. Para ángulos cercanos a 45 grados, la tasa era mayor. Al entender que la corteza visual, responsable de esta increíble acción ingenieril, está compuesta de capas jerárquicas que son sensibles a orientaciones en las imágenes predominantes en el campo visual, estas ideas cimentaron la posibilidad de encontrar una estructura computacional que pudiese artificializarla. En 1989, Kunihiko Fukushima, quien se había sumado a un grupo de investigación en la NHK[1],[2] que estaba compuesto de manera interdisciplinaria, se enteró de los avances en neurociencia,

[1]La BBC[2] nipona.

[2]La TV Pública inglesa.

del trabajo de Wiesel y Hubel y materializó la ida del Neocognitrón, con un fin práctico concreto de identificar caracteres en imágenes, un sistema de OCR.

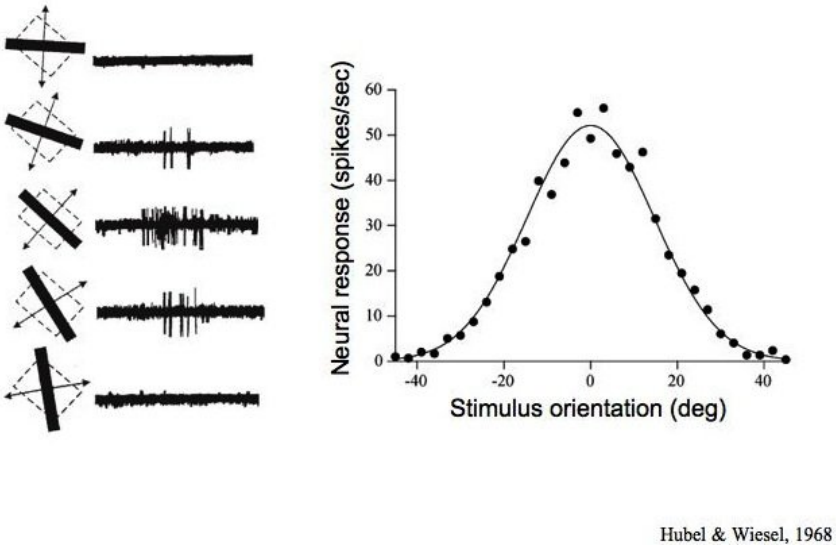


Figura 19.6: En el experimento de Hubel y Wisel, la tasa de disparos neuronales en las diferentes capas de la corteza visual del gato se correlacionan con la inclinación de la barra que le muestran al minino.

En las propuestas del Neocognitrón, y en su formalización matemática por LeCun de manera independiente, se establece la idea de lo que son las redes convolucionales actuales. El quid de la cuestión es modificar un MLP de forma que realice una operación de convolución, de una capa a la otra, en vez de una operación de sumatoria de todas las entradas con pesos diferentes, de forma que la operación además actúe como filtro. Esto permite imitar la misma idea de lo que se observa en la corteza visual, mediante la utilización de la convolución como operación. .

La figura 19.7 muestra la estructura general de una red convolucional, similar a la propuesta por Fukushima/LeCun. La entrada es procesada por capas de convolución donde se aplica la susodicha operación, luego se aplica la operación de *pooling* para finalmente procesarse normalmente por un Perceptrón multicapa. La visión de redes neuronales es que los filtros tradicionales que se usan en visión por computadora son versiones shallow de redes neuronales convolucionales, y en la profundidad reside justamente uno de los éxitos de esta alternativa[140].

19.4 Capa Convolucional

La capa de convolución es central, y sirve entenderla como una operación similar a la del Perceptrón, pero donde los pesos utilizados son compartidos **espacialmente**. Los diferentes componentes se pueden visualizar en la fig 19.8 y en la ecuación 19.1. La operación en cuestión es de la capa q a la capa $q+1$.

$$W^{p,q} = [w_{ijk}^{(p,q)}], h_{ijp}^{(q+1)} = \sum_{r=1}^{Fq} \sum_{s=1}^{Fq} \sum_{k=1}^{dq} w_{rsk}^{(p,q)} h_{i+r-1,j+s-1,k}^{q} \tag{19.1}$$

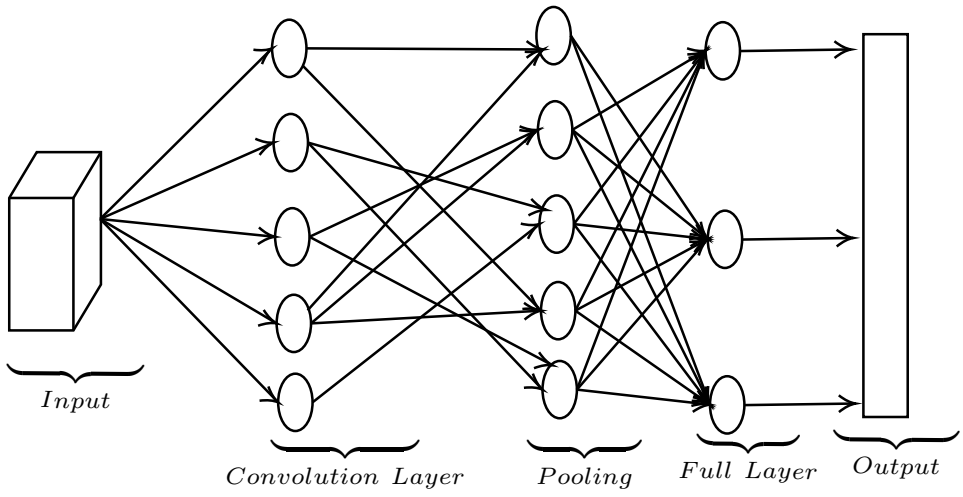


Figura 19.7: Estructura general de una red convolucional con sus diferentes capas.

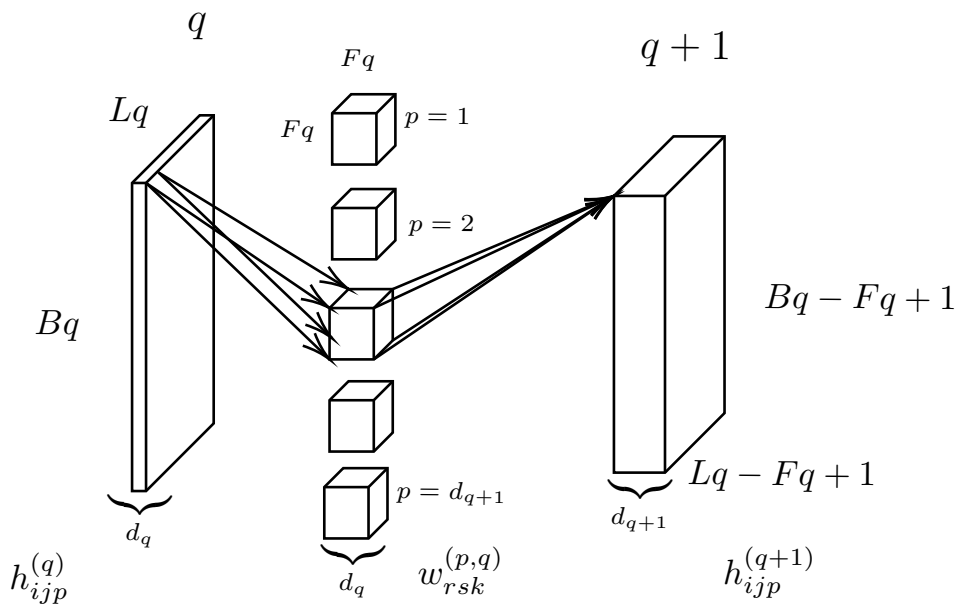


Figura 19.8: Componentes de la capa de convolución

i : height, $\forall i \in \{1,...,Lq - Fq + 1\}$
q : Layer, j : width, $\forall j \in \{1,...,Bq - Fq + 1\}$
k : depth, p : Filter, $\forall p \in \{1,...,d_{q+1}\}$

La imagen de entrada es $h_{ijp}^{(q)}$ de dimensiones $Lq \times Bq$, de ancho y de alto, conformada en d_q colores[3]. De la capa q a $q+1$ se posicionan, por ejemplo, p filtros convolucionales, kernels, que tienen dimensiones de $Fq \times Fq$, por ejemplo 3×3 pero tienen que tener exactamente la misma cantidad de colores que la imágen de entrada, para poder asignar a cada pixel, de cada canal, cada uno de los pesos del kernel. En los kernels es donde está los pesos, los parámetros libres de las redes convolucionales $w_{rsk}^{(p,q)}$. La salida de la capa convolucional, es una nueva imágen, más chica (asumiendo valid como padding), de $Lq - Fq + 1 \times Bq - Fq + 1$, y con una cantidad de *colores* equivalente a la cantidad de filtros en esta capa de convolución d_{q+1}. De esta manera cada filtro trabaja en paralelo, generando un canal más en la imágen de salida.

La no linealidad, al igual que en el Perceptrón multicapa, es aportada por funciones de activación que toman la salida de $h_{ijp}^{(q+1)}$ como entrada. Particularmente en redes convolucionales, pensadas para imágenes donde los valores pueden ser muy grandes, se usan funciones de activación como ReLU que la ventaja que tienen es que no satura tan rápido y permite que el gradiente tenga un rango de valores posibles mucho más grandes, aportando más información, información más fina, para el ajuste de los pesos (que con funciones sigmoideas como la tangencia hiperbólica, rapidamente saturarían llevando todo a un mismo valor).

La definición de Relu es muy simple:

$$f(x) = x^+ = max(0,x) \tag{19.2}$$

$$f(x) = \begin{cases} 0 & x \leq 0 \\ x & x > 0 \end{cases}$$

Al ser una operación de convolución, dos hiperparámetros aparecen nuevos, que son el **stride**, que es el paso del salto espacial que se va haciendo al pasar de un pixel al siguiente (y calcular el promedio ponderado de todos los de alrededor). Y el segundo, por supuesto, es los **efectos de bordes** y como manejar el padding.

19.5 La capa de Pooling

La capa de Pooling está **cableada**, es decir no tiene ningún parámetro libre entrenable. Sirve para capturar invariantes traslacional, y operan de manera similar a una operación de umbralización. Diferentes regiones de la imagen van pasando por capas convolucionales, que son filtros, y enfatizan características particulares de la imagen. La capa de pooling permite que independientemente donde esté localizado ese punto enfatizado, eso se tenga en cuenta en capas sucesivas, y tenga relevancia, para afectar la salida de lo que la red genera. El resultado de esta capa es similar al efecto obtenido mediante el **downsampling**, o mediante la reducción de la resolución de una imagen. La operación de pooling generalmente captura una región de la imagen y la sintetiza en un único pixel, normalmente eligiendo el mayor, conformando el **MaxPooling**, tal como se ve en la figura 19.9. Esta invarianza traslacional puede extenderse a hacer otras características invariantes, dependiendo como se agrupen los filtros y las capas de pooling subsiguientes.

[3]Los colores en las imágenes tradicionales, **R**ed, **G**reen y **B**lue son los utilizados normalmente para el espectro de imágenes visibles por los seres humanos pero perfectamente pueden extenderse a muchos más, es totalmente arbitraria la diferenciación. Muchas imágenes son **hiperespectrales** indicando justamente que tienen muchos canales.

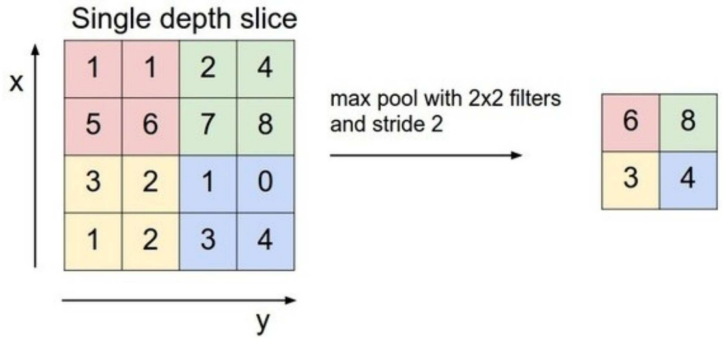


Figura 19.9: La capa de pooling resume la información obtenida en capas anteriores de forma de proveer una invarianza traslacional.

19.6 Full Layer y Softmax

Al final de la Red (fig 19.7) hay una (o más capas) que se encargan de hacer la clasificación final (si ese es el objetivo). Esta subred es tradicionalmente un perceptrón multicapa entrenado para hacer una clasificación multiclase de la imagen de la entrada. En redes convolucionales es muy común utilizar la función de activación Softmax definida en eq. 19.3.

$$\sigma(\vec{V})_i = \frac{e^{V_i}}{\sum_{j=1}^{K} e^{V_i}} \tag{19.3}$$

con V_i como la salida lineal de la neurona de la capa final. Esta función actúa también como un normalizador de la salida, enfatizando las diferencias para dar como resultado una única salida ganadora, aquella que tenga el mayor valor de salida dado por eq. 19.3. Al ser una normalización, la salida de todos los valores suma 1 por lo que se lo puede interpretar como una función de probabilidad.

19.7 Backpropagation

¿Cómo impacta el cambio en la estructura de la red en las capas de convolución para la aplicación del algoritmo de Backpropagation? Lo que se busca siempre es mantener la identificación de la influencia de cada peso al momento de retropropagar los errores. Por eso es que en las capas de convolución, como los pesos son compartidos, es necesario acumular todas las variaciones a la hora de realizar la actualización de los pesos. Existen varias maneras de resolver esto. La primera es la trivial de simplemente desplazar el kernel espacialmente e ir acumulando las modificaciones que se requieren hacer en los pesos. Esto es lo mismo que ocurre en cada época en relación al impacto de cada entrada, pero ahora hay que hacerlo a través de la distribución espacial.

1. Supongamos una red para realizar una clasificación en tres clases.
2. La Red se inicializa al azar, igual que siempre.
3. Se calculan los valores hacia adelante, aplicando la operación de convolución en las capas correspondientes hasta alcanzar los valores de salida, capa a capa.
4. En las capas *Fully Connected* es exactamente igual que el Perceptrón multicapa.
5. En las capas de Pooling, solo se actualizan la contribución de los pesos de la salida ganadora (asumiendo una función identidad).

6. En las capas convolucionales, hay que mantener los pesos de los filtros en la relación espacial.

Backpropagation mediante representación en matrices esparsas

La operación de convolución puede verse como una manera eficiente de realizar una multiplicación de matrices reutilizando pesos. Desde esta perspectiva es posible hacer una representación de la operación de convolución en una matriz esparsa como se detalla en la figura 19.10. Para ello se

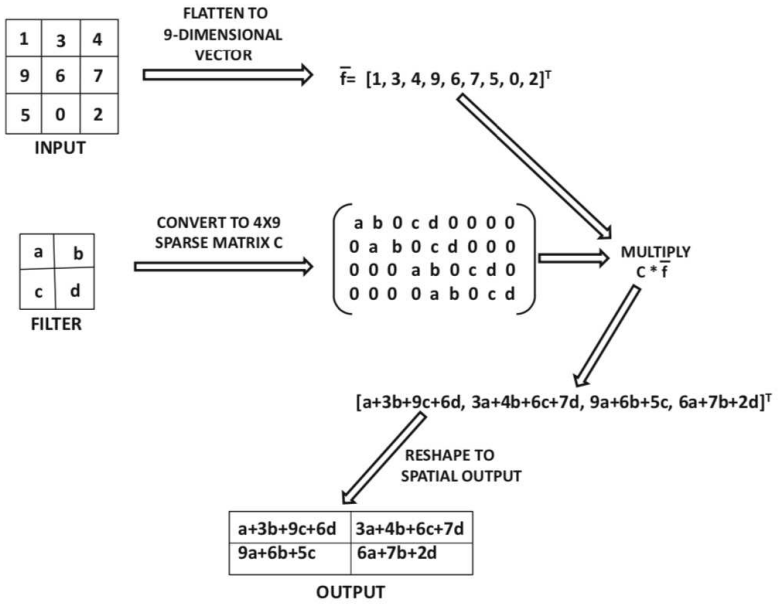


Figura 19.10: La representación esparsa de la operación de convolución permite operar de la misma forma que lo que ocurre en un perceptrón multicapa.

representan los pesos del kernel en representaciones vectoriales unidimensionales del largo de todos los elementos de la matriz de entrada de la operación (en la figura es 9). Y cada peso va ubicado en la misma posición que corresponde a como es la alineación entre el peso y el valor de entrada. Cada fila de esa nueva matriz, representa una alineación de los pesos sobre la entrada. Esto genera una matriz C esparsa donde la mayoría de las entradas son ceros. Luego se puede hacer la multiplicación directa de la entrada, tomada como un vector **chorizeado, flatten** de 9 elementos con la matriz C. El resultado de esto corresponde a lo que sería la matriz de salida de la operación completa de convolución. En ella los pesos quedan como parámetros que pueden utilizarse de esa manera, como una capa de multiplicación lineal, para lo cual aplicar la retropropagación mediante la regla de la cadena es trivial.

19.8 ¿Por qué funcionan?

Las redes convolucionales funcionan muy bien para capturar comportamiento estructurado en jerarquías e invariante a las traslaciones, muy común de imágenes visuales. Las razones son:

- Matrices esparsas son mucho más eficientes: menos cómputos.
- Los parámetros libres son compartidos. Esto hace que se les exige más, que capturen mejor la información. Esto es similar al mecanismo de compresión establecido en un autoencoder en la capa latente, o a adicionar un término regularizador en la función de costo.

- *Equivariante* $g(f(x)) = f(g(x))$, donde f es la traslación. Esto representa que da igual primero trasladar y tratar de identificar que identificar y luego trasladar.
- Jerarquías Espaciales: Ingeniería de Características Jerárquica. Hay una composición jerárquica de estructuras que son similares independientemente de la escala.

19.9 Inteligibilidad

Inteligibilidad y Explicabilidad

La definición dada en 10.1.1 estructura algoritmos que aprenden relaciones entre datos existentes pero no asumen ningún modelo mecanístico en su accionar. Esto implica que por su propia estructura, es difícil entender por qué razón arriban a alguna conclusión particular en cuanto a clasificar o regresionar. Esta es la característica de caja-negra de todos estos algoritmos [141], que ya había sido maravillosamente anticipada por el propio Turing en 1950 como una consecuencia natural de generar un automatismo que tuviese la capacidad de aprender[a]. En contraposición, lo que se requiere de estos algoritmos cuando las decisiones que se toman son relevantes, es dos principios relacionados pero diferentes. El primero es la propiedad de legibilidad o **Inteligible Property** que se refiere a la capacidad del esquema de ofrecer una representación intermedia que le permita a un ser humano entender cuál es la característica que el algoritmo identificó para tomar una decisión. Por ejemplo, en una placa radiográfica remarcar las zonas que caracterizan una neumonía. Por otro lado, la segunda propiedad es la explicabilidad o **Explainability** que es la capacidad del sistema de explicar el paso a paso, la cadena de razonamiento, que lleva a tomar una decisión particular. Esto en modelado directo a como los seres humanos podemos seguir una cadena de razonamiento y dar una explicación. Estos principios suelen englobarse en lo que se denomina **xAI**, Explainable Artificial Intelligence. Dado el alcance e impacto social que estas tecnologías están teniendo, al actuar estos modelos como agentes de decisión, es necesario tener estos puntos en consideración, e incluso ir un paso más allá y considerar esquemas que además sean **Responsible AI**, es decir, diseñar estos sistemas ofreciendo garantías y margenes de funcionamiento, alineandolos con estándares éticos y regulatorios.

[a] "An important feature of a leraning machine is that its teacher will often be very largely ignorant of quite what is going on inside, although he may still be able to some extent to predict his pupil's behaviour [15]"

La posibilidad de la aplicación de filtros convolucionales, que son pesos a los que se les puede asignar un sentido particular como coeficientes de un filtro, permiten que las capas intermedias de la red capturen comportamiento que ayuda tanto a ofrecer legibilidad como eventualmente generar una secuencia de explicabilidad. Esto está bien representado en lo que se denomina los mapas de saliencia o **Saliency Maps**, que capturan cuáles son los elementos de las capas intermedias convolucionales que de variar levemente generan un cambio rotundo en la salida de la red. Al estar representandos en imágenes, se representan como información que puede ser comprendida directamente por seres humanos.

19.10 Arquitecturas Reusables

Un aspecto excelente de las redes convolucionales es que las diferentes combinaciones de capas de convolución, ReLU, pooling y fully connected, permiten armar arquitecturas profundas que pueden reusarse como modulos reutilizables que actúan como componentes de redes más complejas. Esto hace que sean una tecnología muy versátil y explican porque razón están en los corazones. Por ejemplo, los pesos de la capa de convolución se asumen que generan filtros adecuados para procesar imágenes

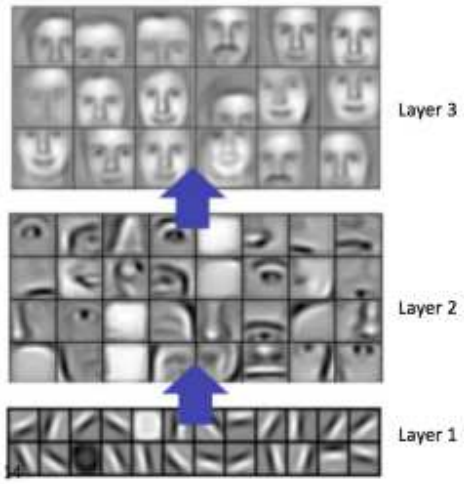


Figura 19.11: Las diferentes capas de una red convolucional capturan capturando características más abstractas de la información contenida en la imagen y que sirven también para explicar el resultado final que ofrece la red.

visuales, se pre-entrenan y se disponibilizan, como el caso de los *Pretrained FC7 Features*, o como los existentes en redes conocidas, descargables y reutilizables como:

- LeNet
- AlexNet
- VGG

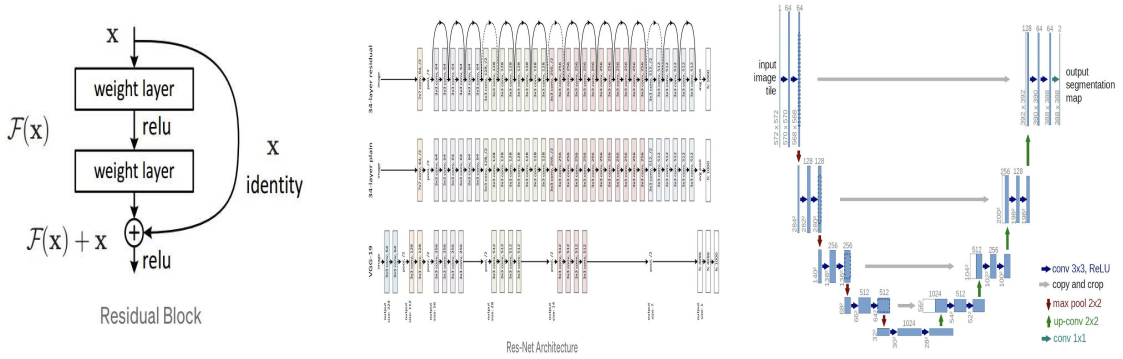
Como siempre, la elección de la arquitectura correcta no escapa a las generalidades de las redes neuronales. Un poco arte, un poco de *educated guesses*, y prueba-y-error.

19.10.1 ResNet

Como las imágenes suelen ser enormes, las redes convolucionales son realmente profundas y el VGP(?) se vuelve muy importante. Una alternativa para reducir sus efectos es utilizar lo que se denomina conexiones residuales. Esto es, al pasar por una capa, se mantiene una conexión residual directa, que actúa como un perceptrón lineal, que se adiciona a la salida propia de la capa de convolución. Esto logra que la red capture el cambio pero minimiza la posibilidad de que el gradiente se haga cero, ya que mantiene los valores cerca de los que fueron tomados en la entrada.

19.10.2 UNet

El primer uso que se les dio a las redes convolucionales fue para clasificación, pero pueden ser muy útiles para hacer segmentación, o incluso alterar una imagen, o completarla como en situaciones de **impainting**. En este caso, se requiere aplicar capas sucesivas que obtengan buenos filtros sobre la imagen, pero luego capas adicionales que permitan recuperar esa información para regenerar una imagen de características similares a la original. En este caso, la idea de ResNet se extiende en las UNets: en estas, las diferentes capas de convolución capturan información importante de una imagen, pero luego se aplica la operación inversa, la deconvolución, que sucesivamente genera imagenes intentando reversar el proceso de deconvolución pero usando las salidas de los filtros. Para que esto tenga sentido, se agregan conexiones residuales que provienen de las imagenes originales de cada capa



(a) Conexiones residuales (b) Estructura de una ResNet (c) Una ResNet en forma de U

Figura 19.12: Extensiones de redes convolucionales

convolutiva y que generan el efecto de recuperar la información que existía en la imagen a ese nivel. Por eso el nombre de esta red, ya que lo que provoca es un dibujo en forma de U donde las salidas de cada capa de la red, se usan para ahondar en más profundidad la aplicación de los filtros, pero a su vez se utilizan también como entrada para regenerar la imagen en el **up-scaling** de deconvolución del otro lado del valle de la U.

19.11 Más allá de las imágenes

Las redes convolucionales son muy versátiles en relación al tipo de datos y problemas en las cuales pueden usarse, dependiendo de cómo se realiza la asignación de las dimensiones. La tabla 19.11 muestra algunos ejemplos.

Dimensiones	Canal único	Multicanal
1-D	Audio	Electroencefalografía Marcadores Captura Movimiento
2-D	Espectrogramas	Imagenes a color
3-D	Tomografía Computada	Video

19.12 Quiero más

- Fukushima Neocognitron [142].
- LeCun 1989, LeNet Architecture [143].
- Libro de Aggarwal Neural Networks and Deep Learning [144].
- Guía para entender la aritmética de las convoluciones [145, 146]

- Estupendo resumen de CNN desde una perspectiva super variada [147]
- AlexNet 2012 [148].
- https://ujjwalkarn.me/2016/08/11/intuitive-explanation-convnets/

20. Generative Adversarial Networks

Las redes GAN, Generative Adversarial Networks, o redes adversariales generativas, surgen aproximadamente al mismo momento que los Autoencoders variacionales, abordando desde una perspectiva distinta la misma problemática: cómo **generar** muestras nuevas de un conjunto X. Fueron propuestas por un grupo de investigadores del núcleo de Deep Learning, entre los que incluye Ian Goodfellow y Yoshua Bengio. Cuenta la historia, que Ian había estado pensando en una red **Discriminadora** que sirviera para evaluar si diferentes mecanismos de síntesis de habla eran buenos o no, y durante una tertulia en un bar con amigos nerds, discutiendo acerca de las complejidades sobre como generar imágenes de manera eficiente, se le cruzó la idea de combinar un generador con un discriminador y usar la información del segundo para precisamente entrenar y ajustar al primero. El remate de la historia es que esa misma noche empezó a codear la idea de las GANs [149].

20.1 Generando gatitos

Típicamente, dado por ejemplo un conjunto de imágenes de gatos, similar a lo que ocurrió el capítulo 18 con Autoencoders, queremos identificar la distribución de probabilidad que caracteriza a las imágenes de los gatos. Es decir queremos encontrar un modelo generativo.

Definición 20.1.1 — Modelo Generativo. Un modelo generativo describe la estructura de generación de un dataset X, en términos de su estructura probabilística. Sampleando de este modelo, es posible generar nuevas muestras X_j que serán probabilísticamente indistinguibles de las muestras originales X_i.

El problema por supuesto es estimar esa distribución. Las redes GAN [150] buscan solucionar esto planteando una interacción entre dos redes. Una primer red es la responsable de discriminar entre las muestras reales pertenecientes al conjunto X, el dataset real, versus muestras artificiales. Este discriminador entonces está sometido a recibir muestras de entrada y de manera supervisada aprender a

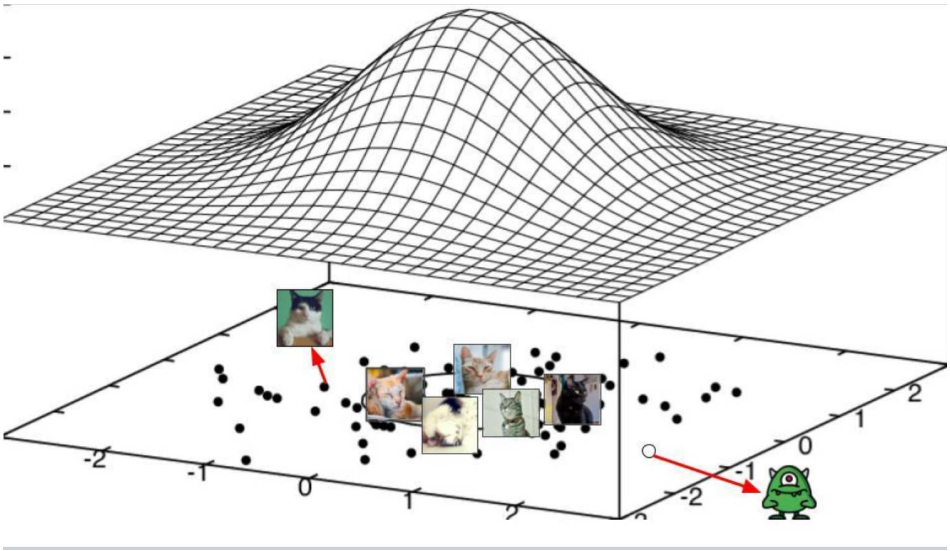


Figura 20.1: Modelo generativo de imágenes de gatitos

decidir si cada una de las muestras produce el label de salida 1 o -1 indicando si pertenecen o no al conjunto real de muestras o son artificiales. A esta red la llamamos **Discriminador** D.

Por otro lado, una segunda red que llamaremos **Generador** va a tener el objetivo de producir las muestras falsas. Esta red actúa de una manera muy similar al decodificar del Autoencoder. De un espacio latente, obtiene una muestra al azar, a la cuál se le agrega ruído, y la misma pasa por una red con pesos libres que va a generar una nueva muestra que será sometida al discriminador para ser juzgada.

El algoritmo opera de la siguiente manera:

1. Dado un dataset X de muestras reales válidas.
2. Se inicializa la red Discriminadora y Generadora.
3. Se utiliza la red Generadora para aumentar el dataset X con un conjunto de muestras artificiales.
4. Se entrena la red Discriminadora de manera supervisada para intentar ajustar los pesos para predecir si la muestra es real o artificial, hasta convergencia.
5. Dejando fijos los pesos del Discriminador, se entrena a la red Generadora para generar muestras de forma que la red discriminadora acepte como verdaderas. Para este paso, la red Generadora + la red Discriminadora se ven como una única red.

El proceso de entrenamiento continua alternando entre el entrenamiento de una red y la otra, y por supuesto lo importante es definir una buena función de costo que capture el objetivo subyacente de este entrenamiento.

20.2 Juego MinMax

Hay un conflicto acá presente. Una red quiere una cosa, y la otra red quiere exactamente la opuesta. El Discriminador quiere identificar con claridad cuáles son las muestras del dataset original, mientras que el Generador quiere generar una muestra que el Discriminador no pueda, justamente, discriminar. Este conflicto se media mediante una función de costo que capture la película completa encuentre un balance.

Los elementos de la ecuación son:

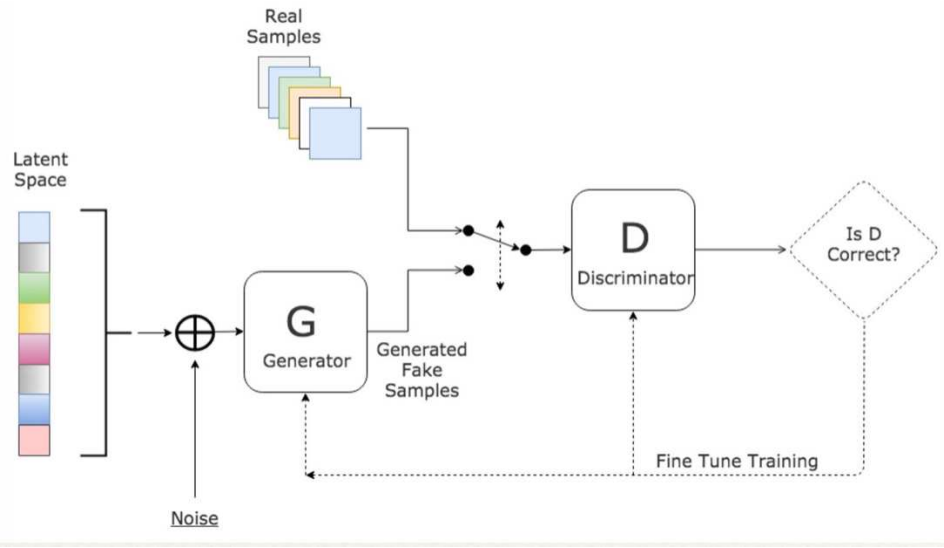


Figura 20.2: Arquitectura de una red GAN, compuesta por una red Generadora y otra Discriminadora que juegan un juego adversarial.

- $D =$ Discriminador
- $G =$ Generador
- $\phi_d =$ Parámetros del Discriminador
- $\phi_g =$ Parámetros del Generador
- $P_z(z) =$ Distribución del ruido de la entrada del generador.
- $P_{data}(x) =$ Distribución de los datos originales.
- $P_g(x) =$ Distribución de los datos generados.

Primero la ecuación desde la perspectiva del Discriminador:

$$L_D = H(y, D(x)) = \frac{1}{N}\{\sum_{i=1}^{N}(y_i)log(D(x_i)) + (1-y_i)log(1-D(x_i))\} \tag{20.1}$$

teniendo en cuenta que y son las etiquetas reales en tanto que $D(x)$ son las etiquetas asignadas por el discriminador, las predicciones.

Lo que el discriminador quiere es que cuando la muestra es real, el primer término sea 1 y el segundo término desaparezca. Esto es que el discriminador acierte, y que $D(x_i) = 1$. Por el contrario, cuando la muestra es falsa, el discriminador quiere que el primer término desaparezca y que $1 - D(x_i) = 1$.

La ecuación 20.1 puede rescribirse como:

$$\max_D H(y, D(x)) = \max_D \ \mathbb{E}_{x\sim P_{data}}\{log(D(x)\} + \mathbb{E}_{z\sim P_z}\{log(1-D(G(z)))\} \tag{20.2}$$

donde lo que el Discriminador quiere es maximizar esa función para justamente acertar cuando corresponde hacerlo.

Por otro lado, lo que el Generador quiere, desde su perspectiva es precisamente minimizar la ecuación 20.2: provocar que el Discriminador no identifique cuando las muestras han sido generados por el Generador.

Esto es, a la vez, optimizar:

$$\min_G \max_D V(D,G) = \min_G \max_D \left\{ \mathbb{E}_{x\sim P_{data}}\{log(D(x)\} + \mathbb{E}_{z\sim P_z}\{log(1-D(G(z)))\} \right\} \tag{20.3}$$

En el paper [150] demuestran que el mínimo de esta ecuación se produce en un punto de equilibrio de Nash [55], donde se alcanza precisamente lo que se planteó al principio: el discriminador no tiene información para poder discriminar una muestra de X de las generadoras por el generador $G(z)$, sólo puede tirar la moneda, mientras que el Generador produce muestras nuevas que tienen la misma distribución estadística que la de los datos originales, es decir, $P_z \rightarrow P_{data}$.

20.3 Colapso Modal

Las redes GAN tienen dos grandes inconvenientes. El primero es que el orden en el que se realiza el entrenamiento alternado del Discriminador y del Generador es muy importante desde un punto de vista pragmático para alcanzar el punto de equilibrio entre las dos redes, y consecuentemente de la red GAN completa. El segundo problema, que está relacionado al anterior, es que puede ocurrir una situación que se denomina *colapso modal* que es donde la red encuentra un punto de solución alternativo donde diferentes puntos del espacio latente generan un único punto, una única muestra en vez de generar un espacio completo o de capturar la verdadera distribución [151].

20.4 El gran simulador

Las GAN presentan un fenómeno interesante. Comienzan a generar imágenes que realmente capturan la escencia presente en los datos, y nos permiten a los seres humanos corroborar eso con nuestros propios ojos. Esto puede visualizarse en la obra de varios artistas digitales [152] tal como puede verse en la figura 20.3.

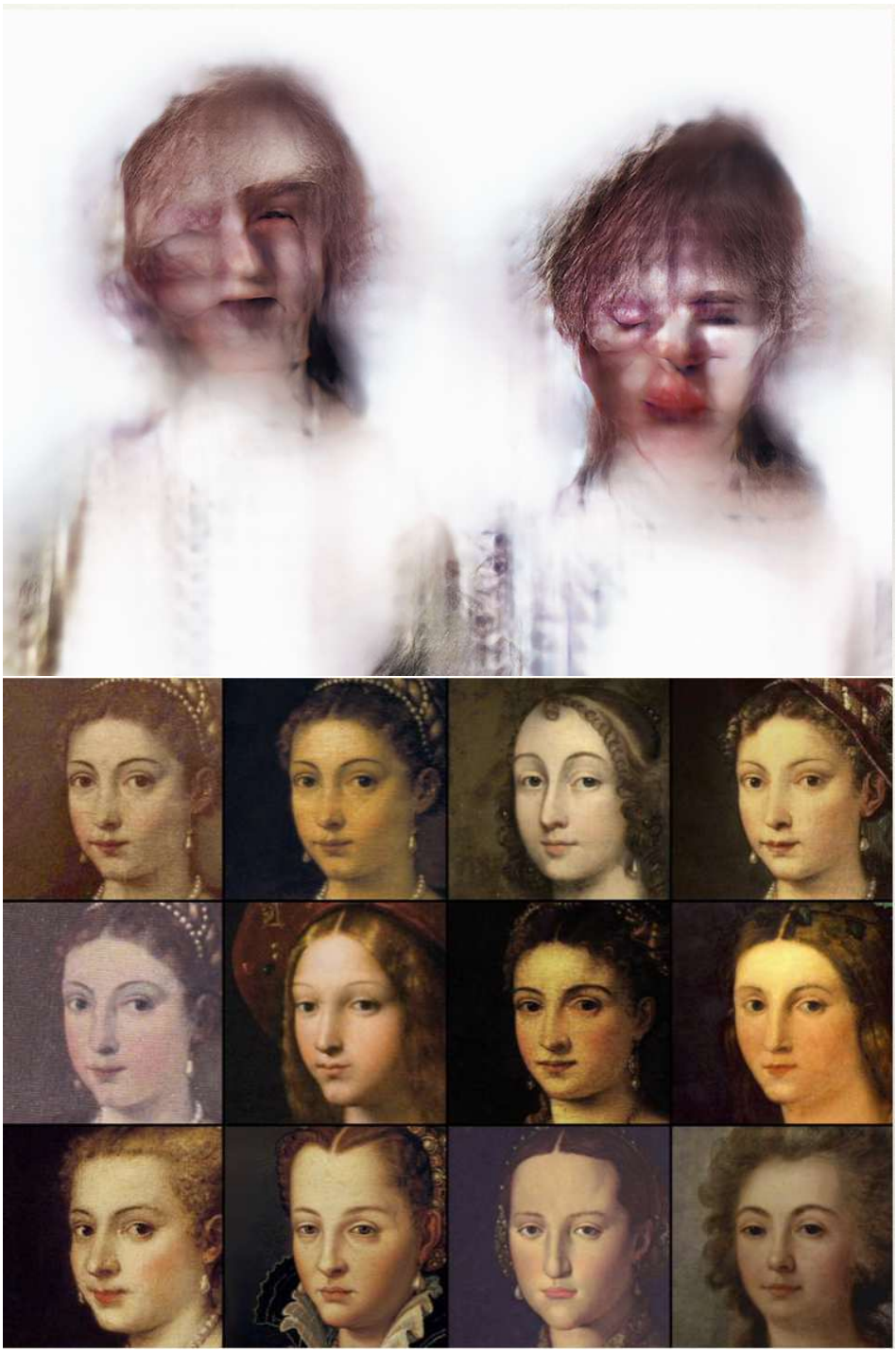


Figura 20.3: Imágenes generadas con redes GANs. En la imagen de la derecha es difícil identificar cuál de esas imágenes es obtenida de un cuadro real o cuáles fueron generadas artificialmente.

21. Transformers

Los transformers fueron propuestos por [37] en el famoso trabajo **Attention is all you need**. Su propuesta era la conclusión en ese momento de muchos años de trabajo e intentos para implementar un mecanismo adecuado para el procesamiento del lenguaje natural, NLP (Natural Language Processing), el procesamiento de tokens de texto, en una secuencia, y solucionando los problemas del *vanishing/exploding gradient*, VGP, de las redes recurrentes y los problemas de las redes LSTM. Un tema curioso, que no puede escapar del propio hype que afecta a toda esta área, es lo opaca que es la literatura en relación a muchos de estos temas. Por eso es que vamos a intentar poner blanco sobre negro en relación a poder identificar de donde vienen los nombres y a qué es que referencian precisamente.

21.1 Las Redes Recurrentes

Las redes neuronales recurrentes tienen fuertes conexiones con sistemas físicos complejos y dinámicos (ver capítulo 14). El modelo de una neurona recurrente puede visualizar en la Figura 21.1 [153].

Las redes recurrentes se caracterizan porque tiene una entrada x_t y una salida y_t, y al igual que en el modelo de neurona tradicional la entrada suele ser vectorial en tanto que la salida escalar. Pero además, las neuronas recurrentes tienen una salida adicional que representa un estado interno, que tras un delay es una entrada adicional al paso de procesamiento que sigue. El esquema dinámico que esto plantea, queda discretizado por el delay recibiendo h_t en la entrada y generando una salida h_{t+1}. Puede ocurrir, como en las redes de Hopfield, que la salida y el estado sean lo mismo y no estén diferenciados. La figura 21.1 muestra el esquema general, donde la entrada y el estado anterior son múltiplicados por matrices de pesos y el estado siguiente y la salida son calculadas después de aplicar una función no lineal.

Las redes recurrentes son la materialización del problema del vanishing/exploding gradient. En la Figura 21.1, del lado derecho, se muestra el proceso de unfolding. Las redes recurrentes, por su

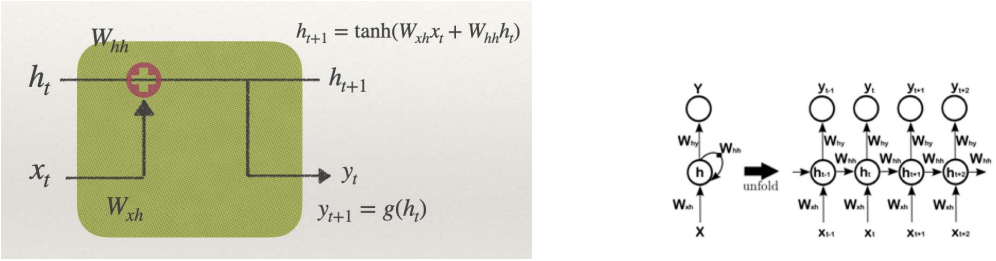


Figura 21.1: RNN, Recurrent Neural Networks, redes recurrentes. La red opera como un modelo con delay, con una salida adicional que representa un estado que sirve de entrada para el paso siguiente.

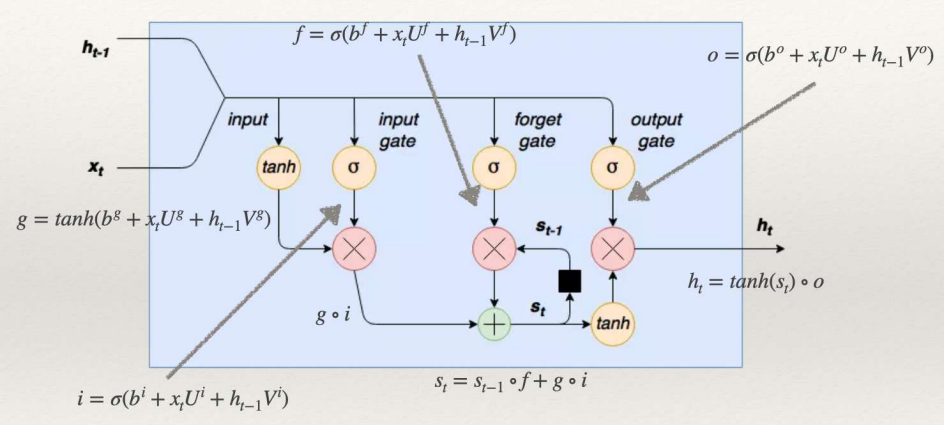


Figura 21.2: Las redes LSTM, Long Short Term Memory, fueron pensadas especificamente para poder superar el problema del vanishing/exploding gradient con un mecanismo de activación de una memoria interna para lograr cadenas de inferencia causales que acepten muchos capas temporales.

caracterización dinámica, se pensaron para procesar información en secuencia, de series de tiempo, o de procesos estocásticos. Por ejemplo, lenguaje escrito. Así el primer token de la frase ingresa como input a la red, afecta la salida en el estado h_{t+1} que a su vez se propaga al próximo estado, donde recibe un nuevo input, que alterna otra vez el estado, y así sucesivamente. Esto, rápidamente, crece en la cantidad de pasos y genera naturalmente una restricción numérica en como retropropagar el error al pasado por muchas, muchas iteraciones[1].

21.1.1 Long Short Term Memory

En 1997 Hochreiter y Schmidhuber[154] proponen una solución a este problema de la redes recurrentes, planteando una alternativa de red que hacía un uso inteligente de compuertas de activación neuronales, que le permitían a la red guardar, recuperar y eliminar información relevante en una memoria, un estado interno de la red, donde la red ajustando automáticamente los pesos aprendía cuando tenía que guardarla, recuperarla o eliminarla, y que el almacenamiento en estado interno resolvía el problema de tener que retropropagar los errores por muchas capas.

Las redes LSTM fueron muy exitosas y fueron el estándar de facto para los teclados predictivos de

[1] De hecho el concepto de Deep de deep learning, hace referencia tanto a la profundidad espacial de las redes, como a la profundidad temporal que se genera en las redes recurrentes y que rápidamente puede crecer de manera muy abrupta y exponencialmente con el problema.

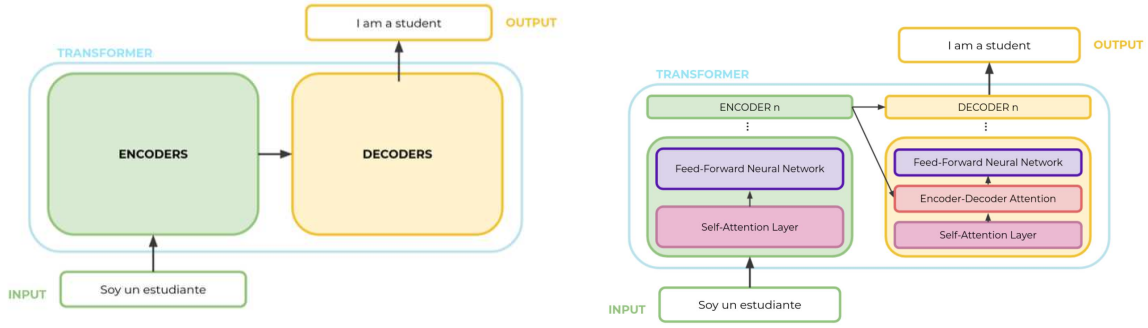


Figura 21.3: Diagrama de componentes básico de un Transformer.

los denominados **feature phone**, los teléfonos celulares de GSM con teclados físicos pequeños, que estaban pensados para mandar mensajes de texto. La limitación que encontraron era que el aprendizaje de la red era complejo, y su generalización muy acotada, ya que el mecanismo de memoria interna le era complejo poder encontrar un gran abanico de abarcabilidad de usos, y se volvía un excesivo cuello de botella.

21.2 Arquitectura de un Transformer

Los Transformers fueron pensados para interpretar secuencia de tokens en texto escrito. Tienen una arquitectura general que está determinada por un **Encoder** y un **Decoder**, influenciado la nomenclatura quizás por la arquitectura de los Autoencoders aunque sin ningún tipo de relación directa. El objetivo del Encoder es procesar las entradas. En tanto que el Decoder recibe información de la entrada pero también de la salida hasta ese momento.

Al pensar al Transformer como una arquitectura para resolver la traducción de un texto de un lenguaje en otro, aparece la justificación de los diferentes componentes.

La red primero tiene un stack de bloques de Encoders y Decoders. En el paper, usan 6 bloques de Encoders y Decoders.

Encoder: El Encoder recibe la información de la cadena de entrada. Tiene dos componentes, el primero corresponde a una capa que implementa el mecanismo de **Atención**, específicamente el mecanismo de **Self-Attention**, que relacionará los lexemas de la entrada entre sí. La segunda capa es un MLP. La salida pasa al Encoder subsiguiente del stack.

Decoder: La salida final del último Encoder es la entrada a cada uno de los Decoders, y reciben una entrada adicional que viene de la salida hasta el momento para el primer Decoder, y de la salida del Decoder anterior para todos los otros. El Decoder tiene una capa de Self-Attention, luego una capa de Encoder-Decoder-Attention (que toma la salida de la capa anterior versus las entradas recibidas del Encoder) y finalmente un MLP. La salida final del último Decoder genera la salida actual, en este caso el token actual de traducción.

Los Encoders tienen el objetivo de encontrar las relaciones entre los lexemas de la entrada. La primera capa, atiende la relación de lexema contra lexema. La segunda capa toma las relaciones entre tuplas de lexemas entre sí. Y así sucesivamente.

Luego los Decoders relacionan las entradas con los elementos de la salida que se ha generado hasta ese momento (por lo que la salida actual es la entrada principal del primer Decoder del Stack). Los Decoders subsiguientes siguen la misma lógica de los Encoders para procesar n-tuplas.

Al final de la cadena de Decoders, se ubica una capa lineal, seguida por una función SoftMax y finalmente se recodifican los lexemas (numéricos) en lexemas reversando la operación de los Embeddings. La capa SoftMax determina cuál es el output de cada uno de los tokens posibles, y la idea es elegir aquel que tenga la salida mayor[2].

21.3 Attention is all you need

El mecanismo de atención es central en los Transformers hasta el punto que es casi un sinónimo que representa principalmente el detalle más fundamental de la arquitectura. Es una capa interna que poseen todos los Encoders y Decoders, cuya función es tener en cuenta otras palabras de la secuencia mientras se está analizando una en particular. En el caso de los Encoders, esta capa le permite poner el foco en cualquier otra palabra de la entrada, en tanto que los Decoder únicamente pueden analizar palabras previas en la secuencia de salida.

La capa de atención permite relacionar los lexemas del lenguaje entre sí. Por ejemplo, de la secuencia "Yo soy un estudiante", la palabra estudiante está referenciando el pronombre *Yo*. Por eso los pesos, los parámetros libres en el mecanismo de atención, deberían ser más fuertes entre estas dos palabras.

Hay tres maneras de ver al mecanismo de atención.

$$SoftMax\left(\frac{QK'}{\sqrt{d_k}}\right) * V \tag{21.1}$$

con la definición de la función de activación SoftMax como $\sigma(\vec{z})_i = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_i}}$.

21.3.1 Una tabla de acceso indexado continua

La nomeclatura de las variables utilizadas en la descripción del mecanismo de atención derivan de este abordaje. Supongamos que tenemos una tabla de acceso indexado, una *hashtable*. Necesitamos obtener los datos de una persona, en base al DNI. El *query* Q representa el dato del DNI de la persona. Este query Q lo usamos para acceder a un índice para obtener el valor del índice, la clave K. Finalmente con K podemos acceder a la tabla real y obtener finalmente V, el valor que necesitamos.

Ahora podemos pensar a este esquema en una variante continua, donde no accedemos a las entradas exactas de la tabla sino establecemos una medida de similitud y vamos a buscar las más parecidas. Con esto, el query Q necesitamos buscar él índice más parecido de la tabla. Usemos el producto interno como medida de similitud(puede haber otras). Entonces para ver cuál de todos los índices k es el más parecido, hacemos

$$Q.k^T$$

. Luego con este resultado, accedemos a la tabla real para encontrar el valor real que se parezca lo más posible a esta medida, y otra vez usamos el producto interno, mediado por una función de activación que rescala los valores para identificar de todos al más parecido

$$(Q.k^T).V$$

.

[2]Softmax puede verse como una probabilidad ya que la salida suma uno pero no corresponde verdaderamente a las probabilidades de salida, ya que no estea calibrada.

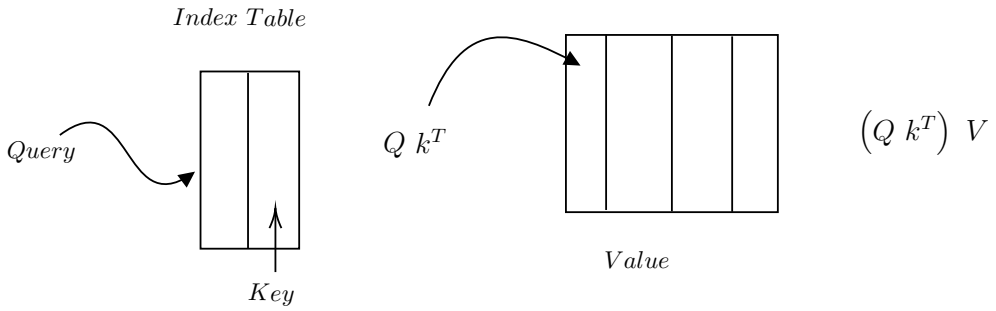


Figura 21.4: Acceso indexado a una tabla de manera continúa.

Finalmente, si consideramos un escalado de los datos y que sean una suma convexa, para distribuir el peso entre todos, nos queda otra vez:

$$\sigma\left(\frac{Q.k^T}{\sqrt{d_k}}\right) * V \tag{21.2}$$

Lo que obtenemos con esto es una manera de poder primero identificar la clave, el índice que es más parecido al query (al dato que tenemos en foco). Luego necesitamos ver cuál es el valor, el Value, que es más parecido a la combinación del query con la clave. La ventaja de esto, es que podemos agregarle pesos, parámetros libres, que van a servir para mediar estos cálculos y así establecer mecanismos de aprendizaje de estas relaciones.

$$\sigma\left(\frac{QW_q.(kW_k)^T}{\sqrt{d_k}}\right) * VW_v \tag{21.3}$$

donde

- Q : Query, dato de entrada en el que está el foco.
- k : Key, clave, los índices, los tokens a los que hay que prestarle atención.
- V : Value, los valores que se van a generar de salida, ponderados por el núcleo.
- W_q, W_k, W_v: pesos que actúan como parámetros libres,

21.3.2 Kernel Smoothing

Supongamos que tenemos un conjunto de datos X y sus supuestas salidas deseadas Y, siendo los primeros la cadena de lexemas de entrada y la segunda la de salida. Dado un x_o de entrada objetivo que queremos traducir, algo básico que se puede usar es utilizar **kNN**, con $k = 1$, identificar el vecino x_i más cercano y su correspondiente y_i. Si eventualmente queremos tener también la información de otros vecinos cercanos, podemos tomar un k mayor, y promediar los valores de y_i que corresponden a varios de esos vecinos

$$\hat{y_o} = \sum_{i=1}^{k} \frac{1}{k} y_i \tag{21.4}$$

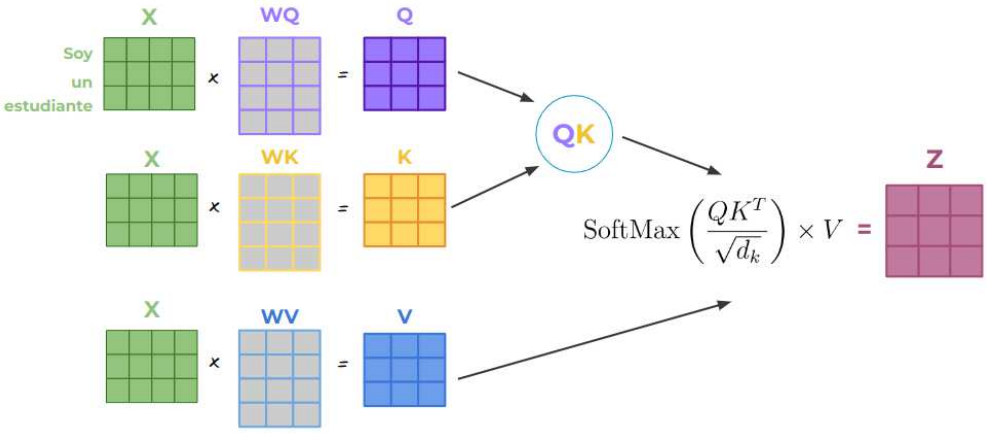


Figura 21.5: Mecanismo básico de Attention y sus componentes.

El problema es que esto no tiene en cuenta cuán cerca cada vecino está del *query* x_o. Lo que podemos hacer entonces es hacer un promedio ponderado, donde le demos más peso a aquellos puntos vecinos que sean más similares a x_o. Para eso agregamos un **kernel** $K(x_i, x_o)$ que nos va a indicar cuán similar es uno del otro. Con ese promedio ponderado, la ecuación anterior queda:

$$\hat{y_o} = \sum_{i=1}^{k} \frac{K(x_i, x_o)}{\sum_{j=1}^{k} K(x_j, x_o)} y_i \tag{21.5}$$

Esta técnica se denomina Kernel Smoothing [155]. Como los valores x son vectores, como función de kernel se puede utilizar cualquier función de similitud vectorial como el producto interno o variantes como $K(x_i, x_o) = exp(x_i \cdot x_o)$ u otra variante donde se puede definir el producto interno en base a una matriz $K(x_i, x_o) = exp(x_i \cdot w_1^T w_2 \cdot x_o)$. Y eventualmente se puede normalizar con la dimensión de los vectores usando $\sqrt{d}$.

Si reemplazamos, $x_o \rightarrow Q$, $x_i \rightarrow k^t$ y $y_i \rightarrow V$, llegamos a la ecuación del mecanismo de atención que desde esta perspectiva se la puede ver como promedio ponderado por kernels de similitud entre la clave k^T y el query Q de valores V.

$$\hat{y_o} = \sum_{i=1}^{k} \frac{K(x_i, x_o)}{\sum_{j=1}^{k} K(x_j, x_o)} y_i \tag{21.6}$$

y a esto se le pueden agregar los pesos como parámetros libres:

$$\left(\frac{exp(QW_q.(kW_k)^T.\sqrt{d_k}^{-1})}{\sum_{i=1}^{N} exp(QW_q.(kW_k)^T.\sqrt{d_k}^{-1}))} \right) * VW_v \tag{21.7}$$

21.3.3 Atención Cognitiva

Este último aspecto se refiere a una biomimesis y paralelo entre el mecanismo de Atención de los Transformers y el mecanismo cognitivo humano de *Atención Cognitiva*: este se refiere al proceso por

el cual los organismos seleccionan un subconjunto de la información disponible para concentrarla, integrarla y procesarla de manera más precisa. Tiene tres aspectos distintivos: la orientación, el filtrado y la búsqueda. La atención está relacionada con la conciencia [156] y a diferencia del mecanismo de **Attention** de los Transformers depende de un estado interno.

Todo esto es muy lindo, pero necesitamos convertir las cadenas de lexemas en números y para eso necesitamos un esquema de codificación.

21.4 Embedding

Los Embeddings son los diferentes esquemas de codificación de los lexemas. El primer proceso consiste en dividir la entrada en componentes. En lenguajes como inglés o español la división es mas directa ya que por ejemplo se podría dividir la entrada en palabras. Este es un proceso denominado **tokenization**. Las palabras (o *tokens*) se transforman en números en base a un diccionario global de codificación. Luego, cada token es representado en un vector numérico de dimensión fija, capturando relaciones semánticas y sintácticas en función de su contexto en el lenguaje. Para ello se aplica un algoritmo de embedding. Existen varias alternativas como *WordToVec*, *CBOW*, *Bag of Words* o incluso SVD.[3]

Este esquema tiene el problema de que el orden es invariante a las permutaciones, lo cuál sabemos que no es algo característico del lenguaje. Por lo tanto también se requiere que la codificación tenga en cuenta la ubicación del token en la secuencia.

Positional encoding: Una posible solución es agregar información a cada palabra sobre su posición en la oración. A esta información se la llama Positional Encoding, lo que es un conjunto de p_i valores que se le suma a cada embedding. Si bien hay diversas formas de abordar este inconveniente, en el paper de [37] se opta por utilizar las siguientes funciones sinusoidales para obtener el vector de posición del token i:

$$PE(pos, 2i) = \sin\left(\frac{pos}{1000^{\frac{2i}{d}}}\right) \tag{21.8}$$

$$PE(pos, 2i+1) = \cos\left(\frac{pos}{1000^{\frac{2i}{d}}}\right) \tag{21.9}$$

donde

- pos es la posición del token en la secuencia.
- i es el índice del vector de posición.
- d es la dimensión de un embedding, la cual será la misma que un vector de posición p_i.

21.5 Entrenamiento

Para el entrenamiento, hay que definir un vocabulario, incluyendo el *EOF*. Además hace falta el vocabulario también en inglés, el idioma destino (e.g.). La estructura del Transfomer es muy versátil y puede adaptarse a muchos tipos de problemas similares.

[3] Tener en cuenta que se está hablando de una ejemplo donde se toma como input un texto pero no necesariamente un token debe ser una palabra e incluso se pueden tener imágenes o señales de audio.

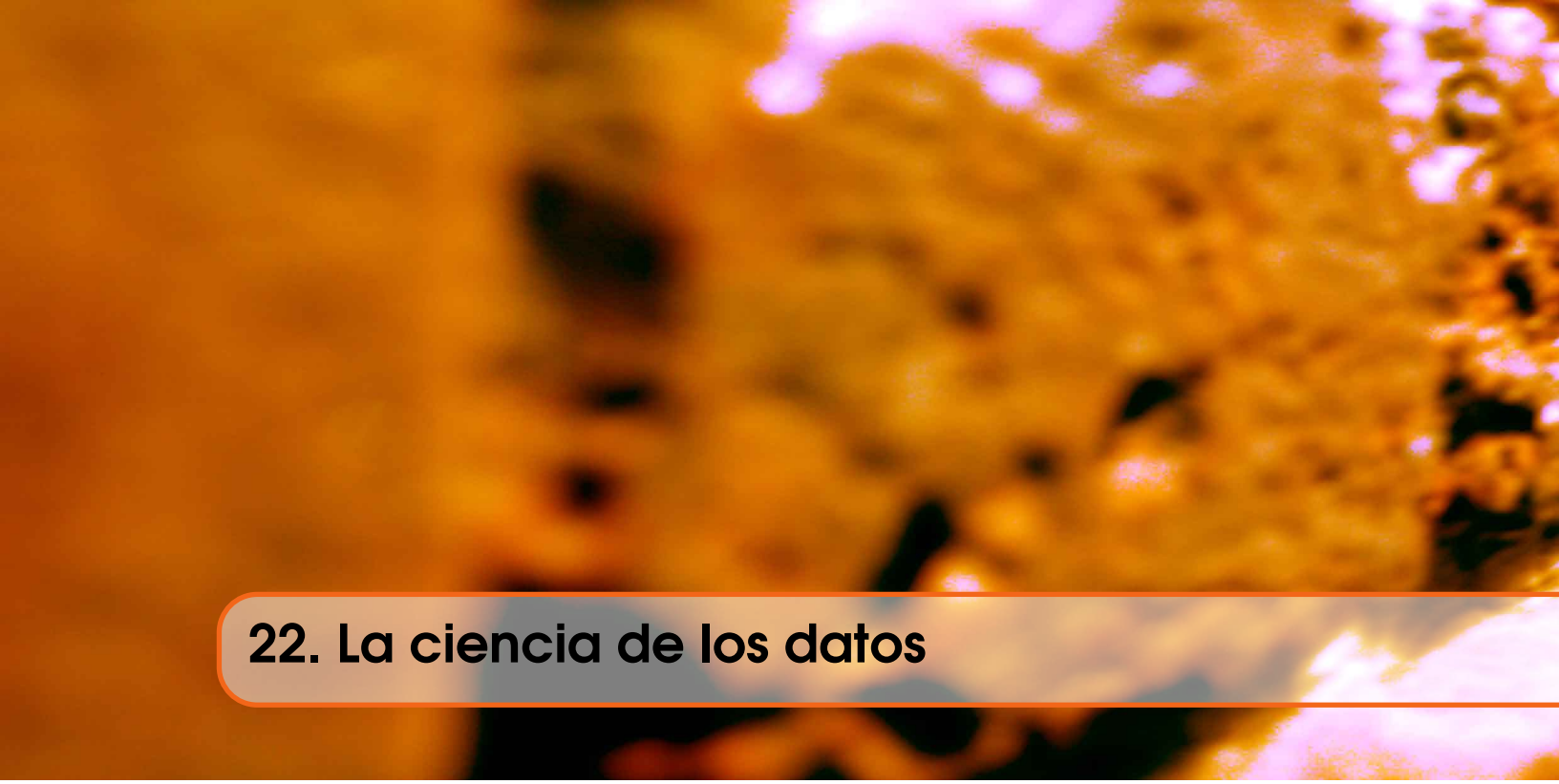
Los pesos se inicializan de forma aleatoria. El modelo sin entrenar devuelve una distribución de probabilidad con valores totalmente arbitrarios. Es necesario entonces seguir iterando, de manera supervisada, para que justamente la salida salga la palabra que se desea y así forzar toda la estructura para aprender [157].

VI Pragmatismo Profesional

22 La ciencia de los datos 203
22.1 El Análisis Cuantitativo
22.2 EDAs y ETLS
22.3 Falacias de la Ciencia de Datos
22.4 Inteligencia Artificial y la Frontera Digital

23 Guía para el apurado 209
23.1 Presentaciones
23.2 Condimentos de una red neuronal
23.3 Infraestructura de Software
23.4 Quiero más

Index 229



22. La ciencia de los datos

22.1 El Análisis Cuantitativo

Durante las primeras dos décadas del siglo 21, la revolución digital hizo que las corporaciones del mundo, de todo tipo, establecieran procesos digitales para llevar adelante sus empréstitos. Esto generó una montaña de datos.

Así surgieron en el mundo de IT, lo que se denomina las áreas de Datamining, Datawarehousing, y el área general de Bussiness Intelligence. Es decir, como almacenar, y explotar esos datos para mejorar la toma de decisión corporativa y los propios procesos internos. En estos desarrollos, generó momento un nuevo enfoque, donde el uso *inteligente* de esta información se fue volviendo central. Y para ello, se tomaron algunas prácticas y conceptos para lidiar con datos, muy típicos de la academia y del método científico (solo algunos).

Esto se da en una confluencia con una segunda rama de desarrollo, ahora revitalizada y generalizada, que son las disciplinas actuariales: un área que pela canas de aplicabilidad de la matemática que desarrollaba modelos para estudiar la incerteza, la esperanza de vida, para medir riesgo crediticio. Surge así el área de la **Ciencia de Datos**, y los roles de ingeniería de datos, analista de datos y cientista de datos.

La figura 22.2 resume un poco las diferentes áreas actuales de una organización desde la perspectiva de este enfoque. Primero el área de data science (DS) se centraliza en los datos, y en ofrecer herramientas para tomar decisiones y derivar de ellos procesos. En las organizaciones actuales, los datos se vuelven centrales. Las técnicas para procesarlos hoy se centran en Machine Larning (ML) que a su vez se divide en Deep Learning (DL), los métodos muy basados en esquemas bioinspirados de mecanismos biológicos, hoy fuertemente determinados por redes neuronales jerarquicas aglomerativas, y los métodos de machine learning que no son deep learning (a veces se los llama *shallow learning*). Este es el terreno donde habitan los **cientista de datos**. El punto central, donde se concentra verdaderamente la mayor parte de la actividad es **MLOps**, un proceso organizacional digital que se centra en el camino entero

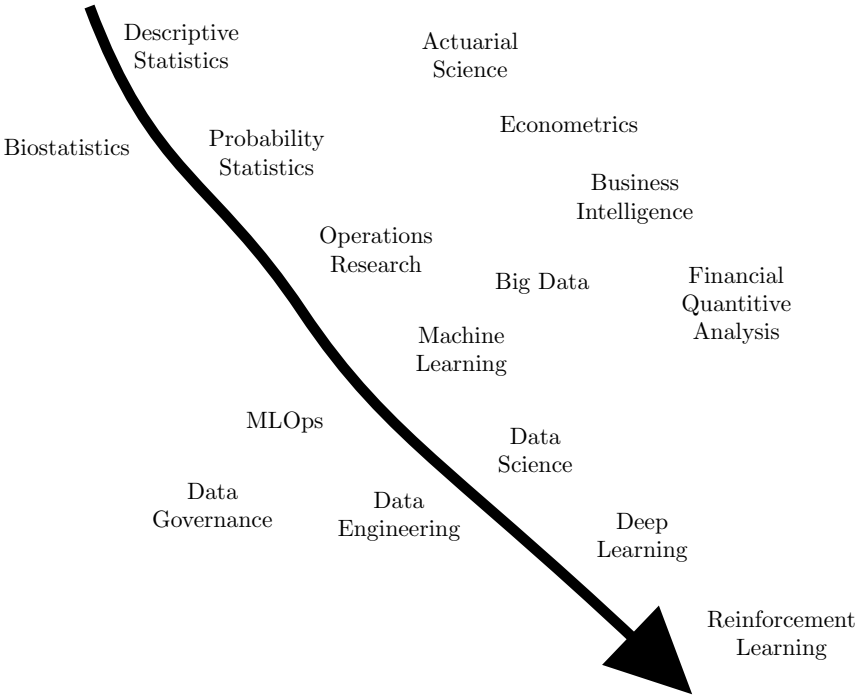


Figura 22.1: Mapa de las diferentes áreas vinculadas con la Ciencia de Datos

que va desde la propia generación de datos, su almacenamiento, su proceamiento, el mantenimiento y *governance* de los modelos y su ejecución. Este es el terreno principal del **ingeniero de datos**.

Finalmente, el producto del análisis de los datos genera **insights** que es donde los datos vuelven al mundo real, y ahí generan ideas reales que son analizadas justamente por los **analistas de datos**: estos son expertos específicos en el dominio del problema, que esos datos representan, pero que mucho más cuantitativamente que antes, análizan la información y la usan para tomar decisiones basadas en datos: **analytics**.

Esta área, corporativamente, requiere una conexión con conceptos de diseño mediante **visualizaciones** de esos datos, porque se requiere justamente poderosas abtracciones visuales para transmitir lo que los datos, fríos, sugieren. Finalmente, el área de **IA** mete la cola en este proceso siendo la encargada de mover las fronteras digitales, capturando datos más frescos, y ejecutando las decisiones en el mundo real de manera directa. Los avances de la Inteligencia Artificial, sobre todo en el NLP, expandieron el área y los roles de Data Science para materializar el rol de **AI Engineer**, quien se especializa justamente en la aplicación de estas tecnologías para la digitalización de procesos corporativos.

22.2 EDAs y ETLS

El trabajo y proceso de Data Science involucra dos étapas muy concretas y relevantes que se destacan del resto. La primera tiene que ver con ETL, *extract tranform and load*, y concierne a las actividades de obtención de datos de diversas y muy variadas fuentes, y su acopió en bases de datos centralizadas y estandarizadas. El proceso de ETL también involucra la compleja y laboriosa actividad

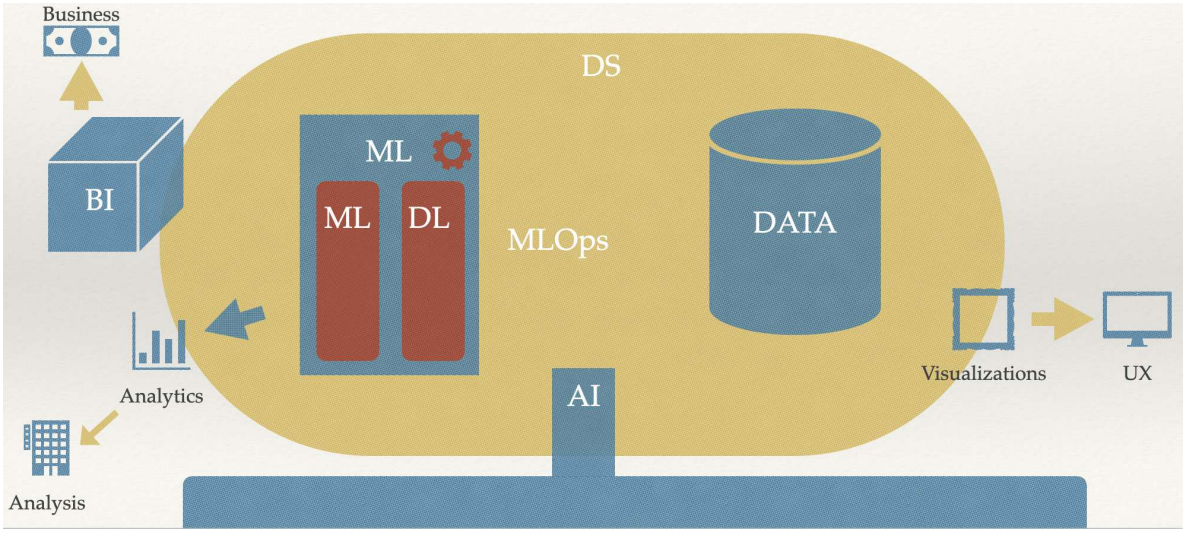


Figura 22.2: La ciencia de datos como núcleo de una organización.

del curado de datos[1]. Puede involucrar el desarrollo de elaboradas consultas de SQL sobre bases de datos, hasta **webcrawlers** o **scrappers** sofisticados, que navegan las páginas web extrayendo la información de ellas obtenida.

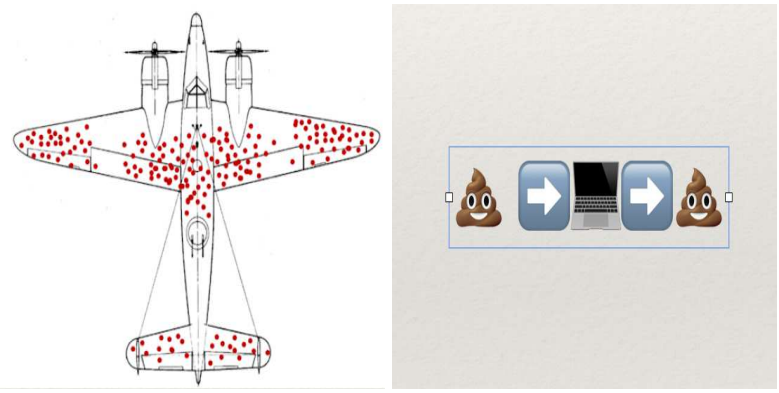
Por otro lado, una vez que los datos ya están más estandarizados y curados, los analistas de datos pueden embarcarse en la actividad identificada como EDA, *exploratory data analysis*. Entender la estructura inherente en los datos, mediante un conjunto de herramientas que busca transformaciones que representen la información de diversas maneras, a la vez, así como también visualizaciones que permitan entender, por un lado la estructura estadística de la información, y por otro, con un contexto específico de lo que los datos representan, ayudar a identificar los posibles patrones que se tienen que encontrar.

22.3 Falacias de la Ciencia de Datos

En Ciencia de Datos existen muchas falacias y puntos de cuidado que se heredan directamente del mundo científico, donde se reconoce la facilidad de arrivar a conclusiones debido a que **nos engañamos a nosotros mismos**. Ahí es donde aparece la necesidad del rigor, que empieza por ser concientes en qué tipo de falacias podemos caer.

- **Biased Cherry Picking**: se refiere a realizar una identificación selectiva de los resultados para que soporten lo que creemos o queremos decir.
- **Data Dredging**: se produce cuando asignamos a una correlación debido al azar mayor importancia que la que verdaderamente tiene.
- **Survivorship bias**: se establecen conclusiones en base a la existencia de muestras que de por sí dejan afuera, porque son las únicas que hay, a un conjunto grande solo por eso. Ejemplo del avión y de los que responden encuestas. Esto se ve en la figura 22.3a
- **Perverse Incentive**: si bien esta no es específica del análisis de datos, si lo es de las decisiones basadas en datos. La falacia se manifiesta cuando se busca maximizar un KPI, se establece una

[1] La mejor palabra en lunfardo es **cocina de datos**



(a) Survivorship Bias (b) Garbage In Garbage Out

Figura 22.3: (a)Los puntos rojos representan las balas recibidas por los aviones que volvían a la base, lo cual puede inferirse erróneamente que esos son los lugares que hay que proteger más. (b) Cualquier modelo puede como máximo hacer lo que los datos de entrada le permiten.

política que busca eso, y precisamente al maximizar el KPI produce el efecto contrario a lo que originalmente se buscaba en el mundo real. Ejemplo de las viboras en india.

- **False causality**: cuando dos cosas ocurren al mismo tiempo y erróneamente se les asigna una relación de causalidad.
- **Garbage in, Garbage out**: este es el mantra de redes neuronales. Lo que apunta es que cualquier implementación basada en datos, sólo va a ser tan buena como la calidad de los datos se lo permitan,
- **Sampling bias**: Técnicas como cross validation o bootstrapping intentan reducir la incidencia de esta falacia.
- **Gamblers fallacy**: asumir que la probabilidad de ocurrencia en sucesos que son independientes aumenta (o varía) porque realmente ocurrieron (es una falacia porque la p condicional es igual a la p). Si llevo una bomba al avión, reduzco la posibilidad de que otro la lleve.
- **Unbalanced datasets**: muy importante en health.
- **Simpsons paradox**: una tendencia aparece cuando los datos están desagrupados pero desaparece totalmente cuando se agrupan.
- **Clever Hans Effect**: muy común en redes neuronales artificiales y MLPs, cuando la red aprende patrones ocultos en los datos, que no son debido al fenómeno de estudio sino a algo indirectamente asociado al resultado. Hay un leakage de información en los datos que está oculto. Por ejemplo, radiografías que revelan la enfermedad del paciente, la etiqueta, porque son tomadas mayoritariamente cuando la persona está acostada [158].

Esta última falacia, muy controversial, queda para el final.

La gran falacia de Data Science

Las Decisiones son basadas en Datos.

La razón por la que es una falacia es porque los datos sin contexto, sin el manto de interpretabilidad, no dicen nada. **Podés acogotar tus datos hasta que te digan lo que vos querés escuchar**. Es decir,

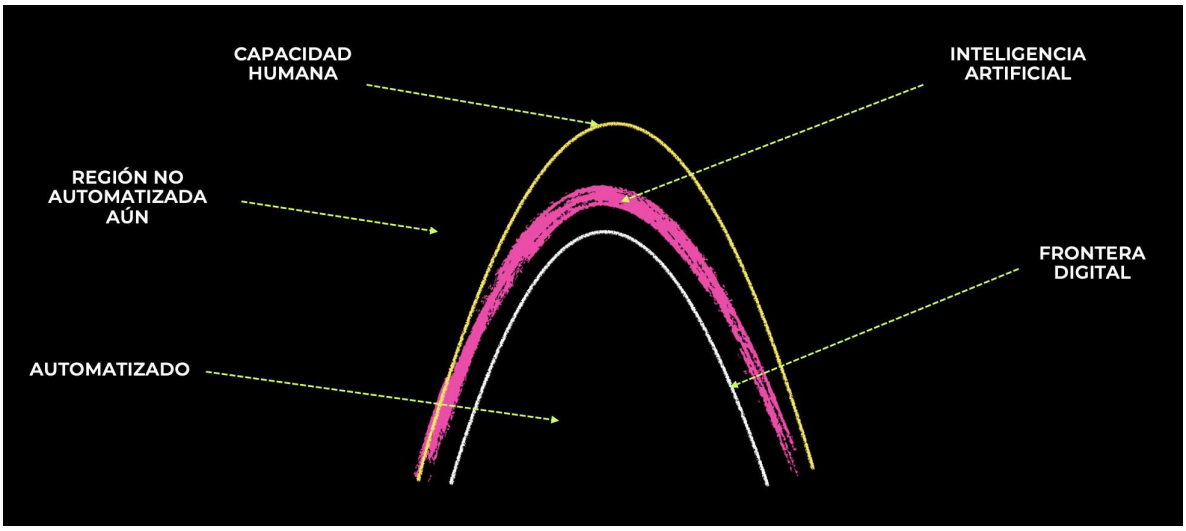


Figura 22.4: La Inteligencia Artificial entendida como la frontera digital que determina el borde entre lo que está y no automatizado.

los datos requieren inevitablemente una interpretación, con un componente de subjetividad que puede y debe reducirse, pero que no es totalmente eliminable. Siempre queda algo remanente. Una situación similar se presenta con los tests estadísticos de ensayo de hipótesis [159].

22.4 Inteligencia Artificial y la Frontera Digital

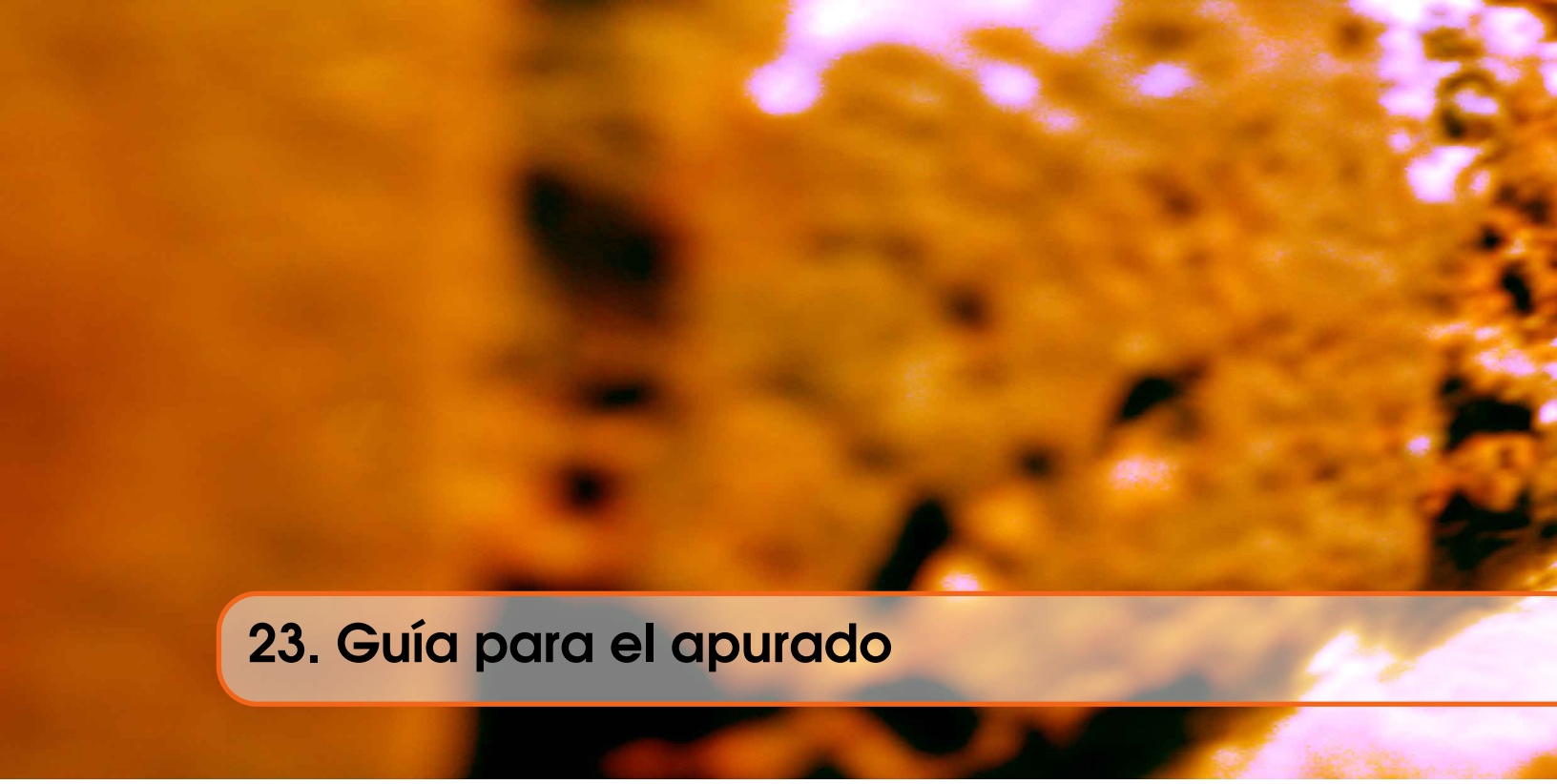
La Inteligencia Artificial, es además de una disciplina científica, un concepto tecnológico y de desarrollo de producto. Desde esta perspectiva, es posible evocar la figura 22.4. La región interna en esa curva son las actividades para los cuales los seres humanos pudimos encontrar un mecanismo de automatización. Es decir, sabemos cómo implementarlas de forma autónoma, como en una fila de dominó que requiere mover una ficha inicial y el resto simplemente es una secuencia de ejecución de pasos totalmente automática. Vamos a llamar a esa frontera, la *Frontera Digital*, porque es hoy mediante la digitalización y las computadoras la manera en la que estamos impulsando el desarrollo de tecnología que permite esos automatismos.

La región externa es la región donde todavía no pudimos encontrar una manera de automatizar, pero no sólo eso, es la región para la cuál existen problemas de los cuáles los seres humanos aún no sabemos como solucionar, o que se encuentran directamente por afuera de nuestra capacidad directa de solucionar (o con las herramientas que tenemos hasta el momento). Aparece entonces una nueva frontera, que es la generada por la *capacidad humana*, lo que nosotros los seres humanos somos capaces de hacer (en promedio). Desde esta perspectiva, la inteligencia artificial es la región inmediatamente posterior a la frontera digital. Es todo aquello que aún no sabemos automatizar, pero que está inmediatamente posterior a lo que sí sabemos, y que por esa razón la ponemos en el foco. Esto es coincidente con la idea que la inteligencia artificial es un *moving goalpost* que ha ido cambiando con el tiempo y que ha referenciado temas distintos a lo largo de estas 7 décadas en las que ha existido [106]. Por ejemplo, el área de Automatas y Compiladores solía enmarcarse dentro del área de inteligencia artificial, hasta que el problema se tecnificó, se encontró una manera de abordarlo efectiva que facilitó las soluciones, tomó cuerpo propio, movió la frontera digital, y dejó de pertenecer a esta área.

Esta perspectiva permite también entender, que puede ocurrir en un momento, que la linea de la frontera digital supera la capacidad humana, y tenemos allí lo que se denomina capacidades *superhuamanas*, es decir mecanismos cognitivos automáticos que tienen un desempeño mejor que el promedio de los seres humanos, o quizás mejor que cualquier ser humano. Por ejemplo, pensemos en un dispositivo diseñado para dada una imagen de una tribuna de un estadio, con miles de personas, reconocer en un minuto todas las caras que se vislumbran. Los seres humanos somos buenos reconociendo caras, pero no tan buenos para lograr algo así.

Sobre la frontera digital, y en el límite de lo automatizable, se ubican siempre las zonas donde se empuja la innovación: estos son los temas donde la tecnificación permite generar un impacto en la sociedad de manera directa, generando productos o servicios que pueden encausarse como desarrollos comerciales. Hoy (circa 2024) los temas que ocupan estos espacios son justamente el área de NLP, CV, Data Science, algoritmos generativos, etc.

En linea con esto y asociado a las ofertas de trabajo, el área de Data Science generó una subvariante denominada "IA Engineering". Esta área se concentra sobre todo en implementaciones que exploran los últimos desarrollos técnicos sobre todo en NLP y Computer Vision (por el lado de las redes generativas). Esto es comprensible desde la perspectiva del desarrollo tecnológico ya que la mayor cantidad de información disponible en las industrias y empresas es en general texto [160].



23. Guía para el apurado

23.1 Presentaciones

- La presentación debe ser de 20 minutos aproximadamente, más un tiempo de preguntas individuales.
- Orientada a los resultados (que a su vez son orientados a los datos (que a su vez son orientados a los experimentos (que a su vez son orientados a las implementaciones (que a su vez son orientadas al entendimiento de los temas (que a su vez son orientados a la presencia en las teóricas o el tiempo invertido en ver los videos)))))
- Evitar mostrar código (y menos capturas).
- Evaluación de los parámetros
- Gráficos representativos de los resultados
- Al ser algoritmos estocásticos, recordar promediar varias ejecuciones y visualizar la varianza.
- Ojo con las escalas en los gráficos. Mantener las mismas al momento de hacer comparaciones.
- Fijar un seed particular para reproducir resultados.
- Las Conclusiones son muy importantes y no pueden faltar.
- Entrega: incluir el repositorio, incluyendo ppt, readme.md y config.json
- AI Coding Literacy: reglas y recomendaciones para sacarle el jugo a los asistentes y agentes de código `https://medium.com/@rramele/ai-coding-literacy-ce650eeb9ade`.
- Más información [161].
- Videos de todas las clases: `https://www.youtube.com/playlist?list=PLJHMIS4ekxNRJlqsVzpmh6pWqBsI2FOGd`.

23.2 Condimentos de una red neuronal

Cuando se usa una red neuronal para solucionar un problema se requieren decidir los siguiente componentes.

Arquitectura: necesitan decidir la arquitectura de la red, decidir la cantidad de capas, y la cantidad de neuronas por capas. Así también como los diferentes componentes y cómo se encadenan.

Función de Activación: dependiendo de la naturaleza del problema, entender cuál debería ser la función de activación más pertinente. Y sobre todo cuáles van a ser los rangos de valores de entrada y de salida de esas funciones.

Función de Costo: elegir la mejor función del costo en base al objetivo que se le dará a la red: clasificar, regresionar, optimizar o generar. Por ejemplo, la función de error tradicional 23.1 es la que se utiliza en MLPs pero no es la única.

23.2.1 Transformación de los datos

Existen diferentes maneras de preprocesar y transformar los datos, y la nomenclatura no es muy clara en relación a qué es cada caso. Acá ofrecemos una aclaración a modo de resumen y de selección arbitraria de la nomenclatura que se utiliza en todo el texto.

- Normalizar
- Min-Max Scaling
- Feature Scaling
- Standardization Z-Score
- Box-Cox.

Definición 23.2.1 — Normalization.

$$X_{new} = \frac{X}{||X||} \tag{23.1}$$

Definición 23.2.2 — Min-Max Scaling.

$$X_{new} = \frac{X_{old} - X_{min}}{X_{max} - X_{min}}(b-a) + a \tag{23.2}$$

Definición 23.2.3 — Feature Scaling.

$$X_{new} = \frac{X_{old}}{X_{max}}, X_{min} = 0, b = 1, a = 0 \tag{23.3}$$

Definición 23.2.4 — Z-Score, Standardization = Statistical Standardization , Estandarizar.

$$X_{new} = \frac{X_{old} - \bar{X}}{X_{std}} \tag{23.4}$$

donde $\bar{X}$ es la media y X_{std} el desvío estándar.

Definición 23.2.5 — Box-Cox Normalization. Convierte los datos mediante una optimización para que luzcan normales (que provienen de una distribución $\mathcal{N}()$

23.2.2 Balanceo de clases

Tanto cuando se utiliza aprendizaje supervisado como no-supervisado, el balanceo de los datos, o el sesgo inherente, son importantes considerarlos para poder evaluar con claridad los resultados [162].

Esto es un aspecto clave bien conocido, pero que sin embargo muchas veces suele olvidarse completo. Por ejemplo, al utilizar una red neuronal con aprendizaje supervisado e implementar un clasificador binario. Si los datos están extremadamente desbalanceados, una población de cientos de datos contra millones entre una clase y la otra, un clasificador que siempre asigna el label de la clase que tiene más muestras tendrá una eficiencia altísima, totalmente inútil.

23.2.3 Funciones de Costo

Existen diferentes funciones de costo que pueden utilizarse en las redes neuronales [163]. La tabla 23.1 muestra algunas de las más utilizadas que hemos tocado en este texto.

Nombre	Expresión
SSE Adaline	$\frac{1}{2}\sum_{\mu=1}^{p}(\zeta^{\mu}-O^{\mu})$
RMS	$\sum_{i,\mu}\left\{\frac{1}{2}(1+\zeta_i^{\mu})\,log\frac{1+\zeta_i^{\mu}}{1+O_i^{\mu}}+\frac{1}{2}(1-\zeta_i^{\mu})\,log\frac{1-\zeta_i^{\mu}}{1-O_i^{\mu}}\right\}$
MAE	$\frac{1}{p}\sum_{\mu=1}^{p}\lvert\zeta^{\mu}-O^{\mu}\rvert$
Binary Cross Entropy	$-\frac{1}{p}\sum_{i=1}^{p}\{(y_i)\log p(y_i)+(1-y_i)\log(1-p(y_i))\}$
Huber	$\begin{cases}\frac{1}{p}\sum_{i=1}^{p}\frac{1}{2}(\zeta^{\mu}-O^{\mu})^2 & \lvert\zeta^{\mu}-O^{\mu}\rvert <= \delta \\ \frac{1}{p}\sum_{i=1}^{p}\delta(\lvert\zeta^{\mu}-O^{\mu}\rvert-\frac{1}{2}\delta) & \lvert\zeta^{\mu}-O^{\mu}\rvert > \delta\end{cases}$

Cuadro 23.1: Diferentes funciones de error

23.2.4 Funciones de Activación

También existen diversas funciones de activación que se utilizan en las redes neuronales, y que son las responsables de agregarle principalmente la no-linealidad. Desde la más simples y tradicionales hasta las más complejas que se utilizan en escenarios muy específicos, como Softplus y Mish que son versiones suavizada de ReLU. La tabla 23.2 detalla las utilizadas en este texto. La derivada acá está expresada en los términos de las entradas no de las salidas (es decir están expresadas en los términos de la entrada de la función de activación no de su salida).

23.3 Infraestructura de Software

Como se expone en este documento, el desarrollo de esta disciplina está cimentado en el hardware que permitió la realización en tiempo y forma de muchas de las ideas y trucos aquí también expuestos. Pero es además menester destacar la infraestructura de software montada también para esta tarea y que permite estructurar un *stack* de procesamiento sobre el que construir hacia arriba. El diagrama en bloques de la figura 23.1 muestra el stack de plataformas y librerías más utilizadas para el desarrollo de modelos y soluciones de Inteligencia Artificial.

CPU, GPU y FPGA

Al sustrato de hardware tradicional de CPUs, más el agregado actual de GPUs, se le suma también la posibilidad de programar en otros como FPGAs. Los **F**ield **P**rogrammable **G**ate **A**rrays son dispositivos

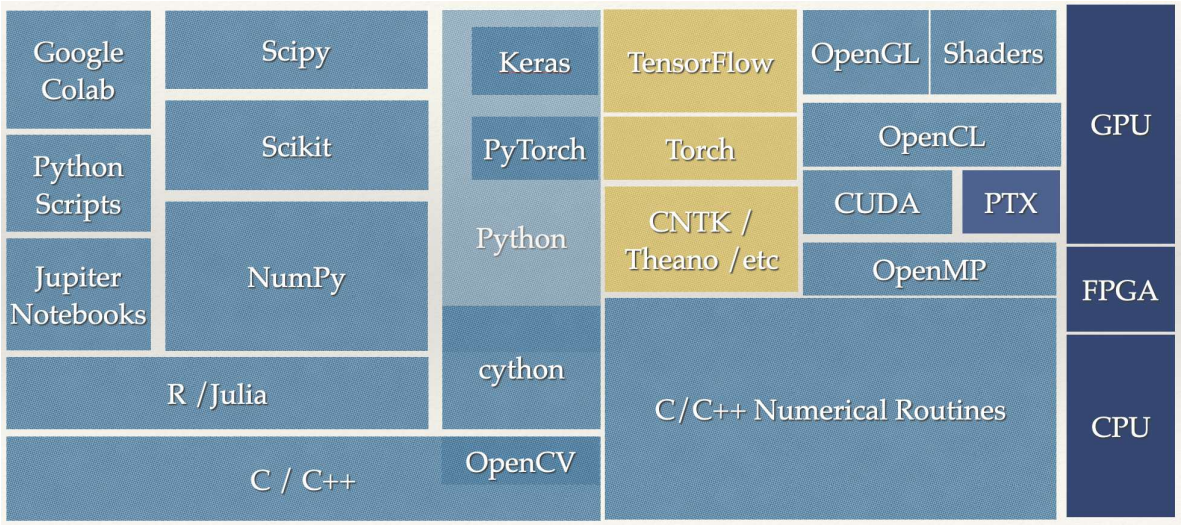


Figura 23.1: Un esbozo de la infraestructura de software y hardware que sustentan el stack que se utiliza para el desarrollo de aplicaciones y sistemas de análisis de datos e inteligencia artificial. Mayoritariamente son librerías de código abierto.

electrónicos de propósito general que pueden modificarse electrónicamente para reestructurar los circuitos de forma que queden de manera específica, cableados, maximizando la eficiencia. Los ASIC, **A**pplication **S**pecific **I**ntegrated **C**ircuit, son una variante que no pueden reprogramarse posteriormente. Este nivel de especialización se materializa con hardware específico para las típicas operaciones de inteligencia artificial, en hardware concreto como los NPU, **N**euro **P**rocessor **U**nits.

OpenGL y Shaders

OpenGL es la librería tradicional abierta para la programación gráfica. Es una vieja librería, muy especializada que está orientada para la creación de gráficos en tres dimensiones. Encapsula la compleja lógica requerida para la programación en 3D en un set de instrucciones de alto nivel. La aparición del hardware específico gráfico, los inicios de las GPUs, forzó a la creación de los shaders de GLSL, **G**raphics **L**ibrary **S**haders **L**anguage, que es código que directamente se compila y corre en el GPU. Esto permitia resolver la lógica dual de procesamiento, donde código general de un juego corre principalmente en un CPU, pero los cálculos gráficos, i.e. multiplicación de matrices, corren directamente manipulado por GLSL en el GPU, y se utilizan interfaces de copia de información estandarizadas entre la memoria uno y otro.

OpenMP y OpenCL

OpenMP es una API que sirve para realizar programación en paralelo, pensada justamente para poder portar de manera simple programas hechos en C++ a cualquier tipo de plataforma. Justamente, es agnóstico del sistema operativo y del hardware, y además está basado en cláusulas para el preprocesador facilitando la modificación de programas existentes. OpenCL por otro lado, fue concebido para poder correr el mismo código en diferentes tipos de procesadores, como ser CPUs, GPUs, FPGAs, etc. Eso lo hace también útil para construir y desarrollar programas que requieren alta capacidad de procesamiento.

CUDA y PTX

CUDA es una de las razones del dominio de NVidia sobre el mercado de los GPUs. Es una plataforma de software muy completa para programar directamente utilizando las capacidades de paralismo de los GPUs de NVidia. Esta librería permite estructurar los programas para operar de manera tradicional en arquitecturas símil x86-64 y sobre eso derivar código que opera sobre la memoria de la GPU, realizando los intercambios de datos entre las diferentes memorias. PTX, **P**arallel **T**hread **E**xecution es el lenguaje de assembler para la máquina virtual en la que corren instrucciones que directamente se mapean a uno o más GPUs de NVidia. Es el lenguaje intermedio utilizado en CUDA.

Python

Este viejo lenguaje de scripting ha encontrado su lugar y acceso al podio máximo como lingua franca para las actividades de data science e inteligencia artificial. Además de su sintaxis simple, una de las claves de este éxito es justamente la librería *cython* que permite interoperabilidad entre librerías numéricas muy optimizadas construidas en Fortran o C++ [164].

NumPy

La más importante de esas librerías en el ecosistema de Python es Numpy [164]. Estas son rutinas de álgebra lineal para multiplicación de matrices, que realizan un manejo muy eficiente de la memoria. Son rutinas optimizadas para correr sobre CPUs.

SciPy

Esta librería es parte del combo con muchas rutinas prácticas para el manejo de datos e información científica, particularmente procesamiento de imágenes y de señales. Es una librería clave que encapsula mucha funcionalidad asociada al preprocesamiento y curado de información científica.

Scikit Learn

Librería base de las rutinas de aprendizaje automático para Python. Si hay una rutina de aprendizaje automático para Python, está en esta librería.

Pandas

Pandas es una librería práctica que facilita el curado de datos, la ingeniería de datos. Modelada y estructurada en base al lenguaje y plataforma R, particularmente el concepto de *dataframe*. Esto facilita el uso de python para aquellas personas que ya conocen esta plataforma, que es fuertemente orientada al análisis actuarial y estadístico. Un aspecto importante es que esta librería hace énfasis en los datos y su metadata asociada.

MatplotLib, Ploty, Seaborn, Altair

Python es un mar de librerías para hacer prácticamente de todo. Una tarea básica que aparece en el procesamiento de los datos, y en IA orientada a datos, es justamente la necesidad de graficar información. **Matplotlib** es la librería tradicional para hacerlo, que aunque cuenta con una interfaz de funciones bastante dura, se puede hacer de todo. **Ploty** por otro lado, es mucho más simple de utilizar pero más limitada en las funcionalidades que implementa. **Seaborn** es mucho más refinada en cuánto a la usabilidad y el diseño gráfico de los elementos gráficos, y a la vez trabaja a la para de **Pandas** con lo que es muy útil para destacar información considerando también la metadata asociada. Sin embargo, un aspecto clave a considerar son las buenas prácticas de diseño asociadas a los gráficos científicos, y desde esa perspectiva, **Altair** es la gran ganadora porque justamente respeta diferentes lineamientos estéticos y de eficiencia comunicacional [165].

TensorFlow y Keras

TensorFlow es una de las librerías principales que permiten la programación de modelos de redes neuronales y su ejecución en CPUs, o GPUs, facilitando la utilización de las librerías de bajo nivel como **CUDA**, **OpenML**, **OpenCL**, y estableciendo las abstracciones de capas, funciones de activación, funciones de costo, etc, necesarias para construir redes neuronales. Tensorflow tiene además un gran secreto, que es **AutoDiff** [166]. Estas librerías, en sus diferentes variantes en las diferentes plataformas, construye el árbol computacional,que determina cómo calcular el gradiente en base a las operaciones que se realizan en las diferentes funciones de un programa, y realiza los cálculos simbólicos y numéricos para determinar el gradiente de la operación. Esto por supuesto, es clave para la construcción de librerías de deep learning. Tensorflow es una librería de más bajo nivel para C++, y su porte a Python es Keras [77].

Torch y PyTorch

Torch es otra popular librería que provee las bases y abstracciones para trabajar con redes neuronales profundas. Su versión para Python, PyTorch es utilizada en muchas implementaciones de redes neuronales cerradas que se distribuyen como componentes, como tal es el caso de redes neuronales convolucionales.

OpenCV

OpenCV es una extraordinaria librería, muy abarcativa, que incluye una numerosa cantidad de algoritmos específicos para el procesamiento de imágenes y de visión por computadora. Entre ellos, incluye muchos paquetes que preceden a las librerías actuales de deep learning pero que tienen el mismo objetivo. Además es una herramienta muy útil para la creación de aplicaciones de todo tipo.

R y Julia

R es un lenguaje pensado y estructurado para hacer análisis estadísticos, y por esa razón es muy popular también en ciencia de datos e inteligencia artificial. *Si una distribución de probabilidad no aparece en R, entonces no existe.* Julia por otro lado, es un lenguaje específicamente creado para el procesamiento de datos y para aprendizaje automático. ¿Será el lenguaje y plataforma del futuro de esta área? Quizás lo sea, o quizás lo sea el mismo lenguaje natural, pero por ahora la escena está dominado por el viejo Python.

23.4 Quiero más

¿Más? Listo por hoy. A la calle, a respirar aire fresco y tomar un poco de sol.

Nombre	$g(h)$	$g'(h)$
Identidad	h	1
Tangente Hiperbólica	$tanh(\beta h) = \frac{e^h - e^{-h}}{e^h + e^{-h}}$	$\beta(1 - g(h)^2)$
Sigmoidea	$\frac{1}{1+exp^{2\beta x}}$	$2\,\beta g(h)(1-g(h))$
Softsign	$\frac{1}{1+\lvert x\rvert}$	$\frac{1}{(1+\lvert x\rvert^2)}$
Función Escalón	$\begin{cases} 0 & h < 0 \\ 0.5 & h = 0 \\ 1 & h > 0 \end{cases}$	Delta de Dirac $del(h)$
Función Signo	$\begin{cases} -1 & h < 0 \\ 0 & h = 0 \\ 1 & h > 0 \end{cases}$	$2\,del(h)$
Lineal por partes	$\begin{cases} 1 & h >= \frac{1}{2} \\ h + \frac{1}{2} & -\frac{1}{2} < h < \frac{1}{2} \\ 0 & h <= -\frac{1}{2} \end{cases}$	$\begin{cases} 0 & h >= \frac{1}{2} \\ 1 & -\frac{1}{2} < h < \frac{1}{2} \\ 0 & h <= -\frac{1}{2} \end{cases}$
ReLU	$max(0,h)$	$\begin{cases} 0 & h < 0 \\ 1 & h >= 0 \end{cases}$
Softplus	$\frac{1}{\beta} ln(1 + e^{\beta\, h})$	$\frac{e^h}{1+e^h}$
Mish	$tanh(softplus(h))$	$\frac{e^h \omega}{\delta^2}, \omega = e^{3h} + e^{2h}(4) + e^h(6 + 4h) + (1+h), \delta = (e^h+1)^2 + 1$

Cuadro 23.2: Funciones de activación simples con sus derivadas.

Nombre	$g(h)$	$g'(h)$
Softmax	$\sigma(\vec{V})_i = \frac{e^{V_i}}{\sum_{j=1}^{K} e^{V_i}}$	$\begin{cases} \sigma(\vec{V})_i(1-\sigma(\vec{V})_i) & i = k \\ -\sigma(\vec{V})_i(\sigma(\vec{V})_i) & i \neq k \end{cases}$

Cuadro 23.3: Funciones de activación simples con sus derivadas (2/2).

Bibliografía

[1] Microsoft, *Neurips Conference Analytics*, 2024 (accessed Feb 27, 2024), https://www.microsoft.com/en-us/research/project/academic/articles/neurips-conference-analytics/.

[2] S. J. Russell and P. Norvig, *Artificial intelligence a modern approach*. London, 2010.

[3] J. A. Hertz, *Introduction to the theory of neural computation*. Crc Press, 2018.

[4] S. Haykin, *Neural networks: a comprehensive foundation*. Prentice Hall PTR, 1998.

[5] M. Mitchell, *An introduction to genetic algorithms*. MIT press, 1998.

[6] I. Goodfellow, Y. Bengio, and A. Courville, *Deep learning*. MIT press, 2016.

[7] C. C. Aggarwal *et al.*, "Neural networks and deep learning," *Springer*, vol. 10, no. 978, p. 3, 2018.

[8] S. Axler, *Linear algebra done right*. Springer Nature, 2023.

[9] J. Gallier and J. Quaintance, "Algebra, topology, differential calculus, and optimization theory for computer science and engineering," 2019.

[10] S. P. Boyd and L. Vandenberghe, *Convex optimization*. Cambridge university press, 2004.

[11] A. M. Turing *et al.*, "On computable numbers, with an application to the entscheidungsproblem," *J. of Math*, vol. 58, no. 345-363, p. 5, 1936.

[12] P. Stone, *The Artificial Intelligence Papers*. Sebtel Press, 2024.

[13] W. S. McCulloch and W. Pitts, "A logical calculus of the ideas immanent in nervous activity," *The bulletin of mathematical biophysics*, vol. 5, pp. 115–133, 1943.

[14] R. E. Brown, "The life and work of donald olding hebb," *Acta Neurologica Taiwanica*, vol. 15, no. 2, p. 127, 2006.

[15] A. M. Turing, *Computing machinery and intelligence*. Springer, 1950.

[16] D. Taraborrelli and R. Gala, "Génesis y actualidad de la inteligencia artificial (ia) en las instituciones públicas de la argentina, una mirada desde los escyt," in *XI Jornadas de Sociología. Facultad de Ciencias Sociales, Universidad de Buenos Aires, Buenos Aires, 2015*. Acta Académica, 2015, p. 01.

[17] P. N. Johnson-Laird and M. Ragni, "What should replace the turing test?" *Intelligent Computing*, vol. 2, p. 0064, 2023.

[18] A. Newell and H. Simon, "The logic theory machine–a complex information processing system," *IRE Transactions on information theory*, vol. 2, no. 3, pp. 61–79, 1956.

[19] P. Stone, R. Brooks, E. Brynjolfsson, R. Calo, O. Etzioni, G. Hager, J. Hirschberg, S. Kalyanakrishnan, E. Kamar, S. Kraus *et al.*, "Artificial intelligence and life in 2030: the one hundred year study on artificial intelligence," 2016.

[20] J. E. Hopcroft and J. D. Ullman, *Formal languages and their relation to automata*. Addison-Wesley Longman Publishing Co., Inc., 1969.

[21] N. Wiener, "Cybernetics or control and communication," *The animal and the machine*, 1948.

[22] R. Solomonoff, "Dartmouth archives," https://raysolomonoff.com/dartmouth/, 2024 (accessed Jan 10, 2025).

[23] B. Bloomfield, "The question of artificial intelligence," https://www-formal.stanford.edu/jmc/reviews/bloomfield/bloomfield.html, 2024 (accessed Jan 10, 2025).

[24] N. Nillson, *The quest for Artificial Intelligence*. Standford University, 2010.

[25] S. L. Brunton and J. N. Kutz, *Data-driven science and engineering: Machine learning, dynamical systems, and control*. Cambridge University Press, 2022.

[26] R. McNaughton, "The theory of automata, a survey," in *Advances in Computers*, ser. Advances in Computers, F. L. Alt, Ed. Elsevier, 1961, vol. 2, pp. 379–421. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0065245808601448

[27] C. Bassett, "The computational therapeutic: exploring weizenbaum's eliza as a history of the present," *AI & SOCIETY*, vol. 34, no. 4, pp. 803–812, 2019.

[28] C. WenBin, L. XiaoLing, L. YiJun, and F. Yu, "A machine learning algorithm for expert system based on mycin model," in *2010 2nd International Conference on Computer Engineering and Technology*, vol. 2. IEEE, 2010, pp. V2–262.

[29] X. Yang and C. Zhu, "Industrial expert systems review: a comprehensive analysis of typical applications," *IEEE Access*, 2024.

[30] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, "Learning representations by back-propagating errors," *nature*, vol. 323, no. 6088, pp. 533–536, 1986.

[31] M. Minsky and S. Papert, "An introduction to computational geometry," *Cambridge tiass., HIT*, vol. 479, no. 480, p. 104, 1969.

[32] A. Clayton, "How eugenics shaped statistics," https://nautil.us/how-eugenics-shaped-statistics-238014/, 2019 (accessed Jul 18, 2024).

[33] T. Haigh, "Between the booms ai in winter," https://cacm.acm.org/opinion/between-the-booms-ai-in-winter/, 2024 (accessed Oct 10, 2024).

[34] J. Pearl, "Probabilistic reasoning in intelligent systems. representation & reasoning," 1988.

[35] V. N. Vapnik, "Statistical learning theory," 1998.

[36] R. Szeliski, *Computer vision: algorithms and applications*. Springer Nature, 2022.

[37] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," *Advances in neural information processing systems*, vol. 30, 2017.

[38] R. Carnota and R. Rodriguez, "Antonio monteiro: pionero de los estudios de computación en argentina," in *Simposio de Historia de la Informá ica en América La ina y el Caribe*, 2012.

[39] P. M. Jacovkis, "Manuel sadosky (1914-2005)," *Revista de la Unión Matemática Argentina*, vol. 46, no. 1, pp. 67–71, 2005.

[40] H. Monteverde, "60 años de sadio," *Electronic Journal of SADIO (EJS)*, vol. 20, no. 2, pp. 2–22, 2021.

[41] I. Daffra, *Historia de la industria informática argentina*. CESSI Argentina, 2014.

[42] R. Wachenchauzer, "The evolution of computer education in latin america: the case of argentina," *ACM Inroads*, vol. 5, no. 1, pp. 70–76, 2014.

[43] J. M. Santos and C. Touzet, "Exploration tuned reinforcement function," *Neurocomputing*, vol. 28, no. 1-3, pp. 93–105, 1999.

[44] C. Hurtado, A. O. Mendelzon, and A. A. Vaisman, "Maintaining data cubes under dimension updates," in *Proceedings 15th International Conference on Data Engineering (Cat. No. 99CB36337)*. IEEE, 1999, pp. 346–355.

[45] A. A. Vaisman and A. O. Mendelzon, "A temporal query language for olap: Implementation and a case study," in *International Workshop on Database Programming Languages*. Springer, 2001, pp. 78–96.

[46] M. C. Parpaglione, "Neural networks applied to fault detection using acoustic emission," in *Non-destructive Testing'92*. Elsevier, 1992, pp. 71–75.

[47] A. C. Frery, H.-J. Muller, C. d. C. F. Yanasse, and S. J. S. Sant'Anna, "A model for extremely heterogeneous clutter," *IEEE transactions on geoscience and remote sensing*, vol. 35, no. 3, pp. 648–659, 1997.

[48] J. Gambini, M. E. Mejail, J. Jacobo-Berlles, and A. C. Frery, "Feature extraction in speckled imagery using dynamic b-spline deformable contours under the model," *International Journal of Remote Sensing*, vol. 27, no. 22, pp. 5037–5059, 2006.

[49] R. Kurzweil, "The singularity is near," in *Ethics and emerging technologies*. Springer, 2005, pp. 393–406.

[50] E. Strickland, "The turbulent past and uncertain future of ai: Is there a way out of ai's boom-and-bust cycle?" *IEEE Spectrum*, vol. 58, no. 10, pp. 26–31, 2021.

[51] T. Urban, *Wait but Why: AI Superintelligence*, 2022 (accessed July 1, 2022), https://waitbutwhy.com/2015/01/artificial-intelligence-revolution-1.html.

[52] S. Legg, M. Hutter *et al.*, "A collection of definitions of intelligence," *Frontiers in Artificial Intelligence and applications*, vol. 157, p. 17, 2007.

[53] A. Ciaunica, A. Safron, and J. Delafield-Butt, "Back to square one: the bodily roots of conscious experiences in early life," *Neuroscience of Consciousness*, vol. 2021, no. 2, p. niab037, 2021.

[54] A. Clark, *Supersizing the mind: Embodiment, action, and cognitive extension*. OUP USA, 2008.

[55] S. Tadelis, *Game theory: an introduction*. Princeton university press, 2013.

[56] P. A. Tipler, *Physics for Scientists and Engineers: Regular Version, Ch. 1-35 and 39*. Macmillan, 1999.

[57] T. Virkkala, "Solving sokoban," Ph.D. dissertation, Master's thesis, University of Helsinki, 2011.

[58] J. H. Holland, "Genetic algorithms and the optimal allocation of trials," *SIAM journal on computing*, vol. 2, no. 2, pp. 88–105, 1973.

[59] N. Ashton and C. Stringer, "Did our ancestors nearly die out?" *Science*, vol. 381, no. 6661, pp. 947–948, 2023.

[60] D. L. Goodstein, *States of matter*. Courier Corporation, 2014.

[61] P. Pedregal, *Introduction to Optimization*. Springer, 2003.

[62] H. A. Taha, *Investigación de operaciones*. Pearson Educación, 2004.

[63] R. P. Brent, *Algorithms for minimization without derivatives*, Prentice-Hall, Ed. Englewood Cliffs, N.J., Prentice-Hall, 1973.

[64] J. A. Snyman, D. N. Wilke *et al.*, *Practical mathematical optimization*. Springer, 2005.

[65] P. Moritz, R. Nishihara, and M. Jordan, "A linearly-convergent stochastic l-bfgs algorithm," in *Artificial Intelligence and Statistics*. PMLR, 2016, pp. 249–258.

[66] L. Bottou, *Online Algorithms and Stochastic Approximations. Online Learning and Neural Networks.*, C. U. Press., Ed. Cambridge University Press., 1998.

[67] M. Lefkowitz, *Perceptron pave way to ai 60 years ago*, 2024 (accessed July 1, 2024), https://news.cornell.edu/stories/2019/09/professors-perceptron-paved-way-ai-60-years-too-soon.

[68] E. Agliari, M. Aquaro, A. Barra, A. Fachechi, and C. Marullo, "From pavlov conditioning to hebb learning," *Neural Computation*, vol. 35, no. 5, pp. 930–957, 2023.

[69] K. Shirriff, "The surprising story of the first microprocessors," *IEEE Spectrum*, vol. 53, no. 9, pp. 48–54, 2016.

[70] D. H. Hubel and T. N. Wiesel, "Receptive fields, binocular interaction and functional architecture in the cat's visual cortex," *The Journal of physiology*, vol. 160, no. 1, p. 106, 1962.

[71] G. Cybenko, "Approximation by superpositions of a sigmoidal function," *Mathematics of control, signals and systems*, vol. 2, no. 4, pp. 303–314, 1989.

[72] K. Hornik, "Approximation capabilities of multilayer feedforward networks," *Neural Networks*, vol. 4, no. 2, pp. 251–257, 1991.

[73] R. A. Fisher, "On the mathematical foundations of theoretical statistics," *Philosophical Transactions of the Royal Society of London. Series A*, vol. 222, pp. 309–368, 1922.

[74] S. Kullback and R. A. Leibler, "On information and sufficiency," *The Annals of Mathematical Statistics*, vol. 22, no. 1, pp. 79–86, 1951.

[75] G. Hinton, N. Srivastava, and K. Swersky, "Neural networks for machine learning lecture 6a overview of mini-batch gradient descent," *Cited on*, vol. 14, no. 8, p. 2, 2012.

[76] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," *arXiv preprint arXiv:1412.6980*, 2014.

[77] F. Chollet, *Deep learning with Python*. Simon and Schuster, 2021.

[78] T. Hastie, R. Tibshirani, J. H. Friedman, and J. H. Friedman, *The elements of statistical learning: data mining, inference, and prediction*. Springer, 2009, vol. 2.

[79] R. Roelofs, V. Shankar, B. Recht, S. Fridovich-Keil, M. Hardt, J. Miller, and L. Schmidt, "A meta-analysis of overfitting in machine learning," *Advances in Neural Information Processing Systems*, vol. 32, 2019.

[80] C. F. Caiafa, E. Salerno, and A. N. Proto, "Blind source separation applied to spectral unmixing: comparing different measures of nongaussianity," in *Knowledge-Based Intelligent Information and Engineering Systems: 11th International Conference, KES 2007, XVII Italian Workshop on Neural Networks, Vietri sul Mare, Italy, September 12-14, 2007, Proceedings, Part III 11*. Springer, 2007, pp. 1–8.

[81] E. Schubert, J. Sander, M. Ester, H. P. Kriegel, and X. Xu, "Dbscan revisited, revisited: why and how you should (still) use dbscan," *ACM Transactions on Database Systems (TODS)*, vol. 42, no. 3, pp. 1–21, 2017.

[82] T. Kohonen, "Self-organized formation of topologically correct feature maps," *Biological Cybernetics*, vol. 1, no. 43, pp. 59–69, 1982.

[83] ——, "The self-organizing map." *Neurocomputing*, pp. 1–6, 1998.

[84] B. Everitt, G. Dunn *et al.*, *Applied multivariate data analysis*. Wiley Online Library, 2001, vol. 2.

[85] R. S. A. of Sciences, *Nobel Prize in Physics 2024*, 2024 (accessed October 17, 2024), https://www.nobelprize.org/prizes/physics/2024/press-release/.

[86] A. K. John Hertz and R. Palmer, *Introduction to the Theory of Neural Computation*, Addison-Wesley, Ed. Addison-Wesley, 1991.

[87] M. Scilipoti, M. Fuster, and R. Ramele, "A strong inductive bias: Gzip for binary image classification," *arXiv preprint arXiv:2401.07392*, 2024.

[88] E. Oja, "Principal components, minor components, and linear neural networks," *Neural networks*, vol. 5, no. 6, pp. 927–935, 1992.

[89] T. D. Sanger, "Optimal unsupervised learning in a singlelayer linear feedforward neural network," *Neural Networks*, vol. 2, no. 6, pp. 459–473, 1989.

[90] P. Dayan and L. F. Abbott, *Theoretical neuroscience: computational and mathematical modeling of neural systems*. MIT press, 2005.

[91] K. Doya, *Bayesian brain: Probabilistic approaches to neural coding*. MIT press, 2007.

[92] P. Shenoy, K. J. Miller, J. G. Ojemann, and R. P. Rao, "Finger movement classification for an electrocorticographic bci," in *2007 3rd International IEEE/EMBS Conference on Neural Engineering*. IEEE, 2007, pp. 192–195.

[93] C. S. Nam, A. Nijholt, and F. Lotte, *Brain–computer interfaces handbook: technological and theoretical advances*. CRC press, 2018.

[94] E. Niedermeyer and F. L. da Silva, *Electroencephalography: basic principles, clinical applications, and related fields*. Lippincott Williams & Wilkins, 2005.

[95] O. Sacks, *The man who mistook his wife for a hat and other clinical tales*. Simon And Schuster, 1985.

[96] R. E. Bryant and D. R. O'Hallaron, *Computer systems: a programmer's perspective*. Prentice Hall, 2011.

[97] M. Krzisch, S. G. Temprana, L. A. Mongiat, J. Armida, V. Schmutz, M. A. Virtanen, J. Kocher-Braissant, R. Kraftsik, L. Vutskits, K.-K. Conzelmann *et al.*, "Pre-existing astrocytes form functional perisynaptic processes on neurons generated in the adult hippocampus," *Brain Structure and Function*, vol. 220, pp. 2027–2042, 2015.

[98] W. Elmenreich, "An introduction to sensor fusion," *Vienna University of Technology, Austria*, vol. 502, pp. 1–28, 2002.

[99] N. Chaffey, "Alberts, b., johnson, a., lewis, j., raff, m., roberts, k. and walter, p. molecular biology of the cell. 4th edn." 2003.

[100] E. M. Izhikevich, *Dynamical systems in neuroscience*. MIT press, 2007.

[101] A. L. Hodgkin and A. F. Huxley, "A quantitative description of membrane current and its application to conduction and excitation in nerve," *The Journal of physiology*, vol. 117, no. 4, p. 500, 1952.

[102] R. Brooks, "An inconvenient truth about ai: Ai won't surpass human intelligence anytime soon," *IEEE Spectrum*, 2021.

[103] A. L. Samuel, "Some studies in machine learning using the game of checkers," *IBM Journal of research and development*, vol. 3, no. 3, pp. 210–229, 1959.

[104] L. Acion, L. Alemany Alonso, E. Ferrante, E. L. Holm, V. Martinez, D. H. Milone, R. Rodriguez, G. Simari, and S. Uchitel, "Desmistificando la inteligencia artificial," *Inteligencia Artificial, Una Mirada Interdisciplinaria X Encuentro Interacademico 2021*, vol. 1, no. 1, pp. 63–86, 2021.

[105] M. Mitchell, "Why AI is harder than we think," *CoRR*, vol. abs/2104.12871, 2021. [Online]. Available: https://arxiv.org/abs/2104.12871

[106] P. Stone, R. Brooks, E. Brynjolfsson, R. Calo, O. Etzioni, G. Hager, J. Hirschberg, S. Kalyanakrishnan, E. Kamar, S. Kraus *et al.*, "Artificial intelligence and life in 2030: the one hundred year study on artificial intelligence," *arXiv preprint arXiv:2211.06318*, 2022.

[107] J. Degrave, F. Felici, J. Buchli, M. Neunert, B. Tracey, F. Carpanese, T. Ewalds, R. Hafner, A. Abdolmaleki, D. de Las Casas *et al.*, "Magnetic control of tokamak plasmas through deep reinforcement learning," *Nature*, vol. 602, no. 7897, pp. 414–419, 2022.

[108] M. Simon, "How a plucky robot found the long-lost endurance shipwreck," https://www.wired.com/story/how-a-plucky-robot-found-the-long-lost-endurance-shipwreck, 2022 (accessed Jul 18, 2024).

[109] A. Fawzi, M. Balog, A. Huang, T. Hubert, B. Romera-Paredes, M. Barekatain, A. Novikov, F. J. R Ruiz, J. Schrittwieser, G. Swirszcz *et al.*, "Discovering faster matrix multiplication algorithms with reinforcement learning," *Nature*, vol. 610, no. 7930, pp. 47–53, 2022.

[110] J. Jumper, R. Evans, A. Pritzel, T. Green, M. Figurnov, O. Ronneberger, K. Tunyasuvunakool, R. Bates, A. Žídek, A. Potapenko *et al.*, "Highly accurate protein structure prediction with alphafold," *nature*, vol. 596, no. 7873, pp. 583–589, 2021.

[111] R. Sutton, "The bitter lesson," http://www.incompleteideas.net/IncIdeas/BitterLesson.html, 2019 (accessed May 29, 2024).

[112] P. Domingos, "A few useful things to know about machine learning," *Communications of the ACM*, vol. 55, no. 10, pp. 78–87, 2012.

[113] O. Boiman, E. Shechtman, and M. Irani, "In defense of nearest-neighbor based image classification," *26th IEEE Conference on Computer Vision and Pattern Recognition, CVPR*, 2008, this paper is the NBNN method which is similar to what I am doing with Sift descriptors.

[114] M. Telgarsky, "Deep learning theory lecture notes," https://mjt.cs.illinois.edu/dlt/, 2021, version: 2021-10-27 v0.0-e7150f2d (alpha).

[115] T. Poggio, H. Mhaskar, L. Rosasco, B. Miranda, and Q. Liao, "Why and when can deep-but not shallow-networks avoid the curse of dimensionality: A review," *International Journal of Automation and Computing*, vol. 14, no. 5, pp. 503–519, 2017. [Online]. Available: https://doi.org/10.1007/s11633-017-1054-2

[116] J. Schmidhuber, "Deep learning in neural networks: An overview," *Neural networks*, vol. 61, pp. 85–117, 2015.

[117] G. E. Hinton, S. Osindero, and Y.-W. Teh, "A fast learning algorithm for deep belief nets," *Neural computation*, vol. 18, no. 7, pp. 1527–1554, 2006.

[118] S. Hooker, "The hardware lottery," *Communications of the ACM*, vol. 64, no. 12, pp. 58–65, 2021.

[119] D. Garisto, "How cutting-edge computer chips are speeding up the ai revolution," *Nature*, vol. 630, no. 8017, pp. 544–546, 2024.

[120] M. Pandey, M. Fernandez, F. Gentile, O. Isayev, A. Tropsha, A. C. Stern, and A. Cherkasov, "The transformational role of gpu computing and deep learning in drug discovery," *Nature Machine Intelligence*, vol. 4, no. 3, pp. 211–221, 2022.

[121] M. A. Kramer, "Nonlinear principal component analysis using autoassociative neural networks," *AIChE journal*, vol. 37, no. 2, pp. 233–243, 1991.

[122] D. P. Kingma and M. Welling, "An introduction to variational autoencoders," *Foundations and Trends in Machine Learning*, vol. 12, no. 4, pp. 307–392, 2019. [Online]. Available: http://dx.doi.org/10.1561/2200000056

[123] K. P. Murphy, *Machine learning: a probabilistic perspective*. MIT press, 2012.

[124] jhu edu., *Variational Tutorial*, 2011 (accessed September 3, 2018), http://www.cs.jhu.edu/~jason/tutorials/variational.html.

[125] C. W. Fox and S. J. Roberts, "A tutorial on variational bayesian inference," *Artificial intelligence review*, vol. 38, no. 2, pp. 85–95, 2012.

[126] M. E. Khan and G. Bouchard, "Variational em algorithms for correlated topic models," Technical report, University of British Columbia, Tech. Rep., 2009.

[127] C. Tang and R. R. Salakhutdinov, "Learning stochastic feedforward neural networks," *Advances in Neural Information Processing Systems*, vol. 26, 2013.

[128] E. W. Weisstein, *Stochastic Function*, 2023 (accessed November 17, 2023), https://mathworld.wolfram.com/StochasticFunction.html.

[129] J. R. Anderson and C. Peterson, "A mean field theory learning algorithm for neural networks," *Complex Systems*, vol. 1, pp. 995–1019, 1987.

[130] M. J. Wainwright and M. I. Jordan, *Graphical models, exponential families, and variational inference*. Now Publishers Inc, 2008.

[131] G. E. Hinton and D. Van Camp, "Keeping the neural networks simple by minimizing the description length of the weights," in *Proceedings of the sixth annual conference on Computational learning theory*, 1993, pp. 5–13.

[132] G. Gundersen., *Reparamatrization Trick*, 2021 (accessed October 2, 2021), https://gregorygundersen.com/blog/2018/04/29/reparameterization/.

[133] B. Rocca, *Stack Exchange, Machine Learning*, 2021 (accessed October 2, 2021), https://stats.stackexchange.com/questions/420974/backpropagation-on-variational-autoencoders.

[134] E. S. Piñeiro, R. Ramele, and J. Gambini, "Variational autoencoder as a data augmentation tool for confocal microscopy images," in *2023 IEEE 36th International Symposium on Computer-Based Medical Systems (CBMS)*. IEEE, 2023, pp. 882–885.

[135] D. M. Blei, A. Kucukelbir, and J. D. McAuliffe, "Variational inference: A review for statisticians," *Journal of the American Statistical Association*, vol. 112, no. 518, pp. 859–877, Feb 2017. [Online]. Available: http://dx.doi.org/10.1080/01621459.2017.1285773

[136] C. Doersch, "Tutorial on variational autoencoders," 2021.

[137] C. C. Aggarwal, *Neural Networks and Deep Learning*. Cham: Springer, 2018.

[138] D. P. Kingma and M. Welling, "Auto-encoding variational bayes," 2014.

[139] J. Jordan, *Autoencoders*, 2022 (accessed Novemenber 1, 2022), https://www.jeremyjordan.me/autoencoders/.

[140] G. W. Lindsay, "Convolutional neural networks as a model of the visual system: Past, present, and future," *Journal of Cognitive Neuroscience*, pp. 1–15, Feb 2020. [Online]. Available: http://dx.doi.org/10.1162/jocn_a_01544

[141] B. E. Ail, R. Ramele, J. Gambini, and J. M. Santos, "An intrinsically explainable method to decode p300 waveforms from eeg signal plots based on convolutional neural networks," *Brain Sciences*, vol. 14, no. 8, p. 836, 2024.

[142] K. Fukushima and N. Wake, "Handwritten alphanumeric character recognition by the neocognitron," *IEEE transactions on Neural Networks*, vol. 2, no. 3, pp. 355–365, 1991.

[143] Y. LeCun, B. Boser, J. S. Denker, D. Henderson, R. E. Howard, W. Hubbard, and L. D. Jackel, "Backpropagation applied to handwritten zip code recognition," *Neural computation*, vol. 1, no. 4, pp. 541–551, 1989.

[144] C. C. Aggarwal *et al.*, *Neural networks and deep learning*. Springer, 2018.

[145] V. Dumoulin and F. Visin, "A guide to convolution arithmetic for deep learning," mar 2016. [Online]. Available: http://arxiv.org/abs/1603.07285

[146] E. Shelhamer, J. Long, and T. Darrell, "Fully Convolutional Networks for Semantic Segmentation," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 39, no. 4, pp. 640–651, nov 2017. [Online]. Available: http://arxiv.org/abs/1411.4038

[147] S. Jiang and V. M. Zavala, "Convolutional Neural Nets: Foundations, Computations, and New Applications," jan 2021. [Online]. Available: http://arxiv.org/abs/2101.04869

[148] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "Imagenet classification with deep convolutional neural networks," in *Advances in neural information processing systems*, 2012, pp. 1097–1105.

[149] D. Foster, *Generative deep learning*. O'Reilly Media, Inc., 2022.

[150] I. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio, "Generative adversarial nets," *Advances in neural information processing systems*, vol. 27, 2014.

[151] M. Kahng, N. Thorat, P. Chau, F. Viégas, and M. Wattenberg, *Gan Lab*, 2023 (accessed November 17, 2023), https://poloclub.github.io/ganlab/.

[152] M. Klingermann, "Quasimondo," https://quasimondo.com/, 2024 (accessed Jul 18, 2024).

[153] H. T. Siegelmann and E. D. Sontag, "On the computational power of neural nets," in *Proceedings of the fifth annual workshop on Computational learning theory*, 1992, pp. 440–449.

[154] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[155] Y.-H. H. Tsai, S. Bai, M. Yamada, L.-P. Morency, and R. Salakhutdinov, "Transformer dissection: a unified understanding of transformer's attention via the lens of kernel," *arXiv preprint arXiv:1908.11775*, 2019.

[156] R. Wright and W. LM, *Orienting of Attention*, 2008.

[157] E. Fedorenko, S. T. Piantadosi, and E. A. Gibson, "Language is primarily a tool for communication rather than thought," *Nature*, vol. 630, no. 8017, pp. 575–586, 2024.

[158] J. Kauffmann, L. Ruff, G. Montavon, and K.-R. Müller, "The clever hans effect in anomaly detection," *arXiv preprint arXiv:2006.10609*, 2020.

[159] D. Huff, *How to lie with statistics*. Penguin UK, 2023.

[160] A. Said and V. Torra, *Data science in practice*. Springer, 2019.

[161] E. Piñeiro, "SIA tips," https://github.com/eugepineiro/sia-tips, 2022 (accessed September 3, 2023).

[162] A. Zhou, F. Tajwar, A. Robey, T. Knowles, G. J. Pappas, H. Hassani, and C. Finn, "Do deep networks transfer invariances across classes?" in *International Conference on Learning Representations*, 2021.

[163] M. Brooks, "Inside the maths that drives ai," *Nature*, vol. 631, no. 8019, pp. 244–246, 2024.

[164] C. R. Harris, K. J. Millman, S. J. Van Der Walt, R. Gommers, P. Virtanen, D. Cournapeau, E. Wieser, J. Taylor, S. Berg, N. J. Smith *et al.*, "Array programming with numpy," *Nature*, vol. 585, no. 7825, pp. 357–362, 2020.

[165] L. Wilkinson, "The grammar of graphics," in *Handbook of computational statistics: Concepts and methods*. Springer, 2011, pp. 375–414.

[166] N. Ketkar, J. Moolayil, N. Ketkar, and J. Moolayil, "Automatic differentiation in deep learning," *Deep Learning with python: learn best practices of deep learning models with PyTorch*, pp. 133–145, 2021.

Índice alfabético

A

Agente 27
Alelo 45
Alta dimensionalidad 141
Aprendizaje Automático 97
aprendizajeincremental 84
aprendizajeporlotes 84
Aprobación 7
Aptitud 45
Aptitud Relativa 45
Autoencoder 147

B

Backtracking 33
BFS 32
Búsqueda 29

C

Coca Cola 53
Comunicación 7
Convolución 168
Cromosoma 44

D

Data Science 197
Datasets Masivos 145
Decomposición Espectral 148
Deep Learning 139
DFS 33

E

Evaluación 5

F

Fenotipo 44
Funciones Estocásticas 154

G

GA 43
GAN 181
Generación 0 45
Genotipo 44
GPUs 145
Graph-Search 31

H

Heurística 33
Hopfield 117

I

Inferencia Variacional 156
Inteligencia 11
Inverse Sampling Theorem 165

K

k-Fold Cross Validation 93
KL 163
Kohonen 113

L

Locus 44

M

MLE 163
MLP 82
MLS 164
Modelo Generativo 181
Momentum 59

O

Oja 123

P

PCA 107
Perceptrón Simple 69
Profundidad 141
Propriocepción 129

R

Reglamento 5
Regularización 163

S

Sanger 126
Simplex 55
Softmax 175
SVD 148

T

Tied Weights 165
Tips 203
Trabajos Prácticos 6
Transformers 187
Travel Salesman Problem 38
Tío y Tía 139

U

UCS 32